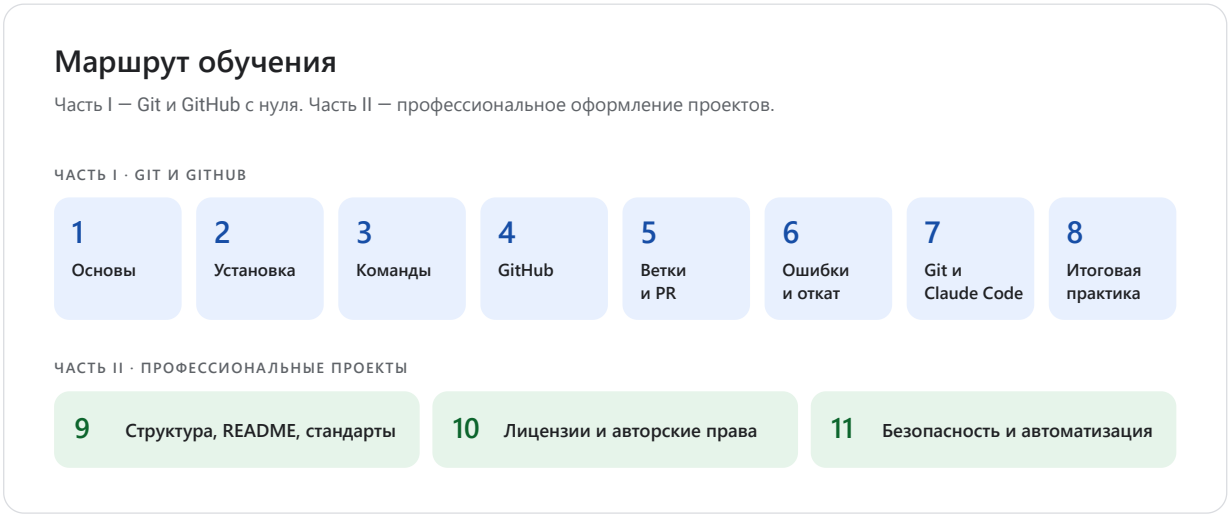
# Git и GitHub с нуля

Методичка для начинающих: от первого коммита до профессионально оформленного проекта, лицензий, безопасности и работы с AI-ассистентами




Сведения проверены по официальным источникам 11 октября 2026 года · Windows 10 и 11 · Git 2.56

# Содержание

**О методичке** 7

ЧАСТЬ I · GIT И GITHUB С НУЛЯ

**Глава 1. Git и GitHub: основы** 14

1.1. Зачем нужен контроль версий 14
1.2. Что такое Git 15
1.3. Что такое GitHub 15
1.4. Git и GitHub: в чём разница 16
1.5. Главные понятия 16
1.6. Бытовая аналогия 18
1.7. Как связаны вы, Git и GitHub 19
1.8. Чего Git не делает 19
Проверьте себя 20
Что дальше 20

**Глава 2. Установка и первая настройка** 21

2.1. Что такое терминал 21
2.2. Установка Git на Windows 23
2.3. Проверка установки 24
2.4. Первая настройка Git 25
2.5. Проверка конфигурации 26
2.6. PowerShell и Git Bash: что выбрать 27
Проверьте себя 28
Что дальше 28

**Глава 3. Основные команды Git** 29

3.1. Три зоны Git 29
3.2. Подготовка учебной папки 30
3.3. git init — создать репозиторий 31
3.4. git status — посмотреть состояние 32
3.5. git add — подготовить изменения 33
3.6. git commit — сохранить версию 34
3.7. git log — посмотреть историю 36
3.8. git diff — посмотреть изменения 37
3.9. git clone — скачать репозиторий 38
3.10. git push — отправить коммиты на GitHub 40
3.11. git pull — получить изменения с GitHub 41
3.12. git fetch — узнать, что нового, ничего не меняя 42
3.13. Шпаргалка 43
Проверьте себя 44
Что дальше 44

**Глава 4. Работа с GitHub** 45
4.1. Аккаунт на GitHub 45
4.2. Настройка приватности почты 46
4.3. Создание репозитория на сайте 46
4.4. Что такое README.md 48
4.5. Работа с файлами через сайт 49
4.6. История изменений на сайте 52
4.7. Скачать проект 52
4.8. Вход в GitHub из терминала 53
4.9. Удалённый репозиторий: git remote 54
4.10. Отправка проекта на GitHub: сценарий А — проект уже на компьютере 55
4.11. Сценарий Б — новый проект с нуля 58
4.12. Если на GitHub уже есть README.md 59
4.13. Ежедневная синхронизация 61
Проверьте себя 61
Что дальше 62
**Глава 5. Ветки и Pull Request** 63
5.1. Что такое ветка 63
5.2. Зачем нужна main 64
5.3. Создание и переключение веток 64
5.4. Объединение веток — git merge 65
5.5. Отправка ветки на GitHub 67
5.6. Что такое Pull Request 67
5.7. Работа через Pull Request: пошагово 68
5.8. Просмотр изменений перед объединением 70
5.9. Командная работа 71
5.10. Шпаргалка 72
Проверьте себя 72
Что дальше 72
**Глава 6. Конфликты, восстановление и типичные ошибки** 73
6.1. Что такое конфликт слияния 73
6.2. Как найти и исправить конфликт 74
6.3. Отменить незакоммиченные изменения 77
6.4. Убрать файл из staging 80
6.5. Исправить последний коммит 80
6.6. Отменить коммит безопасно — git revert 80
6.7. Восстановить предыдущую версию файла 81
6.8. git reset — перемотать историю назад 82
6.9. restore, revert и reset: в чём разница 84
6.10. Спасательный круг: git reflog 85
6.11. Почему нельзя хранить секреты в репозитории 86
6.12. .gitignore: что не должно попасть в Git 86
6.13. Публичные и приватные репозитории: правила безопасности 88
6.14. Если push не проходит 89

6.15. Ошибки авторизации 90
6.16. Как читать сообщения терминала 90
6.17. Ошибка → причина → решение 91
Проверьте себя 92
Что дальше 93

**Глава 7. Git и Claude Code 94**

7.1. Что такое Claude Code 94
7.2. Открыть существующий проект 95
7.3. Проверить состояние Git перед работой с AI 95
7.4. Создать контрольную точку 96
7.5. Поставить задачу AI 96
7.6. Просмотреть изменения, сделанные AI 97
7.7. Запустить тесты 98
7.8. Проверить diff перед коммитом 98
7.9. Сохранить работающую версию 98
7.10. Вернуться к предыдущему состоянию 99
7.11. Ветки для экспериментов с AI 99
7.12. Как не потерять код 100
7.13. Рекомендуемый рабочий цикл 101
7.14. Профессиональное оформление проектов с AI 102
7.15. Пример структуры проекта для разработки с AI 106
Проверьте себя 107
Что дальше 107

**Глава 8. Практика и итоговые проекты 108**

8.1. Упражнения по главам 108
8.2. Проект 1. «Мой первый репозиторий» 109
8.3. Проект 2. «Профессионально оформленный репозиторий» 113
8.4. Чек-лист готовности репозитория к публикации 117
8.5. Контрольный список навыков 118
8.6. Что изучать дальше 119

ЧАСТЬ II · ПРОФЕССИОНАЛЬНЫЕ ПРОЕКТЫ

**Глава 9. Стандарты и оформление проектов 121**

9.1. Стандарты, практики и требования: как их различать 121
9.2. Структура профессионального репозитория 122
9.3. README.md — главная страница проекта 126
9.4. Документы сопровождения 127
9.5. Версионирование: SemVer, теги и релизы 130
9.6. Сообщения коммитов: Conventional Commits 131
9.7. Международные стандарты и инженерные практики 132
9.8. Жизненный цикл программного продукта 139
9.9. Уровни зрелости проекта 140
Проверьте себя 140

Что дальше 141

**Глава 10. Лицензии и авторские права 142**

10.1. Основы: авторское право и лицензия 142
10.2. Виды лицензий 144
10.3. Сравнение лицензий 145
10.4. Когда возникают обязательства 148
10.5. Как выбрать лицензию 149
10.6. Как добавить лицензию в проект 152
10.7. Авторство, NOTICE и сторонние компоненты 153
10.8. Код, созданный с помощью AI 155
10.9. SPDX и REUSE: машиночитаемые сведения о лицензиях 156
10.10. Что зависит от страны и договоров 157
10.11. Практические примеры 158
Проверьте себя 158
Что дальше 159

**Глава 11. Безопасность и автоматизация 160**

11.1. CI и GitHub Actions 160
11.2. Пример workflow для проекта на Python 161
11.3. Линтеры, форматтеры и тесты 164
11.4. Защита основной ветки 164
11.5. Проверка изменений в Pull Request 165
11.6. Токены, ключи и файлы .env 166
11.7. Secret scanning и push protection 167
11.8. Если секрет попал в историю 168
11.9. Зависимости: Dependabot, dependency review и лицензии 169
11.10. CodeQL: поиск уязвимостей в коде 171
11.11. Закрытые сообщения об уязвимостях 171
11.12. Минимальные права доступа 172
11.13. Безопасность GitHub Actions и CI/CD 172
11.14. Чек-лист безопасности репозитория 173
Проверьте себя 173
Что дальше 174

ПРИЛОЖЕНИЯ

**Словарь терминов 176**

**Источники и полезные ссылки 182**

**Пример README учебного проекта tempconv 188**

**Шаблоны документов 192**

README.template.md 192
CONTRIBUTING.template.md 195
SECURITY.template.md 196
CHANGELOG.template.md 197

CODE_OF_CONDUCT.template.md ..... 199
ISSUE_TEMPLATE.md ..... 200
PULL_REQUEST_TEMPLATE.md ..... 201
LICENSE_SELECTION_GUIDE.md ..... 202
PROJECT_RELEASE_CHECKLIST.md ..... 205

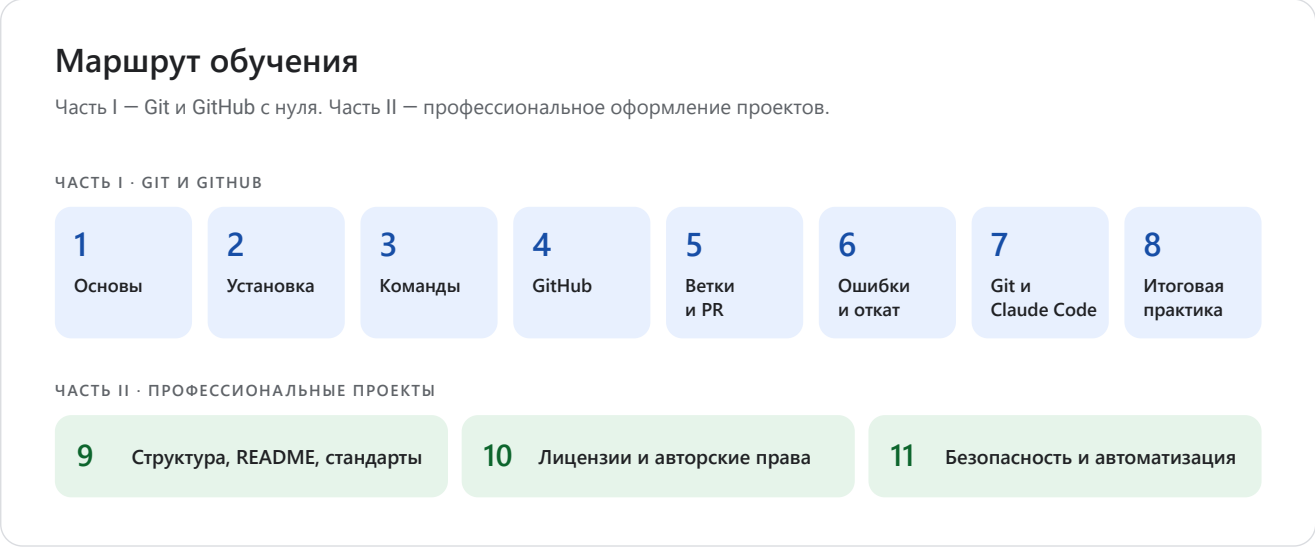
# О методичке

**Бесплатная методичка на русском языке: от первого коммита до профессионально оформленного репозитория.**

Методичка для тех, кто начинает изучать разработку программного обеспечения и никогда не работал с Git. Вы научитесь сохранять версии проекта, работать с GitHub, безопасно использовать AI-ассистентов и оформлять проекты по международным практикам.

## Что внутри

- **С нуля и по шагам.** Для каждой команды — зачем она нужна, как её ввести, что вы увидите и какие ошибки встречаются чаще всего.
- **Проверено на практике.** Выводы команд получены в настоящем Git, факты сверены с официальной документацией **11 октября 2026 года**.
- **Для Windows 10 и 11.** PowerShell и Git Bash, установка Git, вход в GitHub из терминала.
- **Наглядно.** 25 схем и диаграммы Mermaid: три области Git, ветки, Pull Request, восстановление изменений, утечка секрета.
- **С практикой.** Вопросы с ответами в каждой главе, упражнения и два итоговых проекта.
- **Как в профессиональных проектах.** README, лицензии, CONTRIBUTING, SECURITY, SemVer, CI, Dependabot, CodeQL — и готовые шаблоны.
- **Git и AI.** Как работать с AI-ассистентом и не потерять контроль над кодом: ветки, контрольные точки, просмотр изменений, откат.




**После части I вы сможете** самостоятельно создать репозиторий, загрузить в него проект, вносить и сохранять изменения, работать с ветками и Pull Request и при необходимости восстановить предыдущую версию.

**После части II вы сможете** оформить репозиторий как профессиональный проект: README, лицензия, документы сопровождения, версии и релизы, автоматические проверки и настройки безопасности.

## Кому подойдёт

- Студентам и начинающим разработчикам, которые ни разу не работали с Git.
- Тем, кто пользуется Git по памяти и хочет понять, что происходит при каждой команде.
- Авторам проектов, которые готовят репозиторий к публикации.
- Преподавателям и наставникам: материалы можно свободно использовать на занятиях с указанием авторства (см. лицензию).

## Первый коммит за 10 минут

1. Установите Git — способы описаны в главе 2. Через winget:

   ```
   POWERSHELL
   winget install --id Git.Git -e --source winget
   ```

2. Откройте **новое** окно PowerShell и представьтесь Git — эти данные попадут в каждый коммит:

   ```
   POWERSHELL
   git config --global user.name "Иван Петров"
   git config --global user.email "ivan.petrov@example.com"
   git config --global init.defaultBranch main
   ```

3. Создайте учебную папку и репозиторий:

   ```
   POWERSHELL
   mkdir ~\projects\git-practice
   cd ~\projects\git-practice
   git init
   notepad hello.txt
   ```

   В Блокноте нажмите **Да**, напишите `Привет, Git!`, сохраните и закройте.

4. Сохраните первую версию и посмотрите историю:

   ```
   POWERSHELL
   git add hello.txt
   git commit -m "Первый коммит"
   git log --oneline
   ```

Получилось? Теперь разберитесь, что произошло, — начните с главы 1.

## Содержание

### Часть I. Git и GitHub с нуля


| Глава | Тема | Что внутри |
| --- | --- | --- |
| 1. Основы | Git и GitHub: основы | Что такое Git и GitHub, репозиторий, коммит, ветка, история; бытовые аналогии и схемы |

| Глава | Тема | Что внутри |
|---|---|---|
| 2. Установка | Установка и первая настройка | Терминал, PowerShell и Git Bash, установка Git на Windows, имя и почта, проверка конфигурации |
| 3. Команды | Основные команды Git | Рабочая папка, staging area, история; `init`, `status`, `add`, `commit`, `log`, `diff`, `clone`, `push`, `pull`, `fetch` |
| 4. GitHub | Работа с GitHub | Аккаунт и 2FA, репозитории, работа через сайт, вход из терминала, отправка проекта на GitHub: два сценария, случай с README |
| 5. Ветки | Ветки и Pull Request | `branch`, `switch`, `merge`, Pull Request, ревью, GitHub Flow, командная работа |
| 6. Восстановление | Конфликты, восстановление и типичные ошибки | Конфликты слияния, `restore`, `revert`, `reset`, `reflog`, секреты и `.gitignore`, ошибки push и входа, таблица «ошибка → причина → решение» |
| 7. AI | Git и Claude Code | Рабочий цикл с AI, контрольные точки, просмотр изменений, откат, `CLAUDE.md`, оформление AI-проектов |
| 8. Практика | Практика и итоговые проекты | Упражнения по главам, проект «Мой первый репозиторий», проект «Профессионально оформленный репозиторий», чек-лист готовности к публикации, контрольный список навыков |

## Часть II. Как создавать, оформлять и публиковать программные проекты по международным практикам

| Глава | Тема | Что внутри |
|---|---|---|
| 9. Оформление | Стандарты и оформление проектов | Стандарты, практики и требования; структура репозитория; README; CONTRIBUTING, CODE_OF_CONDUCT, SECURITY, CHANGELOG; SemVer, теги, релизы; ISO/IEC 12207, 25010, 29148, SPDX, REUSE, Conventional Commits, Scorecard, SLSA, OWASP |
| 10. Лицензии | Лицензии и авторские права | Авторское право и лицензия; сравнение MIT, Apache-2.0, BSD, GPL, LGPL, AGPL, MPL, Unlicense; выбор и оформление лицензии; NOTICE, сторонние компоненты, CITATION.cff; AI-код |
| 11. Безопасность | Безопасность и автоматизация | GitHub Actions и CI, защита веток, секреты и `.env`, Dependabot, secret scanning, CodeQL, утечка секрета, безопасность CI/CD |

## Справочные материалы

| Материал | Назначение |
|---|---|
| Словарь терминов | Все термины методички с определениями |
| Источники | Официальная документация и дата проверки |
| Шаблоны | Готовые шаблоны документов проекта (список ниже) |
| Учебный проект tempconv | Эталон профессионально оформленного репозитория: код, тесты, CI, документация |
| Проверка качества | Что и как проверено, известные ограничения |
| PDF-версия | Вся методичка одним файлом с оглавлением и нумерацией страниц |

**Шаблоны:** README · CONTRIBUTING · SECURITY · CHANGELOG · CODE_OF_CONDUCT · Issue · Pull Request · Выбор лицензии · Чек-лист публикации

Пояснения в шаблонах записаны служебными комментариями `<!-- ... -->`. На GitHub они скрыты — чтобы их прочитать, откройте файл в режиме исходного текста (кнопка **Code** или **Raw**) или в редакторе.

## Как пользоваться методичкой

1. **Читайте главы по порядку.** Каждая опирается на предыдущие. Часть II можно начинать после глав 1–6, а проект 2 из главы 8 выполнять после глав 9–11.
2. **Повторяйте команды руками** в учебной папке. Чтение без практики почти не работает.
3. **Сверяйтесь с ожидаемым результатом.** Если результат другой — остановитесь и выполните `git status`. Типичные ошибки разобраны в главе 6.
4. **Отвечайте на вопросы в конце главы.** Ответы спрятаны под спойлером «Ответы» или «Ожидаемый результат».
5. **Закрепите всё в главе 8**: два итоговых проекта, чек-лист готовности к публикации и контрольный список навыков.

Примерное время: часть I — 12–15 часов, часть II — 8–10 часов вместе с практикой.

## Обозначения

| Обозначение | Значение |
|---|---|
| Блок `powershell` | Команда, которую нужно ввести в терминале. Приглашение `PS C:\...>` не показано — его не набирают |
| Блок `text` | Ожидаемый вывод. Хеши, даты и пути у вас будут другими |
| `<файл>`, `<коммит>`, `<логин>` | Часть команды или адреса, которую нужно заменить, например `git add <файл>` → `git add hello.txt` |
| `[В КВАДРАТНЫХ СКОБКАХ]` | Заполнитель в шаблонах документов: замените своими данными |
| `OWNER` | Заполнитель в учебном проекте `examples/tempconv`: замените своим логином GitHub |
| ⚠ | Команда может удалить изменения или переписать историю |

Блоки-примечания:

> **ПРИМЕЧАНИЕ**
>
> Полезное уточнение.

> **СОВЕТ**
>
> Совет, который экономит время.

> **ВАЖНО**
> То, что важно не пропустить.

> **ВНИМАНИЕ**
> Команда или действие может привести к потере изменений.

> **ОПАСНО**
> Опасное действие: потеря данных, утечка или необратимые последствия.

Схемы: картинки в папке images/ нарисованы для этой методички. Схемы интерфейса GitHub — **схематические изображения, а не снимки экрана**: они помечены надписью «Схема интерфейса · не скриншот». Диаграммы в блоках `mermaid` GitHub рисует автоматически.

## PDF-версия

Файл git-github-guide.pdf содержит обложку, оглавление с номерами страниц, все главы, словарь, источники и шаблоны. В PDF есть закладки для навигации. Последняя версия файла также прикреплена к релизу.

Как пересобрать PDF после правок — в tools/README.md.

## Как помочь проекту

- **Нашли ошибку или устаревшие сведения?** Создайте Issue — для этого есть готовые формы.
- **Хотите исправить сами?** Прочитайте CONTRIBUTING.md: как предложить правку, правила текста и проверки.
- **Нашли уязвимость в скриптах или учебном проекте?** Сообщите закрыто по правилам SECURITY.md, а не в открытом Issue.
- **Трудно читать с программой экранного доступа или при увеличении экрана?** Что сделано для доступности и как сообщить о барьере — в ACCESSIBILITY.md.
- **Методичка помогла?** Поставьте звезду репозиторию — так её проще найти другим.

Участвуя в проекте, вы соглашаетесь соблюдать кодекс поведения. История версий — в CHANGELOG.md.

## Как цитировать

Сведения для цитирования записаны в CITATION.cff. На странице репозитория нажмите **Cite this repository** справа — GitHub покажет готовую ссылку в форматах APA и BibTeX.

## Лицензия

© 2026 Ruslan Kazyradzi.

| Что | Лицензия | Файл |
| --- | --- | --- |
| Тексты методички, схемы и анимация в `images/`, шаблоны в `templates/`, PDF-версия | CC BY 4.0 — можно копировать, распространять и изменять, в том числе в коммерческих целях, если указать автора, дать ссылку на лицензию и отметить внесённые изменения | LICENSE |
| Скрипты в `tools/` и код учебного проекта в `examples/tempconv/` | MIT | LICENSE-CODE |

Как указать авторство: «Git и GitHub с нуля», Ruslan Kazyradzi, https://github.com/RuslanKazyradzi13/git-github-guide, лицензия CC BY 4.0.

Файл examples/tempconv/LICENSE с заполнителями вместо имени — часть учебного задания (образец оформления), а не лицензия этого репозитория. Кодекс поведения — официальный перевод Contributor Covenant 2.1 (CC BY 4.0).

ЧАСТЬ I

# Git и GitHub с нуля

Как сохранять версии проекта, работать с GitHub, ветками и Pull Request, восстанавливать изменения и безопасно работать с AI.


Глава 1. Git и GitHub: основы

Глава 2. Установка и первая настройка

Глава 3. Основные команды Git

Глава 4. Работа с GitHub

Глава 5. Ветки и Pull Request

Глава 6. Конфликты, восстановление и типичные ошибки

Глава 7. Git и Claude Code

Глава 8. Практика и итоговые проекты

# Глава 1. Git и GitHub: основы

В этой главе нет команд. Здесь вы разберётесь, что такое Git и GitHub, зачем они нужны и какими словами о них говорят разработчики. Все остальные главы опираются на эти понятия.

**Вы научитесь:**

- объяснять, чем Git отличается от GitHub;
- понимать слова «репозиторий», «коммит», «ветка», «история изменений»;
- представлять, как файлы попадают с вашего компьютера на GitHub.

**Время:** около 30 минут. **Понадобится:** ничего, кроме внимания.

## 1.1. Зачем нужен контроль версий

Представьте, что вы пишете диплом. Через месяц в папке лежат файлы:

```
диплом.docx
диплом_новый.docx
диплом_новый_исправленный.docx
диплом_финал.docx
диплом_финал_точно_финал.docx
```

Уже непонятно, чем один файл отличается от другого и какой из них правильный. А если над дипломом работают двое, станет ещё хуже: правки будут теряться при пересылке.

С программами то же самое, только файлов сотни. Поэтому разработчики используют **систему контроля версий**.

> **Система контроля версий** (СКВ, по-английски *version control system*, VCS) — программа, которая запоминает изменения файлов. С её помощью можно посмотреть, кто, когда и что изменил, и вернуться к любой сохранённой версии.

Пример. Вы поменяли цвет кнопки на сайте, и сайт сломался. Система контроля версий покажет, какие именно строки вы изменили, и позволит вернуть файл к вчерашнему состоянию за одну команду.

Что даёт контроль версий:

| Возможность | Что это значит на практике |
|---|---|
| История | Видно каждое сохранённое изменение: кто, когда, зачем |
| Откат | Можно вернуться к рабочей версии, если что-то сломалось |
| Эксперименты | Можно пробовать новое, не трогая рабочую версию |
| Командная работа | Несколько человек меняют один проект, и правки не теряются |
| Резервная копия | Копия проекта хранится на сервере (если вы её туда отправили) |

## 1.2. Что такое Git

> **Git** — бесплатная программа для контроля версий. Её устанавливают на компьютер и управляют ею командами в терминале или через программы с кнопками.

Git создал Линус Торвальдс в 2005 году для разработки ядра Linux. Сегодня это самая распространённая система контроля версий в мире. Git — программа с открытым исходным кодом: её код может прочитать любой.

Главное о Git:

- **Работает на вашем компьютере.** Интернет для сохранения версий не нужен.
- **Сохраняет версии только по вашей команде.** Git не делает это автоматически, как облачный диск.
- **Хранит всю историю внутри проекта** — в скрытой папке `.git`.
- **Распределённый.** У каждого участника есть полная копия проекта со всей историей.

Пример. Вы написали первую страницу сайта и сказали Git: «Сохрани эту версию с подписью „Добавить главную страницу"». Через неделю вы сможете открыть именно эту версию, даже если с тех пор сто раз меняли файл.

## 1.3. Что такое GitHub

> **GitHub** — сайт и облачный сервис, где хранят Git-проекты и работают над ними вместе. Адрес: github.com.

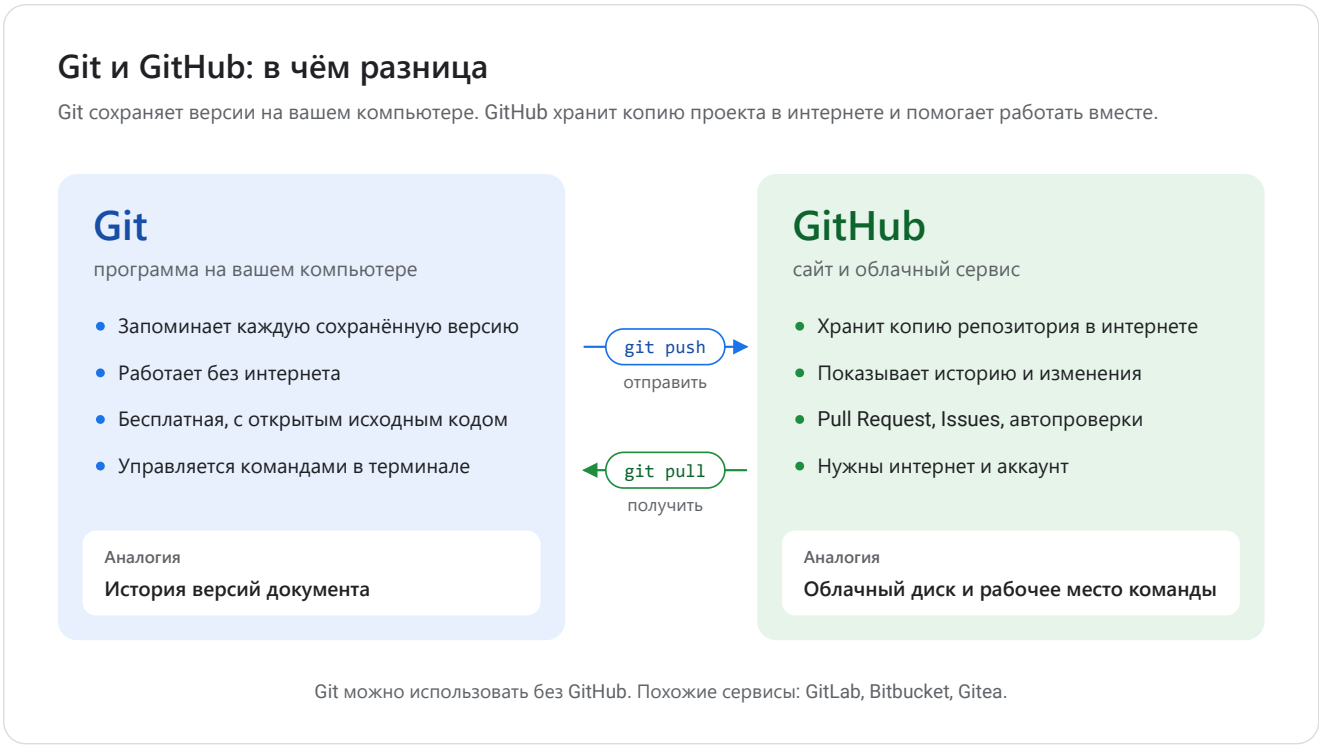
GitHub принадлежит компании Microsoft. Базовые возможности бесплатны.

Что умеет GitHub:

- хранить копию проекта в интернете;
- показывать файлы, историю и изменения прямо в браузере;
- давать доступ коллегам;
- обсуждать изменения перед тем, как принять их (**Pull Request**, глава 5);
- вести список задач и ошибок (**Issues**);
- автоматически проверять код (**GitHub Actions**, глава 11).

Пример. Вы работаете над сайтом дома и в офисе. Дома вы отправляете изменения на GitHub, в офисе скачиваете их оттуда. Проект всегда одинаковый на обоих компьютерах.

## 1.4. Git и GitHub: в чём разница




| | Git | GitHub |
|---|---|---|
| Что это | Программа | Сайт и облачный сервис |
| Где работает | На вашем компьютере | В интернете |
| Нужен интернет | Нет | Да |
| Нужен аккаунт | Нет | Да |
| Главная задача | Сохранять версии файлов | Хранить проекты и помогать работать вместе |
| Аналогия | История версий документа | Облачный диск и рабочее место команды |

Git можно использовать без GitHub. GitHub — один из сервисов для Git-проектов. Похожие сервисы: GitLab, Bitbucket, Gitea. В этой методичке мы используем GitHub.

## 1.5. Главные понятия

Эти слова встретятся в каждой главе. Не нужно заучивать их сейчас: возвращайтесь сюда или в словарь терминов.

### Репозиторий

> **Репозиторий** (*repository*, сокращённо «репо») — папка проекта, за которой следит Git, вместе со всей историей её изменений.

Пример. Папка `my-site` с файлами `index.html` и `style.css` — обычная папка. После команды `git init` в ней появляется скрытая папка `.git`, и `my-site` становится репозиторием.

Репозитории бывают двух видов:

- **Локальный** — на вашем компьютере.
- **Удалённый** (*remote*) — на сервере, например на GitHub. Обычно его называют `origin`.

### Коммит

> **Коммит** (*commit*) — сохранённый снимок проекта. У каждого коммита есть автор, дата, сообщение и уникальный номер.

Пример коммита:

```
commit 3f2a9c1
Автор:     Иван Петров
Дата:      11 октября 2026
Сообщение: Добавить страницу контактов
```

Номер `3f2a9c1` — это начало **хеша**: длинного уникального кода из цифр и букв. По нему Git находит коммит. Обычно хватает первых 7 символов.

Аналогия: коммит похож на сохранение в компьютерной игре. Вы сохранились перед сложным уровнем и всегда можете к этому моменту вернуться.

### История изменений

> **История изменений** — цепочка коммитов от самого первого до последнего.

Пример истории (сначала новые):

```
3f2a9c1 Добавить страницу контактов
b71e0d4 Исправить опечатку в меню
5c9a2e8 Добавить главную страницу
```

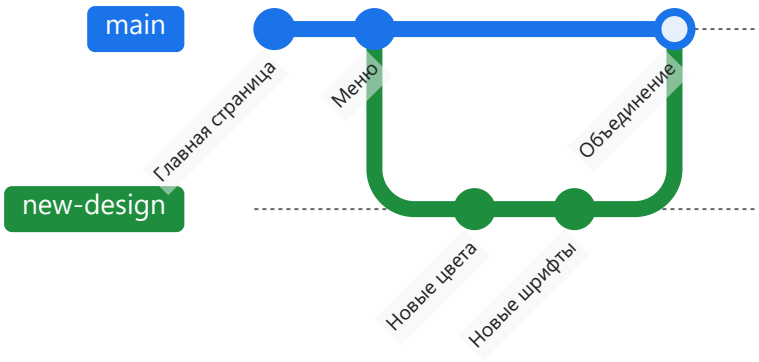
По истории видно, как проект развивался, и можно вернуться к любому коммиту.

### Ветка

> **Ветка** (*branch*) — отдельная линия работы над проектом. Изменения в одной ветке не влияют на другие, пока вы их не объедините.

Основная ветка обычно называется `main`. В ней лежит стабильная, рабочая версия.

Пример. Вы хотите попробовать новый дизайн сайта, но не уверены в результате. Вы создаёте ветку `new-design` и работаете в ней. Если дизайн понравится, вы объедините ветку с `main`. Если нет — просто удалите ветку, и `main` останется нетронутой.




### Отправить и получить изменения

- **Отправить** (*push*, «запушить») — передать ваши коммиты с компьютера на GitHub.
- **Получить** (*pull*, «спулить») — скачать новые коммиты с GitHub к себе.
- **Клонировать** (*clone*) — скачать проект с GitHub целиком, со всей историей.

## 1.6. Бытовая аналогия

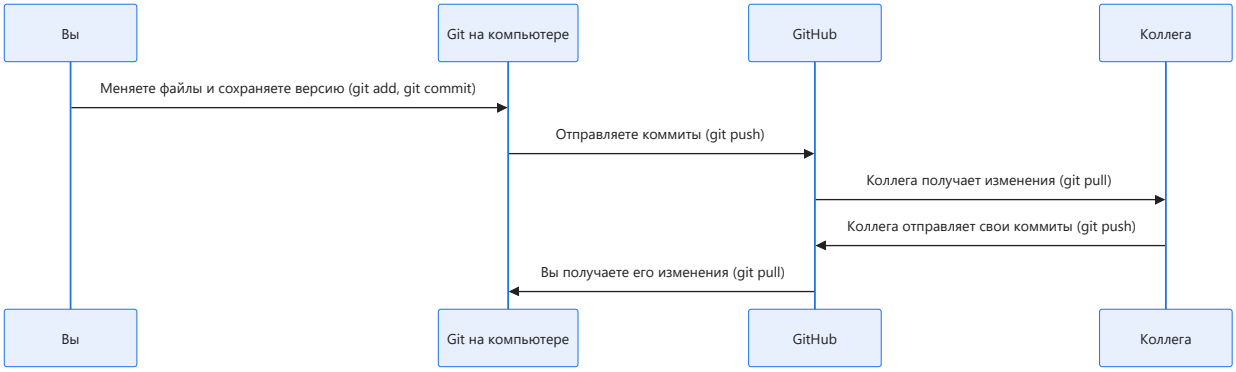
| Понятие Git | На что похоже |
|---|---|
| Git | Функция «История версий» в текстовом редакторе |
| Репозиторий | Папка проекта с журналом всех изменений |
| Коммит | Сохранение в игре с подписью |
| История | Список всех сохранений |
| Ветка | Черновик, где можно пробовать, не портя чистовик |
| Слияние (*merge*) | Перенос удачных правок из черновика в чистовик |
| GitHub | Облачный диск, где команда видит проект и обсуждает правки |
| push | Загрузить свои сохранения в облако |
| pull | Скачать из облака чужие сохранения |

У аналогии есть предел: облачный диск синхронизирует файлы сам, а Git сохраняет и отправляет изменения только по вашей команде.

## 1.7. Как связаны вы, Git и GitHub




Обычный день работы с проектом выглядит так:




Все команды из схемы вы освоите в главах 3 и 4.

## 1.8. Чего Git не делает

Git — надёжный инструмент, но не волшебный. Важно знать его границы:

- **Git не сохраняет автоматически.** В историю попадает только то, что вы закоммитили.
- **Локальный репозиторий — не резервная копия.** Если диск сломается, а вы не отправили коммиты на GitHub, история пропадёт вместе с диском.
- **Git не защищает секреты.** Пароль, попавший в коммит, остаётся в истории (глава 6).
- **Git плохо подходит для больших файлов** — видео, архивов, дистрибутивов. GitHub не принимает файлы больше 100 МБ.
- **Git не проверяет, работает ли программа.** Он сохранит и сломанную версию, если вы её закоммитите.

## Проверьте себя

1. Нужен ли интернет, чтобы сохранить версию проекта в Git?
2. Чем репозиторий отличается от обычной папки?
3. Вы попробовали новый дизайн, и он не понравился. Как ветка помогает в такой ситуации?
4. Чем Git отличается от облачного диска?

**Ответы**

1. Нет. Git работает на компьютере. Интернет нужен только для обмена с GitHub.
2. В репозитории есть скрытая папка `.git` с историей всех коммитов.
3. Вы работали в отдельной ветке, поэтому `main` не изменилась. Ветку можно просто удалить.
4. Облачный диск синхронизирует файлы сам. Git сохраняет версии только по вашей команде и хранит полную историю изменений.

## Что дальше

В главе 2 вы установите Git на Windows и сделаете первую настройку.

# Глава 2. Установка и первая настройка

В этой главе вы научитесь пользоваться терминалом, установите Git на Windows и представитесь ему: укажете имя и почту. Настройку делают один раз на компьютере.

**Вы научитесь:**

- открывать терминал и выполнять в нём команды;
- устанавливать Git и проверять установку;
- настраивать имя, почту и другие параметры Git;
- проверять текущую конфигурацию.

**Время:** 30–45 минут. **Понадобится:** компьютер с Windows 10 или 11 и права на установку программ.

## 2.1. Что такое терминал

> **Терминал** — окно, в котором компьютером управляют текстовыми командами. Вы пишете команду, нажимаете `Enter`, компьютер выполняет её и печатает результат.

Пример. Команда `pwd` выводит адрес папки, в которой вы сейчас находитесь:

```powershell
pwd
```

```
Path
----
C:\Users\Ivan
```

В Windows есть несколько программ-терминалов. Для этой методички подойдут две:

| Программа | Что это | Когда использовать |
|---|---|---|
| **PowerShell** | Стандартная командная оболочка Windows | По умолчанию во всех примерах методички |
| **Git Bash** | Оболочка в стиле Linux, ставится вместе с Git | Если вам привычнее Linux-команды |

> **Оболочка** (*shell*) — программа, которая понимает ваши команды. PowerShell и Git Bash — разные оболочки. Команды Git работают в них одинаково, а команды для работы с файлами немного различаются.

### Как открыть PowerShell

1. Нажмите `Win`.
2. Наберите `Терминал` (в Windows 11) или `PowerShell` (в Windows 10).
3. Нажмите `Enter`.

Откроется окно со строкой вроде этой:

```
PS C:\Users\Ivan>
```

Это **приглашение** (*prompt*). `PS` означает PowerShell, дальше — текущая папка. Команду пишут после знака `>`.

> **ВАЖНО**
>
> В примерах методички приглашение не показано. Набирайте только саму команду: `git status`, а не `PS C:\Users\Ivan> git status`.

### Как открыть терминал сразу в нужной папке

1. Откройте папку проекта в Проводнике.
2. Щёлкните правой кнопкой мыши по пустому месту в папке.
3. Выберите **Открыть в Терминале**.

В Windows 10 этого пункта может не быть. Тогда откройте PowerShell и перейдите в папку командой `cd` (см. ниже).

### Пять команд, которых хватит для начала

| Команда (PowerShell) | Что делает | Пример |
|---|---|---|
| `pwd` | Показывает текущую папку | `pwd` |
| `ls` | Показывает файлы в текущей папке | `ls` |
| `cd <папка>` | Переходит в папку | `cd Documents` |
| `cd ..` | Поднимается на одну папку вверх | `cd ..` |
| `mkdir <имя>` | Создаёт папку | `mkdir projects` |

Ещё три полезные команды:

- `notepad <файл>` — открыть файл в Блокноте (если файла нет, Блокнот предложит его создать);
- `explorer .` — открыть текущую папку в Проводнике (точка означает «текущая папка»);
- `cls` — очистить экран.

> **Путь** — адрес файла или папки, например `C:\Users\Ivan\projects`. Знак `~` — короткая запись домашней папки пользователя: `~\projects` то же самое, что `C:\Users\Ivan\projects`.

### Приёмы, которые экономят время

- Tab — дописать имя файла или папки. Наберите `cd Doc` и нажмите Tab.
- ↑ и ↓ — листать ранее введённые команды.
- Ctrl + C — прервать выполняющуюся команду.
- Вставить текст в терминал: Ctrl + V или правая кнопка мыши.

> **СОВЕТ**
>
> Называйте папки проектов латиницей и без пробелов: `my-site`, а не `Мой сайт`. Так меньше проблем с командами и адресами на GitHub.

## 2.2. Установка Git на Windows

### Способ 1. Установщик с официального сайта (рекомендуется)

1. Откройте страницу git-scm.com/install/windows.
2. Скачайте установщик для вашего компьютера. Почти всегда это **x64**. Для компьютеров на процессоре ARM выберите **ARM64**.
3. Запустите скачанный файл. Windows спросит разрешение на изменения — нажмите **Да**.
4. Проходите страницы мастера кнопкой **Next**. На большинстве страниц оставьте значения по умолчанию. Две страницы стоит изменить — они выделены в таблице.

| Страница установщика | Что выбрать | Почему |
|---|---|---|
| Select Components | По умолчанию | Нужные компоненты уже отмечены |
| **Choosing the default editor used by Git** | **Use Notepad as Git's default editor** (или Visual Studio Code, если он установлен) | По умолчанию стоит Vim — сложный для новичка редактор |
| **Adjusting the name of the initial branch in new repositories** | **Override the default branch name for new repositories**, в поле — `main` | Иначе новые репозитории получат ветку `master`, а GitHub использует `main` |
| Adjusting your PATH environment | Git from the command line and also from 3rd-party software (Recommended) | Git будет доступен в PowerShell |
| Choosing the SSH executable | Use bundled OpenSSH | По умолчанию |
| Choosing HTTPS transport backend | Use the native Windows Secure Channel library | По умолчанию |
| Configuring the line ending conversions | Checkout Windows-style, commit Unix-style line endings | Правильные концы строк для Windows (см. ниже) |
| Configuring the terminal emulator | Use MinTTY | По умолчанию |
| Choose the default behavior of `git pull` | Merge (в установщиках до середины 2026 года — Fast-forward or merge) | По умолчанию |
| Choose a credential helper | Git Credential Manager | Позволяет входить в GitHub через браузер |
| Configuring extra options | По умолчанию | — |

5. Нажмите **Install**, дождитесь окончания и нажмите **Finish**.

> **ПРИМЕЧАНИЕ**
> Названия страниц приведены для Git for Windows 2.56 (октябрь 2026). В других версиях формулировки могут немного отличаться. Если сомневаетесь — оставляйте значение по умолчанию: всё важное мы всё равно настроим командами в разделе 2.4.

> **Концы строк.** В Windows строка текста заканчивается двумя невидимыми символами (CRLF), в Linux и macOS — одним (LF). Выбранная настройка хранит в репозитории формат LF, а в рабочей папке показывает файлы в формате Windows. Поэтому вы иногда увидите предупреждение `LF will be replaced by CRLF` — это нормально.

### Способ 2. Через winget

> **winget** — встроенный в Windows 10 и 11 менеджер программ: он устанавливает программы одной командой.

```powershell
winget install --id Git.Git -e --source winget
```

- `--id Git.Git` — какую программу установить;
- `-e` — искать точное совпадение названия;
- `--source winget` — брать программу из официального каталога winget.

Winget ставит Git с настройками по умолчанию: редактор Vim и ветка `master`. Обязательно выполните настройку из раздела 2.4. Новая версия Git может появиться в каталоге winget с задержкой в несколько дней.

## 2.3. Проверка установки

1. **Закройте и снова откройте** терминал. Новое окно узнает об установленной программе.
2. Выполните:

```powershell
git --version
```

Ожидаемый результат (номер версии может быть новее):

```
git version 2.56.0.windows.2
```

Если вы видите номер версии — Git установлен.

### Если команда не найдена

```
git : Имя "git" не распознано как имя командлета, функции, файла сценария или выполняемой
программы.
```

В англоязычной Windows: `The term 'git' is not recognized...`.

Что сделать:

1. Закройте все окна терминала и откройте новое.
2. Если не помогло — перезагрузите компьютер.
3. Если и это не помогло — запустите установщик ещё раз и на странице **Adjusting your PATH environment** выберите вариант **(Recommended)**.

## 2.4. Первая настройка Git

Git записывает в каждый коммит имя и почту автора. Без них Git откажется сохранять версии. Выполните команды по одной, подставив свои данные.

### Имя

POWERSHELL

```
git config --global user.name "Иван Петров"
```

- `git config` — команда настройки Git;
- `--global` — настройка для всех ваших репозиториев на этом компьютере;
- `user.name` — имя параметра;
- `"Иван Петров"` — значение. Кавычки нужны, потому что в имени есть пробел.

Команда ничего не выводит. Это нормально: в терминале «тишина» обычно означает успех.

### Почта

POWERSHELL

```
git config --global user.email "ivan.petrov@example.com"
```

Укажите почту, которую используете (или будете использовать) на GitHub. Тогда GitHub свяжет ваши коммиты с вашим профилем.

> **ВАЖНО**
>
> Почта из коммитов видна всем, кто видит репозиторий. Если не хотите показывать настоящий адрес, GitHub даёт специальный скрытый адрес вида `12345678+login@users.noreply.github.com`. Как его получить — в главе 4. Почту в настройке можно поменять в любой момент той же командой.

### Имя основной ветки

```
POWERSHELL
git config --global init.defaultBranch main
```

Новые репозитории будут сразу создаваться с веткой `main` — как на GitHub. Если вы выбрали `main` в установщике, команда просто подтвердит выбор.

### Редактор

```
POWERSHELL
git config --global core.editor notepad
```

Иногда Git открывает редактор, чтобы вы написали сообщение (например, при слиянии веток). С этой настройкой откроется Блокнот: напишите текст, сохраните (Ctrl + S) и **закройте окно** — только тогда Git продолжит работу.

Если у вас установлен Visual Studio Code, можно выбрать его:

```
POWERSHELL
git config --global core.editor "code --wait"
```

> **СОВЕТ**
>
> **Если Git всё же открыл Vim** (экран с тильдами `~` слева), выйти можно так: нажмите Esc, наберите `:wq` и нажмите Enter — сохранить и выйти. `:q!` — выйти без сохранения.

### Русские имена файлов

```
POWERSHELL
git config --global core.quotepath false
```

Без этой настройки Git показывает русские имена файлов в виде кодов: `"\320\277\320\273\320\260\320\275.txt"`. С ней — нормально: `план.txt`.

## 2.5. Проверка конфигурации

Посмотрите все глобальные настройки:

```
POWERSHELL
git config --global --list
```

Ожидаемый результат:

```
user.name=Иван Петров
user.email=ivan.petrov@example.com
init.defaultbranch=main
core.editor=notepad
core.quotepath=false
```

Git показывает имена параметров маленькими буквами — `init.defaultbranch`. Регистр в именах параметров не важен.

Узнать одно значение:

POWERSHELL
```
git config user.name
```

```
Иван Петров
```

Узнать, в каком файле записана каждая настройка:

POWERSHELL
```
git config --list --show-origin
```

```
file:C:/Program Files/Git/etc/gitconfig    init.defaultbranch=master
file:C:/Users/Ivan/.gitconfig              user.name=Иван Петров
file:C:/Users/Ivan/.gitconfig              init.defaultbranch=main
...
```

Настройки хранятся на трёх уровнях. Более близкий уровень важнее:

| Уровень | Флаг | Файл | Действует |
|---|---|---|---|
| Системный | `--system` | `C:\Program Files\Git\etc\gitconfig` | Для всех пользователей компьютера |
| Глобальный | `--global` | `C:\Users\<вы>\.gitconfig` | Для всех ваших репозиториев |
| Локальный | `--local` | `.git\config` внутри проекта | Только для одного репозитория |

В примере выше системный файл говорит `master`, а глобальный — `main`. Побеждает глобальный, поэтому новые репозитории получат ветку `main`.

> **ПРИМЕЧАНИЕ**
>
> В Git 2.46 появилась новая запись команд: `git config list`, `git config get user.name`, `git config set --global user.name "..."`. Старая запись, которую использует методичка, тоже работает. Документация рекомендует со временем переходить на новую.

## 2.6. PowerShell и Git Bash: что выбрать

Команды Git одинаковы в обеих оболочках. Различаются команды для работы с файлами и запись путей.

| Действие | PowerShell | Git Bash |
|---|---|---|
| Показать текущую папку | `pwd` | `pwd` |
| Показать файлы, включая скрытые | `ls -Force` | `ls -a` |
| Путь к папке | `C:\Users\Ivan\projects` | `/c/Users/Ivan/projects` |
| Показать содержимое файла | `cat file.txt` | `cat file.txt` |
| Удалить папку со всем содержимым | `Remove-Item -Recurse folder` | `rm -r folder` |
| Создать пустой файл | `New-Item file.txt` | `touch file.txt` |

Рекомендация: начните с PowerShell — все примеры методички написаны для него. Git Bash пригодится, когда вы встретите в интернете инструкции для Linux или macOS.

Чтобы открыть Git Bash: Win → наберите `Git Bash` → Enter.

> **ВНИМАНИЕ**
>
> Команды удаления (`Remove-Item -Recurse`, `rm -r`) удаляют файлы **мимо Корзины**. Восстановить их обычно нельзя. Перед запуском проверьте, в какой папке вы находитесь (`pwd`).

## Проверьте себя

1. Откройте PowerShell и выполните `git --version`. Что вы видите?
2. Выполните `git config --global --list`. Есть ли в списке `user.name`, `user.email` и `init.defaultbranch=main`?
3. Создайте папку `projects` в домашней папке и перейдите в неё.

**Ожидаемый результат**

1. Строка вида `git version 2.56.0.windows.2`.
2. Да, все три параметра с вашими значениями.
3. Команды:

```powershell
cd ~
mkdir projects
cd projects
pwd
```

`pwd` покажет `C:\Users\<ваше имя>\projects`.

## Что дальше

В главе 3 вы создадите первый репозиторий и освоите основные команды Git.

# Глава 3. Основные команды Git

В этой главе вы создадите первый репозиторий и освоите десять команд, которыми разработчики пользуются каждый день. Каждая команда описана по одному шаблону: что делает, когда нужна, как выглядит, пример, результат, проверка и типичные ошибки.

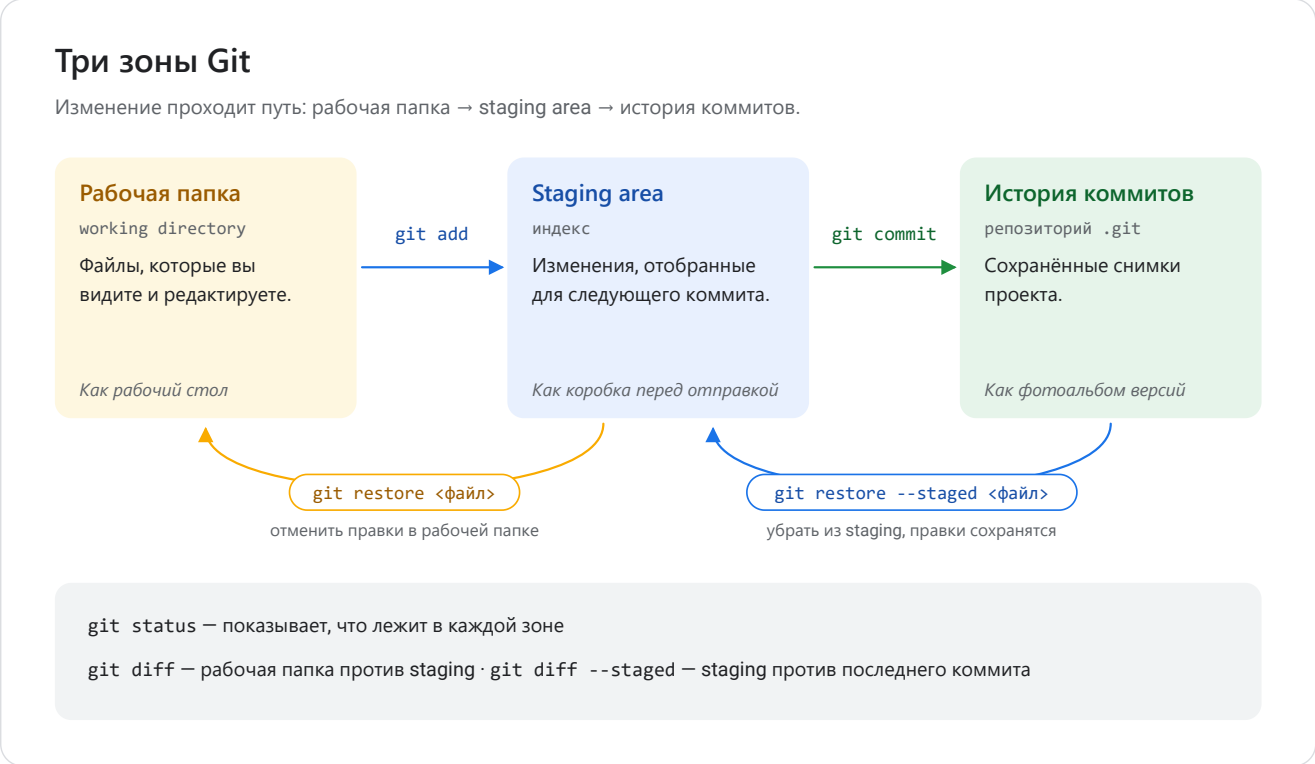
**Вы научитесь:**

- создавать репозиторий и сохранять версии файлов;
- понимать разницу между рабочей папкой, staging area и историей;
- смотреть историю и изменения;
- понимать, как работают `clone`, `push`, `pull` и `fetch`.

**Время:** 1,5–2 часа. **Понадобится:** установленный и настроенный Git (глава 2).

## 3.1. Три зоны Git

Прежде чем запускать команды, разберёмся, где «живут» изменения. Это самая важная идея главы.




> **Рабочая папка** (*working directory*) — обычная папка проекта с файлами, которые вы открываете и редактируете.

> **Staging area** (также «индекс», «область подготовки») — список изменений, которые войдут в следующий коммит. Вы сами решаете, что туда положить.

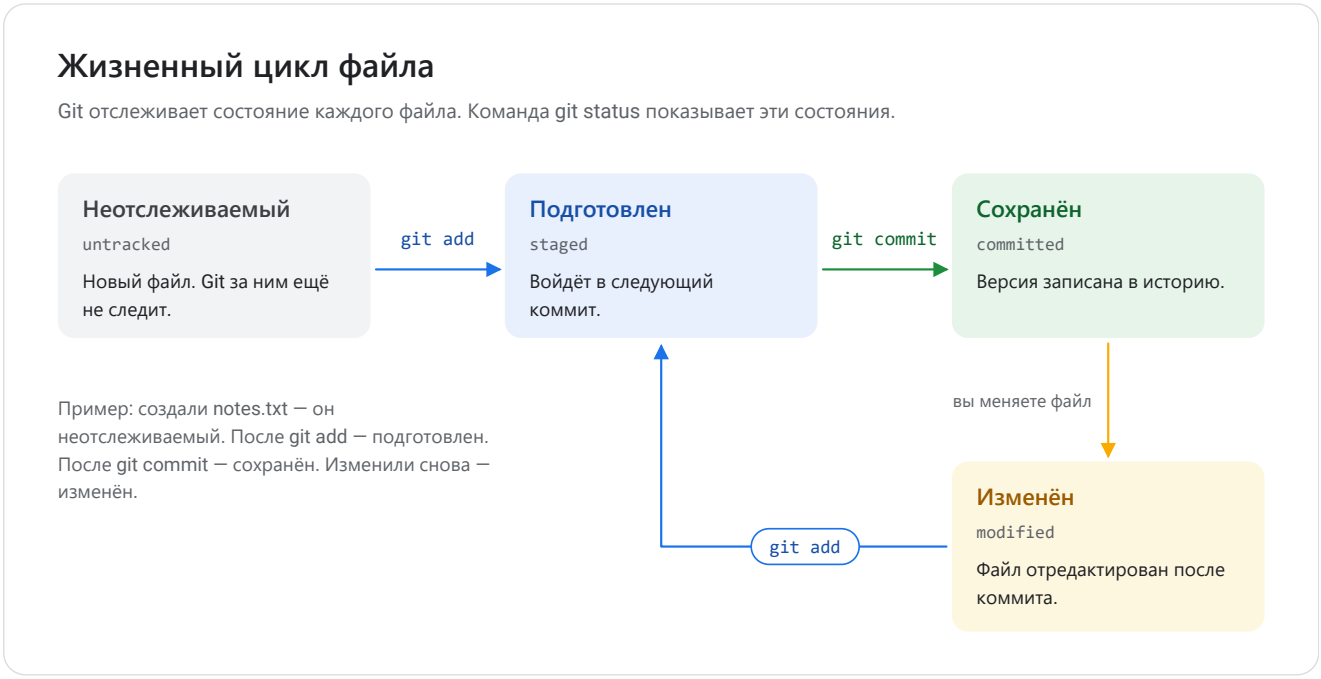
> **История коммитов** — сохранённые снимки проекта внутри папки `.git`.

Аналогия с посылкой:

1. **Рабочая папка** — ваш стол. На нём лежат вещи, некоторые вы меняете.
2. **Staging area** — коробка. Вы кладёте в неё только то, что хотите отправить.
3. **Коммит** — заклеенная и подписанная коробка. Её содержимое больше не меняется.

Зачем нужен промежуточный шаг? Допустим, вы исправили опечатку в одном файле и начали большую новую функцию в другом. Staging area позволяет сохранить опечатку отдельным коммитом, а недоделанную функцию — оставить на столе.

Каждый файл в репозитории находится в одном из состояний:




| Состояние | Английский термин в выводе Git | Что значит |
|---|---|---|
| Неотслеживаемый | `Untracked files` | Новый файл, Git за ним ещё не следит |
| Изменён | `Changes not staged for commit` / `modified` | Файл изменён, но изменения не в staging |
| Подготовлен | `Changes to be committed` | Изменения в staging, войдут в коммит |
| Сохранён | `nothing to commit, working tree clean` | Всё сохранено, отличий нет |

## 3.2. Подготовка учебной папки

Создайте папку для упражнений и перейдите в неё:

```powershell
mkdir ~\projects\git-practice
cd ~\projects\git-practice
```

Создайте первый файл в Блокноте:

```powershell
notepad hello.txt
```

Блокнот спросит, создать ли новый файл. Нажмите **Да**, напишите строку `Привет, Git!`, сохраните (Ctrl + S) и закройте Блокнот.

> **СОВЕТ**
>
> Создавайте и редактируйте текстовые файлы в Блокноте или в редакторе кода. Блокнот Windows 11 сохраняет файлы в кодировке UTF-8, и русский текст в Git отображается правильно.

> **ПРИМЕЧАНИЕ**
>
> Примеры вывода в этой главе получены в Git для Windows. Хеши коммитов, даты и пути у вас будут другими — это нормально.

## 3.3. git init — создать репозиторий

**Что делает.** Превращает обычную папку в Git-репозиторий: создаёт в ней скрытую папку `.git`, где будет храниться история.

**Когда нужна.** Один раз — в начале работы над новым проектом, который ещё не на GitHub.

**Как выглядит.**

```powershell
git init
```

**Пример.**

```powershell
cd ~\projects\git-practice
git init
```

**Что произойдёт.**

```
Initialized empty Git repository in C:/Users/Ivan/projects/git-practice/.git/
```

Перевод: «Создан пустой Git-репозиторий в ...». Файлы проекта не изменились, появилась только папка `.git`.

**Как проверить.**

```powershell
ls -Force
```

В списке появится папка `.git`. Она скрытая, поэтому без `-Force` её не видно.

**Типичные ошибки.**

- **Запуск не в той папке.** Например, в домашней папке `C:\Users\Ivan` — тогда Git начнёт следить за всеми вашими документами. Перед `git init` проверьте папку командой `pwd`. Если ошиблись, удалите появившуюся папку `.git` в Проводнике (включите показ скрытых элементов).
- **Повторный `git init`** в существующем репозитории ничего не ломает, но и не нужен. Git напишет `Reinitialized existing Git repository`.
- **Удаление папки `.git`** удаляет всю историю проекта. Никогда не трогайте её вручную без необходимости.
- **Ветка `master` вместо `main`.** Значит, не выполнена настройка `init.defaultBranch` из главы 2. Переименуйте ветку: `git branch -M main`.

## 3.4. git status — посмотреть состояние

**Что делает.** Показывает, в какой ветке вы находитесь и в каком состоянии файлы: что изменено, что подготовлено к коммиту, что Git ещё не отслеживает.

**Когда нужна.** Постоянно. Перед каждой командой, которая что-то меняет, и после неё. Это самая безопасная команда: она только смотрит.

**Как выглядит.**

```
POWERSHELL
git status
```

**Пример.** В папке `git-practice` есть новый файл `hello.txt`:

```
POWERSHELL
git status
```

**Что произойдёт.**

```
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        hello.txt

nothing added to commit but untracked files present (use "git add" to track)
```

Как читать:

| Строка | Перевод |
|---|---|
| `On branch main` | Вы в ветке `main` |
| `No commits yet` | Коммитов пока нет |

| Строка | Перевод |
| --- | --- |
| `Untracked files:` | Неотслеживаемые файлы |
| `(use "git add <file>..." ...)` | Подсказка: чтобы включить файл в коммит, используйте `git add` |
| `nothing added to commit but untracked files present` | В коммит ничего не добавлено, но есть неотслеживаемые файлы |

**Как проверить.** Команда сама и есть проверка. Если всё сохранено, вы увидите:

```
On branch main
nothing to commit, working tree clean
```

«Сохранять нечего, рабочая папка чистая».

**Типичные ошибки.**

- `fatal: not a git repository (or any of the parent directories): .git` — вы не в репозитории. Перейдите в папку проекта (`cd`) или создайте репозиторий (`git init`).
- Привычка пропускать `git status`. Именно эта команда показывает, что вы сейчас закоммитите.

## 3.5. git add — подготовить изменения

**Что делает.** Кладёт изменения в staging area — «коробку», которая уйдёт в следующий коммит.

**Когда нужна.** Перед каждым коммитом.

**Как выглядит.**

```
POWERSHELL
git add <файл>          # один файл
git add <файл1> <файл2>  # несколько файлов
git add .               # все изменения в текущей папке и вложенных
```

Текст после `#` — комментарий для вас, набирать его не нужно.

**Пример.**

```
POWERSHELL
git add hello.txt
git status
```

**Что произойдёт.**

```
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   hello.txt
```

Файл перешёл в раздел `Changes to be committed` — «изменения, которые будут закоммичены».

**Как проверить.** `git status` показывает файл в разделе `Changes to be committed`.

**Типичные ошибки.**

- **`git add .` без проверки.** Команда добавит всё подряд, включая случайные и секретные файлы. Сначала посмотрите `git status`, а лишнее внесите в `.gitignore` (глава 6).
- **`fatal: pathspec 'helo.txt' did not match any files`** — опечатка в имени файла. Используйте Tab для автодополнения.
- **Предупреждение** `warning: in the working copy of 'hello.txt', LF will be replaced by CRLF the next time Git touches it` — не ошибка. Git сообщает, что приведёт концы строк к формату Windows (см. главу 2).
- **Изменили файл после `git add`.** В коммит попадёт версия на момент `git add`. Чтобы добавить новые правки, выполните `git add` ещё раз.

## 3.6. git commit — сохранить версию

**Что делает.** Создаёт коммит — снимок всего, что лежит в staging area, с вашим сообщением.

**Когда нужна.** Когда закончен логический кусок работы: исправлена ошибка, добавлен раздел, программа снова работает.

**Как выглядит.**

```
POWERSHELL
git commit -m "Сообщение"
```

`-m` (от *message*) — сообщение коммита прямо в команде. Без `-m` Git откроет редактор, чтобы вы написали сообщение там.

**Пример.**

```
POWERSHELL
git commit -m "Добавить файл hello.txt"
```

**Что произойдёт.**

```
[main (root-commit) 3e7e53a] Добавить файл hello.txt
 1 file changed, 1 insertion(+)
 create mode 100644 hello.txt
```

| Часть | Значение |
| --- | --- |
| `main` | Ветка, в которую записан коммит |
| `root-commit` | Самый первый коммит репозитория |
| `3e7e53a` | Короткий хеш — номер коммита (у вас будет другой) |
| `1 file changed, 1 insertion(+)` | Изменён 1 файл, добавлена 1 строка |

**Как проверить.**

```
POWERSHELL
git status
git log --oneline
```

`git status` покажет `nothing to commit, working tree clean`, а `git log --oneline` — ваш коммит.

**Как писать сообщения коммитов.**

- Коротко описывайте, *что* изменено: «Добавить страницу контактов», «Исправить опечатку в меню».
- Начинайте с глагола. Хорошо подходит повелительная форма («Добавить», «Исправить»).
- Укладывайтесь в 50–70 символов. Подробности можно добавить в описание (см. ниже).
- Плохие примеры: «правки», «asdf», «исправил всё».

Сообщение с подробным описанием: каждый `-m` — отдельный абзац.

```
POWERSHELL
git commit -m "Исправить расчёт скидки" -m "Скидка считалась от суммы без учёта доставки."
```

**Типичные ошибки.**

- **`Author identity unknown *** Please tell me who you are.`** — не указаны имя и почта. Выполните настройку из главы 2.
- **`nothing to commit, working tree clean`** или **`no changes added to commit`** — staging area пуста. Сначала `git add`.
- **Открылся редактор, и вы не знаете, что делать.** Вы запустили `git commit` без `-m`. Напишите сообщение в первой строке, сохраните файл и закройте редактор. Если открылся Vim — нажмите Esc, наберите `:wq`, Enter.
- **`Aborting commit due to empty commit message`** — сообщение оказалось пустым, коммит не создан. Повторите с `-m "..."`.

> **СОВЕТ**
>
> Делайте коммиты часто и маленькими порциями. Маленький коммит легче понять, проверить и отменить.

## 3.7. git log — посмотреть историю

**Что делает.** Показывает историю коммитов: от новых к старым.

**Когда нужна.** Чтобы вспомнить, что делали; найти хеш коммита для отмены или сравнения; проверить, что коммит создан.

**Как выглядит.**

```
POWERSHELL
git log                         # подробно
git log --oneline               # одна строка на коммит
git log --oneline --graph --all  # с рисунком веток
git log -3                      # только три последних коммита
git log -- hello.txt            # только коммиты, затронувшие файл
```

**Пример.** Измените `hello.txt` (добавьте вторую строку `Это моя вторая строка.`), затем:

```
POWERSHELL
git add hello.txt
git commit -m "Добавить вторую строку"
git log
```

**Что произойдёт.**

```
commit 554a88af3ad71c11f9f2266871fa35166d108af3 (HEAD -> main)
Author: Иван Петров <ivan.petrov@example.com>
Date:   Sun Oct 11 15:39:31 2026 +0500

    Добавить вторую строку

commit 3e7e53a6756bdfb7b9b64862f7cca442f031f7e5
Author: Иван Петров <ivan.petrov@example.com>
Date:   Sun Oct 11 15:39:30 2026 +0500

    Добавить файл hello.txt
```

`HEAD -> main` означает: вы сейчас здесь, на последнем коммите ветки `main`.

> **HEAD** — указатель «вы находитесь здесь». Обычно он указывает на последний коммит текущей ветки.

Короткий вид:

```
POWERSHELL
git log --oneline
```

```
554a88a (HEAD -> main) Добавить вторую строку
3e7e53a Добавить файл hello.txt
```

**Как проверить.** Количество строк в `git log --oneline` равно количеству коммитов в ветке.

**Типичные ошибки.**

- **Терминал «завис», внизу `:` или `(END)`.** Длинная история открылась в режиме постраничного просмотра. Листайте клавишами ↑ ↓ или пробелом, выход — клавиша Q.
- **`fatal: your current branch 'main' does not have any commits yet`** — коммитов ещё нет. Сначала сделайте первый коммит.

## 3.8. git diff — посмотреть изменения

**Что делает.** Показывает построчно, что изменилось: какие строки добавлены, какие удалены.

**Когда нужна.** Перед `git add` и перед `git commit` — чтобы убедиться, что сохраняете именно то, что хотели. Особенно важно после изменений, сделанных AI (глава 7).

**Как выглядит.**

```powershell
git diff                 # рабочая папка против staging area
git diff --staged        # staging area против последнего коммита
git diff <коммит1> <коммит2>  # разница между двумя коммитами
git diff --stat          # только список файлов и число строк
```

**Пример.** Добавьте в `hello.txt` строку `Третья строка.` и сохраните файл. Затем:

```powershell
git diff
```

**Что произойдёт.**

```diff
diff --git a/hello.txt b/hello.txt
index d404783..8e1f0aa 100644
--- a/hello.txt
+++ b/hello.txt
@@ -1,2 +1,3 @@
 Привет, Git!
 Это моя вторая строка.
+Третья строка.
```

Как читать:

| Фрагмент | Значение |
|---|---|
| `--- a/hello.txt` | Старая версия файла |
| `+++ b/hello.txt` | Новая версия файла |
| `@@ -1,2 +1,3 @@` | Было 2 строки начиная с 1-й, стало 3 |
| Строка с `+` (зелёная) | Добавлена |
| Строка с `-` (красная) | Удалена |

| Фрагмент | Значение |
| --- | --- |
| Строка без знака | Не изменилась, показана для контекста |

Изменённая строка выглядит как пара: удалённая старая (`-`) и добавленная новая (`+`).

**Как проверить.** После `git add hello.txt` команда `git diff` ничего не покажет: изменения ушли в staging. Их видно через `git diff --staged`.

**Типичные ошибки.**

- «**`git diff` ничего не показывает, хотя я менял файл**». Изменения уже в staging — используйте `git diff --staged`. Или файл новый (неотслеживаемый) — `git diff` не показывает новые файлы до `git add`.
- **Длинный вывод «завис»**. Это постраничный просмотр, выход — `Q`.

Завершите пример — сохраните третью строку:

```powershell
git add hello.txt
git commit -m "Добавить третью строку"
```

### Ежедневный цикл

Команды 3.4–3.8 складываются в цикл, который вы будете повторять постоянно:

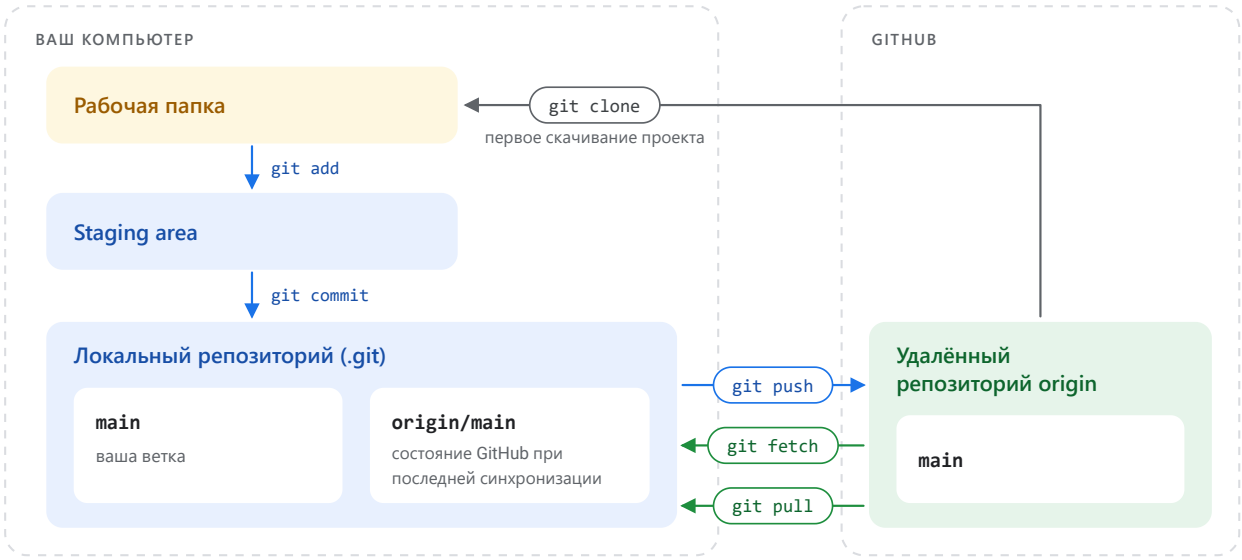



## 3.9. git clone — скачать репозиторий

Следующие четыре команды работают с удалённым репозиторием на GitHub. Попробовать их на практике вы сможете в главе 4, когда создадите аккаунт и репозиторий.

## Локальный и удалённый репозитории

origin — стандартное имя удалённого репозитория, с которым связан ваш проект.




`git push` — отправить ваши коммиты на GitHub.

`git fetch` — скачать новости с GitHub в origin/main, не меняя ваши файлы.

`git pull` — скачать и сразу объединить с вашей веткой (fetch + merge).

**Что делает.** Скачивает репозиторий с GitHub на компьютер целиком: все файлы и всю историю. Заодно запоминает адрес GitHub-репозитория под именем `origin`.

**Когда нужна.** Один раз — чтобы начать работать с проектом, который уже есть на GitHub.

**Как выглядит.**

```powershell
git clone <адрес>              # в новую папку с именем репозитория
git clone <адрес> <папка>      # в папку с другим именем
```

**Пример.**

```powershell
cd ~\projects
git clone https://github.com/ivan-petrov/my-site.git
```

**Что произойдёт.**

```
Cloning into 'my-site'...
remote: Enumerating objects: 12, done.
...
Receiving objects: 100% (12/12), done.
```

Появится папка `my-site` с файлами проекта и папкой `.git`.

**Как проверить.**

```powershell
cd my-site
git log --oneline
git remote -v
```

`git log` покажет историю, `git remote -v` — адрес на GitHub.

**Типичные ошибки.**

- `fatal: destination path 'my-site' already exists and is not an empty directory` — такая папка уже есть. Перейдите в другую папку или укажите другое имя.
- `remote: Repository not found` — опечатка в адресе, или репозиторий приватный, а у вашей учётной записи нет доступа.
- **Клонирование внутрь другого репозитория.** Перед `git clone` выполните `git status`: должно быть `fatal: not a git repository`. Значит, вы вне репозитория.

> **ПРИМЕЧАНИЕ**
> Кнопка **Download ZIP** на GitHub скачивает только файлы — без истории и без связи с GitHub. Чтобы работать с проектом через Git, используйте `git clone`.

## 3.10. git push — отправить коммиты на GitHub

**Что делает.** Отправляет ваши новые коммиты из локального репозитория в удалённый.

**Когда нужна.** После коммитов — чтобы сохранить их на GitHub и показать коллегам.

**Как выглядит.**

```powershell
git push -u origin main   # первый раз для ветки: отправить и запомнить связь
git push                  # в следующие разы
```

- `origin` — имя удалённого репозитория;
- `main` — какую ветку отправить;
- `-u` — запомнить, что локальная `main` связана с `origin/main`. После этого хватает короткого `git push`.

**Пример.**

```powershell
git push
```

**Что произойдёт.**

```
Enumerating objects: 5, done.
...
To https://github.com/ivan-petrov/my-site.git
   554a88a..9b1c2d3  main -> main
```

`554a88a..9b1c2d3` — на GitHub ветка `main` передвинулась с коммита `554a88a` на `9b1c2d3`.

**Как проверить.** Обновите страницу репозитория на GitHub: там появятся новые коммиты. В терминале:

```
POWERSHELL
git status
```

```
On branch main
Your branch is up to date with 'origin/main'.
```

«Ваша ветка совпадает с `origin/main`».

**Типичные ошибки.**

- `! [rejected] main -> main (fetch first)` — на GitHub есть коммиты, которых у вас нет. Сначала `git pull`, потом снова `git push`.
- `error: src refspec main does not match any` — в репозитории нет ветки `main`: либо ещё нет коммитов, либо ветка называется `master`. Проверьте `git status` и `git branch`.
- **Ошибки входа в GitHub** — см. главу 4 и главу 6.

## 3.11. git pull — получить изменения с GitHub

**Что делает.** Скачивает новые коммиты с GitHub и сразу объединяет их с вашей текущей веткой. По сути это две команды подряд: `git fetch` и `git merge`.

**Когда нужна.** Перед началом работы и перед `git push`, если над проектом работает кто-то ещё (или вы сами с другого компьютера).

**Как выглядит.**

```
POWERSHELL
git pull
```

**Пример.** Коллега отправил на GitHub изменения в `README.md`.

```
POWERSHELL
git pull
```

**Что произойдёт.**

```
From https://github.com/ivan-petrov/my-site
   554a88a..9b1c2d3  main       -> origin/main
Updating 554a88a..9b1c2d3
Fast-forward
 README.md | 2 ++
 1 file changed, 2 insertions(+)
```

`Fast-forward` — «перемотка вперёд»: ваша ветка просто догнала GitHub. Если новых коммитов нет, Git напишет `Already up to date.`

**Как проверить.** `git log --oneline` покажет новые коммиты, а `git status` — `Your branch is up to date with 'origin/main'`.

**Типичные ошибки.**

- **Открылся редактор с текстом** `Merge branch 'main' of ...`. И у вас, и на GitHub были разные новые коммиты, и Git создаёт коммит слияния. Сохраните файл и закройте редактор — сообщение уже заполнено.
- `CONFLICT (content): Merge conflict in ...` — вы и коллега изменили одни и те же строки. Как это исправить — в главе 6.
- `fatal: Need to specify how to reconcile divergent branches` — Git не знает, как объединять разошедшиеся ветки. Выполните один раз `git config --global pull.rebase false` и повторите `git pull`.
- `error: Your local changes to the following files would be overwritten by merge` — у вас есть незакоммиченные изменения в тех же файлах. Сначала закоммитьте их (`git add`, `git commit`).

## 3.12. git fetch — узнать, что нового, ничего не меняя

**Что делает.** Скачивает новые коммиты с GitHub, но **не трогает** ваши файлы и вашу ветку. Новые коммиты попадают в `origin/main` — «закладку», которая помнит состояние GitHub.

**Когда нужна.** Чтобы спокойно посмотреть, что изменилось на GitHub, прежде чем объединять.

**Как выглядит.**

```
POWERSHELL
git fetch
```

**Пример.**

```
POWERSHELL
git fetch
git status
```

**Что произойдёт.**

```
From https://github.com/ivan-petrov/my-site
   554a88a..9b1c2d3  main       -> origin/main
```

```
On branch main
Your branch is behind 'origin/main' by 1 commit, and can be fast-forwarded.
  (use "git pull" to update your local branch)
```

«Ваша ветка отстаёт от `origin/main` на 1 коммит».

**Как проверить.** Посмотрите новые коммиты и изменения до объединения:

```
POWERSHELL
git log --oneline main..origin/main   # коммиты, которых у вас ещё нет
git diff main origin/main             # что они меняют
```

Если всё в порядке — `git pull` (или `git merge origin/main`).

**Типичные ошибки.**

- «**Сделал `git fetch`, а файлы не изменились**». Так и задумано: `fetch` только скачивает. Чтобы применить изменения, выполните `git pull`.

| | `git fetch` | `git pull` |
|---|---|---|
| Скачивает коммиты с GitHub | Да | Да |
| Меняет ваши файлы и ветку | Нет | Да |
| Может вызвать конфликт | Нет | Да |
| Когда использовать | Посмотреть, что нового | Получить и сразу применить |

## 3.13. Шпаргалка

| Команда | Назначение |
|---|---|
| `git init` | Создать репозиторий в текущей папке |
| `git status` | Посмотреть состояние файлов |
| `git add <файл>` | Подготовить изменения к коммиту |
| `git add .` | Подготовить все изменения в папке |
| `git commit -m "..."` | Сохранить версию с сообщением |
| `git log --oneline` | Короткая история коммитов |
| `git log --oneline --graph --all` | История с рисунком всех веток |
| `git diff` | Изменения, ещё не добавленные в staging |
| `git diff --staged` | Изменения, которые войдут в коммит |
| `git clone <адрес>` | Скачать репозиторий с GitHub |
| `git push` | Отправить коммиты на GitHub |
| `git pull` | Скачать и применить изменения с GitHub |
| `git fetch` | Скачать изменения, не применяя их |
| `git remote -v` | Показать адреса удалённых репозиториев |

Полную документацию по любой команде открывает `git help <команда>`, например `git help commit`. Краткую справку прямо в терминале — `git commit -h`.

## Проверьте себя

Выполните в папке `git-practice`:

1. Создайте файл `notes.txt` с любым текстом. Какое состояние покажет `git status`?
2. Подготовьте и закоммитьте файл с сообщением «Добавить заметки».
3. Измените `notes.txt`. Что покажут `git diff` и `git diff --staged` до и после `git add`?
4. Сколько коммитов покажет `git log --oneline`?

**Ожидаемый результат**

1. Файл в разделе `Untracked files`.
2. `git add notes.txt`, затем `git commit -m "Добавить заметки"`. Git выведет строку вида `[main 1a2b3c4] Добавить заметки`.
3. До `git add`: `git diff` показывает изменения, `git diff --staged` — ничего. После `git add` — наоборот.
4. Четыре коммита, если вы выполнили все примеры главы: «Добавить файл hello.txt», «Добавить вторую строку», «Добавить третью строку», «Добавить заметки» (пятый появится, если закоммитите изменение из пункта 3).

## Что дальше

В главе 4 вы создадите аккаунт на GitHub и отправите туда свой проект.

# Глава 4. Работа с GitHub

В этой главе вы заведёте аккаунт на GitHub, научитесь работать с репозиторием через сайт и отправите туда проект со своего компьютера.

**Вы научитесь:**

- создавать аккаунт и репозиторий на GitHub;
- добавлять, изменять и удалять файлы через сайт;
- смотреть историю изменений и скачивать проекты;
- входить в GitHub из терминала;
- отправлять на GitHub существующий и новый проект.

**Время:** 1,5–2 часа. **Понадобится:** установленный Git (глава 2), почтовый ящик и браузер.

## 4.1. Аккаунт на GitHub

1. Откройте github.com/signup.
2. Введите почту, придумайте пароль и **имя пользователя** (*username*, логин).
3. Подтвердите почту: GitHub пришлёт код или ссылку. Без подтверждения нельзя, например, создавать репозитории.
4. Пройдите короткую анкету или пропустите её.

Можно также войти через аккаунт Google или Apple — кнопки **Continue with Google** / **Continue with Apple** на странице регистрации.

Советы по логину:

- логин станет частью адресов ваших проектов: `github.com/ivan-petrov/my-site`;
- выбирайте короткий и нейтральный — его увидят будущие работодатели и коллеги;
- допустимы латинские буквы, цифры и дефис.

### Двухфакторная аутентификация

> **Двухфакторная аутентификация** (2FA) — вход с двумя проверками: пароль плюс одноразовый код из приложения на телефоне или физический ключ. Даже если пароль украдут, без второго фактора войти не получится.

GitHub требует 2FA от всех, кто отправляет код на GitHub.com. Когда придёт ваша очередь, GitHub покажет напоминание и даст 45 дней на настройку. Не откладывайте:

1. Установите на телефон приложение-аутентификатор (любое, которое поддерживает одноразовые коды TOTP).
2. На GitHub: аватар в правом верхнем углу → **Settings** → **Password and authentication** → **Enable two-factor authentication**.
3. Отсканируйте QR-код приложением и введите код из него.

4. **Сохраните коды восстановления** (*recovery codes*) в надёжном месте. Без них при потере телефона доступ к аккаунту восстановить очень трудно.

Источник: About mandatory two-factor authentication.

## 4.2. Настройка приватности почты

Почта из настроек Git попадает в каждый коммит. В публичном репозитории её видят все. Чтобы скрыть настоящий адрес:

1. **Settings** → **Emails**.
2. Включите **Keep my email addresses private**. Ниже GitHub покажет ваш скрытый адрес вида `12345678+ivan-petrov@users.noreply.github.com`.
3. Включите **Block command line pushes that expose my email** — GitHub не примет push, если в коммите окажется ваш настоящий адрес.
4. Укажите скрытый адрес в Git:

```
POWERSHELL
git config --global user.email "12345678+ivan-petrov@users.noreply.github.com"
```

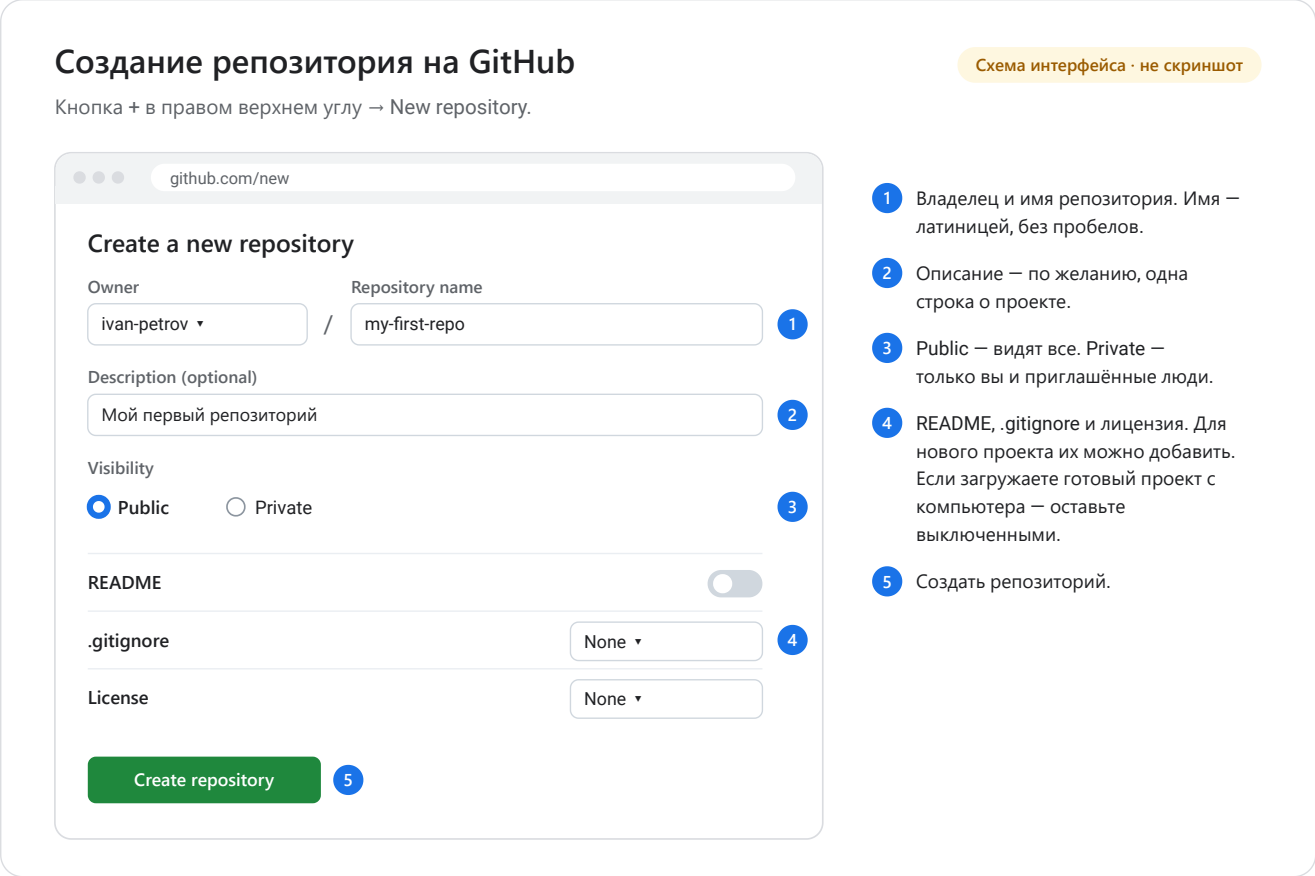
Новые коммиты будут подписаны скрытым адресом, и GitHub всё равно свяжет их с вашим профилем.

> **ПРИМЕЧАНИЕ**
> Коммиты, сделанные раньше, сохранят старый адрес. Если GitHub отклоняет push из-за почты, исправьте адрес в последнем коммите: `git commit --amend --reset-author --no-edit` (только для коммита, который ещё не отправлен).

Источник: Email addresses reference.

## 4.3. Создание репозитория на сайте

1. Нажмите **+** в правом верхнем углу → **New repository**.
2. Заполните форму:




*Схематическое изображение, а не снимок экрана. Реальный интерфейс GitHub может выглядеть иначе.*

3. Нажмите **Create repository**.

## Public или Private

| | Public (публичный) | Private (приватный) |
|---|---|---|
| Кто видит | Все в интернете | Только вы и приглашённые люди |
| Когда выбирать | Открытые проекты, портфолио, учебные проекты без личных данных | Рабочие проекты, личные заметки, всё, что не должно быть открытым |
| Что важно помнить | Видны все файлы и **вся история**, включая старые коммиты | Приватность не отменяет правил: секреты в Git хранить нельзя и здесь |

Видимость можно изменить позже: **Settings** → раздел **Danger Zone** → **Change repository visibility**.

> **ВНИМАНИЕ**
>
> Публичный репозиторий виден сразу и всем. Перед тем как сделать репозиторий публичным, проверьте всю историю на пароли, ключи и личные данные (глава 6).

## README, .gitignore и лицензия

В форме есть три необязательных пункта:

- **README** — создаёт файл `README.md` с названием проекта (см. раздел 4.4).

- **.gitignore** — список файлов, которые Git должен игнорировать. GitHub предлагает готовые шаблоны для языков программирования (Python, Node и другие).
- **License** — лицензия: условия, на которых другие могут использовать ваш код. Выбор лицензии подробно разобран в главе 10.

Как поступить:

| Ваша ситуация | README, .gitignore, лицензия |
|---|---|
| Новый проект, вы начнёте его с GitHub (сценарий Б1) | Включите README; .gitignore и лицензию — по желанию |
| Проект уже лежит на компьютере (сценарий А) | **Оставьте всё выключенным** — репозиторий должен быть пустым |

Почему это важно, объяснено в разделе 4.12.

### Создать репозиторий из терминала

Если установлен GitHub CLI (раздел 4.8, способ 2), пустой репозиторий на GitHub можно создать и связать с папкой проекта одной командой. Выполните её в папке, где уже есть хотя бы один коммит:

POWERSHELL

```
gh repo create my-site --private --source=. --remote=origin --push
```

| Часть | Значение |
|---|---|
| `gh repo create my-site` | Создать на GitHub репозиторий `my-site` |
| `--private` или `--public` | Видимость |
| `--source=.` | Взять код из текущей папки |
| `--remote=origin` | Добавить удалённый репозиторий с именем `origin` |
| `--push` | Сразу отправить коммиты |

Это короткий вариант сценария А (раздел 4.10). Новичку полезно сначала пройти сценарий пошагово, чтобы понимать, что делает каждая команда. Справка: gh repo create.

## 4.4. Что такое README.md

> **README.md** — главная страница проекта. GitHub автоматически показывает её под списком файлов. Обычно в README рассказывают, что это за проект, как его установить и как им пользоваться.

`.md` означает **Markdown** — простой язык разметки текста. Вы пишете обычный текст с несколькими значками, а GitHub превращает его в оформленную страницу:

| Вы пишете | GitHub показывает |
|---|---|
| `# Заголовок` | Крупный заголовок |
| `## Подзаголовок` | Заголовок поменьше |
| `**жирный**` | **жирный** |
| `- пункт` | Маркированный список |
| `1. пункт` | Нумерованный список |
| `[текст](https://example.com)` | Ссылка |
| `` `код` `` | `код` |

Пример минимального README:

```markdown
# Мой сайт

Учебный сайт-визитка.

## Как открыть

Откройте файл `index.html` в браузере.
```

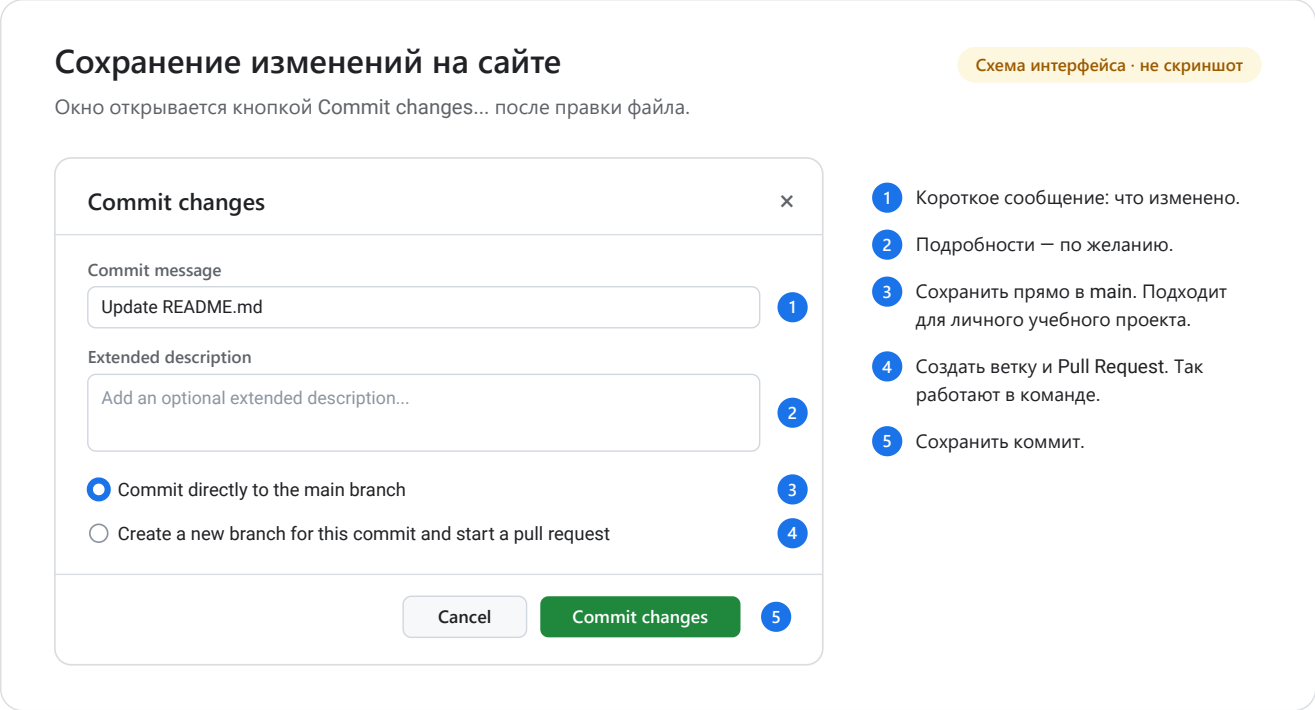
Как написать профессиональный README, разобрано в главе 9.

## 4.5. Работа с файлами через сайт

Небольшие правки удобно делать прямо в браузере. Каждое изменение на сайте — это коммит, как в терминале.

### Создать файл

1. На странице репозитория: **Add file** → **Create new file**.
2. Введите имя файла. Чтобы создать папку, напишите имя с косой чертой: `docs/notes.md`.
3. Напишите содержимое.
4. Нажмите **Commit changes...**, введите сообщение коммита и нажмите **Commit changes**.




*Схематическое изображение, а не снимок экрана. Реальный интерфейс GitHub может выглядеть иначе.*

### Загрузить файлы с компьютера

1. **Add file** → **Upload files**.
2. Перетащите файлы в окно или нажмите **choose your files**.
3. Внизу введите сообщение и нажмите **Commit changes**.

Ограничения загрузки через браузер: до 25 МБ на файл и до 100 файлов за раз. Для больших проектов используйте Git в терминале.

### Изменить файл

1. Откройте файл и нажмите значок карандаша (**Edit this file**).
2. Внесите правки. Вкладка **Preview** покажет результат для Markdown.
3. **Commit changes...** → сообщение → **Commit changes**.

### Удалить файл

1. Откройте файл.
2. Нажмите **...** (три точки) в правом верхнем углу файла → **Delete file**.
3. **Commit changes...** → сообщение → **Commit changes**.

> ВАЖНО
>
> Удалённый файл исчезает только из текущей версии. В истории он остаётся, и любой, кто видит репозиторий, может открыть старый коммит. Поэтому удаление файла не спасает от утечки секрета (глава 11).

### После правок на сайте

Изменения, сделанные на сайте, есть только на GitHub. Чтобы получить их на компьютере, выполните `git pull` в папке проекта.

### То же через терминал

Всё, что вы делаете с файлами на сайте, можно сделать в терминале в папке склонированного репозитория. Разница в том, что в терминале вы сначала сохраняете изменения у себя (коммит), а потом отправляете их на GitHub ( `git push` ).

| Действие | На сайте | В терминале |
|---|---|---|
| Создать файл | **Add file** → **Create new file** | `notepad notes.md` , затем `git add notes.md` , `git commit -m "Добавить заметки"` , `git push` |
| Загрузить файлы | **Add file** → **Upload files** | Скопируйте файлы в папку проекта, затем `git add .` , `git commit` , `git push` |
| Изменить файл | Значок карандаша | Отредактируйте файл, затем `git add` , `git commit` , `git push` |
| Удалить файл | **...** → **Delete file** | `git rm notes.md` , `git commit -m "Удалить заметки"` , `git push` |
| Переименовать файл | Измените имя в режиме редактирования | `git mv old.md new.md` , `git commit` , `git push` |
| Посмотреть историю | Ссылка с числом коммитов | `git log --oneline` |

> **`git rm <файл>`** удаляет файл из папки и сразу подготавливает удаление к коммиту. Файл можно вернуть, пока вы не сделали коммит: `git restore --staged <файл>` , затем `git restore <файл>` . После коммита старая версия остаётся в истории.

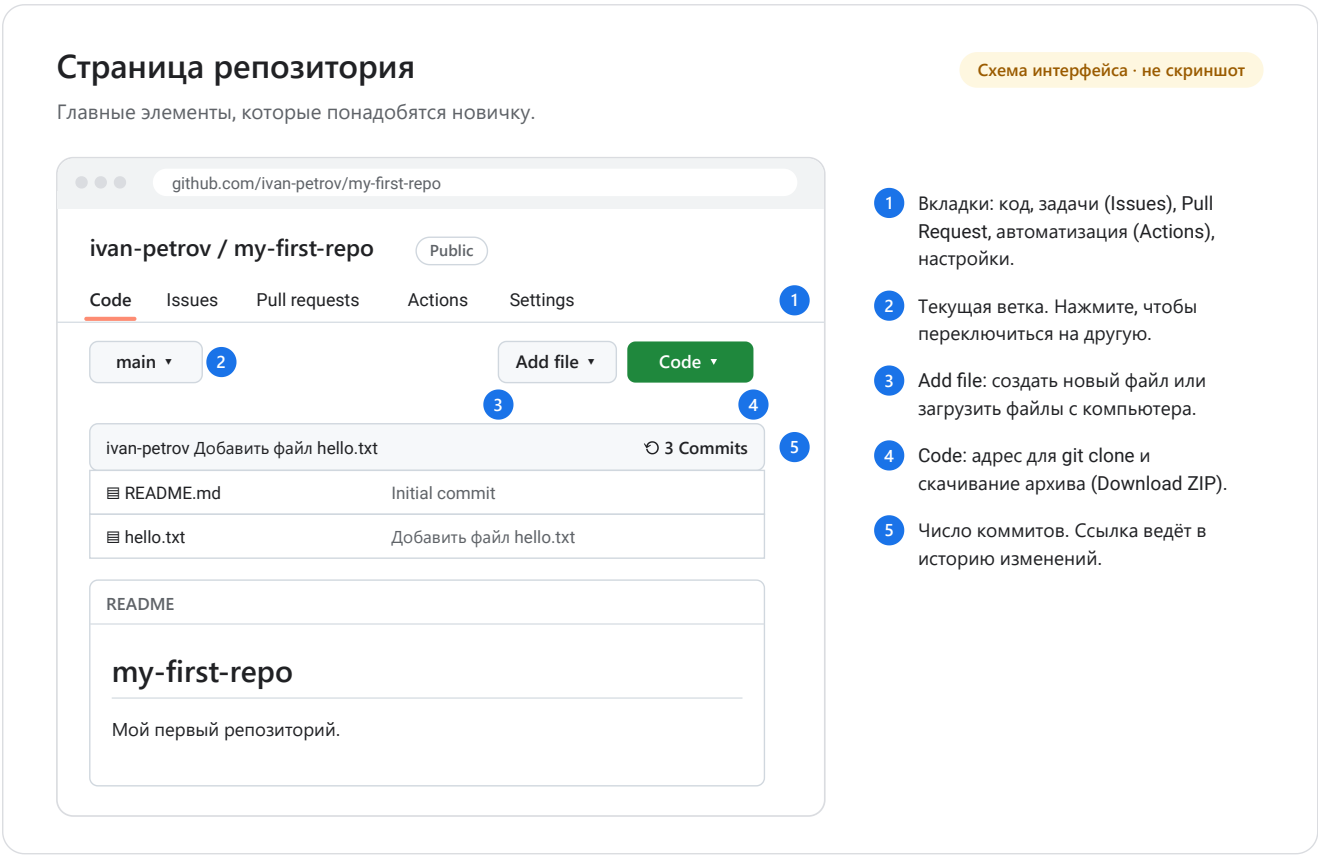
Пример удаления через терминал:

```powershell
git rm notes.md
git status
git commit -m "Удалить заметки"
git push
```

```
rm 'notes.md'
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        deleted:    notes.md
```

## 4.6. История изменений на сайте




*Схематическое изображение, а не снимок экрана. Реальный интерфейс GitHub может выглядеть иначе.*

- **История всего проекта.** Нажмите на число коммитов над списком файлов (например, **3 Commits**). Откроется список коммитов, как в `git log`.
- **Один коммит.** Нажмите на сообщение коммита — увидите, какие строки изменились, как в `git diff`. Добавленные строки зелёные, удалённые — красные.
- **История файла.** Откройте файл и нажмите **History**.
- **Кто изменил строку.** Откройте файл и переключитесь в режим **Blame**: рядом с каждой строкой будет коммит, который её изменил последним.

## 4.7. Скачать проект

Есть два способа, и они дают разный результат.

| | Download ZIP | git clone |
|---|---|---|
| Где | **Code → Download ZIP** | `git clone <адрес>` в терминале |
| Что получите | Только файлы текущей версии | Файлы и всю историю |
| Связь с GitHub | Нет | Да, можно делать `pull` и `push` |
| Когда использовать | Просто посмотреть или запустить чужой проект | Работать с проектом через Git |

Адрес для клонирования: **Code** → вкладка **HTTPS** → кнопка копирования. Адрес выглядит так: `https://github.com/ivan-petrov/my-site.git`.

```
POWERSHELL
cd ~\projects
git clone https://github.com/ivan-petrov/my-site.git
cd my-site
```

## 4.8. Вход в GitHub из терминала

Когда вы отправляете изменения (`git push`) или работаете с приватным репозиторием, GitHub спрашивает, кто вы. **Пароль от аккаунта для этого не подходит**: с 13 августа 2021 года GitHub не принимает пароли в командах Git.

Если вы попробуете ввести пароль, увидите:

```
remote: Support for password authentication was removed on August 13, 2021.
```

Есть три способа войти.

### Способ 1. Git Credential Manager — вход через браузер (рекомендуется)

**Git Credential Manager** (GCM) — программа, которая хранит данные для входа. Она устанавливается вместе с Git для Windows.

1. Выполните первую команду, которой нужен GitHub, например `git push`.
2. Откроется окно GCM. Выберите вход через браузер (**Sign in with your browser**).
3. В браузере войдите в GitHub и подтвердите доступ (**Authorize**).
4. Вернитесь в терминал — команда продолжит работу.

GCM запомнит вход в **Диспетчере учётных данных Windows**. В следующий раз ничего вводить не придётся.

### Способ 2. GitHub CLI

**GitHub CLI** (`gh`) — официальная программа GitHub для терминала. Кроме входа, она умеет создавать репозитории и Pull Request.

```
POWERSHELL
winget install --id GitHub.cli -e --source winget
```

Закройте и снова откройте терминал, чтобы он увидел новую программу, затем:

```
POWERSHELL
gh auth login
```

Отвечайте на вопросы: `GitHub.com` → `HTTPS` → `Yes` (настроить Git) → `Login with a web browser`. Скопируйте показанный код и вставьте его в открывшемся браузере.

### Способ 3. Персональный токен доступа

> **Персональный токен доступа** (*personal access token*, PAT) — длинный секретный ключ, который заменяет пароль в командах Git. Токен можно ограничить: дать доступ только к нужным репозиториям, только на чтение, на определённый срок.

GitHub рекомендует **fine-grained** токены (они начинаются с `github_pat_`). Создать: **Settings** → **Developer settings** → **Personal access tokens** → **Fine-grained tokens** → **Generate new token**. Для отправки изменений токену нужно разрешение на запись содержимого репозитория: **Repository permissions** → **Contents** → **Read and write**. При первом `git push` откроется окно Git Credential Manager — выберите вход по токену (**Token**) и вставьте его. Если GCM не установлен, Git спросит логин и пароль в терминале — вместо пароля вставьте токен.

> ОПАСНО
>
> Токен — такой же секрет, как пароль. Не храните его в файлах проекта, не отправляйте в чатах и не показывайте AI-ассистентам. Если токен утёк — сразу удалите его на GitHub и создайте новый.

### А как же SSH?

**SSH-ключ** — ещё один способ входа: пара файлов-ключей на компьютере, публичную часть которых вы добавляете на GitHub. Он удобен опытным пользователям, но требует больше настройки. Для начала хватит способа 1. Инструкция: Connecting to GitHub with SSH.

## 4.9. Удалённый репозиторий: git remote

> **Удалённый репозиторий** (*remote*) — копия репозитория на сервере, с которой ваш локальный репозиторий обменивается коммитами. У него есть короткое имя и адрес.

По соглашению главный удалённый репозиторий называют `origin`. `git clone` настраивает его сам, а при `git init` его нужно добавить вручную.

| Команда | Что делает |
|---|---|
| `git remote add origin <адрес>` | Запомнить адрес GitHub-репозитория под именем `origin` |
| `git remote -v` | Показать удалённые репозитории и их адреса (`-v` — *verbose*, подробно) |
| `git remote set-url origin <адрес>` | Заменить адрес, если ошиблись |
| `git remote remove origin` | Забыть удалённый репозиторий (на GitHub ничего не удаляется) |

Пример вывода `git remote -v`:

```
origin  https://github.com/ivan-petrov/my-site.git (fetch)
origin  https://github.com/ivan-petrov/my-site.git (push)
```

`(fetch)` — адрес для скачивания, `(push)` — для отправки. Обычно они совпадают.

### Что значит git push -u origin main

```powershell
git push -u origin main
```

| Часть | Значение |
|---|---|
| `git push` | Отправить коммиты |
| `-u` | Запомнить связь: локальная `main` ↔ `origin/main` (полная форма — `--set-upstream`) |
| `origin` | Куда отправить — в удалённый репозиторий с именем `origin` |
| `main` | Какую ветку отправить |

`-u` нужен один раз для каждой новой ветки. Потом достаточно `git push` и `git pull` без параметров: Git уже знает, куда и откуда.

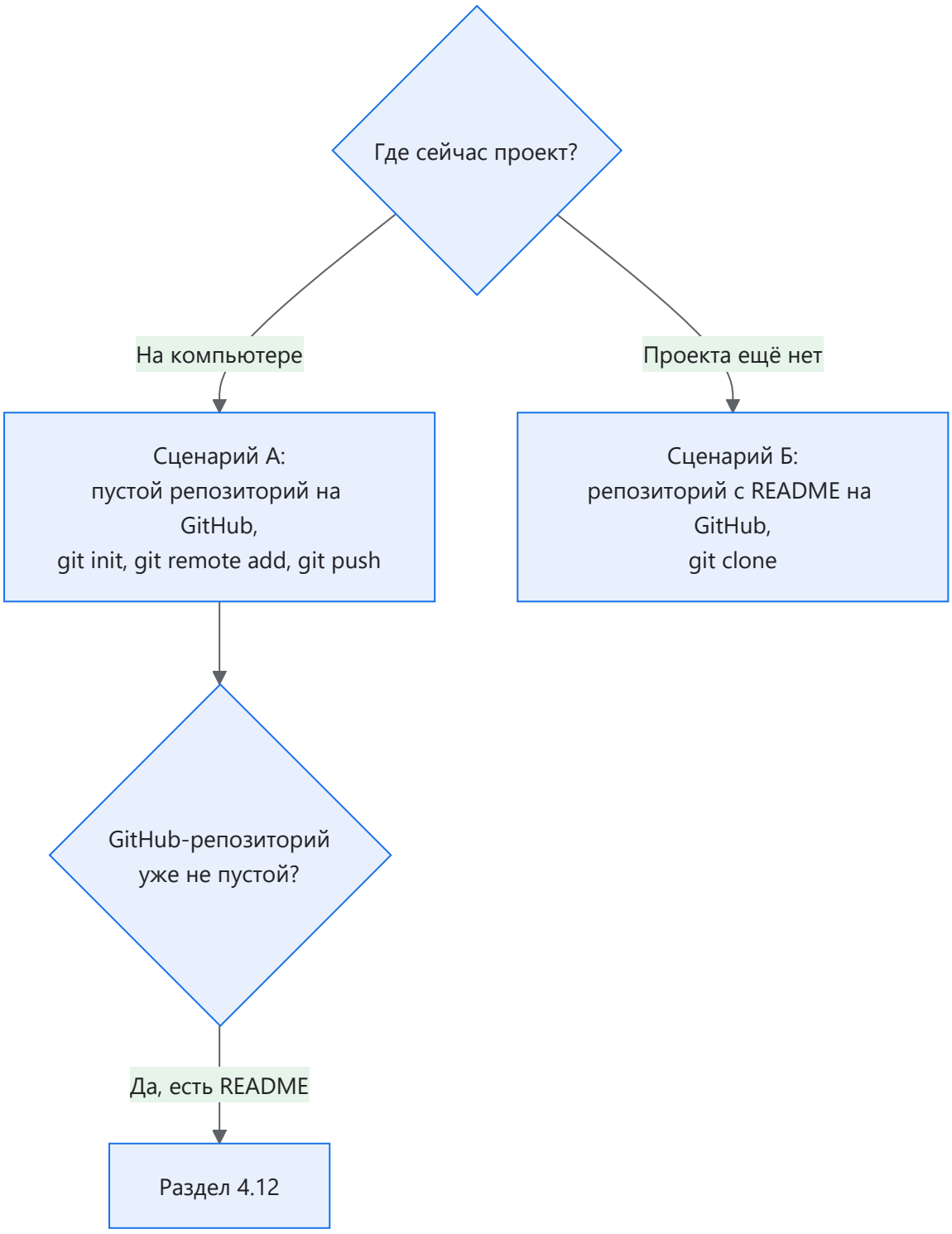
### Что значит git branch -M main

```powershell
git branch -M main
```

Переименовывает текущую ветку в `main` (`-M` — переименовать, даже если ветка с таким именем уже есть). Нужна, если ваш репозиторий создан с веткой `master`. Если ветка уже называется `main`, команда ничего не меняет — её можно выполнить для надёжности.

## 4.10. Отправка проекта на GitHub: сценарий А — проект уже на компьютере

У вас есть папка с проектом, например `C:\Users\Ivan\projects\my-site`. Отправим её на GitHub.




**Шаг 1. Откройте терминал в папке проекта.**

```
POWERSHELL
cd ~\projects\my-site
```

**Шаг 2. Создайте репозиторий** (если папка ещё не репозиторий — проверьте командой `git status`):

```
POWERSHELL
git init
```

**Шаг 3. Создайте `.gitignore`**, чтобы не отправить лишнее:

```powershell
New-Item .gitignore
notepad .gitignore
```

Впишите в него то, что не должно попасть на GitHub, например:

```.gitignore
# Секреты и личные настройки
.env
*.key

# Временные и служебные файлы
*.log
Thumbs.db
desktop.ini
```

Подробнее о `.gitignore` — в главе 6.

**Шаг 4. Проверьте, что будет сохранено.**

```powershell
git add .
git status
```

Внимательно просмотрите список `Changes to be committed`. Если там есть пароли, ключи, личные документы или большие файлы — уберите их из staging командой `git rm --cached <файл>` (в репозитории ещё нет коммитов, поэтому `git restore --staged` здесь не сработает), добавьте их в `.gitignore` и снова выполните `git add .` и `git status`.

**Шаг 5. Сделайте первый коммит.**

```powershell
git commit -m "Первый коммит"
```

**Шаг 6. Создайте пустой репозиторий на GitHub.** **+** → **New repository** → имя (например, `my-site`) → видимость → README, .gitignore и лицензию **не добавляйте** → **Create repository**. GitHub покажет страницу **Quick setup** с адресом репозитория. Скопируйте адрес HTTPS.

**Шаг 7. Свяжите локальный репозиторий с GitHub.**

```powershell
git remote add origin https://github.com/ivan-petrov/my-site.git
git remote -v
```

Ожидаемый результат:

```
origin  https://github.com/ivan-petrov/my-site.git (fetch)
origin  https://github.com/ivan-petrov/my-site.git (push)
```

**Шаг 8. Убедитесь, что ветка называется main.**

```
POWERSHELL
git branch -M main
```

**Шаг 9. Отправьте проект.**

```
POWERSHELL
git push -u origin main
```

При первом push откроется окно входа (раздел 4.8). Ожидаемый результат:

```
Enumerating objects: 5, done.
...
To https://github.com/ivan-petrov/my-site.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

**Шаг 10. Проверьте.** Обновите страницу репозитория в браузере — там появятся ваши файлы. В терминале:

```
POWERSHELL
git status
```

```
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
```

Все команды сценария А подряд:

```
POWERSHELL
cd ~\projects\my-site
git init
# создать и заполнить .gitignore
git add .
git status
git commit -m "Первый коммит"
# создать ПУСТОЙ репозиторий на GitHub и скопировать адрес
git remote add origin https://github.com/ivan-petrov/my-site.git
git remote -v
git branch -M main
git push -u origin main
```

## 4.11. Сценарий Б — новый проект с нуля

### Вариант Б1. Начать с GitHub (рекомендуется для нового проекта)

Это самый простой путь: репозиторий сразу правильно связан, и конфликтов с README не будет.

1. На GitHub: **+** → **New repository** → имя `my-new-project` → **включите README** → **Create repository**.
2. Скопируйте адрес: **Code** → **HTTPS**.
3. Склонируйте репозиторий:

```
POWERSHELL
cd ~\projects
git clone https://github.com/ivan-petrov/my-new-project.git
cd my-new-project
```

4. Создайте файлы проекта, например `index.html`.
5. Сохраните и отправьте:

```
POWERSHELL
git add .
git status
git commit -m "Добавить главную страницу"
git push
```

`-u` не нужен: после `git clone` связь с `origin/main` уже настроена.

### Вариант Б2. Начать на компьютере

Создайте папку, файлы и дальше действуйте по сценарию А:

```
POWERSHELL
mkdir ~\projects\my-new-project
cd ~\projects\my-new-project
git init
notepad README.md
git add .
git commit -m "Первый коммит"
git remote add origin https://github.com/ivan-petrov/my-new-project.git
git branch -M main
git push -u origin main
```

Репозиторий на GitHub в этом варианте тоже должен быть **пустым**.

## 4.12. Если на GitHub уже есть README.md

Частая ситуация: вы создали репозиторий на GitHub с README, а потом пытаетесь отправить туда проект с компьютера (сценарий А). Push не проходит:

```
To https://github.com/ivan-petrov/my-site.git
 ! [rejected]        main -> main (fetch first)
error: failed to push some refs to 'https://github.com/ivan-petrov/my-site.git'
hint: Updates were rejected because the remote contains work that you do not
hint: have locally. ...
```

**Почему.** На GitHub есть коммит с README, которого нет у вас. Ваш проект и репозиторий на GitHub — две независимые истории. Git не перезаписывает чужую историю молча.

Выберите один из способов.

### Способ 1. Пересоздать репозиторий (если на GitHub ничего ценного)

1. На GitHub: **Settings** → **Danger Zone** → **Delete this repository**.
2. Создайте репозиторий заново, **без README**.
3. Выполните `git push -u origin main`.

> **ВНИМАНИЕ**
>
> Удаление репозитория на GitHub необратимо: пропадут все файлы, Issues и Pull Request. Используйте этот способ, только если репозиторий только что создан и в нём ничего нет, кроме автоматического README.

### Способ 2. Объединить две истории

```powershell
git pull origin main --allow-unrelated-histories --no-rebase --no-edit
```

| Параметр | Зачем |
|---|---|
| `origin main` | Откуда взять изменения |
| `--allow-unrelated-histories` | Разрешить объединить две независимые истории. Без него Git ответит `fatal: refusing to merge unrelated histories` |
| `--no-rebase` | Объединить слиянием (merge) — самый понятный для новичка способ |
| `--no-edit` | Не открывать редактор, взять стандартное сообщение коммита слияния |

Если README.md есть и у вас, и на GitHub, Git сообщит о конфликте:

```
Auto-merging README.md
CONFLICT (add/add): Merge conflict in README.md
Automatic merge failed; fix conflicts and then commit the result.
```

Откройте `README.md` в Блокноте. Вы увидите обе версии между маркерами:

```
<<<<<<< HEAD
# Мой сайт
Описание
=======
# my-site
>>>>>>> b9a3c4f2e69e8f1a7796471dda0a3b0812e6919d
```

Оставьте нужный текст, удалите строки с `<<<<<<<`, `=======`, `>>>>>>>`, сохраните файл. Затем:

```powershell
git add README.md
git commit -m "Объединить с README из GitHub"
git push -u origin main
```

Подробнее о конфликтах — в главе 6.

### Способ 3. Склонировать и перенести файлы

1. Склонируйте GitHub-репозиторий в **новую** папку: `git clone <адрес> my-site-new`.
2. Скопируйте в неё файлы проекта через Проводник (кроме папки `.git`!).
3. В новой папке: `git add .`, `git commit -m "Добавить файлы проекта"`, `git push`.

Минус способа: история старого локального репозитория не перенесётся.

> **ОПАСНО**
>
> В интернете часто советуют `git push --force`. Эта команда **перезапишет** историю на GitHub вашей версией, и всё, чего нет у вас, пропадёт. Не используйте её, пока не понимаете последствий (глава 6).

## 4.13. Ежедневная синхронизация

Когда проект уже на GitHub, рабочий день выглядит так:

```powershell
# начало работы
git pull

# работа: изменения, проверки, коммиты
git status
git add <файлы>
git commit -m "Что сделано"

# конец работы или законченная задача
git push
```

Правило: **pull перед началом работы, push после коммитов**. Так меньше конфликтов, а GitHub всегда хранит свежую копию.

## Проверьте себя

1. Создайте на GitHub публичный репозиторий `git-practice` **без** README.
2. Отправьте туда учебный репозиторий из главы 3 по сценарию А.
3. Отредактируйте `hello.txt` прямо на сайте и закоммитьте изменение.
4. Получите это изменение на компьютере.
5. Посмотрите историю на сайте и в терминале. Совпадает ли она?

> **Ожидаемый результат**
>
> 1. Пустой репозиторий со страницей **Quick setup**.
> 2. `git remote add origin https://github.com/<логин>/git-practice.git`, `git branch -M main`, `git push -u origin main`. На сайте видны `hello.txt`, `notes.txt` и все коммиты.

3. Новый коммит в истории на сайте.
4. `git pull` выводит `Fast-forward` и имя изменённого файла.
5. Да. `git log --oneline` показывает те же коммиты и хеши, что и список коммитов на сайте.

## Что дальше

В главе 5 вы научитесь работать с ветками и Pull Request.

# Глава 5. Ветки и Pull Request

Ветки позволяют пробовать новое, не ломая рабочую версию. Pull Request — способ показать изменения другим и обсудить их до того, как они попадут в основную ветку. Так работают почти все команды разработчиков.

**Вы научитесь:**

- создавать ветки и переключаться между ними;
- объединять ветки командой `git merge`;
- отправлять ветку на GitHub и открывать Pull Request;
- просматривать изменения перед объединением.

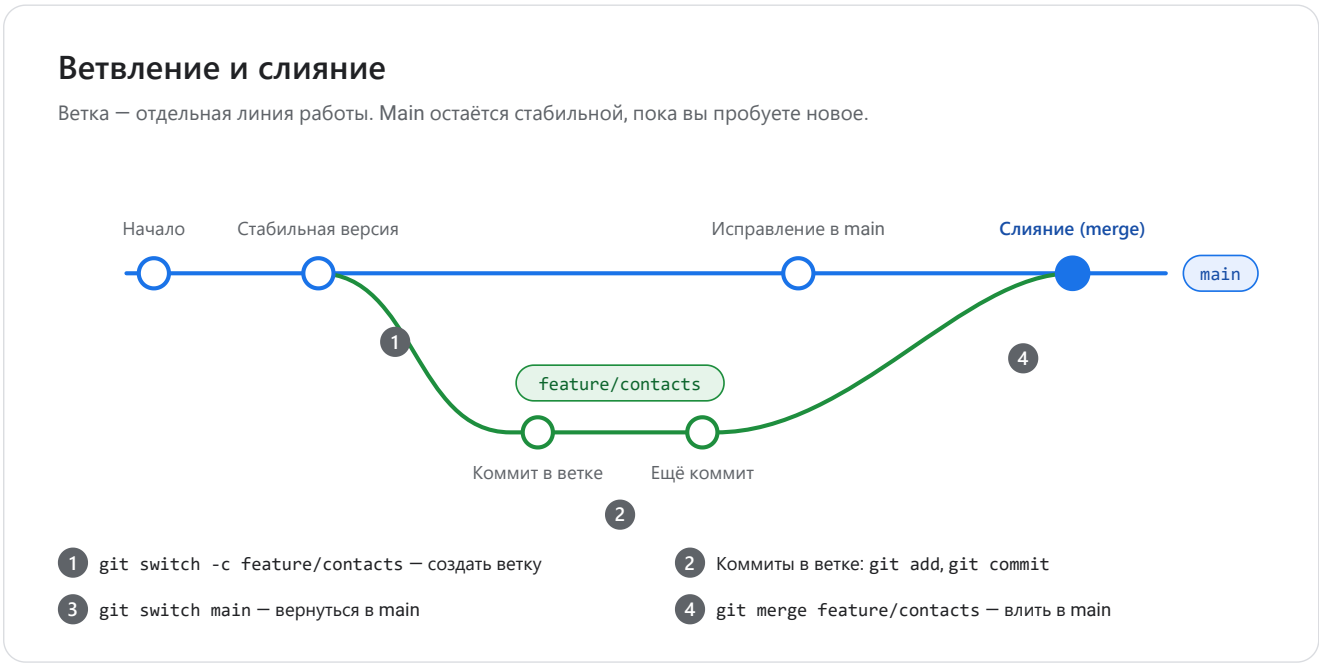
**Время:** 1,5 часа. **Понадобится:** репозиторий, связанный с GitHub (глава 4).

## 5.1. Что такое ветка

> **Ветка** (*branch*) — независимая линия разработки. Коммиты в одной ветке не появляются в другой, пока вы не объедините ветки.

Технически ветка — это просто подвижная метка, которая указывает на последний коммит линии. Когда вы делаете новый коммит, метка передвигается на него. Поэтому ветки в Git создаются мгновенно и почти не занимают места.

Пример. Сайт работает, а вам нужно добавить страницу контактов. Вы создаёте ветку `feature/contacts` и работаете в ней. Пока работа не закончена, сайт в ветке `main` остаётся рабочим.



## 5.2. Зачем нужна main

> **main** — основная ветка репозитория. В ней лежит стабильная версия проекта: та, что работает и которую можно показать пользователям.

Правила, которых придерживаются почти все команды:

- **В `main` не экспериментируют.** Новая работа идёт в отдельной ветке.
- **`main` всегда рабочая.** Изменения попадают в неё только после проверки.
- **Одна задача — одна ветка.** Так изменения легче проверять и отменять.

> **ПРИМЕЧАНИЕ**
>
> В старых проектах основная ветка называется `master`. Это то же самое, просто другое имя. С 2020 года GitHub называет основную ветку новых репозиториев `main`.

## 5.3. Создание и переключение веток

### Посмотреть ветки — git branch

```powershell
git branch
```

```
* main
```

Звёздочка `*` отмечает текущую ветку. Ветки на GitHub показывает `git branch -a` (они начинаются с `remotes/origin/`).

### Создать ветку и перейти в неё — git switch -c

```powershell
git switch -c feature/contacts
```

```
Switched to a new branch 'feature/contacts'
```

- `switch` — переключиться;
- `-c` (от *create*) — сначала создать ветку.

Новая ветка начинается с того коммита, на котором вы находились.

### Переключиться на существующую ветку — git switch

```powershell
git switch main
```

```
Switched to branch 'main'
```

При переключении Git меняет файлы в рабочей папке: они становятся такими, какими были в последнем коммите выбранной ветки. Файл, созданный в другой ветке, «исчезнет» из папки и вернётся, когда вы переключитесь обратно.

> **ПРИМЕЧАНИЕ**
>
> В старых инструкциях вместо `git switch` встречается `git checkout`. `git checkout feature/contacts` — то же, что `git switch feature/contacts`, а `git checkout -b имя` — то же, что `git switch -c имя`. Команда `switch` проще и безопаснее, используйте её.

### Как называть ветки

Имя ветки — латиница, цифры, дефисы и косая черта, без пробелов. Распространённые префиксы:

| Префикс | Для чего | Пример |
|---|---|---|
| `feature/` | Новая возможность | `feature/contacts` |
| `fix/` | Исправление ошибки | `fix/menu-typo` |
| `docs/` | Документация | `docs/readme-install` |
| `experiment/` | Проба идеи, которую могут выбросить | `experiment/new-design` |

Это соглашение, а не правило Git. В команде договоритесь о своём варианте.

### Если Git не даёт переключиться

```
error: Your local changes to the following files would be overwritten by checkout:
        index.html
Please commit your changes or stash them before you switch branches.
Aborting
```

Перевод: «Ваши незакоммиченные изменения в `index.html` будут перезаписаны. Закоммитьте их или отложите (stash) перед переключением». Git защищает вашу работу. Варианты:

- закоммитить изменения: `git add`, `git commit`;
- временно отложить: `git stash` — подробнее в главе 6.

## 5.4. Объединение веток — git merge

> **Слияние** (*merge*) — перенос изменений из одной ветки в другую.

Сливают всегда **в текущую ветку**. Поэтому сначала переключитесь туда, *куда* нужно влить изменения.

**Пример.** В ветке `feature/contacts` есть коммит «Добавить страницу контактов». Вливаем его в `main`:

```powershell
git switch main
git merge feature/contacts
```

```
Updating 08d86d6..611fbba
Fast-forward
 contacts.html | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 contacts.html
```

### Два вида слияния

**Fast-forward** («перемотка вперёд»). Пока вы работали в ветке, в `main` не было новых коммитов. Git просто передвигает метку `main` вперёд. Новый коммит не создаётся.

**Коммит слияния** (*merge commit*). Пока вы работали в ветке, в `main` появились другие коммиты. Git объединяет обе линии и создаёт отдельный коммит слияния:

```
Merge made by the 'ort' strategy.
 menu.html | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 menu.html
```

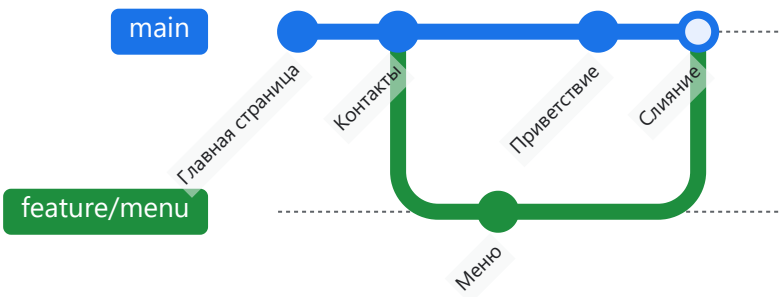
`ort` — название алгоритма слияния, ничего делать не нужно. Если Git открыл редактор с сообщением `Merge branch 'feature/menu'`, сохраните файл и закройте редактор.

История после такого слияния:

```powershell
git log --oneline --graph --all
```

```
*   ceacf4c (HEAD -> main) Merge branch 'feature/menu'
|\
| * 068b99c (feature/menu) Добавить меню
* | 29dba4b Добавить приветствие
|/
* 611fbba Добавить страницу контактов
* 08d86d6 Добавить главную страницу
```

То же в виде диаграммы:



Если обе ветки изменили одни и те же строки, возникнет **конфликт**. Как его решить — в главе 6.

### Удалить ветку после слияния

```powershell
git branch -d feature/contacts
```

```
Deleted branch feature/contacts (was 611fbba).
```

Коммиты не пропадают: они уже в `main`. Удаляется только метка.

Если ветка ещё не слита, Git откажется её удалять:

```
error: the branch 'trial' is not fully merged
hint: If you are sure you want to delete it, run 'git branch -D trial'
```

> **ВНИМАНИЕ**
>
> `git branch -D` (с большой буквы) удаляет ветку принудительно, даже если её коммиты больше нигде нет. Используйте, только если действительно хотите выбросить эту работу. Восстановить её в течение некоторого времени можно через `git reflog` (глава 6).

## 5.5. Отправка ветки на GitHub

Ветка, созданная на компьютере, существует только у вас. Чтобы показать её на GitHub:

```powershell
git push -u origin feature/contacts
```

```
...
remote: Create a pull request for 'feature/contacts' on GitHub by visiting:
remote:      https://github.com/ivan-petrov/my-site/pull/new/feature/contacts
remote:
To https://github.com/ivan-petrov/my-site.git
 * [new branch]      feature/contacts -> feature/contacts
branch 'feature/contacts' set up to track 'origin/feature/contacts'.
```

Строки с `remote:` — сообщение от GitHub: он сразу предлагает ссылку для создания Pull Request.

Дальнейшие коммиты в этой ветке отправляйте короткой командой `git push`.

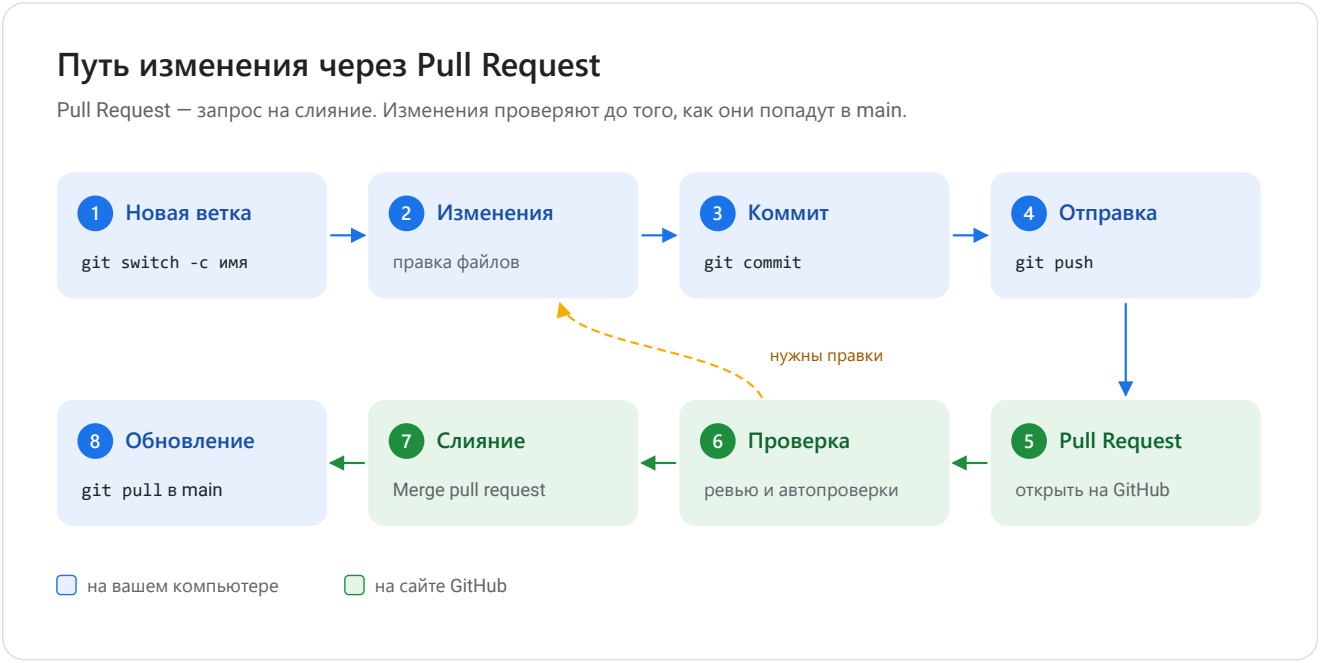
## 5.6. Что такое Pull Request

> **Pull Request** (PR, «пул-реквест», «запрос на слияние») — предложение влить изменения из одной ветки в другую на GitHub. В нём видно, что изменилось; там же изменения обсуждают, проверяют и принимают.

Зачем PR, если можно выполнить `git merge` самому:

- **Второй взгляд.** Коллега находит ошибки, которые автор не заметил.
- **Автоматические проверки.** GitHub Actions запускает тесты до слияния (глава 11).
- **История решений.** Через год видно не только *что* изменили, но и *почему*.
- **Защита `main`.** Можно запретить изменения в `main` без PR (глава 11).

PR полезен и тем, кто работает один: это удобный способ ещё раз просмотреть свои изменения перед слиянием.




## 5.7. Работа через Pull Request: пошагово

**Шаг 1. Создайте ветку и сделайте изменения.**

```powershell
git switch main
git pull
git switch -c feature/contacts
notepad contacts.html
git add contacts.html
git commit -m "Добавить страницу контактов"
```

`git pull` в начале — чтобы ветка началась с самой свежей версии `main`.

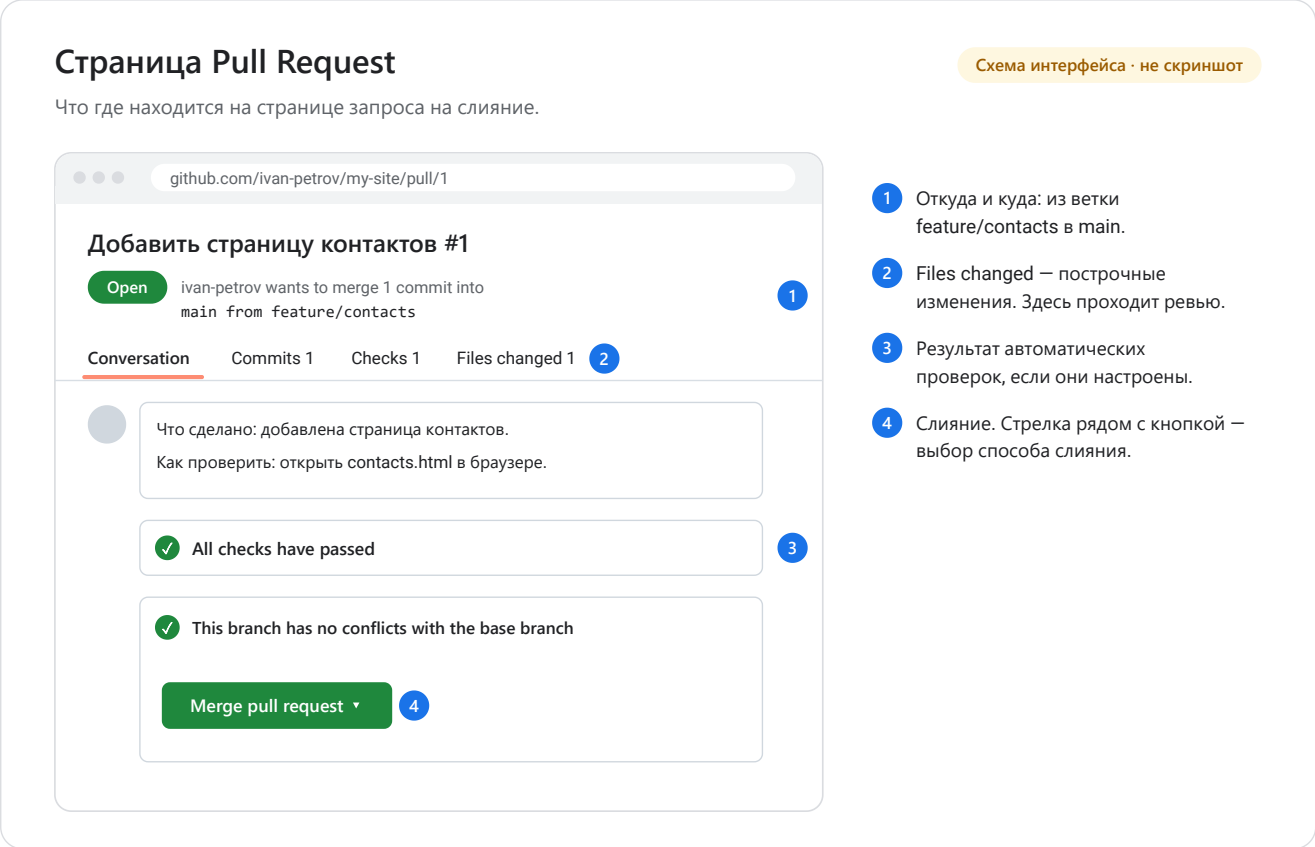
**Шаг 2. Отправьте ветку.**

```powershell
git push -u origin feature/contacts
```

**Шаг 3. Откройте Pull Request.**

1. Откройте репозиторий на GitHub. Вверху появится жёлтая плашка с кнопкой **Compare & pull request** — нажмите её. Если плашки нет: вкладка **Pull requests** → **New pull request** → в поле **compare** выберите свою ветку.
2. Проверьте направление: **base: main** ← **compare: feature/contacts**. Base — куда вливаем, compare — откуда.
3. Заполните **заголовок** (что сделано) и **описание** (зачем и как проверить).
4. Нажмите **Create pull request**.

Если работа ещё не закончена, выберите в выпадающем меню кнопки **Create draft pull request** — черновик нельзя случайно слить.




*Схематическое изображение, а не снимок экрана. Реальный интерфейс GitHub может выглядеть иначе.*

**Шаг 4. Просмотрите изменения.** Вкладки PR:

| Вкладка | Что там |
|---|---|
| **Conversation** | Описание, обсуждение, результаты проверок, кнопка слияния |
| **Commits** | Список коммитов из ветки |
| **Checks** | Результаты автоматических проверок, если они настроены |
| **Files changed** | Построчные изменения во всех файлах — главное место ревью |

**Шаг 5. Ревью.** Ревьюер (или вы сами) открывает **Files changed**, оставляет комментарии к строкам и выбирает итог: **Comment** (просто комментарий), **Approve** (одобрить) или **Request changes** (попросить исправить).

> **Ревью кода** (*code review*) — чтение изменений другим человеком перед их принятием.

Если нужны исправления — сделайте новые коммиты в ту же ветку и отправьте их `git push`. PR обновится сам, создавать новый не нужно.

**Шаг 6. Слияние.** На вкладке **Conversation** нажмите **Merge pull request**, затем **Confirm merge**. В выпадающем меню кнопки есть три способа:

| Способ | Что делает | Когда выбрать |
|---|---|---|
| **Create a merge commit** | Переносит все коммиты и добавляет коммит слияния | По умолчанию, сохраняет полную историю |
| **Squash and merge** | Склеивает все коммиты ветки в один | Когда в ветке много мелких коммитов «поправил опечатку» |
| **Rebase and merge** | Переносит коммиты по одному без коммита слияния | Когда команда хочет линейную историю |

Для начала подойдёт любой. Главное — чтобы в команде все делали одинаково.

**Шаг 7. Удалите ветку на GitHub.** После слияния появится кнопка **Delete branch**. Работа уже в `main`, ветка больше не нужна.

**Шаг 8. Обновите `main` у себя.**

```powershell
git switch main
git pull
git branch -d feature/contacts
```

Теперь ваша локальная `main` содержит слитые изменения, а лишняя ветка удалена.

## 5.8. Просмотр изменений перед объединением

Смотреть изменения можно и до создания PR, прямо в терминале.

Какие коммиты есть в ветке, но ещё нет в `main`:

```powershell
git log --oneline main..feature/contacts
```

Что изменится в файлах после слияния:

```powershell
git diff main...feature/contacts
```

Три точки означают «изменения ветки с момента, когда она отделилась от `main`». Это то же, что показывает вкладка **Files changed** в PR.

Только список изменённых файлов:

```powershell
git diff --stat main...feature/contacts
```

На что смотреть при ревью:

- изменено только то, что заявлено в описании;
- нет случайных файлов: временных, личных настроек, паролей и ключей;
- нет закомментированного или отладочного кода;
- проверки (если есть) прошли;
- понятно, как проверить изменения вручную.

## 5.9. Командная работа

### Как дать доступ коллеге

1. Откройте репозиторий → **Settings** → **Collaborators** (в некоторых версиях — **Collaborators and teams**).
2. Нажмите **Add people** и найдите коллегу по логину или почте.
3. Коллега получит приглашение на почту и должен его принять.

Участник с доступом (*collaborator*) личного репозитория может отправлять ветки и сливать PR. Не путайте это с соавторством в смысле авторского права (глава 10): доступ к репозиторию не делает человека правообладателем. Тонкая настройка прав доступна в репозиториях организаций.

### GitHub Flow

Рабочий процесс, который описан в документации GitHub, называется **GitHub Flow**:




Источник: GitHub flow.

### Как предложить изменение в чужой проект

У вас нет прав на запись в чужой репозиторий. Поэтому:

1. Нажмите **Fork** — GitHub создаст вашу копию репозитория.
2. Склонируйте свою копию, создайте ветку, сделайте коммиты, отправьте.
3. Откройте Pull Request из своей копии в исходный репозиторий.

> **Форк** (*fork*) — копия чужого репозитория в вашем аккаунте GitHub, связанная с оригиналом.

Перед этим прочитайте файл `CONTRIBUTING.md` проекта, если он есть: там описаны правила участия (глава 9).

## 5.10. Шпаргалка

| Команда | Назначение |
|---|---|
| `git branch` | Список локальных веток |
| `git branch -a` | Локальные и удалённые ветки |
| `git switch -c <ветка>` | Создать ветку и перейти в неё |
| `git switch <ветка>` | Перейти в ветку |
| `git merge <ветка>` | Влить ветку в текущую |
| `git branch -d <ветка>` | Удалить слитую ветку |
| `git push -u origin <ветка>` | Отправить новую ветку на GitHub |
| `git log --oneline main..<ветка>` | Коммиты ветки, которых нет в `main` |
| `git diff main...<ветка>` | Изменения ветки относительно `main` |

## Проверьте себя

1. Создайте ветку `feature/about`, добавьте файл `about.txt` и закоммитьте его.
2. Переключитесь на `main`. Есть ли в папке `about.txt`?
3. Влейте ветку в `main`. Какое слово Git напишет в выводе?
4. Удалите ветку `feature/about`.

**Ожидаемый результат**

1. `git switch -c feature/about`, создать файл, `git add about.txt`, `git commit -m "Добавить страницу о себе"`.
2. Нет. Файл есть только в ветке `feature/about`.
3. `git switch main`, затем `git merge feature/about`. В выводе будет `Fast-forward`, потому что в `main` не было новых коммитов.
4. `git branch -d feature/about` → `Deleted branch feature/about (was ...)`.

## Что дальше

В главе 6 вы научитесь решать конфликты, отменять изменения и разбираться в ошибках.

# Глава 6. Конфликты, восстановление и типичные ошибки

Ошибаются все. Хорошая новость: Git почти всегда позволяет вернуться назад — если знать, какую команду выбрать. В этой главе вы научитесь решать конфликты, отменять изменения, защищать секреты и разбираться в сообщениях об ошибках.

**Вы научитесь:**

- находить и исправлять конфликты слияния;
- отменять изменения до и после коммита;
- восстанавливать старые версии файлов;
- выбирать между `git restore`, `git revert` и `git reset`;
- не допускать утечки паролей и ключей;
- понимать сообщения терминала и исправлять частые ошибки.

**Время:** 2 часа. **Понадобится:** учебный репозиторий из глав 3–5.

> ВАЖНО
>
> В этой главе есть команды, которые **удаляют изменения без возможности восстановления** или **переписывают историю**. Они отмечены предупреждениями. Перед такими командами всегда выполняйте `git status` и думайте, что именно пропадёт.

## 6.1. Что такое конфликт слияния

> **Конфликт слияния** (*merge conflict*) — ситуация, когда Git не может сам объединить изменения, потому что обе ветки по-разному изменили одно и то же место в файле.

Почему он возникает. Вы в ветке `main` написали «Молоко — 2 литра», а коллега в ветке `feature/milk` в той же строке — «Молоко — 1 литр». Обе правки разумны, и Git не знает, какая правильная. Он останавливается и просит решить человека.

Конфликт — не ошибка и не поломка. Это нормальная часть совместной работы.

Когда конфликт **не** возникает:

- изменены разные файлы;
- изменены разные места одного файла;
- изменения совпадают.

Конфликт может появиться при `git merge`, `git pull`, `git revert` и других командах, которые объединяют изменения.

## 6.2. Как найти и исправить конфликт

**Шаг 1. Git сообщает о конфликте.**

```
Auto-merging shopping.txt
CONFLICT (content): Merge conflict in shopping.txt
Automatic merge failed; fix conflicts and then commit the result.
```

Перевод: «Конфликт содержимого в `shopping.txt`. Автоматическое слияние не удалось: исправьте конфликты и сделайте коммит».

**Шаг 2. Найдите файлы с конфликтом.**

POWERSHELL

```
git status
```

```
On branch main
You have unmerged paths.
  (fix conflicts and run "git commit")
  (use "git merge --abort" to abort the merge)

Unmerged paths:
  (use "git add <file>..." to mark resolution)
        both modified:   shopping.txt
```

`both modified` — файл изменён в обеих ветках.

**Шаг 3. Откройте файл.** Git вставил в него обе версии и маркеры:

### Как выглядит конфликт слияния

Обе ветки изменили одну и ту же строку. Git не знает, какую версию оставить, и просит выбрать вас.

shopping.txt

```
Список покупок
<<<<<<< HEAD
Молоко — 2 литра
=======
Молоко — 1 литр
>>>>>>> feature/milk
Хлеб
```

**Ваша версия**
из текущей ветки (HEAD)

**Разделитель**
между двумя версиями

**Другая версия**
из ветки feature/milk

КАК ИСПРАВИТЬ

1. Оставьте нужный текст
2. Удалите строки-маркеры
3. `git add shopping.txt`
4. `git commit`

| Маркер | Значение |
| --- | --- |
| `<<<<<<< HEAD` | Начало вашей версии (текущая ветка) |
| `=======` | Граница между версиями |
| `>>>>>>> feature/milk` | Конец версии из сливаемой ветки |

**Шаг 4. Решите, что оставить.** Можно оставить одну версию, другую или объединить их. Удалите все три строки-маркера. Результат, например:

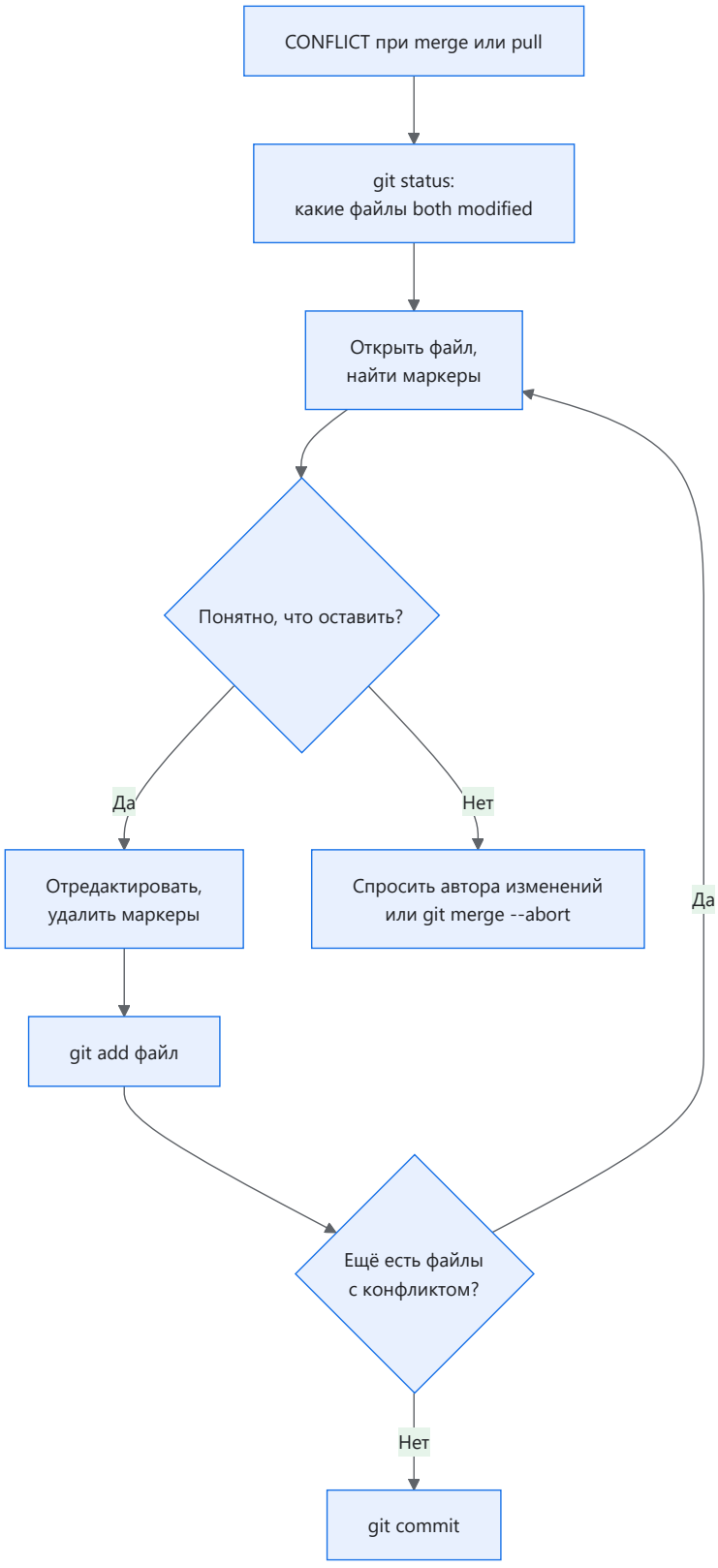
```
Список покупок
Молоко — 2 литра
Хлеб
```

**Шаг 5. Отметьте конфликт решённым и завершите слияние.**

```powershell
git add shopping.txt
git commit --no-edit
```

`--no-edit` — взять стандартное сообщение коммита слияния. Можно написать своё: `git commit -m "Объединить ветку feature/milk"`.

**Шаг 6. Проверьте.** `git status` должен показать `nothing to commit, working tree clean`, а `git log --oneline --graph` — коммит слияния.




## Как передумать

Если запутались, отмените слияние — всё вернётся к состоянию до `git merge`:

```powershell
git merge --abort
```

### Подсказки

- **Ищите маркеры поиском.** В Блокноте: Ctrl + F → `<<<<<<<`. Маркеров не должно остаться ни в одном файле.
- **Редакторы кода помогают.** Visual Studio Code подсвечивает конфликты и показывает кнопки **Accept Current Change**, **Accept Incoming Change**, **Accept Both Changes**.
- **Проверьте результат.** После решения конфликта запустите программу или тесты: Git проверяет только текст, а не работоспособность.

### Конфликт в Pull Request на GitHub

Если в PR написано **This branch has conflicts that must be resolved**:

- **Простой конфликт** (изменены одни и те же строки текста) можно решить на сайте: **Resolve conflicts** → отредактировать файл → **Mark as resolved** → **Commit merge**.
- **Сложный конфликт** решают на компьютере: влейте `main` в свою ветку и отправьте результат.

```powershell
git switch feature/contacts
git pull origin main --no-rebase
# решить конфликты, как описано выше
git add <файлы>
git commit --no-edit
git push
```

PR обновится сам.

## 6.3. Отменить незакоммиченные изменения

Карта всех способов отмены:

## Как отменить изменения

Найдите свою ситуацию. Цвет показывает, насколько безопасна команда.

| СИТУАЦИЯ | КОМАНДА | РИСК |
|---|---|---|
| Изменили файл, git add не делали. Хотите вернуть как было. | `git restore <файл>`<br>правки в файле будут удалены | Удаляет правки |
| Сделали git add, но передумали. | `git restore --staged <файл>`<br>файл останется изменённым | Безопасно |
| Нужна версия файла из прошлого коммита. | `git restore --source=<коммит> <файл>`<br>текущее содержимое файла заменится | Осторожно |
| Ошибочный коммит уже отправлен на GitHub. | `git revert <коммит>`<br>создаёт новый коммит-отмену | Безопасно |
| Последний коммит ещё не отправлен. Хотите его переделать. | `git reset --soft HEAD~1`<br>коммит отменён, правки остаются | Меняет историю |
| Хотите удалить все незакоммиченные правки. | `git reset --hard`<br>вернуть правки будет нельзя | Опасно |
| Коммит «потерялся» после reset. | `git reflog`<br>журнал прошлых положений веток | Безопасно |

● данные не теряются ● меняет файл или историю ● изменения можно потерять навсегда

## Вернуть файл к последней сохранённой версии — git restore

```powershell
git restore hello.txt
```

Файл станет таким, каким был в последнем коммите (точнее — в staging area, если вы делали `git add`).

> **ВНИМАНИЕ**
>
> `git restore <файл>` **безвозвратно удаляет** незакоммиченные правки в файле. Git их нигде не сохранял, поэтому `reflog` не поможет. Если правки могут пригодиться — используйте `git stash` (ниже).

Отменить правки во **всех** файлах:

```powershell
git restore .
```

`git restore .` возвращает файлы к версии из staging area: удаляются правки, которые вы ещё не добавили через `git add`. Чтобы сбросить и подготовленные правки — вернуть всё к последнему коммиту, — используйте `git restore --staged --worktree .`.

> **ОПАСНО**
>
> Обе команды **безвозвратно удаляют** правки во всех отслеживаемых файлах папки. Перед запуском выполните `git status` и `git diff`, чтобы увидеть, что пропадёт.

### Отложить изменения — git stash

> **Stash** («тайник», «заначка») — временное хранилище для незакоммиченных изменений. Изменения убираются из рабочей папки, но не теряются.

```
POWERSHELL
git stash -u
```

```
Saved working directory and index state WIP on main: 7a1b353 Вторая версия
```

`-u` — отложить и новые (неотслеживаемые) файлы тоже. Рабочая папка станет чистой.

Посмотреть отложенное и вернуть его:

```
POWERSHELL
git stash list
git stash pop
```

```
stash@{0}: WIP on main: 7a1b353 Вторая версия
```

`git stash pop` возвращает изменения и удаляет запись из тайника. `git stash drop` удаляет запись без возврата — осторожно.

Когда пригодится stash: нужно срочно переключиться на другую ветку, а текущая работа не готова к коммиту; или AI внёс изменения, которые не хочется применять, но и выбрасывать жалко.

### Удалить новые неотслеживаемые файлы — git clean

`git restore` не трогает новые файлы, которые Git ещё не отслеживает. Для них есть `git clean`.

Сначала **всегда** пробный запуск:

```
POWERSHELL
git clean -n
```

```
Would remove tmp.log
```

`-n` (*dry run*) — только показать, что будет удалено. Если список правильный:

```
POWERSHELL
git clean -f
```

> **ОПАСНО**
>
> `git clean -f` удаляет файлы с диска **мимо Корзины**. Добавление `-d` удаляет и папки, `-x` — даже файлы из `.gitignore` (например, `.env` с вашими настройками). Никогда не запускайте `git clean` без предварительного `git clean -n`.

## 6.4. Убрать файл из staging

Вы сделали `git add`, но передумали включать файл в коммит:

```
POWERSHELL
git restore --staged secret.txt
```

Это **безопасно**: файл и правки в нём остаются, просто не войдут в коммит. После этого добавьте файл в `.gitignore`, если он не должен попадать в Git.

В новом репозитории, где ещё нет ни одного коммита, `git restore --staged` завершится ошибкой `fatal: could not resolve 'HEAD'`. В этом случае используйте `git rm --cached <файл>` — эту команду и подсказывает `git status`.

## 6.5. Исправить последний коммит

Забыли файл или опечатались в сообщении последнего коммита:

```
POWERSHELL
git add forgotten.txt
git commit --amend -m "Исправленное сообщение"
```

`--amend` заменяет последний коммит новым: с добавленными файлами и новым сообщением. Чтобы оставить старое сообщение — `git commit --amend --no-edit`.

> **ВНИМАНИЕ**
>
> `--amend` создаёт новый коммит с новым хешем — то есть **переписывает историю**. Используйте только для коммита, который ещё **не отправлен** на GitHub. Если коммит уже отправлен, сделайте обычный новый коммит с исправлением.

## 6.6. Отменить коммит безопасно — git revert

> **git revert** создаёт новый коммит, который делает обратное указанному: удаляет добавленные строки и возвращает удалённые. Старый коммит остаётся в истории.

Это главный способ отменить коммит, который **уже на GitHub** или в общей ветке: никто не теряет историю.

**Шаг 1. Найдите хеш коммита.**

```
POWERSHELL
git log --oneline
```

```
7a1b353 (HEAD -> main) Вторая версия
f271226 Первая версия
```

**Шаг 2. Отмените коммит.**

```
POWERSHELL
git revert --no-edit 7a1b353
```

```
[main d36110f] Revert "Вторая версия"
 1 file changed, 1 deletion(-)
```

Без `--no-edit` Git откроет редактор с сообщением `Revert "Вторая версия"` — сохраните и закройте его.

**Шаг 3. Проверьте и отправьте.**

```
POWERSHELL
git log --oneline
git push
```

```
d36110f (HEAD -> main) Revert "Вторая версия"
7a1b353 Вторая версия
f271226 Первая версия
```

Отменить последний коммит можно короче: `git revert --no-edit HEAD`.

Если `revert` вызвал конфликт (более поздние коммиты меняли те же строки), решите его как в разделе 6.2 и завершите командой `git revert --continue`. Передумать — `git revert --abort`.

## 6.7. Восстановить предыдущую версию файла

**Шаг 1. Найдите коммит с нужной версией.** История только этого файла:

```
POWERSHELL
git log --oneline -- hello.txt
```

**Шаг 2. Посмотрите, как файл выглядел тогда** (ничего не меняя):

```
POWERSHELL
git show f271226:hello.txt
```

**Шаг 3. Восстановите файл из этого коммита.**

```
POWERSHELL
git restore --source=f271226 hello.txt
```

Файл в рабочей папке станет таким, каким был в коммите `f271226`. Остальные файлы и история не меняются.

> **ВНИМАНИЕ**
> Текущее содержимое файла будет заменено. Если в нём есть незакоммиченные правки — они пропадут. Сначала закоммитьте или отложите их (`git stash`).

**Шаг 4. Сохраните восстановленную версию** как новый коммит:

```
POWERSHELL
git add hello.txt
git commit -m "Вернуть hello.txt к версии f271226"
```

Посмотреть весь проект на момент старого коммита, ничего не меняя, можно так: `git switch --detach f271226`. Вернуться — `git switch main`.

> **Detached HEAD** («отсоединённый HEAD») — состояние, когда вы смотрите на конкретный коммит, а не на ветку. Коммиты в этом состоянии легко потерять. Если хотите что-то поменять, создайте ветку: `git switch -c имя-ветки`.

## 6.8. git reset — перемотать историю назад

> **git reset** передвигает текущую ветку на другой коммит. Коммиты «после» него перестают быть частью ветки.

У команды три режима. Они различаются тем, что происходит с изменениями из «отменённых» коммитов:

| Режим | Ветка | Staging area | Файлы в папке | Риск |
|---|---|---|---|---|
| `--soft` | Переносится | Изменения остаются подготовленными | Не меняются | Меняет историю |
| `--mixed` (по умолчанию) | Переносится | Сбрасывается к выбранному коммиту | Не меняются | Меняет историю |
| `--hard` | Переносится | Сбрасывается к выбранному коммиту | **Перезаписываются** | **Удаляет правки** |

### Безопасный сценарий: переделать последний неотправленный коммит

```
POWERSHELL
git reset --soft HEAD~1
```

`HEAD~1` — «на один коммит назад от текущего». Последний коммит отменён, а все его изменения остались подготовленными. Теперь можно поправить файлы и закоммитить заново.

### Опасный сценарий: выбросить всё до последнего коммита

```
POWERSHELL
git reset --hard
```

> **ОПАСНО**
>
> `git reset --hard` **безвозвратно удаляет** все незакоммиченные правки в отслеживаемых файлах. `git reset --hard HEAD~3` вдобавок убирает из ветки три последних коммита. Закоммиченное можно спасти через `reflog` (раздел 6.10), незакоммиченное — нельзя.

### Когда reset использовать нельзя

- **Коммиты уже на GitHub.** После `reset` ваша история разойдётся с GitHub, и обычный `git push` не пройдёт. Пришлось бы делать `git push --force`, который сотрёт эти коммиты у всех. Используйте `git revert`.
- **Общая ветка** (`main` в команде). Коллеги уже получили эти коммиты.
- **Вы не уверены, что делаете.** Начните с `git stash` или `git revert`.

### Страховка перед reset

Перед любой рискованной операцией создайте ветку-закладку — это бесплатно и мгновенно:

```
POWERSHELL
git branch backup-before-reset
git reset --hard HEAD~2   # ⚠ убирает два коммита из ветки и удаляет незакоммиченные правки
```

Если что-то пошло не так: `git reset --hard backup-before-reset` вернёт всё как было. Когда закладка больше не нужна — `git branch -D backup-before-reset`.

### Про git push --force

> **ОПАСНО**
>
> `git push --force` заменяет историю на GitHub вашей локальной. Всё, чего нет у вас, будет удалено с GitHub, а у коллег возникнут проблемы. Если без принудительной отправки своей ветки не обойтись, используйте более осторожный вариант `git push --force-with-lease`: он откажет, если на GitHub появились коммиты, которых вы не видели. Никогда не применяйте force к общей ветке без договорённости с командой. Очистка секрета из истории — особый случай со своей процедурой (глава 11, раздел 11.8).

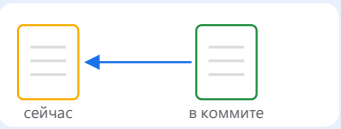
# 6.9. restore, revert и reset: в чём разница

## restore, revert и reset: в чём разница

Три команды отменяют разное. Перед выбором ответьте: изменения уже в коммите? Коммит уже на GitHub?

### git restore

**Отменяет правки в файлах**



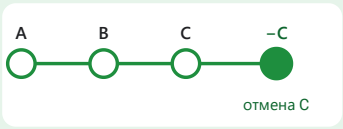

Что меняет
файлы в рабочей папке или staging

История
не меняется

Когда
правки ещё не закоммичены

### git revert

**Отменяет коммит новым коммитом**



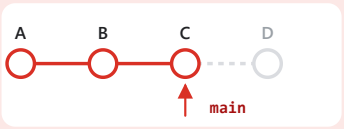

Что меняет
добавляет коммит, обратный старому

История
дополняется, ничего не стирается

Когда
коммит уже на GitHub

### git reset

**Переносит ветку назад**




Что меняет
положение ветки, иногда файлы

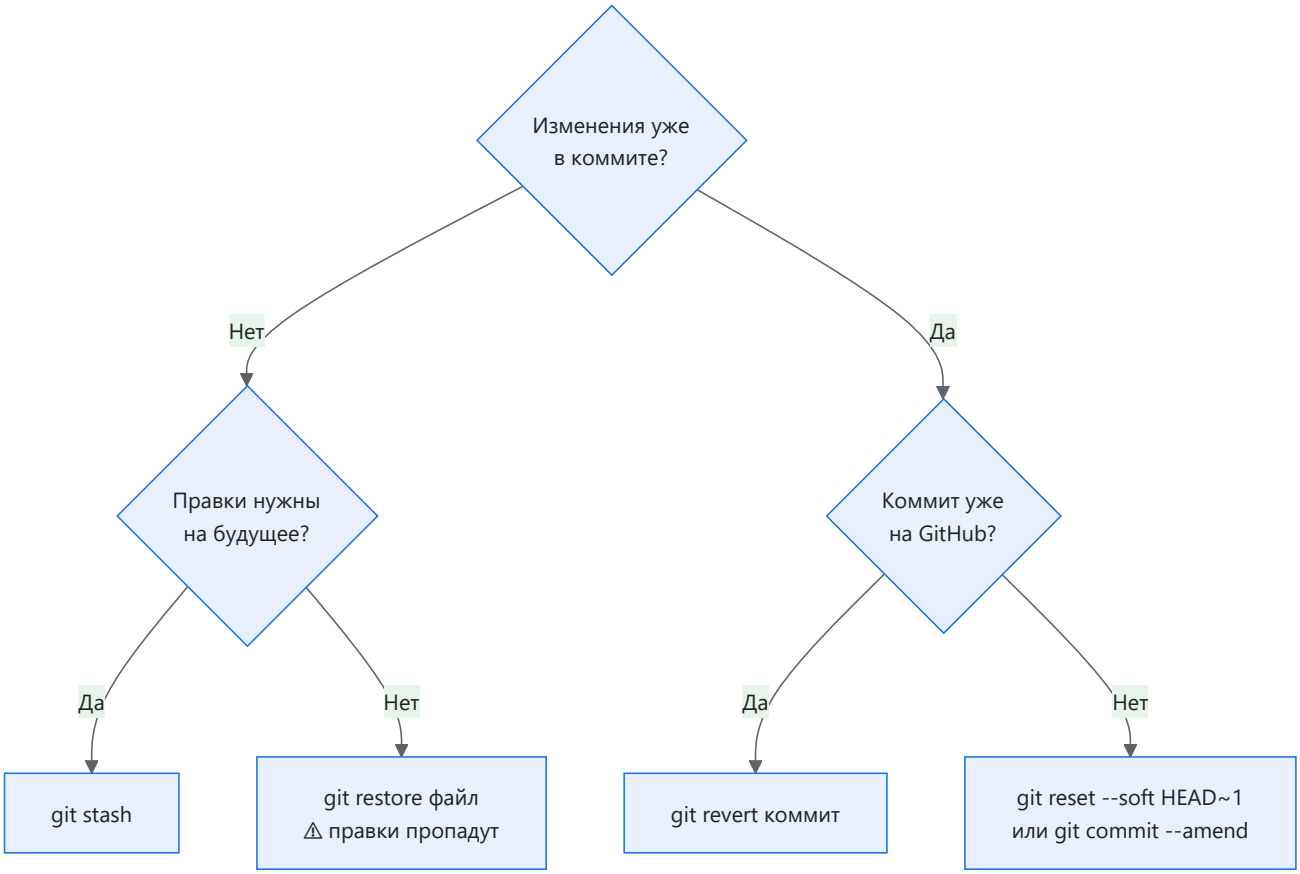
История
переписывается

Когда
только неотправленные коммиты

⚠ `--hard` удаляет незакоммиченные правки

| | `git restore` | `git revert` | `git reset` |
|---|---|---|---|
| Что отменяет | Правки в файлах | Целый коммит | Один или несколько коммитов |
| Как | Перезаписывает файл версией из staging или коммита | Создаёт новый коммит-отмену | Передвигает ветку назад |
| История | Не меняется | Дополняется | Переписывается |
| Безопасно для отправленных коммитов | Не относится к коммитам | **Да** | **Нет** |
| Может удалить правки навсегда | Да (незакоммиченные) | Нет | Да (в режиме `--hard`) |
| Типичный случай | «Испортил файл, верните как было» | «Коммит на GitHub оказался ошибочным» | «Последний коммит ещё у меня, хочу переделать» |

Правило выбора:




## 6.10. Спасательный круг: git reflog

> **Reflog** (*reference log*, журнал ссылок) — локальный журнал, куда Git записывает каждое перемещение `HEAD`: коммиты, переключения веток, `reset`, слияния.

Если после `git reset --hard` или удаления ветки «пропали» коммиты, скорее всего, они ещё существуют — на них просто ничего не указывает.

POWERSHELL
```
git reflog
```

```
f271226 (HEAD -> main) HEAD@{0}: reset: moving to HEAD~2
d36110f HEAD@{1}: revert: Revert "Вторая версия"
7a1b353 HEAD@{2}: commit: Вторая версия
f271226 (HEAD -> main) HEAD@{3}: commit (initial): Первая версия
```

Читать так: последнее действие — `reset` на два коммита назад; перед ним был коммит `d36110f`. Найдите строку с состоянием, которое нужно вернуть, и создайте на нём ветку:

POWERSHELL
```
git branch restored d36110f
```

Или верните текущую ветку туда целиком: `git reset --hard d36110f` (осторожно: незакоммиченные правки пропадут).

Ограничения reflog:

- хранит только то, что **было закоммичено**. Незакоммиченные правки, удалённые `restore` или `reset --hard`, не восстановить;
- он **локальный**: только на вашем компьютере, на GitHub его нет, при новом `git clone` он пустой;
- записи устаревают: по умолчанию через 90 дней, а для недоступных коммитов — через 30 дней.

## 6.11. Почему нельзя хранить секреты в репозитории

> **Секрет** — любые данные, дающие доступ: пароли, API-ключи, токены, ключи SSH, строки подключения к базам данных, файлы `.env` с настройками.

> **API-ключ** — секретная строка, по которой сервис (оплата, почта, AI, карты) узнаёт вашу программу. Кто знает ключ, тот действует от вашего имени и за ваш счёт.

Почему секретам не место в Git:

- **История вечна.** Удалённый файл остаётся в старых коммитах. Его видит каждый, кто может открыть репозиторий.
- **Копии расходятся.** Клоны, форки, резервные копии и кэши сохраняют историю независимо от вас.
- **Публичные репозитории сканируют.** Автоматические программы, в том числе злоумышленников, ищут утёкшие ключи в публичных репозиториях.
- **Приватность временна.** Приватный репозиторий могут сделать публичным, к нему добавят новых людей, его могут скопировать.

Где хранить секреты вместо кода:

- **локально** — в файле `.env`, который внесён в `.gitignore`;
- **в GitHub Actions** — в разделе **Settings → Secrets and variables → Actions**;
- **в программе** — читать из переменных окружения, а не писать в код.

Подробно о работе с `.env` и о том, что делать при утечке, — в главе 11.

## 6.12. .gitignore: что не должно попасть в Git

> `.gitignore` — текстовый файл в корне репозитория со списком файлов и папок, которые Git должен игнорировать: не показывать в `git status` и не добавлять при `git add .`.

### Как создать

Если файла `.gitignore` ещё нет:

POWERSHELL

```
New-Item .gitignore
notepad .gitignore
```

Если он уже есть (например, вы создали его в главе 4), просто откройте его: `notepad .gitignore`.

> **СОВЕТ**
>
> Не создавайте `.gitignore` через «Создать → Текстовый документ» в Проводнике: Windows может назвать файл `.gitignore.txt`, и Git его не увидит. Команда `New-Item` создаёт файл с правильным именем.

## Синтаксис

| Строка | Что игнорируется |
|---|---|
| `.env` | Файл `.env` в любой папке |
| `*.log` | Все файлы с расширением `.log` |
| `build/` | Папка `build` целиком |
| `/config.local.json` | Только файл в корне репозитория |
| `!keep.log` | Исключение: этот файл **не** игнорировать |
| `# текст` | Комментарий |

## Пример для учебного проекта

.GITIGNORE

```
# Секреты и локальные настройки
.env
.env.*
!.env.example
*.pem
*.key

# Служебные файлы Windows и macOS
Thumbs.db
desktop.ini
.DS_Store

# Редакторы
.vscode/
.idea/

# Логи и временные файлы
*.log
*.tmp

# Python
__pycache__/
.venv/
```

```
# Node.js
node_modules/
```

Готовые шаблоны для разных языков: github.com/github/gitignore (например, `Python.gitignore`, `Node.gitignore`).

### Проверка

Почему файл игнорируется (или нет):

```
POWERSHELL
git check-ignore -v .env
```

```
.gitignore:2:.env       .env
```

Файл `.env` игнорируется из-за строки 2 файла `.gitignore`.

Показать игнорируемые файлы: `git status --ignored`.

### Если файл уже попал в Git

`.gitignore` действует только на файлы, которые Git **ещё не отслеживает**. Если файл уже закоммичен, добавление его в `.gitignore` ничего не изменит. Уберите файл из отслеживания, оставив его на диске:

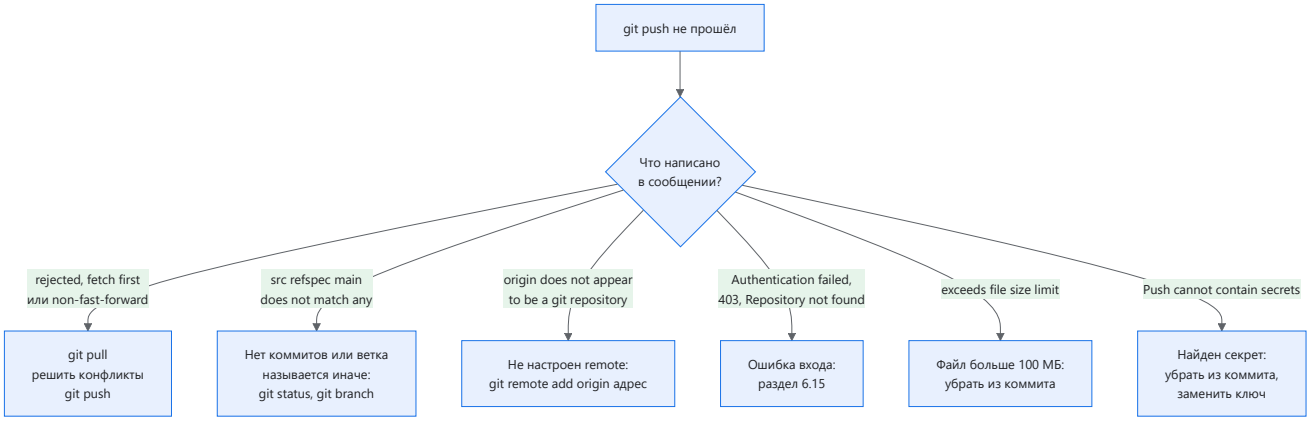
```
POWERSHELL
git rm --cached .env
git commit -m "Перестать отслеживать .env"
```

> **ВНИМАНИЕ**
>
> `git rm --cached` убирает файл из будущих коммитов, но **не из истории**. Если в файле был секрет, считайте его утёкшим: замените пароль или ключ (глава 11).

## 6.13. Публичные и приватные репозитории: правила безопасности

- Перед первым коммитом проверяйте `git status` и `git diff --staged`, а перед первым `git push` в публичный репозиторий — `git log --stat`: какие файлы попадут на GitHub.
- Перед тем как сделать приватный репозиторий публичным, проверьте **всю историю**, а не только текущие файлы.
- Не публикуйте личные данные: паспорта, адреса, телефоны, переписку, базы клиентов.
- Не добавляйте в участники приватного репозитория (*collaborators*) тех, кому не нужен доступ. Убирайте доступ, когда человек больше не работает над проектом.
- Включите двухфакторную аутентификацию в аккаунте GitHub.
- Помните: приватный репозиторий защищает от посторонних глаз, но не от утечки через ваш компьютер, токены и людей с доступом.

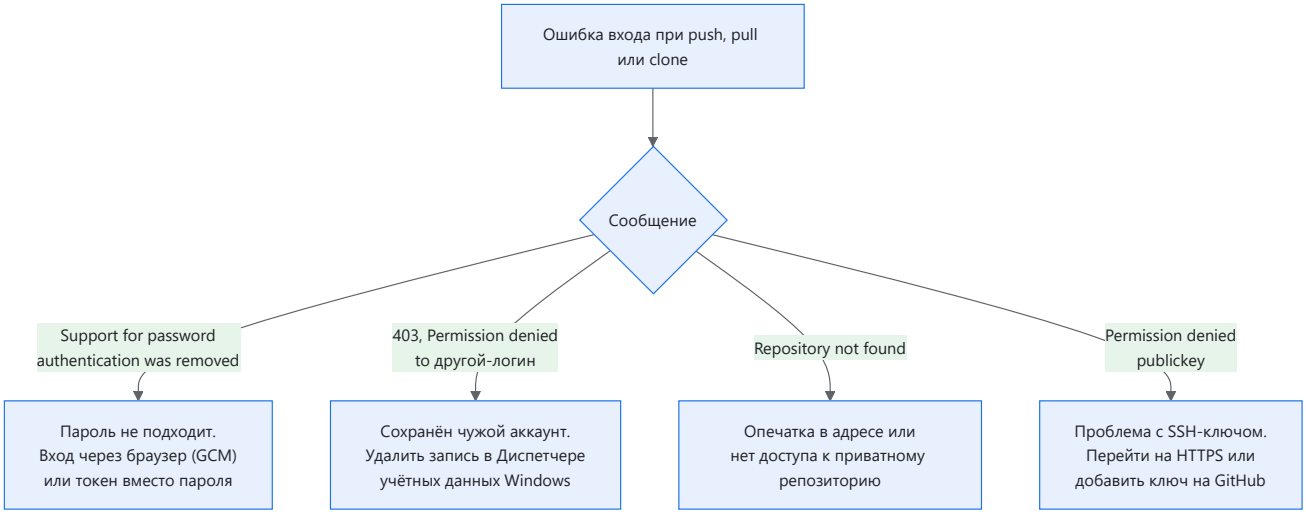
## 6.14. Если push не проходит




| Сообщение | Причина | Решение |
|---|---|---|
| `! [rejected] main -> main (fetch first)` | На GitHub есть коммиты, которых нет у вас | `git pull`, при конфликте — раздел 6.2, затем `git push` |
| `! [rejected] main -> main (non-fast-forward)` | Ваша ветка и GitHub разошлись | То же: `git pull`, затем `git push` |
| `error: src refspec main does not match any` | Нет коммитов или ветка не `main` | `git status`; сделайте коммит; `git branch -M main` |
| `fatal: No configured push destination` | Нет удалённого репозитория | `git remote add origin <адрес>` |
| `fatal: 'origin' does not appear to be a git repository` | Нет remote с именем `origin` | `git remote -v`; `git remote add origin <адрес>` |
| `fatal: The current branch feature has no upstream branch` | Новая ветка ещё не связана с GitHub | `git push -u origin <ветка>` |
| `remote: error: File big.zip is 120.00 MB; this exceeds GitHub's file size limit of 100.00 MB` | Файл больше 100 МБ | Уберите файл из коммита и добавьте в `.gitignore`; для больших файлов существует Git LFS |
| `remote: error: GH013: Repository rule violations found` и `Push cannot contain secrets` | Сработала защита от секретов (*push protection*) | Уберите секрет из коммитов, замените ключ (глава 11) |

> **СОВЕТ**
>
> Большой файл, попавший в последний **неотправленный** коммит, убирают так: `git rm --cached big.zip`, добавить `big.zip` в `.gitignore`, затем `git commit --amend --no-edit`. Если файл в более ранних коммитах — понадобится переписать историю (глава 11).

## 6.15. Ошибки авторизации




| Сообщение | Причина | Решение |
|---|---|---|
| `remote: Support for password authentication was removed on August 13, 2021.` | Ввели пароль от аккаунта | Войдите через браузер (Git Credential Manager) или используйте токен вместо пароля (глава 4) |
| `fatal: Authentication failed for 'https://github.com/...'` | Неверный или просроченный токен | Создайте новый токен; удалите старую запись входа (см. ниже) |
| `remote: Permission to ivan-petrov/my-site.git denied to other-user.` и `The requested URL returned error: 403` | Git вошёл под другим аккаунтом | Удалите сохранённую запись и войдите заново (см. ниже) |
| `remote: Repository not found.` | Опечатка в адресе, или репозиторий приватный, а у аккаунта нет доступа | Проверьте адрес (`git remote -v`); попросите владельца добавить вас в участники репозитория (*collaborators*) |
| `git@github.com: Permission denied (publickey).` | SSH-ключ не добавлен на GitHub или не загружен | Используйте HTTPS-адрес (`git remote set-url origin https://...`) или настройте SSH по документации GitHub |

### Как удалить сохранённый вход в Windows

1. Win → наберите **Диспетчер учётных данных** → Enter.
2. Выберите **Учётные данные Windows**.
3. Найдите запись `git:https://github.com` → **Удалить**.
4. Повторите `git push` — Git Credential Manager снова предложит войти.

## 6.16. Как читать сообщения терминала

Сообщения Git на английском, но устроены по одному образцу. Первое слово строки подсказывает, насколько всё серьёзно:

| Начало строки | Значение | Что делать |
|---|---|---|
| `hint:` | Подсказка | Прочитать: часто там готовое решение |
| `warning:` | Предупреждение, команда выполнена | Обычно можно продолжать, но стоит понять причину |
| `error:` | Ошибка, часть действия не выполнена | Прочитать и исправить |
| `fatal:` | Критическая ошибка, команда не выполнена совсем | Исправить причину и повторить команду |
| `remote:` | Сообщение от сервера (GitHub) | Прочитать: там бывают ссылки и причина отказа |
| `CONFLICT` | Конфликт слияния | Раздел 6.2 |

Как разбираться:

1. **Читайте снизу вверх.** Последние строки обычно самые важные.
2. **Ищите `fatal:` и `error:`** — это причина. Строки `hint:` — подсказка, как исправить.
3. **Переводите ключевые слова:** `rejected` — отклонено, `not found` — не найдено, `denied` — запрещено, `already exists` — уже существует, `conflict` — конфликт, `untracked` — не отслеживается, `unmerged` — не слито, `upstream` — связанная ветка на сервере.
4. **Выполните `git status`.** Он почти всегда подсказывает следующий шаг.
5. **Ищите точный текст ошибки** в документации или поисковике. Можно спросить AI-ассистента, но не вставляйте в запрос пароли и токены.

## 6.17. Ошибка → причина → решение

| Ошибка или ситуация | Причина | Решение |
|---|---|---|
| `fatal: not a git repository` | Вы не в папке репозитория | `cd` в папку проекта или `git init` |
| `Author identity unknown` | Не указаны имя и почта | `git config --global user.name ...`, `user.email ...` |
| `nothing added to commit but untracked files present` | Файлы не добавлены в staging | `git add <файл>` |
| `nothing to commit, working tree clean` | Нечего коммитить — всё сохранено | Ничего делать не нужно |
| `fatal: pathspec 'x' did not match any files` | Опечатка в имени файла | Проверьте имя, используйте Tab |
| `error: Your local changes ... would be overwritten` | Незакоммиченные правки мешают переключению или pull | Закоммитьте или `git stash` |
| `CONFLICT (content): Merge conflict in ...` | Обе ветки изменили одни строки | Раздел 6.2 |
| `fatal: refusing to merge unrelated histories` | Две независимые истории (обычно README на GitHub) | Глава 4, раздел 4.12 |

| Ошибка или ситуация | Причина | Решение |
|---|---|---|
| `fatal: Need to specify how to reconcile divergent branches` | Не настроен способ `git pull` | `git config --global pull.rebase false`, повторить `git pull` |
| `error: remote origin already exists` | `origin` уже настроен | `git remote -v`; при ошибке в адресе — `git remote set-url origin <адрес>` |
| `error: the branch 'x' is not fully merged` | Удаляете неслитую ветку | Сначала `git merge`, или `git branch -D` — если работа не нужна |
| `fatal: destination path 'x' already exists` | Папка для `git clone` уже есть | Клонируйте в другую папку |
| `warning: ... LF will be replaced by CRLF` | Git приводит концы строк к формату Windows | Ничего делать не нужно |
| Терминал «завис», внизу `:` или `(END)` | Открыт постраничный просмотр | Нажмите `Q` |
| Открылся Vim | Не настроен редактор | `Esc`, `:wq`, `Enter`; затем `git config --global core.editor notepad` |
| `Aborting commit due to empty commit message` | Пустое сообщение коммита | `git commit -m "сообщение"` |
| `You are in 'detached HEAD' state` | Вы переключились на коммит, а не на ветку | `git switch main` или `git switch -c новая-ветка` |
| Русские имена файлов как `\320\277...` | Не настроен вывод путей | `git config --global core.quotepath false` |
| `fatal: unable to access ...: Could not resolve host: github.com` | Нет интернета или проблема с DNS/прокси | Проверьте подключение к интернету |
| Ошибки push и авторизации | — | Разделы 6.14 и 6.15 |

## Проверьте себя

Выполните в учебном репозитории `git-practice`:

1. Испортите `hello.txt` (удалите все строки) и верните его командой `git restore`.
2. Сделайте коммит «Тестовая правка», затем отмените его через `git revert`. Сколько коммитов добавилось?
3. Создайте ветку `conflict-test`, измените в ней первую строку `hello.txt` и закоммитьте. В `main` измените ту же строку иначе и закоммитьте. Влейте ветку и решите конфликт.
4. Создайте файл `.env` со строкой `API_KEY=test`. Добавьте его в `.gitignore`. Что покажет `git status`?

**Ожидаемый результат**

1. После `git restore hello.txt` файл снова содержит все строки, `git status` — `working tree clean`.

2. Два: «Тестовая правка» и `Revert "Тестовая правка"`. Исходный коммит остался в истории.
3. `git merge conflict-test` выводит `CONFLICT (content): Merge conflict in hello.txt`. После правки файла: `git add hello.txt`, `git commit --no-edit`. `git log --oneline --graph` показывает коммит слияния.
4. `.env` не появляется в списке, виден только изменённый `.gitignore`. `git check-ignore -v .env` подтверждает, что файл игнорируется.

## Что дальше

В главе 7 вы примените всё изученное к работе с AI-ассистентом Claude Code.

# Глава 7. Git и Claude Code

AI-ассистенты пишут код быстро — и так же быстро могут его сломать. Git превращает работу с AI в безопасный процесс: каждое изменение можно просмотреть, проверить, сохранить или отменить. В этой главе — рабочий цикл с Claude Code и правила оформления проектов, которые создаются с помощью AI.

**Вы научитесь:**

- готовить репозиторий перед работой с AI;
- создавать контрольные точки и ветки для экспериментов;
- просматривать и проверять изменения, сделанные AI;
- возвращаться к рабочей версии;
- оформлять AI-проекты: `CLAUDE.md`, документация, тесты, проверка зависимостей и лицензий.

**Время:** 1,5 часа. **Понадобится:** главы 3–6, подписка Claude с доступом к Claude Code.

## 7.1. Что такое Claude Code

> **Claude Code** — AI-ассистент для программирования от Anthropic. Он читает файлы проекта, предлагает и вносит изменения, запускает команды и тесты. Работает в терминале, в приложении Claude для компьютера (вкладка **Code**) и в расширениях для редакторов VS Code и JetBrains.

Что важно знать до начала:

- **Нужна учётная запись.** Claude Code доступен в платных планах Claude (Pro, Max, Team, Enterprise) или через аккаунт Claude Console. В бесплатном плане его нет.
- **Claude Code меняет настоящие файлы** на вашем компьютере. Поэтому Git здесь не дополнение, а основа безопасности.
- **Действия требуют разрешения или проверки.** В Claude Code есть режимы разрешений: в ручном режиме (*default*) он спрашивает перед изменением файлов и запуском команд; в режиме *acceptEdits* правит файлы сам; в режиме планирования (*plan*) только исследует и предлагает план; в режиме *auto* действия проверяет встроенный классификатор. Текущий режим виден в интерфейсе, переключение в терминале — Shift + Tab.

### Установка на Windows

Рекомендуемый способ — официальный установщик. В PowerShell:

```powershell
irm https://claude.ai/install.ps1 | iex
```

- `irm` (*Invoke-RestMethod*) скачивает скрипт установки с сайта Anthropic;
- `| iex` (*Invoke-Expression*) выполняет его.

> **ВАЖНО**
> Команда вида «скачать и выполнить» запускает код из интернета. Выполняйте её, только если адрес точно официальный и совпадает с документацией: Claude Code — Set up.

Другие способы: `winget install --id Anthropic.ClaudeCode -e --source winget` или `npm install -g @anthropic-ai/claude-code` (нужен Node.js 22+). Проверка — в новом окне терминала:

```powershell
claude --version
```

Git для Windows рекомендуется: если он установлен, Claude Code использует Git Bash для выполнения команд.

## 7.2. Открыть существующий проект

```powershell
cd ~\projects\my-site
claude
```

Claude Code начнёт сессию в текущей папке и будет работать с её файлами. В приложении Claude: вкладка **Code** → выберите папку проекта.

> **СОВЕТ**
> Запускайте Claude Code в **корневой папке репозитория** — там, где лежит папка `.git`. Не запускайте его в домашней папке или на диске `C:\` целиком.

## 7.3. Проверить состояние Git перед работой с AI

Перед каждой задачей убедитесь, что точка старта сохранена.

```powershell
git status
git log --oneline -3
git branch
```

Что должно быть:

| Проверка | Ожидание | Если не так |
|---|---|---|
| `git status` | `nothing to commit, working tree clean` | Закоммитьте текущую работу или отложите её (`git stash -u`) |
| `git log` | Последний коммит — рабочая версия | Сначала доведите проект до рабочего состояния |
| `git branch` | Вы в нужной ветке | `git switch <ветка>` |

Почему чистая рабочая папка так важна: после задачи `git diff` покажет **только** изменения AI. Если перед этим были ваши незакоммиченные правки, они смешаются с правками AI, и откатить одно без другого будет трудно.

## 7.4. Создать контрольную точку

> **Контрольная точка** — сохранённое состояние, к которому можно вернуться. В Git это коммит, ветка или тег (постоянная метка на коммите, подробнее — в главе 9).

Перед крупной задачей (новая функция, рефакторинг, обновление библиотек):

**1. Закоммитьте рабочую версию.**

```powershell
git add .
git commit -m "Рабочая версия перед добавлением формы обратной связи"
```

**2. Создайте ветку для задачи.**

```powershell
git switch -c ai/feedback-form
```

Префикс `ai/` — соглашение, а не правило: так видно, какие ветки — эксперименты с AI.

**3. (Необязательно) Отправьте ветку на GitHub** — так контрольная точка сохранится и вне компьютера:

```powershell
git push -u origin ai/feedback-form
```

> **ПРИМЕЧАНИЕ**
>
> У Claude Code есть собственные контрольные точки (*checkpoints*): двойное нажатие `Esc` или команда `/rewind` возвращают файлы к состоянию до определённого запроса. Но они отслеживают только правки, сделанные инструментами редактирования Claude. Изменения, сделанные командами в терминале (удаление, перемещение файлов) или вами вручную, туда не попадают, а сами точки хранятся ограниченное время. В документации прямо сказано, что это не замена системе контроля версий. Источник: Checkpointing.

## 7.5. Поставить задачу AI

Хорошая задача снижает число ошибок и лишних изменений.

| Плохо | Хорошо |
|---|---|
| «Сделай форму» | «Добавь на страницу `contacts.html` форму обратной связи: имя, почта, сообщение. Не меняй другие страницы.» |
| «Исправь баг» | «При пустом поле „почта“ форма отправляется. Нужна проверка на клиенте. Добавь тест.» |

| Плохо | Хорошо |
| --- | --- |
| «Улучши код» | «Вынеси повторяющийся код из `app.py` в функцию. Поведение не меняй. Тесты должны проходить.» |

Что включать в задачу:

- **цель** — что должно получиться;
- **границы** — какие файлы можно и нельзя трогать;
- **критерий готовности** — как проверить результат (тесты, ручная проверка);
- **ограничения** — без новых библиотек, без изменения структуры данных и т. п.

Для больших задач попросите сначала план: переключитесь в режим планирования (Shift + Tab до режима plan) или напишите «сначала покажи план, ничего не меняй».

## 7.6. Просмотреть изменения, сделанные AI

После того как Claude Code закончил:

```
POWERSHELL
git status
```

Какие файлы изменены, добавлены, удалены. Неожиданные файлы — повод насторожиться.

```
POWERSHELL
git diff --stat
```

Сколько строк изменено в каждом файле. Если в маленькой задаче изменены десятки файлов — что-то пошло не так.

```
POWERSHELL
git diff
```

Построчные изменения. Читайте их целиком.

Где ещё смотреть изменения:

- команда `/diff` внутри Claude Code;
- расширение VS Code показывает изменения рядом с исходным текстом;
- в приложении Claude индикатор вида `+12 -1` открывает панель изменений.

Новые файлы `git diff` не показывает, пока они не добавлены в staging. Просмотрите их отдельно или выполните `git add -N <файл>` (*intent to add*), чтобы они появились в `git diff`.

**Чек-лист просмотра:**

☐ изменены только те файлы, о которых шла речь;
☐ нет удалённого кода, который был нужен;
☐ нет паролей, ключей и личных данных;

☐ нет новых библиотек, о которых вы не просили;

☐ понятна каждая изменённая строка — если нет, попросите Claude объяснить.

## 7.7. Запустить тесты

> **Тест** — небольшая программа, которая проверяет, что код работает как задумано. Например: «функция перевода 0 °C в Фаренгейты возвращает 32».

Запустите тесты проекта. Команда зависит от языка, например:

```
POWERSHELL
python -m unittest discover -s tests   # Python
npm test                               # JavaScript / Node.js
```

Если тестов нет — проверьте работу вручную (откройте страницу, запустите программу) и попросите Claude добавить тесты.

Документация Claude Code советует давать ассистенту способ проверить свою работу: тесты, сборку, скриншот. С тестами AI сам находит и исправляет часть ошибок. Источник: Best practices.

## 7.8. Проверить diff перед коммитом

Подготовьте изменения и посмотрите финальную версию того, что войдёт в коммит:

```
POWERSHELL
git add contacts.html tests/test_form.py
git diff --staged
```

Добавляйте файлы **по именам**, а не `git add .`, если в папке есть что-то, что вы не проверили.

## 7.9. Сохранить работающую версию

```
POWERSHELL
git commit -m "Добавить форму обратной связи на страницу контактов"
git push
```

Claude Code умеет сам делать коммиты и открывать Pull Request (с установленным GitHub CLI `gh`), если попросить. По умолчанию он добавляет в коммит строку `Co-Authored-By` с указанием модели. Даже в этом случае **решение о коммите принимаете вы** — после просмотра diff и тестов.

Для командной работы отправьте ветку и откройте Pull Request (глава 5): второй человек и автоматические проверки посмотрят изменения до слияния в `main`.

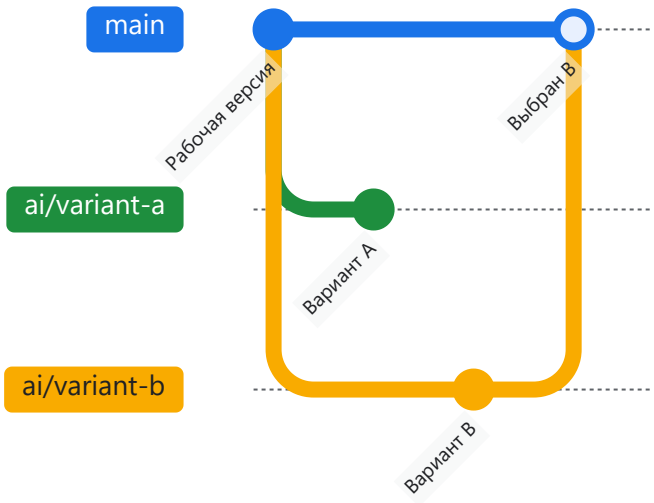
## 7.10. Вернуться к предыдущему состоянию

| Ситуация | Что сделать |
|---|---|
| AI ещё работает и делает не то | Остановите его клавишей `Esc` |
| Нужно откатить последний шаг Claude | `Esc` `Esc` или `/rewind` (только правки инструментов Claude) |
| Изменения не закоммичены, хочу всё вернуть, но сохранить на всякий случай | `git stash -u` |
| Изменения не закоммичены, выбросить их совсем | `git restore --staged --worktree .`, затем `git clean -n` и `git clean -f` для новых файлов ⚠ |
| Неудачные изменения уже в коммите, коммит не отправлен | `git reset --soft HEAD~1` или `git revert` |
| Неудачный коммит уже на GitHub | `git revert <коммит>` |
| Весь эксперимент в ветке не удался | `git switch main`, затем `git branch -D ai/feedback-form` ⚠ |

Отметка ⚠ — команды, которые удаляют изменения безвозвратно (глава 6).

## 7.11. Ветки для экспериментов с AI

Ветка — самый простой способ дать AI свободу без риска:




- Попробуйте два подхода в двух ветках и оставьте лучший.
- Неудачную ветку просто удалите — `main` не пострадает.

Для параллельной работы нескольких сессий Claude Code есть **рабочие деревья** (*git worktrees*): отдельные папки с разными ветками одного репозитория. Claude Code создаёт их командой:

```powershell
claude --worktree feedback-form
```

Папки создаются внутри `.claude/worktrees/`. Добавьте эту папку в `.gitignore`. Источник: Worktrees.

## 7.12. Как не потерять код

**Правила:**

1. Начинайте каждую задачу с чистого `git status`.
2. Коммитьте каждую проверенную рабочую версию.
3. Экспериментируйте в отдельных ветках.
4. Регулярно отправляйте коммиты на GitHub.
5. Читайте diff перед каждым коммитом.
6. Не разрешайте AI выполнять команды, смысл которых вам непонятен: `git reset --hard`, `git push --force`, `git clean -fdx`, удаление папок.
7. Храните секреты вне репозитория и закройте их от чтения AI (раздел 7.14).

**Ограничения — о них важно помнить:**

- **Git защищает только закоммиченное.** Незакоммиченные правки, удалённые `git restore` или `reset --hard`, не восстановить.
- **Локальный репозиторий — не резервная копия.** Без `git push` история пропадёт вместе с диском.
- **Файлы из `.gitignore` не сохраняются в Git.** Тот же `.env` нужно хранить отдельно (например, в менеджере паролей).
- **GitHub — не единственная резервная копия.** Аккаунт можно потерять, репозиторий — удалить. Для важных проектов держите независимую копию.
- **Контрольные точки Claude Code временные** и не охватывают изменения, сделанные командами терминала.
- **Git не проверяет правильность кода.** Он сохранит и сломанную версию. Проверка — тесты, ревью и ваше внимание.

# 7.13. Рекомендуемый рабочий цикл

## Рабочий цикл с Claude Code и Git

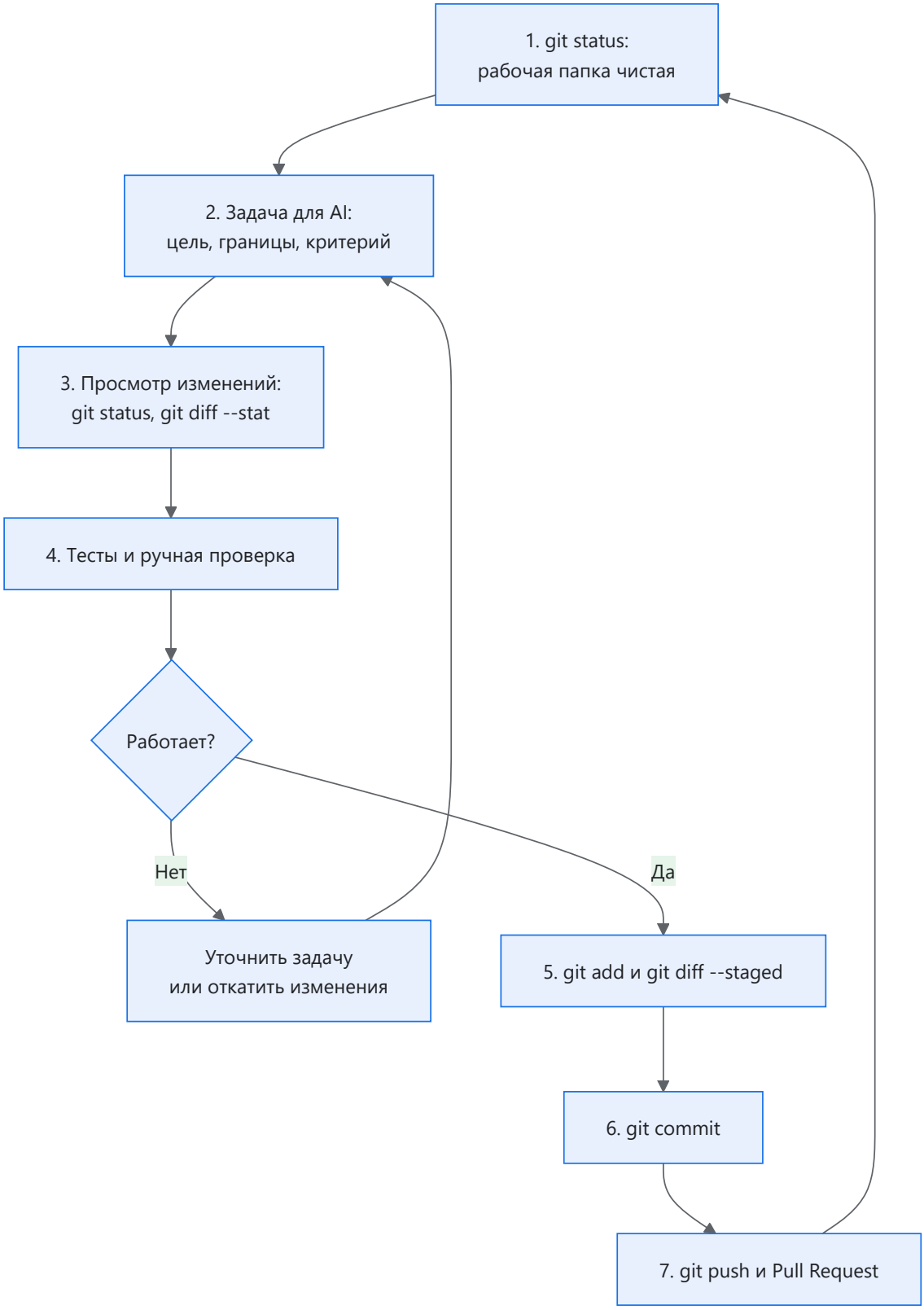
Каждая задача для AI начинается и заканчивается проверенной версией, сохранённой в Git.




Главное правило: коммит делаете вы, после проверки. Git хранит только то, что вы закоммитили.

**Результат не устроил?**

Не коммитьте. Верните файлы к последнему коммиту (глава 6) и уточните задачу.




## 7.14. Профессиональное оформление проектов с AI

Проект, написанный с помощью AI, должен соответствовать тем же требованиям, что и любой другой: понятная структура, документация, тесты, лицензия, проверка зависимостей. AI ускоряет работу, но ответственность за результат остаётся на человеке.

### Начинайте с оформленного репозитория

До первой задачи для AI создайте:

1. `README.md` — что за проект и как его запустить (глава 9).
2. `.gitignore` — чтобы AI не закоммитил лишнего.
3. `LICENSE` — если проект будет публичным (глава 10).
4. `CLAUDE.md` — инструкции для AI (ниже).
5. Папки `src/` и `tests/` и команду запуска тестов.

Так AI с первого запроса видит правила проекта и следует им.

## CLAUDE.md — инструкции для AI

> **CLAUDE.md** — файл с инструкциями, который Claude Code читает в начале каждой сессии. Туда пишут то, что AI не может понять из кода сам: команды сборки и тестов, правила оформления, договорённости команды.

Где хранить:

| Файл | Для чего | В Git |
|---|---|---|
| `./CLAUDE.md` или `./.claude/CLAUDE.md` | Правила проекта для всей команды | **Да**, закоммитьте |
| `~/.claude/CLAUDE.md` | Ваши личные правила для всех проектов | Нет, это вне проекта |
| `./CLAUDE.local.md` | Ваши личные правила для этого проекта | **Нет** — добавьте в `.gitignore` сами |

Создать заготовку: команда `/init` в Claude Code проанализирует проект и предложит `CLAUDE.md`. Отредактируйте её вручную. Открыть файлы памяти — `/memory`.

Пример `CLAUDE.md` для учебного проекта на Python:

MARKDOWN

```
# Инструкции для Claude Code

## Проект
Консольный конвертер температур. Код — в src/tempconv/, тесты — в tests/.

## Команды
- Тесты: python -m unittest discover -s tests -v
- Проверка стиля: ruff check .

## Правила
- Только стандартная библиотека Python. Новые зависимости — только после согласования.
- Каждое изменение поведения сопровождается тестом.
- После изменения поведения обнови README.md и раздел [Unreleased] в CHANGELOG.md.
- Не читай и не изменяй файлы .env и папку secrets/.
- Не делай коммиты и push без явной просьбы.
```

Советы из документации: держите файл коротким (до ~200 строк), проверяйте каждую строку вопросом «если её убрать, AI начнёт ошибаться?», не пишите то, что видно из кода. `CLAUDE.md` — это контекст, а не принуждение: для правил, которые должны выполняться всегда, используют настройки и хуки (*hooks* — команды, которые Claude Code запускает автоматически при определённых событиях, например форматирование после каждой правки файла). Источник: Memory.

Для других AI-инструментов есть похожие файлы (например, `AGENTS.md` или файлы правил конкретного редактора). Проверьте документацию своего инструмента.

### Закройте секреты от чтения

В файле `.claude/settings.json` (общий для проекта, его коммитят) можно запретить Claude Code читать секретные файлы:

```JSON
{
  "permissions": {
    "deny": [
      "Read(./.env)",
      "Read(./.env.*)",
      "Read(./secrets/**)"
    ]
  }
}
```

Личные настройки хранят в `.claude/settings.local.json`; Claude Code сам исключает его из Git, если создаёт этот файл. Такие запреты действуют на инструменты чтения файлов Claude Code; это дополнительная мера, а не замена `.gitignore` и аккуратности. Источник: Settings.

### Документируйте архитектуру

> **Архитектура** — как устроена программа: из каких частей она состоит и как они связаны.

- `docs/architecture.md` — короткое описание частей программы и схема (можно в Mermaid).
- `docs/adr/` — **записи архитектурных решений** (*Architecture Decision Records*, ADR): по одному файлу на важное решение. Формат: контекст → решение → последствия.

Пример ADR:

```markdown
# 0001. Хранить настройки в переменных окружения

Статус: принято
Дата: 2026-10-11

## Контекст
Программе нужен ключ API погодного сервиса. Ключ нельзя хранить в коде.

## Решение
Читать ключ из переменной окружения WEATHER_API_KEY. В репозитории — только .env.example.

## Последствия
Перед запуском нужно создать .env. Это описано в README.
```

Документация нужна и людям, и AI: в следующих сессиях Claude прочитает её и не будет «изобретать» архитектуру заново. Попросите: «Прочитай docs/architecture.md и следуй ему».

### Просите тесты и обновление документации

Добавляйте к задачам:

- «Добавь тесты на новое поведение и на граничные случаи (пустой ввод, отрицательные числа)».
- «Запусти тесты и покажи результат».
- «Обнови README.md, если изменились установка или использование».
- «Добавь запись в раздел [Unreleased] файла CHANGELOG.md».

Проверяйте сами тесты: AI может написать тест, который проверяет не то или проходит всегда.

### Проверяйте зависимости и лицензии

> **Зависимость** — чужая библиотека, которую использует ваш проект.

AI охотно добавляет библиотеки. Перед тем как принять новую зависимость:

1. **Нужна ли она?** Может, задачу решает стандартная библиотека.
2. **Существует ли она?** AI иногда придумывает несуществующие пакеты. Злоумышленники регистрируют пакеты с такими «придуманными» именами. Найдите пакет на официальном сайте реестра (pypi.org, npmjs.com) и в его репозитории.
3. **Поддерживается ли?** Дата последнего релиза, открытые проблемы, число пользователей.
4. **Какая лицензия?** Совместима ли она с вашей (глава 10).
5. **Есть ли известные уязвимости?** `pip-audit` для Python, `npm audit` для Node.js, Dependabot на GitHub (глава 11).

### Проверяйте происхождение кода

AI обучен на большом объёме кода и теоретически может воспроизвести фрагмент существующей программы. Правовой статус результатов работы AI зависит от страны и продолжает меняться (глава 10).

Практические правила:

- насторожитесь, если AI выдал большой готовый фрагмент с чужими комментариями, именами авторов или уведомлением о лицензии;
- если код явно взят из конкретного проекта — соблюдайте его лицензию или замените код;
- фиксируйте происхождение заимствованного кода в `THIRD_PARTY_NOTICES.md` (глава 10);
- соблюдайте правила работодателя и заказчика об использовании AI.

### Делайте осмысленные коммиты

- **Одна задача — один коммит** (или несколько логичных). Не сохраняйте результат двухчасовой сессии одним коммитом «изменения от AI».
- **Сообщение описывает результат,** а не процесс: «Добавить проверку почты в форме», а не «попросил Claude».
- По желанию используйте соглашение **Conventional Commits**: `feat: добавить форму обратной связи` (глава 9).

### Проверяйте изменения перед слиянием

- Откройте Pull Request и прочитайте вкладку **Files changed** так, будто код написал незнакомый человек.
- Дождитесь автоматических проверок CI (глава 11).
- Можно попросить Claude Code сделать ревью: команда `/code-review` ищет ошибки в изменениях ветки, `/security-review` — проблемы безопасности. Это помощь, а не замена вашей проверке.

### Почему AI-код нельзя считать автоматически правильным

- **Ошибки в логике.** Код выглядит убедительно, но считает неправильно в редких случаях.
- **Выдуманные функции и библиотеки.** Вызовы того, чего не существует, или устаревших версий.
- **Уязвимости.** Например, отсутствие проверки ввода или секреты в коде.
- **Лицензионные риски.** Фрагменты чужого кода без соблюдения лицензии.
- **Лишние изменения.** «Заодно улучшенные» файлы, о которых не просили.

Относитесь к коду от AI как к коду от нового коллеги: полезно, но обязательно проверять.

## 7.15. Пример структуры проекта для разработки с AI

```
my-project/
├── README.md              # что это, как установить и запустить
├── LICENSE                # лицензия
├── CHANGELOG.md           # журнал изменений
├── CLAUDE.md              # инструкции для AI: команды, правила
├── .gitignore             # .env, CLAUDE.local.md, .claude/worktrees/, кэши
├── .env.example           # образец настроек без секретов
├── .claude/
│   └── settings.json      # общие настройки Claude Code: запрет чтения .env
├── docs/
│   ├── architecture.md    # как устроена программа
│   └── adr/
```

```
│       └── 0001-config-from-env.md
├── src/                    # исходный код
├── tests/                  # тесты — главный способ проверить работу AI
└── .github/
    ├── workflows/ci.yml    # автоматические тесты на каждый PR
    └── pull_request_template.md
```

Работающий пример такого репозитория — в папке examples/tempconv.

## Проверьте себя

1. Почему перед задачей для AI рабочая папка должна быть чистой?
2. AI изменил 15 файлов, хотя вы просили поправить один. Ваши действия?
3. Чем контрольные точки Claude Code отличаются от коммитов Git?
4. Что написать в `CLAUDE.md`, а чего писать не нужно?
5. AI предложил установить пакет, о котором вы не слышали. Что проверить?

**Ответы**

1. Тогда `git diff` покажет только изменения AI, и их можно откатить, не задев вашу работу.
2. Не коммитить. Посмотреть `git diff --stat` и `git diff`, понять причину. Если изменения лишние — откатить: `git stash -u` (изменения сохранятся в тайнике) или `git restore --staged --worktree .` (⚠ удалит правки безвозвратно), — и поставить задачу точнее, с границами.
3. Контрольные точки отслеживают только правки инструментов Claude, хранятся ограниченное время и существуют только локально. Коммиты охватывают все изменения, хранятся постоянно и отправляются на GitHub.
4. Писать: команды сборки и тестов, правила, которые AI не выведет из кода. Не писать: то, что видно из кода, длинные описания, секреты.
5. Существует ли пакет в официальном реестре, нужен ли он, поддерживается ли, какая у него лицензия, есть ли известные уязвимости.

## Что дальше

В главе 8 — упражнения и итоговые проекты, которые закрепят всё изученное.

# Глава 8. Практика и итоговые проекты

Знания закрепляются только руками. В этой главе — короткие упражнения по каждой главе и два итоговых проекта: первый репозиторий от папки до отмены изменений и профессионально оформленный репозиторий с лицензией, CI и релизом.

**Как работать с главой:**

- выполняйте шаги по порядку и сверяйтесь с ожидаемым результатом;
- если результат отличается — остановитесь, выполните `git status` и найдите причину в главе 6;
- хеши коммитов, даты и логин у вас будут свои — это нормально.

**Время:** проект 1 — 1,5 часа, проект 2 — 3–4 часа.

**Когда выполнять:** упражнения и проект 1 — сразу после глав 1–7. Проект 2 опирается на часть II — выполняйте его после глав 9–11.

## 8.1. Упражнения по главам

| Глава | Задание | Ожидаемый результат |
|---|---|---|
| 1. Основы | Объясните своими словами разницу между Git и GitHub, коммитом и веткой | Git — программа на компьютере, GitHub — сервис в интернете; коммит — сохранённая версия, ветка — линия работы |
| 2. Установка | Выполните `git --version` и `git config --global --list` | Версия Git; `user.name`, `user.email`, `init.defaultbranch=main` |
| 3. Команды | Создайте репозиторий, сделайте три коммита, найдите в `git log` второй и посмотрите его изменения через `git show <хеш>` | Три строки в `git log --oneline`; `git show` выводит сообщение и diff коммита |
| 3. Команды | Измените файл, подготовьте его, затем измените снова. Сравните `git diff` и `git diff --staged` | Первая правка видна в `--staged`, вторая — в `git diff` |
| 4. GitHub | Создайте файл через сайт, затем получите его командой `git pull` | Файл появился в папке, `git log` показывает коммит с сайта |
| 5. Ветки | Создайте две ветки, в каждой по коммиту, влейте обе в `main` | `git log --oneline --graph --all` показывает слияние |
| 5. Pull Request | Откройте PR из ветки, оставьте комментарий к строке во **Files changed**, слейте PR | PR в статусе **Merged**, ветка удалена |
| 6. Восстановление | Создайте конфликт и решите его; затем отмените коммит через `git revert` | В истории коммит слияния и коммит `Revert "..."` |
| 6. Безопасность | Создайте `.env` и убедитесь, что Git его игнорирует | `git check-ignore -v .env` показывает правило из `.gitignore` |

| Глава | Задание | Ожидаемый результат |
|---|---|---|
| 7. AI | Поручите Claude Code маленькую правку в отдельной ветке, просмотрите `git diff`, закоммитьте или откатите | Чистый `git status` в конце; осознанное решение по каждому изменению |

## 8.2. Проект 1. «Мой первый репозиторий»

Цель: пройти полный путь от пустой папки до отмены изменения. Понадобятся главы 2–6.

### Шаг 1. Создайте папку проекта

```powershell
mkdir ~\projects\my-first-repo
cd ~\projects\my-first-repo
pwd
```

**Ожидается:** `pwd` показывает `C:\Users\<вы>\projects\my-first-repo`.

### Шаг 2. Создайте простой файл

```powershell
notepad README.md
```

Нажмите **Да** в вопросе о создании файла и напишите:

```markdown
# Мой первый репозиторий

Учусь работать с Git и GitHub.
```

Сохраните и закройте Блокнот.

**Ожидается:** `ls` показывает `README.md`.

### Шаг 3. Инициализируйте Git

```powershell
git init
git status
```

**Ожидается:**

```
Initialized empty Git repository in C:/Users/<вы>/projects/my-first-repo/.git/
```

```
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        README.md
```

Если вместо `main` написано `master` — выполните `git branch -M main` и проверьте настройку `init.defaultBranch` (глава 2).

### Шаг 4. Создайте первый коммит

```powershell
git add README.md
git status
git commit -m "Добавить README"
```

**Ожидается:** `git status` показывает `new file: README.md` в разделе `Changes to be committed`, коммит выводит строку `[main (root-commit) ...] Добавить README`.

### Шаг 5. Создайте репозиторий на GitHub

1. **+** → **New repository**.
2. Имя: `my-first-repo`. Видимость: **Public** или **Private**.
3. README, .gitignore и лицензию **не добавляйте**.
4. **Create repository** → скопируйте HTTPS-адрес со страницы **Quick setup**.

**Ожидается:** пустой репозиторий с инструкциями Quick setup.

### Шаг 6. Свяжите локальный проект с GitHub

```powershell
git remote add origin https://github.com/<логин>/my-first-repo.git
git remote -v
```

**Ожидается:** две строки с вашим адресом — `(fetch)` и `(push)`.

### Шаг 7. Отправьте изменения

```powershell
git push -u origin main
```

При первом push войдите в GitHub через браузер (глава 4).

**Ожидается:** `* [new branch] main -> main` и `branch 'main' set up to track 'origin/main'`. На сайте виден `README.md` с оформленным заголовком.

### Шаг 8. Создайте отдельную ветку

```powershell
git switch -c feature/about
git branch
```

**Ожидается:** `Switched to a new branch 'feature/about'`; в списке веток звёздочка у `feature/about`.

### Шаг 9. Измените файл и сделайте коммит

Откройте `README.md` (`notepad README.md`) и добавьте в конец:

```markdown
## Обо мне

Начинающий разработчик. Изучаю Git, GitHub и работу с AI-ассистентами.
```

```powershell
git diff
git add README.md
git commit -m "Добавить раздел «Обо мне»"
```

**Ожидается:** `git diff` показывает добавленные строки со знаком `+`; коммит создан в ветке `feature/about`.

### Шаг 10. Объедините изменения

**Вариант А — через Pull Request (рекомендуется):**

```powershell
git push -u origin feature/about
```

На GitHub: **Compare & pull request** → проверьте **base: main** ← **compare: feature/about** → **Create pull request** → вкладка **Files changed** → **Merge pull request** → **Confirm merge** → **Delete branch**. Затем на компьютере:

```powershell
git switch main
git pull
git branch -d feature/about
```

**Вариант Б — локально:**

```powershell
git switch main
git merge feature/about
git push
git branch -d feature/about
```

**Ожидается:** в `main` есть раздел «Обо мне». При локальном слиянии Git пишет `Fast-forward`.

### Шаг 11. Посмотрите историю

```powershell
git log --oneline --graph --all
```

**Ожидается** (вариант Б):

```
* 4c1d2e9 (HEAD -> main, origin/main) Добавить раздел «Обо мне»
* 9a8b7c6 Добавить README
```

При варианте А появится ещё коммит слияния `Merge pull request #1 ...`, а у удалённой ветки, которую вы удалили на сайте, может остаться локальная «закладка» `origin/feature/about`. Уберите её командой `git fetch --prune`. На GitHub откройте список коммитов (ссылка с числом коммитов) — он совпадает с выводом `git log`.

### Шаг 12. Внесите тестовое изменение и отмените его

**12.1. Отмена незакоммиченной правки.** Допишите в `README.md` строку `Ошибочная строка` и сохраните. Затем:

```powershell
git status
git diff
git restore README.md
git status
```

**Ожидается:** сначала `modified: README.md` и строка `+Ошибочная строка` в diff; после `git restore` — `nothing to commit, working tree clean`, строки в файле нет.

**12.2. Отмена отправленного коммита.** Снова допишите `Ошибочная строка`, затем:

```powershell
git add README.md
git commit -m "Тестовое изменение"
git push
git revert --no-edit HEAD
git push
git log --oneline
```

**Ожидается** (вариант Б; при варианте А между коммитами будет ещё `Merge pull request #1 ...`):

```
b3c4d5e (HEAD -> main, origin/main) Revert "Тестовое изменение"
a1b2c3d Тестовое изменение
4c1d2e9 Добавить раздел «Обо мне»
9a8b7c6 Добавить README
```

Строки нет в файле, а в истории видно и ошибку, и её отмену. Именно так безопасно отменяют коммиты, которые уже на GitHub.

### Проверка проекта 1

- ☐ На GitHub есть репозиторий `my-first-repo` с `README.md`.
- ☐ В истории не меньше четырёх коммитов, включая `Revert "Тестовое изменение"`.
- ☐ `git status` показывает `Your branch is up to date with 'origin/main'` и чистую рабочую папку.
- ☐ Ветка `feature/about` удалена локально и на GitHub.

## 8.3. Проект 2. «Профессионально оформленный репозиторий»

Цель: оформить небольшой программный проект так, как это делают в профессиональных командах. Понадобятся главы 9–11.

Эталонное решение — папка examples/tempconv: консольный конвертер температур на Python. Можно повторить его или оформить свой проект по тем же шагам.

> **ПРИМЕЧАНИЕ**
>
> Если вы не программировали на Python, это не страшно: код уже готов в примере. Задача проекта — **оформление репозитория**, а не написание программы. Для запуска проверок понадобится Python 3.11+ с python.org.

### Шаг 1. Создайте репозиторий

1. На GitHub: **New repository** → имя `tempconv` → **Public** → включите **README** → **Create repository**.
2. Склонируйте:

```
POWERSHELL
cd ~\projects
git clone https://github.com/<логин>/tempconv.git
cd tempconv
```

3. Скопируйте в папку файлы `src/`, `tests/`, `pyproject.toml`, `requirements-dev.txt` из примера (через Проводник).
4. Ваш проект только начинается, поэтому задайте начальную версию: в `pyproject.toml` (строка `version = "1.0.0"`) и в `src/tempconv/__init__.py` (строка `__version__ = "1.0.0"`) замените `1.0.0` на `0.1.0`.
5. Первый коммит:

```
POWERSHELL
git add .
git commit -m "feat: добавить конвертер температур"
git push
```

**Ожидается:** на GitHub видны папки `src` и `tests`.

### Шаг 2. Оформите README

Возьмите шаблон templates/README.template.md и заполните разделы: описание, проблема, возможности, установка, быстрый старт, примеры, настройка, тестирование, статус, лицензия. Сверьтесь с README примера.

```
POWERSHELL
git add README.md
git commit -m "docs: описать установку и использование в README"
git push
```

**Ожидается:** на главной странице репозитория — оформленный README с содержанием и примерами команд.

### Шаг 3. Выберите лицензию по условиям

**Условия задачи:**

- весь код написан вами, чужих фрагментов нет, работодатель прав на код не имеет;
- вы хотите, чтобы код могли использовать кто угодно и где угодно, включая коммерческие продукты;
- вы не требуете открывать исходный код производных программ;
- достаточно, чтобы сохранялось уведомление о вашем авторстве;
- лицензия должна быть короткой и понятной.

**Решение.** Подходят разрешительные лицензии. Под условия «коротко и просто» лучше всего подходит **MIT**. Если бы для вас была важна явная патентная лицензия — стоило бы рассмотреть **Apache-2.0** (глава 10).

**Как добавить:**

1. На GitHub: **Add file** → **Create new file** → имя `LICENSE` → **Choose a license template** → **MIT License** → **Review and submit**.
2. Проверьте строку `Copyright (c) <год> <имя>`: год и **ваше** имя или имя правообладателя.
3. **Commit changes**, затем получите новый коммит на компьютере: `git pull`. Так как все ваши коммиты уже отправлены, Git просто «перемотает» ветку вперёд (`Fast-forward`).

Проверьте, что в `pyproject.toml` есть строка `license = "MIT"` (SPDX-идентификатор; в файле из примера она уже есть), и добавьте в README раздел «Лицензия»:

```
POWERSHELL
git add README.md pyproject.toml
git commit -m "docs: указать лицензию MIT в README"
git push
```

**Ожидается:** справа на странице репозитория появилась отметка **MIT license**.

### Шаг 4. Добавьте .gitignore, .editorconfig и .gitattributes

Скопируйте файлы из примера или создайте свои (глава 9).

```powershell
git add .gitignore .editorconfig .gitattributes
git commit -m "chore: добавить .gitignore, .editorconfig и .gitattributes"
git push
```

**Ожидается:** `git status --ignored` показывает `.venv/` и `__pycache__/` в игнорируемых, если они есть.

### Шаг 5. Создайте CONTRIBUTING.md и SECURITY.md

Возьмите шаблоны templates/CONTRIBUTING.template.md и templates/SECURITY.template.md. Заполните заполнители в квадратных скобках своими данными. Укажите реальный способ закрыто сообщить об уязвимости и включите **Private vulnerability reporting** (глава 11).

```powershell
git add CONTRIBUTING.md SECURITY.md
git commit -m "docs: добавить CONTRIBUTING и SECURITY"
git push
```

**Ожидается:** при создании Issue GitHub показывает ссылку на правила участия; в разделе безопасности репозитория есть политика.

### Шаг 6. Ведите CHANGELOG.md

Создайте `CHANGELOG.md` по шаблону templates/CHANGELOG.template.md. Запишите изменения в раздел `## [Unreleased]`, а перед релизом перенесите их в раздел версии:

```markdown
## [0.1.0] - 2026-10-11

### Added

- Перевод температур между шкалами C, F и K.
```

```powershell
git add CHANGELOG.md
git commit -m "docs: добавить CHANGELOG"
git push
```

### Шаг 7. Пишите понятные сообщения коммитов

Используйте формат Conventional Commits (глава 9):

| Тип | Когда | Пример |
|---|---|---|
| `feat` | Новая возможность | `feat: добавить шкалу Ранкина` |
| `fix` | Исправление ошибки | `fix: принимать шкалу с пробелами` |
| `docs` | Документация | `docs: добавить пример в README` |
| `test` | Тесты | `test: проверить отрицательные значения` |

| Тип | Когда | Пример |
|---|---|---|
| `ci` | Автоматические проверки | `ci: добавить Python 3.15 в матрицу` |
| `chore` | Служебное | `chore: обновить .gitignore` |

**Ожидается:** `git log --oneline` читается как список понятных изменений.

### Шаг 8. Настройте базовый workflow проверки

1. Скопируйте `.github/workflows/ci.yml` и `.github/dependabot.yml` из примера.
2. Отправьте их через ветку и Pull Request:

```
POWERSHELL
git switch -c ci/add-checks
git add .github
git commit -m "ci: добавить проверку стиля и тесты"
git push -u origin ci/add-checks
```

3. Откройте PR. Через минуту во вкладке **Checks** появятся проверки для четырёх версий Python.
4. Когда они станут зелёными, слейте PR и обновите `main` у себя (`git switch main`, `git pull`).

**Ожидается:** во вкладке **Actions** — успешный запуск workflow **CI**; в README можно добавить значок CI.

### Шаг 9. Создайте тег и релиз

```
POWERSHELL
git switch main
git pull
git tag -a v0.1.0 -m "Версия 0.1.0"
git push origin v0.1.0
```

> **Тег** (*tag*) — постоянная метка на коммите, обычно номер версии. Аннотированный тег (`-a`) хранит автора, дату и сообщение — такие используют для релизов.

На GitHub: **Releases** → **Draft a new release** → **Choose a tag**: `v0.1.0` → заголовок `v0.1.0` → вставьте раздел версии из `CHANGELOG.md` или нажмите **Generate release notes** → **Publish release**.

**Ожидается:** на странице репозитория справа появился блок **Releases** с версией `v0.1.0`; к релизу автоматически приложены архивы исходного кода.

### Шаг 10. Проверьте безопасность и готовность к публикации

1. Включите защиту ветки `main`: обязательный Pull Request и успешные проверки (глава 11).
2. Проверьте, что включены Dependabot alerts и secret scanning (глава 11).
3. Просмотрите историю на секреты: `git log -p -i -G "password|token|secret|key"` покажет коммиты, где добавлялись или удалялись строки с этими словами.
4. Пройдите чек-лист templates/PROJECT_RELEASE_CHECKLIST.md — уровни «Базовый» и «Профессиональный».

**Ожидается:** все пункты базового и профессионального уровня отмечены или осознанно пропущены с пояснением.

### Проверка проекта 2

☐ README с установкой, примерами, настройкой, тестированием и лицензией.

☐ `LICENSE` с годом и именем правообладателя; GitHub распознаёт лицензию.

☐ `.gitignore`, `.editorconfig`, `.gitattributes`.

☐ `CONTRIBUTING.md`, `SECURITY.md`, `CHANGELOG.md`.

☐ Сообщения коммитов по Conventional Commits.

☐ Workflow CI проходит в Pull Request.

☐ Тег `v0.1.0` и релиз на GitHub.

☐ Защита `main`, Dependabot и secret scanning включены.

## 8.4. Чек-лист готовности репозитория к публикации

Краткая версия. Полная, с пунктами для отметки и итоговой таблицей, — templates/PROJECT_RELEASE_CHECKLIST.md. Уровни накопительные.

**Уровень 1. Базовый — небольшой личный проект**

☐ README: что это, как установить и запустить; команды работают.

☐ `.gitignore`; в репозитории нет временных и служебных файлов.

☐ В коде и истории нет паролей, токенов, ключей и личных данных.

☐ У вас есть право публиковать весь код.

☐ Для публичного кода — файл `LICENSE` с правообладателем.

☐ Понятные сообщения коммитов; всё отправлено на GitHub.

**Уровень 2. Профессиональный — документация, тесты, контроль качества, сопровождение**

☐ README с требованиями, настройкой, тестированием, статусом и ограничениями.

☐ Тесты и линтер запускаются в CI для каждого Pull Request.

☐ `CHANGELOG.md`, версия по SemVer, тег `vX.Y.Z` и релиз.

☐ `CONTRIBUTING.md`, `SECURITY.md`, шаблон Pull Request.

☐ `.editorconfig`, `.gitattributes`.

☐ Ветка `main` защищена; Dependabot включён; действия в workflow закреплены хешами.

☐ Лицензии зависимостей проверены.

**Уровень 3. Публичный open-source — сообщество, безопасность, лицензирование, релизы**

☐ `CODE_OF_CONDUCT.md` с реальным контактом для жалоб.

☐ Шаблоны Issue; условия приёма вклада (лицензия проекта, DCO или CLA).

☐ Закрытые сообщения об уязвимостях, secret scanning, push protection, CodeQL.

☐ Проверка OpenSSF Scorecard.

☐ SPDX-идентификатор лицензии; `THIRD_PARTY_NOTICES.md` или `NOTICE` для встроенных компонентов.

☐ Релизы собираются в CI, заметки о выпуске перечисляют несовместимые изменения и исправления безопасности.

## 8.5. Контрольный список навыков

Отметьте, что умеете делать самостоятельно, без подсказок.

**Основы и настройка**

☐ Объяснить разницу между Git и GitHub, коммитом, веткой и репозиторием.
☐ Открыть терминал в нужной папке и перемещаться по папкам.
☐ Установить Git и настроить имя, почту, ветку `main`, редактор.

**Ежедневная работа**

☐ Создать репозиторий (`git init`) или склонировать его (`git clone`).
☐ Проверить состояние (`git status`) и изменения (`git diff`, `git diff --staged`).
☐ Подготовить и сохранить изменения (`git add`, `git commit`) с понятным сообщением.
☐ Посмотреть историю (`git log --oneline --graph --all`).
☐ Отправить и получить изменения (`git push`, `git pull`, `git fetch`).

**GitHub**

☐ Создать репозиторий и выбрать Public или Private.
☐ Добавить, изменить и удалить файл через сайт.
☐ Отправить существующий проект на GitHub, в том числе в репозиторий с README.
☐ Войти в GitHub из терминала без пароля от аккаунта.

**Ветки и командная работа**

☐ Создать ветку, переключиться, слить и удалить её.
☐ Открыть Pull Request, просмотреть **Files changed**, слить PR.
☐ Решить конфликт слияния.

**Восстановление и безопасность**

☐ Отменить незакоммиченные правки (`git restore`, `git stash`).
☐ Отменить отправленный коммит (`git revert`).
☐ Восстановить старую версию файла (`git restore --source`).
☐ Объяснить разницу между `restore`, `revert` и `reset` и риски `reset --hard` и `push --force`.
☐ Найти «потерянный» коммит через `git reflog`.
☐ Настроить `.gitignore` и не допустить секреты в репозиторий.

**Работа с AI**

☐ Начинать задачу для AI с чистого `git status` и отдельной ветки.
☐ Проверять изменения AI через `git diff` и тесты перед коммитом.
☐ Откатывать неудачные изменения AI.

**Оформление проектов**

- ☐ Оформить README, LICENSE, CHANGELOG, CONTRIBUTING, SECURITY.
- ☐ Выбрать лицензию по целям проекта и проверить лицензии зависимостей.
- ☐ Настроить базовый CI и Dependabot.
- ☐ Выпустить версию: тег и релиз на GitHub.

## 8.6. Что изучать дальше

- **GitHub Skills** — бесплатные интерактивные курсы прямо в GitHub: learn.github.com/skills. Начните с Introduction to GitHub, затем Review pull requests, Resolve merge conflicts, Hello GitHub Actions и Secure your repository's supply chain.
- **Книга Pro Git на русском** — бесплатно на git-scm.com/book/ru/v2. Главы 1–3 повторяют и углубляют эту методичку.
- **Темы следующего уровня:** `git rebase`, подписание коммитов, Git LFS, GitHub Pages, GitHub Projects.

Все источники — в файле sources.md.

ЧАСТЬ II

# Как создавать, оформлять и публиковать программные проекты по международным практикам

Структура репозитория, документация, версии, лицензии, стандарты, безопасность и автоматические проверки.

Глава 9. Стандарты и оформление проектов

Глава 10. Лицензии и авторские права

Глава 11. Безопасность и автоматизация

# Глава 9. Стандарты и оформление проектов

Глава открывает часть II методички «Как создавать, оформлять и публиковать программные проекты по международным практикам». Эта часть — о том, как превратить «папку с кодом» в проект, которым удобно пользоваться, который легко развивать и не страшно публиковать. В этой главе — структура репозитория, README, документы сопровождения, версионирование и обзор международных стандартов и инженерных практик.

**Вы научитесь:**

- отличать международные стандарты от практик сообщества, рекомендаций платформ и требований закона;
- выбирать структуру репозитория и нужные файлы;
- писать README, CONTRIBUTING, CODE_OF_CONDUCT, SECURITY и CHANGELOG;
- выпускать версии по Semantic Versioning с тегами и релизами;
- понимать, когда и как применять ISO/IEC-стандарты, SPDX, REUSE, OpenSSF Scorecard, SLSA и OWASP.

**Время:** 2–3 часа. **Понадобится:** главы 1–6 части I. Лицензиям посвящена глава 10, безопасности и автоматизации — глава 11.

## 9.1. Стандарты, практики и требования: как их различать

Слово «стандарт» часто используют для всего подряд. Разберёмся, что есть что.

| Вид | Кто устанавливает | Обязателен ли | Примеры |
|---|---|---|---|
| **Международный стандарт** | ISO, IEC, IEEE — через национальные органы стандартизации | Добровольный. Обязательным становится через закон, договор или требование заказчика | ISO/IEC/IEEE 12207, ISO/IEC 25010, ISO/IEC/IEEE 29148, ISO/IEC 5962 (SPDX 2.2.1) |
| **Открытая спецификация или практика сообщества** | Сообщества и фонды: OpenSSF, OWASP, Linux Foundation, FSFE | Добровольная | SemVer, Keep a Changelog, Conventional Commits, SPDX 3, REUSE, SLSA, OpenSSF Scorecard, OWASP ASVS и SAMM |
| **Рекомендация платформы** | Компания — владелец сервиса | Добровольная, но от неё зависит работа функций платформы | GitHub Flow, файлы сообщества GitHub, Dependabot |
| **Юридическое требование** | Законы конкретной страны, договоры | Обязательно в своей юрисдикции | Авторское право, условия лицензий, договор с работодателем, регламенты вроде EU Cyber Resilience Act |

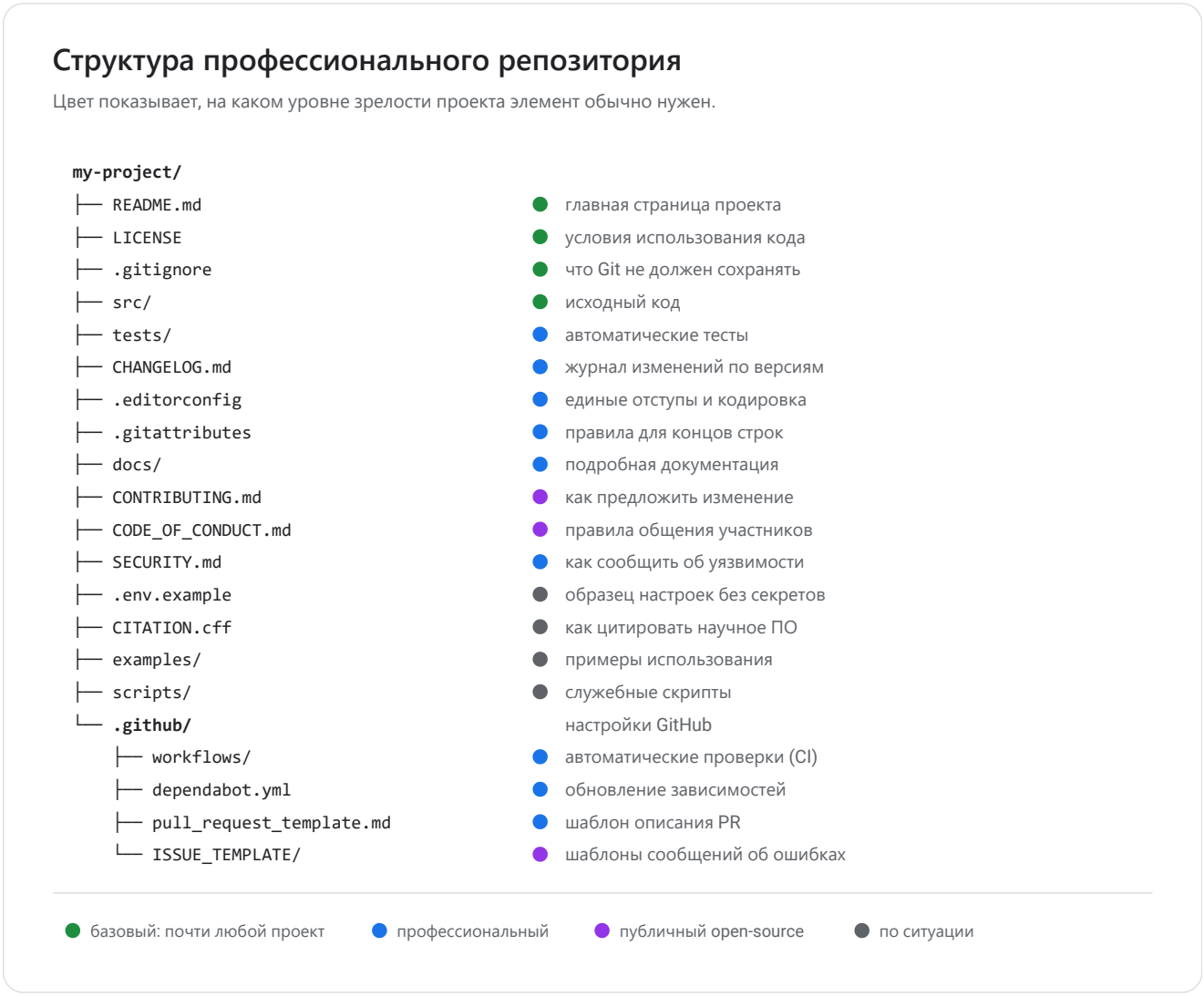
Три важных уточнения:

1. **Применять ≠ соответствовать ≠ быть сертифицированным.** Если вы взяли из стандарта несколько идей, это не значит, что проект ему соответствует. Соответствие требует выполнить все обязательные требования стандарта. Сертификация — это проверка независимым органом; для

многих стандартов из этой главы её нет вовсе.

2. **Тексты ISO в основном платные.** На iso.org бесплатно доступны название, описание и статус документа. Во многих странах есть национальные версии стандартов, их редакции могут отставать от международных.
3. **Практики сообщества меняются.** Проверяйте версию спецификации и дату обновления на официальном сайте.

## 9.2. Структура профессионального репозитория




Пример универсальной структуры:

```
my-project/
├── README.md
├── LICENSE
├── CONTRIBUTING.md
├── CODE_OF_CONDUCT.md
├── SECURITY.md
├── CHANGELOG.md
├── .gitignore
├── .editorconfig
├── .gitattributes
```

```
├── .env.example
├── CITATION.cff
├── docs/
├── src/
├── tests/
├── examples/
├── scripts/
└── .github/
    ├── ISSUE_TEMPLATE/
    ├── workflows/
    ├── pull_request_template.md
    └── dependabot.yml
```

> **ВАЖНО**
>
> Не создавайте все файлы сразу «для галочки». Пустой `CODE_OF_CONDUCT.md` без человека, который следит за его соблюдением, или `SECURITY.md` с несуществующим адресом хуже, чем их отсутствие. Добавляйте файл, когда понимаете, зачем он нужен и кто за него отвечает.

## Назначение каждого элемента

| Элемент | Назначение | Когда нужен |
|---|---|---|
| `README.md` | Главная страница: что это, как установить и использовать | Всегда |
| `LICENSE` | Условия использования кода (глава 10) | Если кодом будут пользоваться другие; без лицензии копирование, изменение и распространение кода по умолчанию запрещены |
| `CONTRIBUTING.md` | Как предложить изменение | Командный или открытый проект |
| `CODE_OF_CONDUCT.md` | Правила общения участников | Открытый проект с сообществом |
| `SECURITY.md` | Как закрыто сообщить об уязвимости | Публичный проект, которым пользуются; профессиональные проекты |
| `CHANGELOG.md` | Изменения по версиям — для пользователей | Когда есть версии и пользователи |
| `.gitignore` | Что Git не должен сохранять | Всегда |
| `.editorconfig` | Единые отступы, кодировка и концы строк во всех редакторах | Больше одного участника или редактора |
| `.gitattributes` | Как Git обрабатывает концы строк и двоичные файлы | Участники на разных ОС; двоичные файлы; Git LFS |
| `.env.example` | Образец настроек без секретов | Программа читает настройки или ключи из окружения |

| Элемент | Назначение | Когда нужен |
|---|---|---|
| `CITATION.cff` | Как цитировать проект в научных работах | Научное и исследовательское ПО |
| `docs/` | Подробная документация: архитектура, руководства, решения | Когда README становится слишком длинным |
| `src/` | Исходный код | Почти всегда; в некоторых экосистемах принята другая раскладка — следуйте ей |
| `tests/` | Автоматические тесты | Профессиональный проект |
| `examples/` | Примеры использования | Библиотеки и инструменты |
| `scripts/` | Служебные скрипты: сборка, установка, миграции | По ситуации |
| `.github/ISSUE_TEMPLATE/` | Шаблоны сообщений об ошибках и идей | Открытый проект |
| `.github/workflows/` | Автоматические проверки GitHub Actions | Профессиональный проект |
| `.github/pull_request_template.md` | Шаблон описания Pull Request | Командная работа |
| `.github/dependabot.yml` | Автоматические предложения обновить зависимости | Есть зависимости или GitHub Actions |

GitHub ищет файлы сообщества (`CONTRIBUTING.md`, `CODE_OF_CONDUCT.md`, `SECURITY.md`, `SUPPORT.md`, шаблоны) в папке `.github/`, затем в корне, затем в `docs/`. `LICENSE` должен лежать в каждом репозитории. Источник: Creating a default community health file.

## Обязательные и рекомендуемые файлы по уровням

Обозначения: ● — обязательно на этом уровне, ○ — рекомендуется, — — обычно не нужно.

| Файл | Базовый (личный проект) | Профессиональный | Публичный open-source |
|---|---|---|---|
| `README.md` | ● | ● | ● |
| `.gitignore` | ● | ● | ● |
| `LICENSE` | ● если код публичный | ● | ● |
| `CHANGELOG.md` | — | ● | ● |
| `tests/` и CI | ○ | ● | ● |
| `.editorconfig`, `.gitattributes` | ○ | ● | ● |
| `CONTRIBUTING.md` | — | ● в команде | ● |
| `SECURITY.md` | — | ● | ● |

| Файл | Базовый (личный проект) | Профессиональный | Публичный open-source |
|---|---|---|---|
| `.github/dependabot.yml` | ○ | ● | ● |
| `pull_request_template.md` | — | ● в команде | ● |
| `ISSUE_TEMPLATE/` | — | ○ | ● |
| `CODE_OF_CONDUCT.md` | — | — | ● |
| `docs/` | — | ○ | ○ |
| `.env.example` | ● если есть настройки | ● если есть настройки | ● если есть настройки |
| `CITATION.cff` | по ситуации | по ситуации | по ситуации |
| `NOTICE`, `THIRD_PARTY_NOTICES.md` | по ситуации (глава 10) | по ситуации | по ситуации |

Это рекомендации методички, а не требования стандарта. Подробный чек-лист — templates/PROJECT_RELEASE_CHECKLIST.md.

## .editorconfig и .gitattributes

> **EditorConfig** — соглашение о файле `.editorconfig`, в котором записаны правила форматирования: отступы, кодировка, концы строк. Редакторы применяют их автоматически. Многие редакторы понимают его сами; VS Code, Notepad++ и Sublime Text — через расширение. Источник: editorconfig.org.

```ini
root = true

[*]
charset = utf-8
end_of_line = lf
insert_final_newline = true
trim_trailing_whitespace = true
indent_style = space
indent_size = 4

[*.md]
trim_trailing_whitespace = false
```

> **`.gitattributes`** — файл с правилами для Git: какие файлы текстовые, какие двоичные, какие концы строк хранить. Строка `* text=auto` просит Git самому определять текстовые файлы и хранить их с концами строк LF. Это убирает «изменения» из-за разницы Windows и Linux.

.GITATTRIBUTES

```
* text=auto
*.ps1 text eol=crlf
*.png binary
```

Полные примеры — в examples/tempconv.

## 9.3. README.md — главная страница проекта

README читают первым. За минуту человек должен понять, что это за проект, подходит ли он ему и как его запустить.

### Из чего состоит хороший README

| Раздел | Что написать | Обязателен |
|---|---|---|
| Название и краткое описание | Одно предложение: что делает программа | Да |
| Проблема | Какую задачу решает и для кого | Да |
| Возможности | 3–7 пунктов | Да |
| Скриншоты или демонстрация | Как выглядит результат | Если есть интерфейс или наглядный вывод |
| Архитектура и схема работы | Из чего состоит, как связаны части | Для нетривиальных проектов; подробности — в `docs/` |
| Требования к окружению | ОС, версия языка, внешние сервисы | Да |
| Установка | Пошаговые команды, которые можно скопировать | Да |
| Быстрый старт | Минимальный пример «запустить и увидеть результат» | Да |
| Примеры использования | Типичные сценарии | Да |
| Настройка через переменные окружения | Таблица: переменная, значение по умолчанию, описание | Если есть настройки |
| Тестирование | Как запустить тесты | Для проектов с тестами |
| Документация | Ссылки на `docs/`, сайт, API | Если есть |
| Статус и ограничения | Стабильность, версия, чего программа не умеет | Да |
| Как сообщить об ошибке и предложить улучшение | Ссылки на Issues, CONTRIBUTING, SECURITY | Для публичных проектов |
| Лицензия | Название и ссылка на `LICENSE` | Да, если есть лицензия |

Полный пример — README учебного проекта tempconv. Шаблон с пояснениями — templates/README.template.md.

### Когда уместны дополнительные элементы

| Элемент | Уместно | Неуместно |
|---|---|---|
| **Бейджи** (значки статуса CI, версии, лицензии) | 2–4 бейджа с полезной информацией, которая обновляется сама | Десяток значков ради украшения; бейджи сломанных сервисов |
| **Содержание** | README длиннее двух-трёх экранов | Короткий README |
| **GIF-анимация** | Интерфейс или процесс, который трудно описать словами | Когда хватает текста или скриншота; тяжёлые файлы на десятки мегабайт |
| **Скриншоты** | Программа с интерфейсом; результат работы | Скриншоты кода — код лучше вставлять текстом |
| **Архитектурная диаграмма** | Несколько компонентов или сервисов | Одна функция в одном файле |
| **Ссылка на демоверсию** | Веб-приложение, которое можно открыть в браузере | Демо, которое давно не работает |

> СОВЕТ
>
> Диаграммы в README удобно рисовать в Mermaid прямо в Markdown — GitHub отображает их автоматически. Изображения храните в репозитории (например, в `docs/images/`) и подписывайте альтернативным текстом: `![Схема работы](docs/images/flow.png)`.

### Правила хорошего README

- Пишите для человека, который видит проект впервые.
- Команды должны работать, если их скопировать. Проверьте установку на «чистом» компьютере или в CI.
- Не обещайте того, чего нет. Раздел «Ограничения» вызывает доверие.
- Обновляйте README вместе с кодом — это можно включить в чек-лист Pull Request.

## 9.4. Документы сопровождения

Шаблоны всех документов — в папке templates/. В каждом есть пояснения и заполнители в квадратных скобках, например `[ИМЯ ПРАВООБЛАДАТЕЛЯ]`. Не оставляйте заполнители в опубликованных файлах.

### CONTRIBUTING.md — как участвовать

Описывает путь от идеи до принятого изменения:

- как предложить изменение (сначала Issue или сразу Pull Request);
- как подготовить окружение разработчика;
- как запустить тесты и проверки;
- правила оформления кода и сообщений коммитов;
- как создать Pull Request и сколько ждать ревью;
- на каких условиях принимается вклад (обычно — на условиях лицензии проекта).

**Когда применять:** в любом проекте, где изменения вносит больше одного человека. GitHub показывает ссылку на этот файл при создании Issue и Pull Request. Шаблон: templates/CONTRIBUTING.template.md.

### CODE_OF_CONDUCT.md — правила общения

Кодекс поведения описывает, какое поведение в сообществе приемлемо, какое нет, куда сообщать о нарушениях и что за них бывает.

- **Готовый текст.** Самый распространённый — Contributor Covenant. Актуальная версия — 3.0 (2025). На GitHub шаблон можно вставить так: **Add file** → **Create new file** → имя `CODE_OF_CONDUCT.md` → **Choose a code of conduct template**. Сверьте версию шаблона с сайтом Contributor Covenant.
- **Обязательно укажите реальный контакт** для жалоб и человека, который их рассматривает.
- **Когда применять:** открытый проект с внешними участниками. В личном проекте без сообщества он не нужен.

Шаблон с пояснениями: templates/CODE_OF_CONDUCT.template.md.

### SECURITY.md — как сообщить об уязвимости

> **Уязвимость** — ошибка, которая позволяет нарушить безопасность: получить чужие данные, выполнить свой код, обойти проверку доступа.

> **Ответственное (координированное) раскрытие** — порядок, при котором исследователь сначала закрыто сообщает об уязвимости разработчикам, а публикует детали только после выхода исправления или по согласованию.

Что написать:

- **поддерживаемые версии** — для каких версий выпускаются исправления;
- **закрытый канал** для сообщений: функция GitHub **Private vulnerability reporting** или почта;
- **что указать** в сообщении: версия, шаги воспроизведения, последствия;
- **сроки** ответа и порядок публикации.

Почему детали нельзя публиковать сразу: пока исправления нет, публичное описание — готовая инструкция для атаки на всех пользователей.

**Когда применять:** любой публичный проект, которым кто-то пользуется, и все профессиональные проекты. Шаблон: templates/SECURITY.template.md. Как включить закрытые сообщения на GitHub — глава 11.

### CHANGELOG.md — журнал изменений

> **Журнал изменений** (*changelog*) — файл, где для каждой версии записано, что изменилось: понятным пользователю языком.

Чем он отличается от `git log`: история коммитов — для разработчиков, в ней «исправил опечатку» и «слил ветку». Журнал изменений — выжимка для пользователей: что нового и что может сломаться при обновлении.

Принципы Keep a Changelog 1.1.0:

- журнал — для людей, а не машин;

- запись для каждой версии, новые версии — сверху;
- дата выпуска в формате ГГГГ-ММ-ДД (ISO 8601);
- изменения сгруппированы по типам;
- вверху раздел `[Unreleased]` — изменения, которые войдут в следующую версию;
- отозванные версии помечаются `[YANKED]`.

| Тип | Что туда писать |
|---|---|
| `Added` | Новые возможности |
| `Changed` | Изменения существующей функциональности |
| `Deprecated` | Возможности, которые скоро удалят |
| `Removed` | Удалённые возможности |
| `Fixed` | Исправленные ошибки |
| `Security` | Исправления уязвимостей |

Как отличать типы изменений:

- **исправление** (*fix*) — программа начинает работать так, как было обещано → `Fixed`, версия PATCH;
- **новая возможность** — появилось то, чего не было, старое работает как раньше → `Added`, версия MINOR;
- **несовместимое изменение** (*breaking change*) — то, что работало у пользователей, перестанет работать без их правок → `Changed` или `Removed` с явной пометкой, версия MAJOR.

Шаблон: templates/CHANGELOG.template.md. Пример: CHANGELOG учебного проекта.

## Другие полезные файлы

| Файл | Назначение |
|---|---|
| `SUPPORT.md` | Где получить помощь: форум, чат, платная поддержка |
| `.github/CODEOWNERS` | Кто автоматически назначается ревьюером для частей кода |
| `.github/FUNDING.yml` | Кнопка **Sponsor** со ссылками на способы поддержать проект |
| `AUTHORS`, `CONTRIBUTORS`, `NOTICE`, `THIRD_PARTY_NOTICES.md` | Авторство и сторонние компоненты — глава 10 |
| `CITATION.cff` | Как цитировать проект — глава 10 |

# 9.5. Версионирование: SemVer, теги и релизы

## Семантическое версионирование

> **Semantic Versioning** (SemVer, «семантическое версионирование») — соглашение о номерах версий вида `MAJOR.MINOR.PATCH`, в котором каждое число сообщает, насколько изменилась программа. Спецификация: semver.org, версия 2.0.0.




| Что изменилось | Какое число увеличить | Пример |
|---|---|---|
| Исправлена ошибка, поведение для пользователей не меняется | PATCH | `1.4.2` → `1.4.3` |
| Добавлена возможность, старое работает как прежде | MINOR (PATCH обнуляется) | `1.4.2` → `1.5.0` |
| Изменение может сломать код или сценарии пользователей | MAJOR (MINOR и PATCH обнуляются) | `1.4.2` → `2.0.0` |

Важные правила спецификации:

- **`0.y.z` — начальная разработка.** Всё может меняться в любой момент, публичный интерфейс нестабилен. Версия 0.x уместна, пока вы ищете форму проекта.
- **`1.0.0` определяет публичный интерфейс.** Если программой уже пользуются в работе, ей, скорее всего, пора быть 1.0.0.
- **Выпущенную версию не меняют.** Любое исправление — новая версия.
- **Предварительные версии** пишутся через дефис: `2.0.0-alpha.1`, `2.0.0-rc.1`. Они «младше» итоговой `2.0.0`.
- **Метаданные сборки** — через плюс: `1.0.0+20261011`. При сравнении версий они не учитываются.
- **Буква `v`** в `v1.2.3` — распространённое соглашение для тегов, а не часть версии. Семантическая версия здесь — `1.2.3`.

> **Публичный интерфейс** (*public API*) — то, на что опираются пользователи программы: команды и их параметры, функции библиотеки, формат файлов. SemVer имеет смысл, только если вы определили, что входит в публичный интерфейс.

### Теги Git

```powershell
git tag -a v1.0.0 -m "Версия 1.0.0"   # создать аннотированный тег на текущем коммите
git push origin v1.0.0                 # отправить тег на GitHub
git tag                                # список тегов
git show v1.0.0                        # информация о теге и коммите
```

Документация Git рекомендует аннотированные теги (`-a`) для релизов: они хранят автора, дату и сообщение. Лёгкие теги (без `-a`) — для личных временных меток. `git push` по умолчанию теги не отправляет — их нужно отправлять явно.

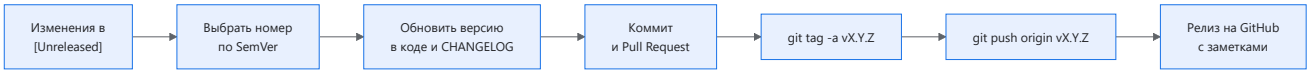
### Релиз на GitHub

> **Релиз** (*release*) — страница на GitHub, связанная с тегом: номер версии, описание изменений (*release notes*) и файлы для скачивания.

1. **Releases** (справа на странице репозитория) → **Draft a new release**.
2. **Choose a tag** → выберите существующий тег или создайте новый (**Create new tag**) на нужной ветке.
3. Заголовок — номер версии, например `v1.0.0`.
4. Описание: скопируйте раздел версии из `CHANGELOG.md` или нажмите **Generate release notes** — GitHub соберёт список слитых Pull Request.
5. Если это предварительная версия (например, `v2.0.0-rc.1`), отметьте соответствующий пункт (*pre-release*).
6. **Publish release**.

Категории автоматических заметок настраиваются в файле `.github/release.yml`. Источник: Managing releases.

### Порядок выпуска версии




## 9.6. Сообщения коммитов: Conventional Commits

> **Conventional Commits** — соглашение о формате сообщений коммитов, удобном и людям, и программам. Спецификация 1.0.0: conventionalcommits.org.

Формат:

```
<тип>[область]: <описание>

[тело — подробности, необязательно]

[сноски, например BREAKING CHANGE: ...]
```

Примеры:

```
feat: добавить шкалу Ранкина
fix(cli): принимать шкалу с пробелами
docs: описать переменную TEMPCONV_PRECISION
feat!: удалить устаревший параметр --round
```

| Элемент | Значение | Связь с SemVer |
|---|---|---|
| `feat` | Новая возможность | MINOR |
| `fix` | Исправление ошибки | PATCH |
| `!` после типа или сноска `BREAKING CHANGE:` | Несовместимое изменение | MAJOR |
| `(cli)` | Область изменения — необязательно | — |

Спецификация обязывает только `feat`, `fix` и обозначение несовместимых изменений. Типы `build`, `chore`, `ci`, `docs`, `style`, `refactor`, `perf`, `test`, `revert` — распространённое соглашение (например, в инструменте commitlint), а не требование спецификации.

**Нужно ли новичку:** не обязательно. Но формат приучает писать понятные сообщения и позволяет автоматически собирать CHANGELOG и вычислять номер версии.

## 9.7. Международные стандарты и инженерные практики

### Сводная таблица

| Документ | Вид | Для чего | Новичку |
|---|---|---|---|
| ISO/IEC/IEEE 12207 | Международный стандарт | Процессы жизненного цикла ПО | Знать о существовании |
| ISO/IEC 25010 | Международный стандарт | Модель качества продукта | Полезно как список вопросов о качестве |
| ISO/IEC/IEEE 29148 | Международный стандарт | Инженерия требований | Полезно: как писать требования |
| SPDX | Спецификация (2.2.1 — стандарт ISO/IEC 5962) | Идентификаторы лицензий, сведения о составе ПО | Да: идентификаторы лицензий |
| REUSE | Практика FSFE | Лицензия и авторство в каждом файле | По желанию, для open-source |
| Semantic Versioning | Спецификация сообщества | Номера версий | Да |

| Документ | Вид | Для чего | Новичку |
|---|---|---|---|
| Conventional Commits | Спецификация сообщества | Сообщения коммитов | По желанию |
| Keep a Changelog | Практика сообщества | Журнал изменений | Да |
| GitHub Flow | Рекомендация платформы | Работа через ветки и PR | Да |
| OpenSSF Scorecard | Инструмент сообщества | Автоматическая оценка практик безопасности | По желанию, для open-source |
| SLSA | Спецификация сообщества | Целостность и происхождение сборок | Знать о существовании |
| OWASP SAMM | Модель зрелости | Оценка практик безопасности в организации | Знать о существовании |
| OWASP ASVS | Стандарт сообщества | Проверяемые требования к безопасности приложений | Полезно для веб-приложений |

> **ВАЖНО**
>
> Использование отдельных идей из этих документов не означает соответствия им и не даёт права заявлять о сертификации. ISO не выдаёт сертификаты сама; OWASP не сертифицирует продукты, специалистов и проверяющих.

## ISO/IEC/IEEE 12207 — процессы жизненного цикла ПО

| | |
|---|---|
| **Что это** | Международный стандарт «Системная и программная инженерия — Процессы жизненного цикла программного обеспечения». Текущая редакция — **ISO/IEC/IEEE 12207:2026** (выпущена в апреле 2026 года и заменила редакцию 2017 года) |
| **Задача** | Общий словарь и набор процессов для приобретения, поставки, разработки, эксплуатации, сопровождения и вывода ПО из эксплуатации. Подходит и для гибких, и для формальных подходов; не навязывает конкретную модель жизненного цикла |
| **Нужно ли новичку** | Нет. Достаточно понимать, что у ПО есть жизненный цикл: требования, проектирование, разработка, проверка, выпуск, сопровождение, вывод из эксплуатации |
| **Как внедрить в небольшом проекте** | Использовать как список вопросов: где описаны требования? как принимаются решения? как управляем версиями и конфигурацией? как сопровождаем после выпуска? В редакции 2017 года процессы делились на группы: соглашения, организационное обеспечение проекта, техническое управление и технические процессы; детали актуальной редакции уточняйте по тексту стандарта |
| **Помощники** | GitHub Issues и Projects (планирование), Pull Request (контроль изменений), теги и релизы (управление конфигурацией), `docs/adr/` (решения) |
| **Официальный источник** | iso.org/standard/90219.html |

Это стандарт процессов, а не система менеджмента: организация может заявлять о соответствии, но схемы сертификации, подобной ISO 9001, для него нет.

## ISO/IEC 25010 — модель качества продукта

| | |
|---|---|
| **Что это** | Стандарт серии SQuaRE «Модель качества продукта». Текущая редакция — **ISO/IEC 25010:2023** |
| **Задача** | Определяет, из чего складывается качество ПО: девять характеристик, каждая делится на подхарактеристики. Помогает формулировать требования к качеству и проверять их |
| **Нужно ли новичку** | Полезно как чек-лист «о чём ещё подумать», кроме того, что программа просто работает |
| **Как внедрить в небольшом проекте** | Для каждой характеристики задать себе вопрос и записать ответ в README или `docs/`: например, «сопровождаемость — есть ли тесты и понятная структура?», «защищённость — где хранятся секреты?» |
| **Помощники** | Тесты и CI (надёжность, функциональная пригодность), линтеры (сопровождаемость), CodeQL и Dependabot (защищённость) |
| **Официальный источник** | iso.org/standard/78176.html |

Девять характеристик редакции 2023 года:

| Характеристика | Вопрос для небольшого проекта |
|---|---|
| Функциональная пригодность (*functional suitability*) | Делает ли программа то, что нужно, и правильно ли? |
| Эффективность производительности (*performance efficiency*) | Достаточно ли быстро работает и экономно ли расходует ресурсы? |
| Совместимость (*compatibility*) | Уживается ли с другими программами и форматами? |
| Способность к взаимодействию (*interaction capability*, ранее *usability*) | Удобно ли пользоваться, понятны ли сообщения? |
| Надёжность (*reliability*) | Работает ли без сбоев, переживает ли ошибки ввода? |
| Защищённость (*security*) | Защищены ли данные и доступ? |
| Сопровождаемость (*maintainability*) | Легко ли понять, изменить и проверить код? |
| Гибкость (*flexibility*, ранее *portability*) | Можно ли запустить в другой среде и адаптировать? |
| Безопасность (*safety*) | Не создаёт ли программа угрозы людям и имуществу? |

Качество при использовании (*quality in use*) в редакции 2023 года вынесено в отдельный стандарт ISO/IEC 25019:2023.

## ISO/IEC/IEEE 29148 — инженерия требований

| | |
|---|---|
| **Что это** | Стандарт «Процессы жизненного цикла — Инженерия требований». Действующая редакция — **ISO/IEC/IEEE 29148:2018**; новая редакция находится в разработке |
| **Задача** | Процессы работы с требованиями и состав документов с требованиями; характеристики хорошо сформулированного требования |
| **Нужно ли новичку** | Полезно в упрощённом виде: умение чётко записать, что должна делать программа, экономит время и вам, и AI-ассистенту |
| **Как внедрить в небольшом проекте** | Записывать требования в Issues или `docs/requirements.md`. Проверять каждое требование по характеристикам ниже. Связывать требования с тестами |

| | |
|---|---|
| **Помощники** | Шаблоны Issue, критерии готовности в Pull Request, тесты как проверка требований |
| **Официальный источник** | iso.org/standard/72089.html |

Характеристики отдельного требования, которые обычно приводят по стандарту: **необходимое, уместное, однозначное, полное, единичное** (одно требование — одна мысль), **выполнимое, проверяемое, корректное, соответствующее** принятым правилам. Текст стандарта платный; перечень дан по общедоступным описаниям.

Пример. Плохо: «Программа должна быстро конвертировать температуру». Хорошо: «Команда `tempconv 100 C F` выводит `100 °C = 212.00 °F` и завершается с кодом 0». Второе требование однозначно и проверяемо тестом.

## SPDX — идентификаторы лицензий и состав ПО

| | |
|---|---|
| **Что это** | Открытый стандарт обмена сведениями о составе ПО, лицензиях и авторских правах. Версия 2.2.1 утверждена как международный стандарт **ISO/IEC 5962:2021**; актуальная версия спецификации — **SPDX 3.0.1**. Список лицензий SPDX даёт каждой лицензии короткий идентификатор: `MIT`, `Apache-2.0`, `GPL-3.0-or-later` |
| **Задача** | Однозначно указывать лицензии в файлах и метаданных; описывать состав ПО (SBOM — *Software Bill of Materials*, «спецификация компонентов») |
| **Нужно ли новичку** | Да, в части идентификаторов: они нужны в `pyproject.toml`, `package.json` и заголовках файлов |
| **Как внедрить в небольшом проекте** | Указать идентификатор лицензии в манифесте пакета; при желании добавить в начало файлов строку `SPDX-License-Identifier: MIT` |
| **Помощники** | GitHub распознаёт лицензию и умеет экспортировать SBOM (вкладка **Insights** → **Dependency graph** → **Export SBOM**) |
| **Официальный источник** | spdx.dev, список лицензий, ISO/IEC 5962:2021 |

Подробнее о выражениях SPDX — в главе 10.

## REUSE — лицензия в каждом файле

| | |
|---|---|
| **Что это** | Практика Free Software Foundation Europe. Спецификация **REUSE 3.3** |
| **Задача** | Чтобы для **каждого** файла было однозначно известно, кто правообладатель и какая лицензия |
| **Нужно ли новичку** | Нет. Полезно для open-source-проектов с внешними участниками и заимствованными файлами |
| **Как внедрить в небольшом проекте** | Три шага: положить тексты лицензий в папку `LICENSES/`; добавить в файлы заголовки `SPDX-FileCopyrightText` и `SPDX-License-Identifier` (или описать их в `REUSE.toml`); проверить командой `reuse lint` |
| **Помощники** | Инструмент `reuse` (команды `reuse download`, `reuse annotate`, `reuse lint`), его можно запускать в CI |

| | |
|---|---|
| **Официальный источник** | reuse.software |

## Semantic Versioning

| | |
|---|---|
| **Что это** | Спецификация номеров версий **SemVer 2.0.0** |
| **Задача** | Номер версии сообщает о совместимости изменений (раздел 9.5) |
| **Нужно ли новичку** | Да, как только у проекта появляются версии |
| **Как внедрить в небольшом проекте** | Начать с `0.1.0`; определить публичный интерфейс; выпустить `1.0.0`, когда им начнут пользоваться; повышать номер по правилам; ставить теги `vX.Y.Z` |
| **Помощники** | Теги и релизы GitHub, CHANGELOG, Conventional Commits |
| **Официальный источник** | semver.org |

## Conventional Commits

| | |
|---|---|
| **Что это** | Спецификация сообщений коммитов **1.0.0** (раздел 9.6) |
| **Задача** | Единый формат сообщений; автоматический CHANGELOG и вычисление версии |
| **Нужно ли новичку** | По желанию |
| **Как внедрить в небольшом проекте** | Описать формат в `CONTRIBUTING.md` и следовать ему; при необходимости проверять в CI |
| **Помощники** | commitlint, release-please, semantic-release |
| **Официальный источник** | conventionalcommits.org |

## Keep a Changelog

| | |
|---|---|
| **Что это** | Рекомендации по журналу изменений **1.1.0** (раздел 9.4) |
| **Задача** | Понятный пользователю список изменений по версиям |
| **Нужно ли новичку** | Да, когда у проекта есть пользователи |
| **Как внедрить в небольшом проекте** | Создать `CHANGELOG.md` с разделом `[Unreleased]`, добавлять запись в каждом Pull Request, переносить в раздел версии при релизе |
| **Помощники** | Пункт в шаблоне Pull Request; release notes на GitHub |
| **Официальный источник** | keepachangelog.com |

## GitHub Flow

| | |
|---|---|
| **Что это** | Лёгкий рабочий процесс на основе веток, описанный в документации GitHub |
| **Задача** | Порядок внесения изменений: ветка → коммиты → Pull Request → ревью → слияние → удаление ветки |
| **Нужно ли новичку** | Да (глава 5) |

| | |
|---|---|
| **Как внедрить в небольшом проекте** | Не коммитить напрямую в `main`; каждая задача — ветка и PR; защитить `main` правилами |
| **Помощники** | Защита веток и rulesets, шаблон PR, CI |
| **Официальный источник** | GitHub flow |

## OpenSSF Scorecard

| | |
|---|---|
| **Что это** | Инструмент Open Source Security Foundation, который автоматически проверяет практики безопасности open-source-проекта и ставит оценку от 0 до 10 по каждой проверке |
| **Задача** | Быстро увидеть слабые места: нет защиты веток, не закреплены версии действий, нет политики безопасности, не обновляются зависимости |
| **Нужно ли новичку** | По желанию; полезно как список идей для улучшения |
| **Как внедрить в небольшом проекте** | Посмотреть оценку публичного репозитория на scorecard.dev или подключить GitHub Action `ossf/scorecard-action`; исправлять пункты по одному |
| **Помощники** | Проверки: Binary-Artifacts, Branch-Protection, CI-Tests, CII-Best-Practices, Code-Review, Contributors, Dangerous-Workflow, Dependency-Update-Tool, Fuzzing, License, Maintained, Packaging, Pinned-Dependencies, SAST, Security-Policy, Signed-Releases, Token-Permissions, Vulnerabilities (Webhooks и SBOM — экспериментальные) |
| **Официальный источник** | scorecard.dev, github.com/ossf/scorecard |

Проверки Scorecard — эвристики: бывают ложные срабатывания и пропуски. Высокая итоговая оценка не доказывает, что проект безопасен.

## SLSA — целостность сборок

| | |
|---|---|
| **Что это** | Спецификация OpenSSF «Supply-chain Levels for Software Artifacts». Текущая версия — **SLSA v1.2** |
| **Задача** | Защитить цепочку «исходный код → сборка → готовый пакет» от подмены. Главная идея — **происхождение** (*provenance*): проверяемые сведения о том, кто, из чего и как собрал файл |
| **Нужно ли новичку** | Нет; важно для проектов, которые выпускают пакеты и бинарные файлы для других |
| **Как внедрить в небольшом проекте** | Собирать релизы только в CI, а не на своём компьютере; добавлять к ним аттестации происхождения. Уровни трека сборки: L0 — нет гарантий, L1 — происхождение есть, L2 — подписанное происхождение с размещённой платформы сборки, L3 — защищённая платформа сборки. В v1.2 добавлен трек исходного кода (*Source track*) |
| **Помощники** | GitHub Artifact Attestations (действие `actions/attest`) дают уровень Build L2; проверка — `gh attestation verify`. Для публичных репозиториев доступно во всех планах |
| **Официальный источник** | slsa.dev |

## OWASP SAMM — зрелость практик безопасности

| | |
|---|---|
| **Что это** | Модель зрелости OWASP Software Assurance Maturity Model (версия 2) |

| | |
|---|---|
| **Задача** | Оценить и спланировать развитие практик безопасной разработки в команде или организации. 5 бизнес-функций (управление, проектирование, реализация, проверка, эксплуатация) × 3 практики = 15 практик, у каждой три уровня зрелости |
| **Нужно ли новичку** | Нет. Это инструмент для команд и организаций |
| **Как внедрить в небольшом проекте** | Пройти самооценку по таблице SAMM Toolbox, выбрать 2–3 практики для улучшения (например, управление дефектами и безопасная сборка) |
| **Помощники** | SAMM Toolbox (таблица самооценки) |
| **Официальный источник** | owaspsamm.org |

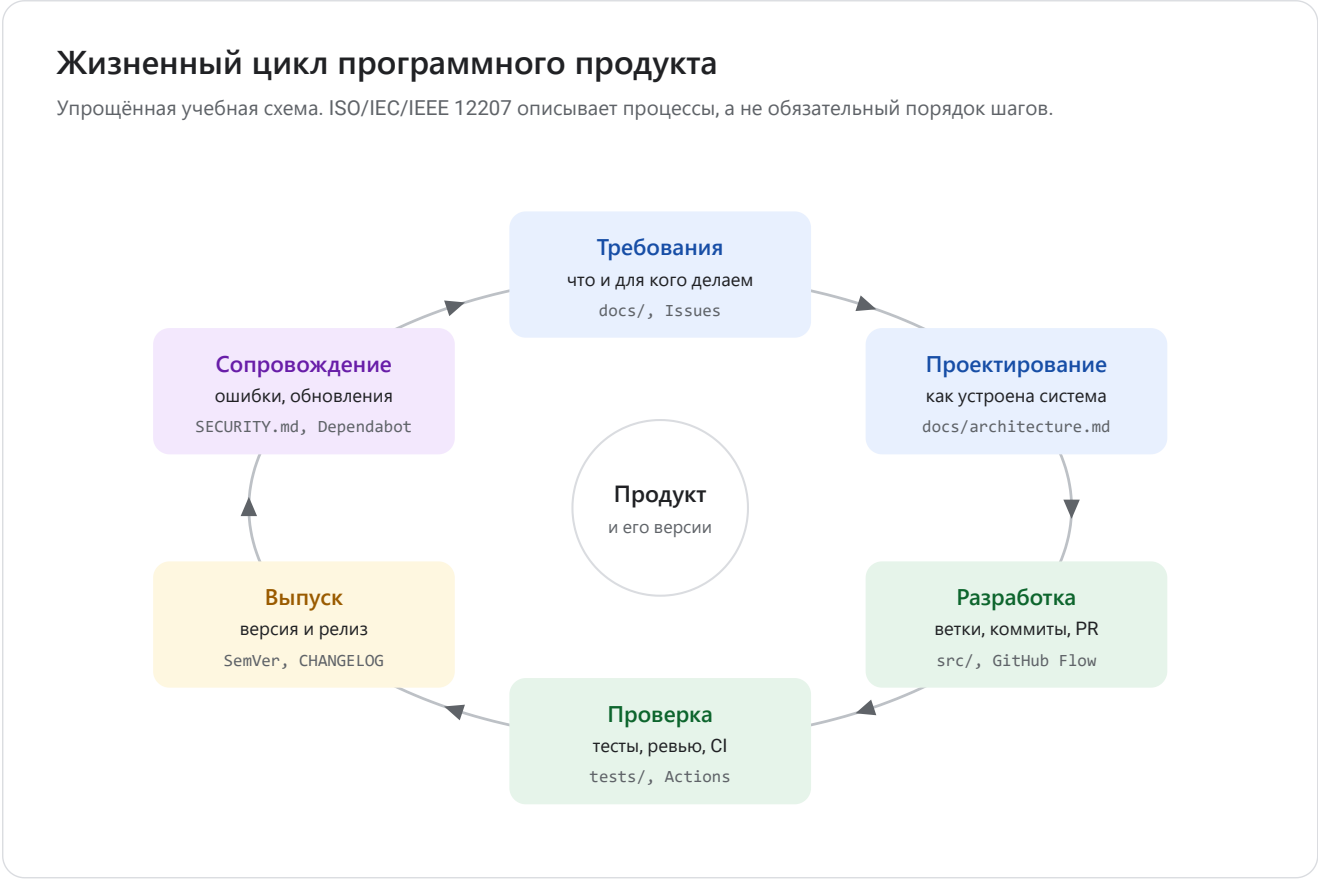
## OWASP ASVS — требования к безопасности приложений

| | |
|---|---|
| **Что это** | OWASP Application Security Verification Standard — открытый стандарт проверяемых требований к безопасности приложений. Текущая версия — **ASVS 5.0.0** (май 2025), 17 разделов |
| **Задача** | Список конкретных требований для разработки и проверки: аутентификация, сессии, контроль доступа, криптография, обработка ошибок и др. |
| **Нужно ли новичку** | Полезно для веб-приложений и API; для консольной утилиты без сети — в основном нет |
| **Как внедрить в небольшом проекте** | Выбрать уровень: L1 — минимальный набор требований, L2 — рекомендуемый для большинства приложений, L3 — для наиболее критичных. Отобрать применимые разделы и использовать как чек-лист; ссылаться на требования по идентификатору вида `v5.0.0-1.2.5` |
| **Помощники** | Списки требований в форматах CSV и JSON; OWASP Top 10 (2025) — обзор самых частых рисков |
| **Официальный источник** | owasp.org/www-project-application-security-verification-standard |

## Связанные документы

| Документ | Что это |
|---|---|
| ISO/IEC 5230:2020 (OpenChain) | Международный стандарт требований к программе соблюдения лицензий open source в организации |
| ISO/IEC 18974:2023 | Международный стандарт требований к обеспечению безопасности open-source-компонентов в организации |
| OpenSSF Best Practices Badge | Самооценка open-source-проекта с бейджем: уровни passing, silver, gold и уровни Baseline |
| OpenSSF OSPS Baseline | Минимальные меры безопасности для open-source-проектов, три уровня |
| OWASP Top 10:2025 | Обзор самых распространённых рисков безопасности веб-приложений |
| NIST SSDF (SP 800-218) | Рекомендации правительства США по безопасной разработке (версия 1.1; версия 1.2 — в проекте) |

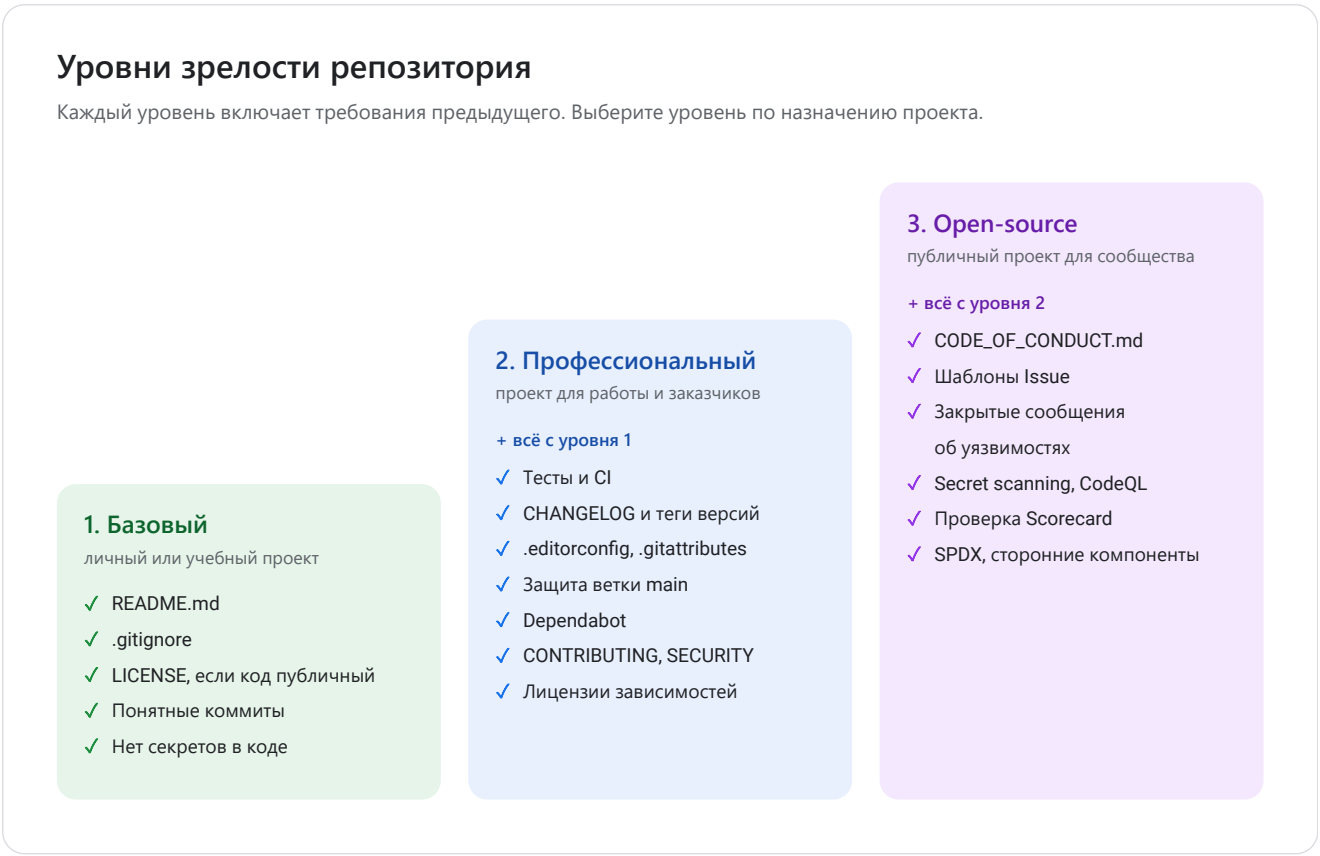
# 9.8. Жизненный цикл программного продукта




| Этап | Что происходит | Что в репозитории |
|---|---|---|
| Требования | Понять, что и для кого делаем | Issues, `docs/requirements.md` |
| Проектирование | Решить, как устроена система | `docs/architecture.md`, `docs/adr/` |
| Разработка | Писать код небольшими проверяемыми шагами | Ветки, коммиты, Pull Request |
| Проверка | Тесты, ревью, автоматические проверки | `tests/`, `.github/workflows/` |
| Выпуск | Версия, заметки, артефакты | Теги, Releases, `CHANGELOG.md` |
| Сопровождение | Исправления, обновления зависимостей, ответы пользователям | Issues, Dependabot, `SECURITY.md` |
| Вывод из эксплуатации | Проект больше не развивается | Пометка в README, архивирование репозитория (**Settings → Archive this repository**) |

Это упрощённая учебная схема. ISO/IEC/IEEE 12207 описывает процессы, которые могут выполняться параллельно и повторяться, а не обязательную последовательность шагов.

## 9.9. Уровни зрелости проекта




- **Базовый** — личный или учебный проект: README, `.gitignore`, лицензия для публичного кода, понятные коммиты, никаких секретов.
- **Профессиональный** — проект для работы и заказчиков: тесты и CI, CHANGELOG и версии, единое форматирование, защита `main`, обновление зависимостей, правила участия, политика безопасности (`SECURITY.md`), проверка лицензий зависимостей.
- **Публичный open-source** — проект для сообщества: кодекс поведения, шаблоны Issue, закрытые сообщения об уязвимостях, secret scanning и CodeQL, проверка Scorecard, учёт сторонних компонентов (SPDX, `THIRD_PARTY_NOTICES.md`).

Полный чек-лист по трём уровням — templates/PROJECT_RELEASE_CHECKLIST.md.

## Проверьте себя

1. Чем международный стандарт отличается от практики сообщества? Приведите по примеру.
2. Ваш проект использует две отметки из ISO/IEC 25010. Можно ли написать в README «соответствует ISO/IEC 25010»?
3. Вы исправили ошибку в версии `2.3.1`. Какой будет новая версия? А если добавили новую команду?
4. В какой раздел CHANGELOG записать «удалён параметр `--round`»? Какую часть версии увеличить?
5. Какие файлы нужны личному учебному проекту, а какие — публичной библиотеке?

Ответы

1. Международный стандарт принимают организации по стандартизации (ISO/IEC/IEEE 12207), практику сообщества — открытое сообщество (Semantic Versioning). Оба вида добровольны, пока их не требует закон или договор.
2. Нет. Соответствие требует выполнить все обязательные требования стандарта. Корректно: «при оценке качества мы опираемся на характеристики ISO/IEC 25010».
3. `2.3.2`. При новой совместимой возможности — `2.4.0`.
4. В `Removed`. Это несовместимое изменение — увеличить MAJOR.
5. Учебному проекту: README, `.gitignore`, лицензия, если код публичный. Публичной библиотеке дополнительно: CHANGELOG, тесты и CI, CONTRIBUTING, CODE_OF_CONDUCT, SECURITY, шаблоны Issue и PR, учёт лицензий зависимостей.

## Что дальше

В главе 10 — лицензии, авторские права и учёт стороннего кода.

# Глава 10. Лицензии и авторские права

Код, опубликованный на GitHub, не становится «общим». Кто и как может его использовать, определяют авторское право и лицензия. В этой главе — что такое лицензия, чем отличаются популярные лицензии, как выбрать и правильно оформить лицензию и как учитывать чужой код, шрифты и изображения.

**Вы научитесь:**

- понимать, кто правообладатель и что разрешает лицензия;
- сравнивать MIT, Apache-2.0, BSD, GPL, LGPL, AGPL, MPL и Unlicense;
- понимать, когда возникают обязательства по лицензии;
- выбирать лицензию по целям проекта;
- оформлять `LICENSE`, `NOTICE`, `THIRD_PARTY_NOTICES.md`, `CITATION.cff` и сведения SPDX.

**Время:** 2 часа.

> ВАЖНО
>
> **Эта глава — учебный материал, а не юридическая консультация.** Правила зависят от страны, договора с работодателем или заказчиком, лицензий сторонних компонентов и конкретного способа использования программы. Для коммерческих продуктов и спорных случаев обращайтесь к юристу, который специализируется на интеллектуальной собственности. Сведения проверены по официальным источникам на 11 октября 2026 года.

## 10.1. Основы: авторское право и лицензия

### Авторское право возникает автоматически

> **Авторское право** — права автора на созданное им произведение. Программа для компьютера охраняется как литературное произведение.

Права возникают **в момент создания** кода — регистрировать ничего не нужно. Так устроено в большинстве стран. Например, в Гражданском кодексе РФ программы для ЭВМ охраняются как литературные произведения (ст. 1259, ст. 1261), а для возникновения прав не требуется регистрация или иные формальности (ст. 1259, п. 4).

Пример. Вы написали скрипт и выложили его в публичный репозиторий без лицензии. Права на скрипт принадлежат вам. Другие люди могут его прочитать, но по закону не могут свободно копировать, изменять и распространять.

### Кто правообладатель

> **Правообладатель** — тот, кому принадлежат исключительные права на произведение: право разрешать или запрещать его использование.

Правообладателем может быть не автор:

- **Код, написанный по работе.** Во многих странах права на служебное произведение принадлежат работодателю, если договор не говорит иное. Например, в РФ — ст. 1295 ГК РФ; в США — правило *work made for hire*.
- **Код по договору с заказчиком.** Права могут перейти заказчику по договору.
- **Код соавторов.** Если в проекте есть чужие правки, правообладателей несколько.

> **ВНИМАНИЕ**
>
> Прежде чем публиковать код, написанный на работе или по заказу, проверьте трудовой договор и договор с заказчиком. Опубликовать чужой (работодателя) код под открытой лицензией без разрешения — нарушение.

Кроме исключительных прав есть **личные неимущественные права**: право авторства и право на имя. Во многих странах от них нельзя отказаться. Например, в РФ отказ от этих прав ничтожен (ст. 1228, п. 2 и ст. 1265 ГК РФ).

### Что такое лицензия

> **Лицензия на ПО** — разрешение правообладателя использовать программу на определённых условиях: что можно делать, что нужно соблюдать и чего лицензия не гарантирует.

| | Авторское право | Лицензия |
|---|---|---|
| Что это | Права, которые принадлежат правообладателю | Разрешение, которое правообладатель даёт другим |
| Как возникает | Автоматически, при создании | Правообладатель выбирает и публикует условия |
| Кто решает | Закон | Правообладатель в рамках закона |
| Пример | «Код мой» | «Можно использовать, если сохраните моё имя» |

Лицензия не передаёт права на код — правообладатель остаётся прежним. Она лишь разрешает другим то, что без неё было бы запрещено.

### Почему публичный репозиторий не означает разрешения

На GitHub, сделав репозиторий публичным, вы по Условиям использования GitHub разрешаете другим пользователям **просматривать и делать форк** репозитория в рамках сервиса GitHub (раздел D.5 «License Grant to Other Users»). Больше ничего: ни копировать код в свои проекты, ни распространять, ни изменять вне GitHub без лицензии нельзя.

Документация GitHub: без лицензии действуют обычные законы об авторском праве — никто не может воспроизводить, распространять и создавать производные работы. Источники: Licensing a repository, GitHub Terms of Service, No License — choosealicense.com.

### Что лицензия даёт и что требует

**Права пользователя** (зависят от лицензии): использовать, в том числе коммерчески; изменять; распространять копии и изменённые версии; использовать в закрытых продуктах; иногда — использовать патенты участников.

**Обязанности при использовании чужого кода** (зависят от лицензии):

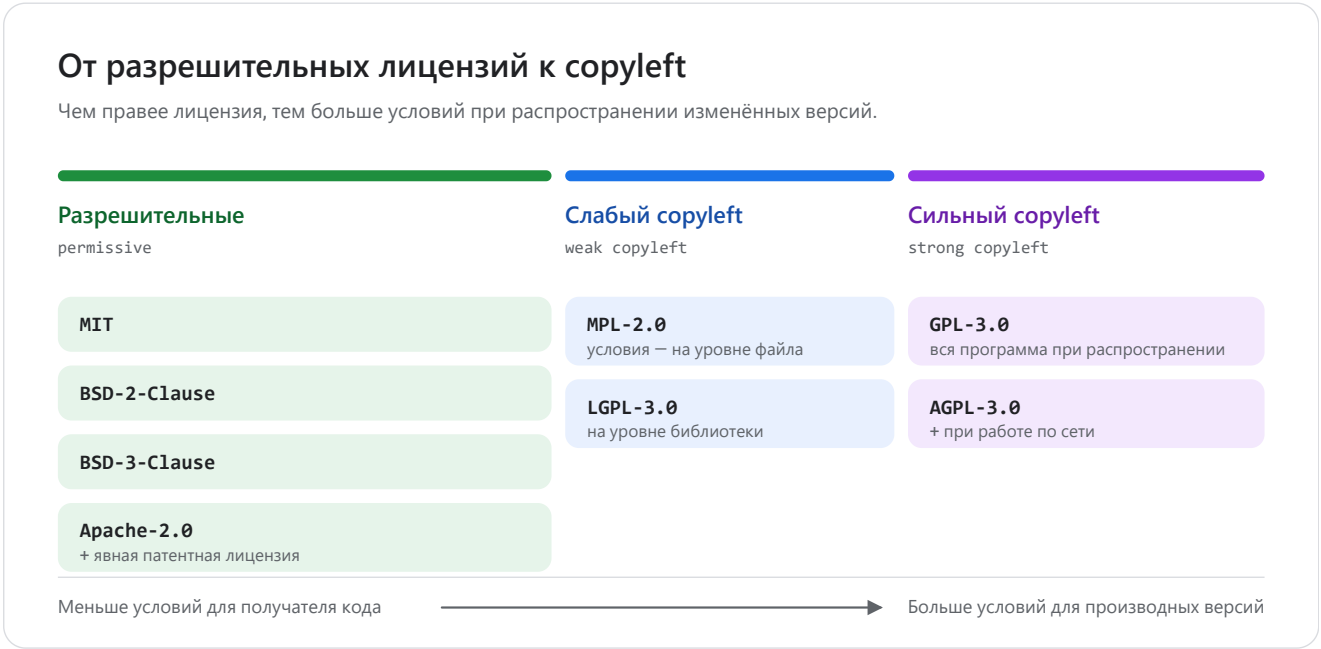
- сохранять уведомление об авторских правах и текст лицензии;
- отмечать внесённые изменения;
- предоставлять исходный код производной работы на тех же условиях (copyleft);
- не использовать имена авторов и товарные знаки для рекламы;
- соблюдать условия при работе программы по сети (AGPL).

Почти все открытые лицензии **отказываются от гарантий и ответственности**: программа предоставляется «как есть».

### Почему нельзя просто скопировать лицензию из чужого проекта

- В чужом `LICENSE` — **чужое имя правообладателя**. Скопировав его, вы заявите, что код принадлежит другому человеку.
- Вы можете взять лицензию с условиями, которых не понимаете, например **copyleft**, и неожиданно обязать себя и пользователей открывать код.
- Лицензия проекта должна быть **совместима с лицензиями зависимостей** — иначе распространять проект на выбранных условиях нельзя.
- Некоторые лицензии требуют особого оформления: заголовков в файлах, файла `NOTICE`, указания версии («только 3.0» или «3.0 и новее»).

## 10.2. Виды лицензий



| Вид | Суть | Примеры |
|---|---|---|
| **Разрешительные** (*permissive*) | Можно почти всё, включая закрытые коммерческие продукты; нужно сохранить уведомление об авторстве | MIT, BSD-2-Clause, BSD-3-Clause, Apache-2.0 |
| **Слабый copyleft** | Изменения самого компонента (файла или библиотеки) остаются открытыми; остальной продукт может быть закрытым | MPL-2.0 (на уровне файла), LGPL-3.0 (на уровне библиотеки) |
| **Сильный copyleft** | Распространяемая производная программа целиком должна быть под той же лицензией с исходным кодом | GPL-3.0 |
| **Сетевой copyleft** | Как сильный copyleft, плюс обязанность предложить исходный код изменённой версии пользователям, работающим с ней по сети | AGPL-3.0 |
| **Отказ от прав в пользу общественного достояния** | Автор пытается отказаться от всех прав; на случай, если это невозможно, даётся максимально широкая лицензия | Unlicense, CC0-1.0 |
| **Без лицензии** | Действует авторское право по умолчанию: всё запрещено, кроме прямо разрешённого законом | — |

> **Copyleft** («копилефт») — условие лицензии: если вы распространяете изменённую версию, вы должны распространять её на тех же условиях, вместе с исходным кодом.

## 10.3. Сравнение лицензий

Таблицы — упрощение. Точные условия — только в текстах лицензий (ссылки в разделе 10.6 и в sources.md). Идентификаторы — из списка лицензий SPDX.

### Права и базовые условия

| Лицензия (SPDX) | 1. Коммерческое использование | 2. Изменение кода | 3. Распространение изменённой версии | 4. Сохранять уведомление и лицензию |
|---|---|---|---|---|
| **MIT** (`MIT`) | Да | Да | Да; свои изменения можно распространять на любых условиях | Да |
| **Apache License 2.0** (`Apache-2.0`) | Да | Да | Да; на свои изменения можно дать другие условия, соблюдая Apache-2.0 для исходного кода; изменённые файлы нужно пометить | Да, плюс атрибуции из файла `NOTICE`, если он есть |
| **BSD 2-Clause** (`BSD-2-Clause`) | Да | Да | Да; свои изменения можно распространять на любых условиях | Да |

| Лицензия (SPDX) | 1. Коммерческое использование | 2. Изменение кода | 3. Распространение изменённой версии | 4. Сохранять уведомление и лицензию |
|---|---|---|---|---|
| **BSD 3-Clause** (`BSD-3-Clause`) | Да | Да | Да; свои изменения можно распространять на любых условиях | Да; нельзя использовать имена авторов для продвижения без разрешения |
| **GNU GPL v3** (`GPL-3.0-only`, `GPL-3.0-or-later`) | Да | Да | Да, только под GPL-3.0 и с исходным кодом; изменения нужно отметить | Да |
| **GNU LGPL v3** (`LGPL-3.0-only`, `LGPL-3.0-or-later`) | Да | Да | Изменённая библиотека — под LGPL с исходным кодом; программа, которая её использует, — на ваших условиях при соблюдении LGPL | Да |
| **GNU AGPL v3** (`AGPL-3.0-only`, `AGPL-3.0-or-later`) | Да | Да | Да, только под AGPL-3.0 и с исходным кодом | Да |
| **Mozilla Public License 2.0** (`MPL-2.0`) | Да | Да | Изменённые файлы MPL — под MPL с исходным кодом; остальной продукт — на ваших условиях | Да |
| **The Unlicense** (`Unlicense`) | Да | Да | Да, без условий | Нет |
| **Нет лицензии** | Нет, без разрешения правообладателя | Нет | Нет | — |

## Исходный код, сеть, патенты и применение

| Лицензия | 5. Исходный код при распространении производной работы | 6. Особые требования при работе по сети | 7. Патенты | 8. Где может быть уместна |
|---|---|---|---|---|
| **MIT** | Не требуется | Нет | Явной патентной лицензии нет | Небольшие библиотеки и утилиты, учебные проекты, максимально широкое использование |
| **Apache-2.0** | Не требуется | Нет | Явная патентная лицензия от участников; прекращается для того, кто подаёт патентный иск о нарушении | Крупные проекты, корпоративная среда, когда важны патенты |
| **BSD-2-Clause** | Не требуется | Нет | Явной патентной лицензии нет | Как MIT |

| Лицензия | 5. Исходный код при распространении производной работы | 6. Особые требования при работе по сети | 7. Патенты | 8. Где может быть уместна |
|---|---|---|---|---|
| **BSD-3-Clause** | Не требуется | Нет | Явной патентной лицензии нет | Как MIT, плюс защита имени автора от использования в рекламе |
| **GPL-3.0** | Да: полный исходный код всей производной программы под GPL | Нет: работа программы на сервере без передачи копий не считается распространением | Явная патентная лицензия от участников; запрет патентных исков о нарушении программой | Приложения, все распространяемые версии которых должны оставаться свободными |
| **LGPL-3.0** | Исходный код библиотеки и её изменений; для программы с библиотекой — возможность заменить или пересобрать библиотеку | Нет | Как GPL-3.0 | Библиотеки, которые можно использовать в закрытых программах, но улучшения самой библиотеки должны быть открыты |
| **AGPL-3.0** | Да, как GPL-3.0 | **Да:** если вы изменили программу, пользователям, работающим с ней по сети, нужно предложить исходный код изменённой версии | Как GPL-3.0 | Серверные и веб-приложения, где важно, чтобы изменения сервиса были открыты |
| **MPL-2.0** | Исходный код файлов под MPL (включая изменённые) | Нет: работа только на сервере не считается распространением | Явная патентная лицензия от участников; прекращается при патентном иске | Компромисс: открытые файлы компонента внутри закрытого продукта |
| **Unlicense** | Не требуется | Нет | Патенты не упоминаются | Когда автор хочет передать код без условий; см. оговорку ниже |
| **Нет лицензии** | — | — | — | Закрытые и приватные проекты; это не стратегия защиты публичного кода |

## Оговорка о The Unlicense

The Unlicense пытается передать код в **общественное достояние** (*public domain*). Но не во всех странах автор может отказаться от авторских прав: например, в Германии авторское право не передаётся (§ 29 UrhG), а в РФ отказ от личных неимущественных прав ничтожен. Поэтому в тексте Unlicense есть и широкое разрешение на использование — GNU описывает его как запасную разрешительную

лицензию на случай, если отказ от прав не сработает. OSI одобрила Unlicense в 2020 году; FSF считает её свободной и совместимой с GPL. Если нужна предсказуемость в разных странах, многие выбирают MIT.

Похожая лицензия-отказ CC0-1.0 не одобрена OSI, и OSI не рекомендует её для ПО: она прямо не предоставляет патентных прав.

## 10.4. Когда возникают обязательства

Обязательства лицензии почти всегда зависят от того, **что вы делаете с программой**.

| Что вы делаете | MIT, BSD, Apache-2.0 | MPL-2.0 | LGPL-3.0 | GPL-3.0 | AGPL-3.0 |
|---|---|---|---|---|---|
| Используете и меняете только у себя | Обязательств нет | Обязательств нет | Обязательств нет | Обязательств нет | Обязательств нет |
| Запускаете на своём сервере, пользователи работают по сети | Обязательств нет | Обязательств нет | Обязательств нет | Обязательств нет | Если программа изменена — предложить исходный код пользователям сети |
| Передаёте копии другим (продаёте, публикуете, отдаёте заказчику) | Сохранить уведомления (Apache — ещё `NOTICE` и пометки изменений) | Открыть исходный код файлов под MPL | Открыть код библиотеки; дать возможность заменить библиотеку | Вся производная программа — под GPL с исходным кодом | Как GPL-3.0 |

Пояснения:

- «**Распространение**» в GPL-3.0 называется *conveying* — любая передача копий другим лицам. Взаимодействие с пользователем по сети без передачи копии распространением не считается. Использование и изменение у себя — не распространение.
- **JavaScript, который сайт отправляет в браузер,** по позиции FSF (GPL FAQ) передаётся пользователю — то есть распространяется.
- **LGPL** допускает подключение библиотеки к закрытой программе, но пользователь должен иметь возможность заменить библиотеку своей изменённой версией: например, через динамическое подключение (*shared library*). При статическом связывании требований больше.
- **MPL-2.0** работает на уровне файлов: новые файлы, в которые не скопирован код под MPL, могут быть под другой лицензией.
- **Внутри одной организации** правила передачи копий между юрлицами и подрядчиками сложны — это вопрос для юриста.

### Совместимость лицензий

> **Совместимость лицензий** — возможность объединить код под разными лицензиями в одну программу и распространять её, соблюдая все лицензии сразу.

Факты из списка лицензий GNU:

- MIT (в терминах GNU — Expat), BSD-2-Clause и BSD-3-Clause совместимы с GPL.
- Apache-2.0 совместима с GPL-3.0, но **не совместима с GPL-2.0** (из-за условий о патентах).
- MPL-2.0 совместима с GPL, если автор не пометил код как «Incompatible With Secondary Licenses».
- GPL-2.0 (только 2.0, без «или новее») и GPL-3.0 между собой **не совместимы**.
- Unlicense совместима с GPL.

Правило на практике: **как правило, код под разрешительной лицензией можно включать в проект под copyleft-лицензией (с учётом исключений выше, например Apache-2.0 и GPL-2.0), но не наоборот** — нельзя взять код под GPL и распространять его в составе проекта под MIT.

Источник: Various Licenses and Comments about Them — GNU.

### Патенты

> **Патент** — исключительное право на изобретение, например на способ обработки данных. Авторское право защищает текст кода, а патент — идею способа.

Явная патентная лицензия (Apache-2.0, GPL-3.0, LGPL-3.0, AGPL-3.0, MPL-2.0) означает: участники проекта разрешают использовать свои патенты, которые нарушаются их вкладом. В лицензиях есть защита от патентных исков: в Apache-2.0 тот, кто подаёт патентный иск о нарушении, теряет полученную по лицензии патентную лицензию; в MPL-2.0 у него прекращаются права, предоставленные участниками; GPL-3.0 запрещает такие иски как условие лицензии. Точные условия — в тексте каждой лицензии. В MIT и BSD явной патентной лицензии нет и явного отказа тоже нет — вопрос о подразумеваемом разрешении решается по праву конкретной страны.

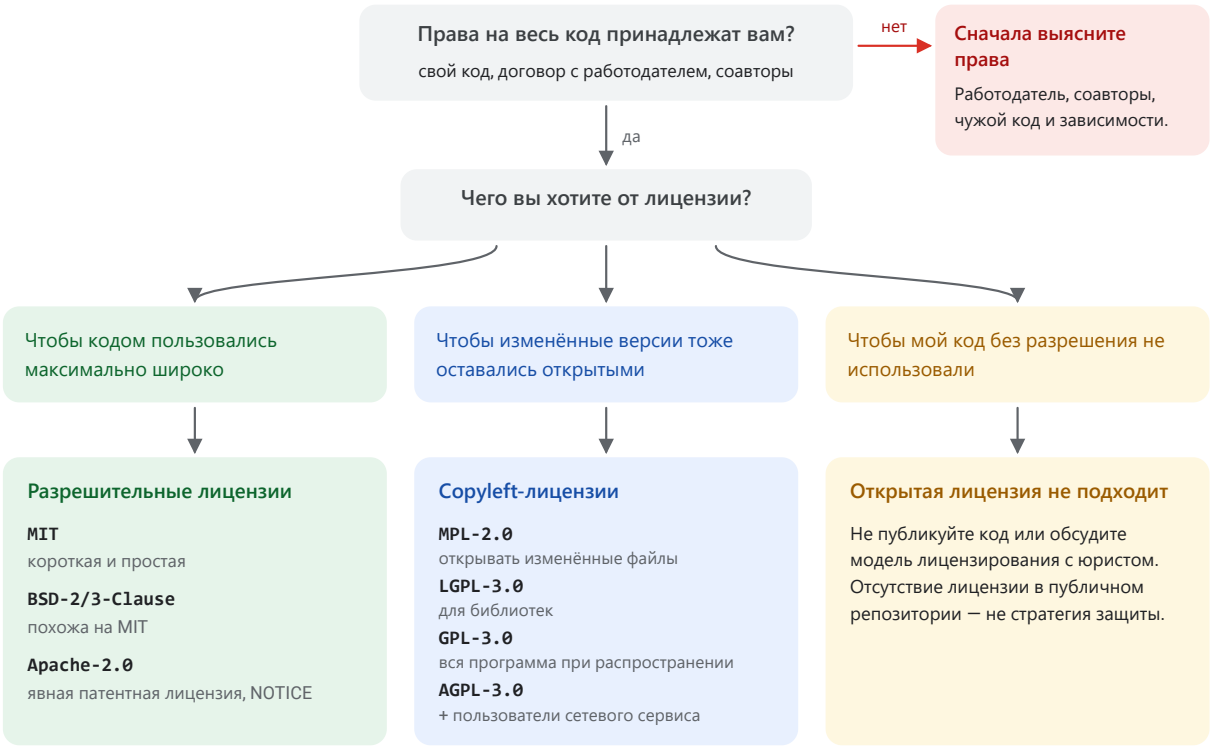
## 10.5. Как выбрать лицензию

Лицензию выбирает **правообладатель**, исходя из своих целей. Методичка не выбирает лицензию за вас. Ответьте на вопросы:

1. **Кому принадлежат права** на весь код? Нет ли кода работодателя, заказчика, соавторов, чужих фрагментов?
2. **Чего вы хотите?** Максимального распространения? Чтобы улучшения возвращались в открытый доступ? Зарабатывать на закрытом продукте?
3. **Какие лицензии у зависимостей?** Они могут ограничить выбор.
4. **Какие лицензии приняты в вашей экосистеме?** Например, в экосистемах JavaScript и Python популярны MIT и Apache-2.0, в Linux — GPL.
5. **Нужна ли явная защита от патентных претензий?**

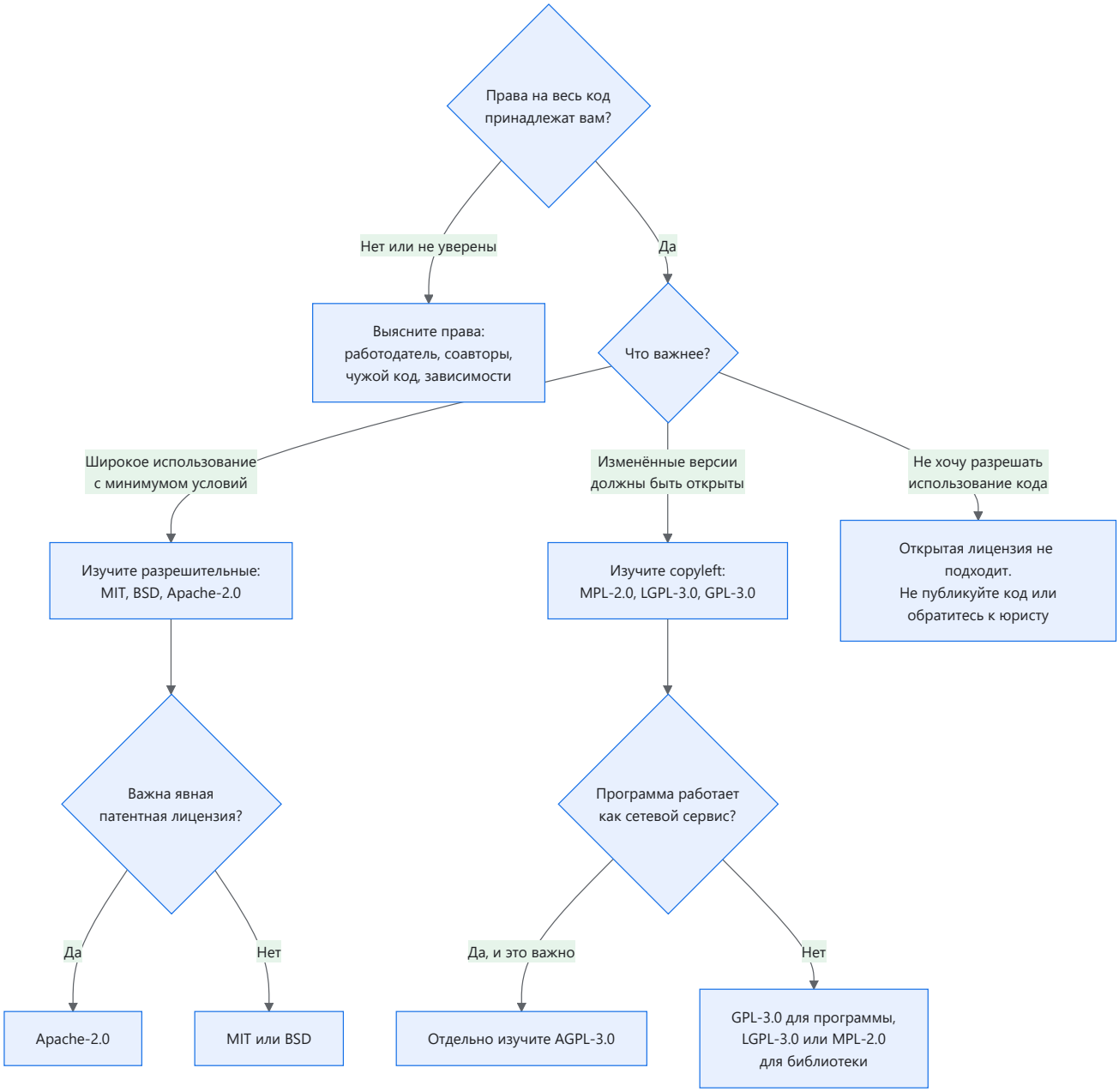
## Как выбрать лицензию

Схема помогает сузить выбор. Это учебный материал, а не юридическая консультация.




Перед выбором проверьте лицензии зависимостей: они могут ограничить ваш выбор.

Подробности и сравнение — в главе 10 и в templates/LICENSE_SELECTION_GUIDE.md.




## Четыре типичные цели

**Хочу разрешить широкое повторное использование с минимальными условиями.** Изучите разрешительные лицензии. MIT — самая короткая и популярная. Apache-2.0 длиннее, но содержит явную патентную лицензию и правила о `NOTICE`; её часто выбирают для крупных и корпоративных проектов.

**Хочу, чтобы производные версии распространялись на условиях copyleft.** Изучите GPL-3.0 для программ. Для библиотек, которые могут использовать закрытые программы, — LGPL-3.0 или MPL-2.0. Решите, указывать «только эта версия» (`-only`) или «эта версия и новее» (`-or-later`).

**Хочу учесть работу программы как сетевого сервиса.** Под GPL изменённую программу можно запустить на сервере и не открывать код. Если это для вас важно, отдельно изучите AGPL-3.0. Учтите: многие компании избегают AGPL-зависимостей.

**Не хочу разрешать произвольное использование исходного кода.** Открытые лицензии этого не дают. Отсутствие лицензии в публичном репозитории — **не готовая стратегия защиты**: код видят все, его можно скопировать вопреки закону, а доказывать нарушение придётся вам. Варианты: держать репозиторий приватным; разработать коммерческое лицензионное соглашение с юристом; рассмотреть модели, где часть продукта открыта, а часть нет. Это решение требует юридической консультации.

Удобный помощник для первого знакомства: choosealicense.com. Подробный опросник — в шаблоне templates/LICENSE_SELECTION_GUIDE.md.

## 10.6. Как добавить лицензию в проект

**Шаг 1. Выберите лицензию** (раздел 10.5).

**Шаг 2. Возьмите официальный текст.** Не переписывайте и не переводите лицензию: юридическую силу имеет официальный текст.

| Лицензия | Официальный текст |
|---|---|
| MIT | opensource.org/license/mit |
| Apache-2.0 | apache.org/licenses/LICENSE-2.0 |
| BSD-2-Clause, BSD-3-Clause | opensource.org/license/bsd-2-clause, opensource.org/license/bsd-3-clause |
| GPL-3.0, LGPL-3.0, AGPL-3.0 | gnu.org/licenses |
| MPL-2.0 | mozilla.org/MPL/2.0 |
| Unlicense | unlicense.org |
| Все лицензии с идентификаторами | spdx.org/licenses |

**Шаг 3. Создайте файл `LICENSE`** в корне репозитория.

Через GitHub: **Add file** → **Create new file** → имя `LICENSE` → **Choose a license template** → выберите лицензию → **Review and submit** → **Commit changes**. Источник: Adding a license to a repository.

Вручную: скопируйте официальный текст в файл `LICENSE` (без расширения или `LICENSE.md`, `LICENSE.txt`).

**Шаг 4. Укажите год и правообладателя**, если лицензия этого требует. Например, MIT:

```
Copyright (c) 2026 Иван Петров
```

- **Год** — год первой публикации; можно указать диапазон `2024–2026`.
- **Правообладатель** — вы, ваша компания или формулировка «авторы проекта» со ссылкой на файл `AUTHORS`.
- В Apache-2.0 текст лицензии не меняют, а строку `Copyright [yyyy] [name of copyright owner]` добавляют в заголовки файлов или в `NOTICE` (см. приложение к лицензии).
- GPL-семейство рекомендует уведомление в начале каждого файла исходного кода (образец — в конце текста лицензии, раздел «How to Apply These Terms to Your New Programs»).

**Шаг 5. Отразите лицензию в README:**

```markdown
## Лицензия

Распространяется по лицензии MIT — см. файл [LICENSE](LICENSE).
```

**Шаг 6. Укажите идентификатор SPDX в манифесте пакета**, если он есть: `license = "MIT"` в `pyproject.toml`, `"license": "MIT"` в `package.json`.

**Шаг 7. Учтите сторонние материалы** (раздел 10.7).

> **ВАЖНО**
>
> Файл `LICENSE` сам по себе **не подтверждает**, что вы обладаете правами на все файлы проекта. Лицензия действует только на то, чем вы вправе распоряжаться. Чужой код, шрифты, изображения и данные остаются под своими лицензиями.

## 10.7. Авторство, NOTICE и сторонние компоненты

### LICENSE и уведомления об авторских правах

- **`LICENSE`** — текст лицензии проекта.
- **Уведомление об авторских правах** (*copyright notice*) — строка вида `Copyright (c) 2026 Иван Петров`. Её размещают в `LICENSE`, а при желании или по требованию лицензии — в заголовках файлов. Чужие уведомления **нельзя удалять**: этого требуют почти все лицензии.

### NOTICE — когда он нужен

> **`NOTICE`** — файл с обязательными атрибуциями, предусмотренный лицензией Apache-2.0 (раздел 4(d)).

- Если вы **распространяете** производную работу от компонента под Apache-2.0, в котором есть файл `NOTICE`, вы обязаны сохранить его атрибуции: в своём `NOTICE`, в документации или в интерфейсе программы.
- Если ваш собственный проект под Apache-2.0, `NOTICE` не обязателен. Apache Software Foundation советует не добавлять в него ничего, что не требуется юридически.
- Содержимое `NOTICE` информационное и не меняет условий лицензии.

Пример (с заполнителями):

```
[НАЗВАНИЕ ПРОЕКТА]
Copyright [ГОД] [ИМЯ ПРАВООБЛАДАТЕЛЯ]

This product includes software developed by [ПРОЕКТ] ([ССЫЛКА]),
licensed under the Apache License, Version 2.0.
```

### AUTHORS и CONTRIBUTORS

Необязательные файлы со списком авторов и участников. Полезны, если в уведомлении указано «Copyright [год] авторы проекта» или если хочется поблагодарить участников. Историю авторства и так хранит Git. **Не добавляйте в эти файлы людей без их согласия и не придумывайте авторов.**

### CITATION.cff — как цитировать научное ПО

> **CITATION.cff** — файл в формате Citation File Format (версия 1.2.0), который сообщает, как ссылаться на программу в научных работах. GitHub показывает для него ссылку **Cite this repository** с форматами APA и BibTeX.

Обязательные поля: `cff-version`, `message`, `title`, `authors`. Пример с заполнителями:

```yaml
cff-version: 1.2.0
message: "Если вы используете эту программу, процитируйте её, как указано ниже."
title: "[НАЗВАНИЕ ПРОЕКТА]"
authors:
  - family-names: "[ФАМИЛИЯ]"
    given-names: "[ИМЯ]"
version: "[ВЕРСИЯ, например 1.0.0]"
date-released: "[ГГГГ-ММ-ДД]"
license: MIT
repository-code: "https://github.com/[ВЛАДЕЛЕЦ]/[РЕПОЗИТОРИЙ]"
```

**Когда нужен:** научное и исследовательское ПО, которое цитируют в публикациях. Сгенерировать файл помогает cffinit. Источник: About CITATION files.

### THIRD_PARTY_NOTICES.md — учёт сторонних компонентов

Единого стандарта названия нет: встречаются `THIRD_PARTY_NOTICES.md`, `ThirdPartyNotices.txt`, `NOTICE`. Смысл один — перечислить сторонние компоненты, которые **входят в распространяемый продукт**, с их лицензиями и уведомлениями.

| Компонент | Версия | Лицензия | Источник | Где используется |
|---|---|---|---|---|
| [библиотека] | [версия] | MIT | [ссылка] | Встроена в `vendor/` |
| [шрифт] | [версия] | OFL-1.1 | [ссылка] | `assets/fonts/` |
| [иконки] | — | CC-BY-4.0 | [ссылка] | `assets/icons/` |

Шаблон есть в templates/LICENSE_SELECTION_GUIDE.md.

Что учитывать:

- **Библиотеки**, скопированные в репозиторий или включённые в сборку. Многие разрешительные лицензии (MIT, BSD) требуют приложить их текст и уведомление к **каждой копии** продукта — в том числе к закрытой программе.
- **Шрифты.** Часто под SIL Open Font License (OFL-1.1) или коммерческими лицензиями с ограничениями.

- **Изображения, иконки, звуки, видео.** Часто под лицензиями Creative Commons: CC BY требует указать автора, CC BY-NC запрещает коммерческое использование, CC BY-ND — изменения.
- **Данные и тексты** — свои условия использования.
- **Фрагменты кода из интернета** (форумы, статьи, ответы AI) — см. ниже.

### Происхождение кода

Записывайте, откуда пришёл заимствованный код:

- в коммите: `Добавить функцию разбора дат (адаптировано из [проект], [лицензия], [ссылка])`;
- в комментарии рядом с кодом и в `THIRD_PARTY_NOTICES.md`;
- для крупных заимствований — сохраните исходное уведомление об авторских правах.

Если происхождение фрагмента неизвестно и его нельзя проверить — лучше переписать его самостоятельно.

### DCO и CLA — права на вклад участников

Когда в проект присылают код другие люди, важно, на каких условиях.

- **DCO** (*Developer Certificate of Origin*) — короткое заявление участника, что он имеет право отправить код на условиях лицензии проекта. Подтверждается строкой `Signed-off-by` в каждом коммите: `git commit -s`. Текст: developercertificate.org.
- **CLA** (*Contributor License Agreement*) — отдельное соглашение, по которому участник предоставляет проекту права на свой вклад. Требует юридического оформления.
- **По умолчанию на GitHub** вклад в репозиторий предоставляется на условиях лицензии этого репозитория (принцип *inbound* = *outbound* в Условиях использования GitHub), если нет отдельного соглашения.

## 10.8. Код, созданный с помощью AI

- **Правовой статус различается по странам и меняется.** Например, Бюро авторского права США считает, что охраняется только то, что создано человеком: одни лишь запросы (промпты), как правило, не делают пользователя автором результата; материал, где человек определил творческие элементы, может охраняться. Источники: Copyright Registration Guidance (2023), Copyright and Artificial Intelligence, Part 2: Copyrightability (2025).
- **Риск совпадения с чужим кодом.** AI может воспроизвести фрагмент существующего кода вместе с его лицензионными обязательствами.
- **Условия AI-сервиса и работодателя.** Прочитайте условия использования инструмента и внутренние правила компании.
- **Ответственность остаётся на человеке.** Публикуя код, вы отвечаете за соблюдение лицензий, даже если код написал AI.

Практические меры перечислены в главе 7.

## 10.9. SPDX и REUSE: машиночитаемые сведения о лицензиях

### Идентификаторы и выражения SPDX

> **Идентификатор SPDX** — короткое стандартное имя лицензии из списка лицензий SPDX: `MIT`, `Apache-2.0`, `GPL-3.0-or-later`.

Для лицензий GNU используйте полные идентификаторы с `-only` («только эта версия») или `-or-later` («эта версия и новее»). Старые формы `GPL-3.0` и `GPL-3.0+` устарели.

Заголовок в начале файла исходного кода:

```python
# SPDX-License-Identifier: MIT
```

```javascript
// SPDX-License-Identifier: Apache-2.0
```

**Выражения лицензий:**

| Выражение | Значение |
|---|---|
| `MIT OR Apache-2.0` | Пользователь выбирает одну из лицензий |
| `MIT AND CC-BY-4.0` | Действуют обе лицензии (например, код и изображения) |
| `GPL-2.0-or-later WITH Classpath-exception-2.0` | Лицензия с исключением |
| `LicenseRef-Company-EULA` | Лицензия, которой нет в списке SPDX |

Источник: SPDX License Expressions.

### REUSE

REUSE — практика, при которой для каждого файла указаны правообладатель и лицензия:

```python
# SPDX-FileCopyrightText: 2026 [ИМЯ ПРАВООБЛАДАТЕЛЯ]
# SPDX-License-Identifier: MIT
```

Три шага REUSE:

1. **Выбрать и положить тексты лицензий** в папку `LICENSES/` с именами по SPDX: `LICENSES/MIT.txt`.
2. **Добавить сведения в каждый файл** — заголовком в комментарии, файлом `*.license` рядом с двоичным файлом или записью в `REUSE.toml`.
3. **Проверить** командой `reuse lint`.

Инструмент `reuse` устанавливается из PyPI (`pip install reuse`). Полезные команды: `reuse download --all` (скачать тексты лицензий), `reuse annotate --copyright="[ИМЯ]" --license=MIT <файлы>` (добавить заголовки), `reuse lint` (проверить).

### Проверка лицензий зависимостей

> **Зависимость** — чужая библиотека, которую использует проект. Прямые зависимости вы подключили сами, транзитивные — пришли вместе с ними.

1. **Составьте список** зависимостей: файл зависимостей (`requirements.txt`, `pyproject.toml`, `package.json`), граф зависимостей GitHub (**Insights** → **Dependency graph**) или выгрузка SBOM.
2. **Определите лицензию** каждой зависимости: в её репозитории, в реестре пакетов (pypi.org, npmjs.com), в SBOM.
3. **Проверьте совместимость** с лицензией и моделью распространения вашего проекта. Особое внимание — GPL и AGPL в закрытых продуктах и проектах под разрешительной лицензией.
4. **Автоматизируйте**: действие `actions/dependency-review-action` умеет не пропускать Pull Request с зависимостями под лицензиями, которых нет в списке разрешённых (параметр `allow-licenses`, глава 11); для больших проектов существуют сканеры вроде ScanCode Toolkit и OSS Review Toolkit.
5. **Повторяйте** при каждом добавлении или обновлении зависимостей.

Сведения о лицензиях в реестрах и автоматических инструментах бывают неполными или ошибочными. Для важных решений проверяйте текст лицензии в самом пакете.

## 10.10. Что зависит от страны и договоров

| Вопрос | От чего зависит | Пример |
|---|---|---|
| Кто правообладатель кода, написанного на работе | Закон страны и трудовой договор | РФ: служебное произведение (ст. 1295 ГК РФ) |
| Можно ли отказаться от авторских прав | Закон страны | В РФ отказ от личных неимущественных прав ничтожен; в Германии авторское право не передаётся |
| Как оформляется открытая лицензия | Закон страны | В РФ есть понятие открытой лицензии (ст. 1286.1 ГК РФ): если срок не указан, для программ она действует весь срок исключительного права, если территория не указана — на территории всего мира |
| Охраняется ли результат работы AI | Закон и практика страны | США: требуется человеческое авторство |
| Обязанности производителя ПО | Отраслевое регулирование | ЕС: Регламент (ЕС) 2024/2847 (Cyber Resilience Act) — для продуктов, поставляемых на рынок ЕС в рамках коммерческой деятельности; обязанности сообщать об активно эксплуатируемых уязвимостях действуют с 11 сентября 2026 года, основные требования — с 11 декабря 2027 года; некоммерческое open-source ПО в целом вне сферы действия |
| Права на код по заказу | Договор с заказчиком | Права могут перейти заказчику |
| Использование чужих компонентов | Лицензии этих компонентов | GPL-библиотека в закрытом продукте |

Источники по РФ: Гражданский кодекс РФ, часть четвёртая. По ЕС: Регламент (ЕС) 2024/2847.

## 10.11. Практические примеры

### Учебный проект в портфолио

**Ситуация:** публичный репозиторий с учебной программой, весь код ваш, хотите, чтобы его могли свободно использовать.

**Решение:** MIT. Файл `LICENSE` с вашим именем и годом, раздел «Лицензия» в README. Если в проекте есть чужие изображения или шрифты — проверьте их лицензии и перечислите в `THIRD_PARTY_NOTICES.md`. Если репозиторий приватный и никому не передаётся, лицензия не обязательна.

### Коммерческий продукт с закрытым кодом

**Ситуация:** компания продаёт программу, исходный код не публикуется.

**Решение:**

- открытая лицензия на собственный код не нужна; условия для клиентов описывает лицензионное соглашение, которое готовит юрист;
- в репозитории можно указать уведомление вида `Copyright (c) [ГОД] [НАЗВАНИЕ КОМПАНИИ]. All rights reserved.` и сделать репозиторий приватным;
- **проверьте лицензии всех зависимостей**: GPL и AGPL могут потребовать открыть код продукта;
- соберите `THIRD_PARTY_NOTICES` для поставки: даже разрешительные лицензии (MIT, BSD, Apache-2.0) требуют приложить их тексты и уведомления к продукту;
- закрепите права на код сотрудников и подрядчиков в договорах.

### Open-source-библиотека

**Ситуация:** библиотека для других разработчиков, хотите широкого распространения, в том числе в компаниях.

**Решение:** часто выбирают Apache-2.0 (явная патентная лицензия) или MIT (простота). Если важно, чтобы улучшения самой библиотеки оставались открытыми, но использовать её могли и закрытые программы, — изучите MPL-2.0 или LGPL-3.0. Дополнительно:

- SPDX-идентификатор в манифесте пакета и, по желанию, в заголовках файлов (REUSE);
- `CONTRIBUTING.md` с условиями приёма вклада (например, DCO);
- проверка лицензий зависимостей в CI;
- `NOTICE`, если выбрана Apache-2.0 и есть обязательные атрибуции.

## Проверьте себя

1. Вы выложили код в публичный репозиторий без лицензии. Может ли другой человек использовать его в своём коммерческом продукте?
2. Почему нельзя скопировать файл `LICENSE` из чужого проекта и закоммитить к себе?
3. Компания запускает изменённую программу под GPL-3.0 на своём сервере, пользователи работают с ней через браузер. Должна ли она открыть исходный код? А если программа под AGPL-3.0?

4. Можно ли включить код под GPL-3.0 в проект под MIT и распространять результат под MIT?
5. Чем Apache-2.0 отличается от MIT с точки зрения патентов?
6. Вы написали программу на работе. Можно ли выложить её под MIT?

**Ответы**

1. Нет. Без лицензии действует авторское право по умолчанию. GitHub разрешает другим только просматривать репозиторий и делать форк на GitHub.
2. В нём чужое имя правообладателя, а условия лицензии могут не соответствовать вашим целям и лицензиям зависимостей.
3. По GPL-3.0 — нет: работа на сервере без передачи копий не считается распространением (если JavaScript с GPL-кодом не отправляется в браузер). По AGPL-3.0 — да: изменённая версия должна предлагать пользователям сети исходный код.
4. Нет. Объединённая программа при распространении должна соответствовать GPL-3.0.
5. Apache-2.0 содержит явную патентную лицензию от участников и прекращает её для того, кто подаёт патентный иск. В MIT явных положений о патентах нет.
6. Только с разрешения правообладателя. Права на служебный код часто принадлежат работодателю — проверьте трудовой договор и спросите работодателя.

## Что дальше

В главе 11 — безопасность репозитория и автоматические проверки на GitHub.

# Глава 11. Безопасность и автоматизация

Автоматические проверки ловят ошибки до того, как они попадут в `main`, а настройки безопасности защищают репозиторий, ключи и пользователей. В этой главе — GitHub Actions и CI, защита веток, работа с секретами, Dependabot, secret scanning, CodeQL и порядок действий при утечке.

**Вы научитесь:**

- понимать, что такое CI и как устроен workflow GitHub Actions;
- добавлять в проект автоматические проверки стиля и тестов;
- защищать основную ветку правилами;
- хранить токены и ключи вне репозитория;
- включать Dependabot, secret scanning, CodeQL и закрытые сообщения об уязвимостях;
- действовать правильно, если секрет попал в Git.

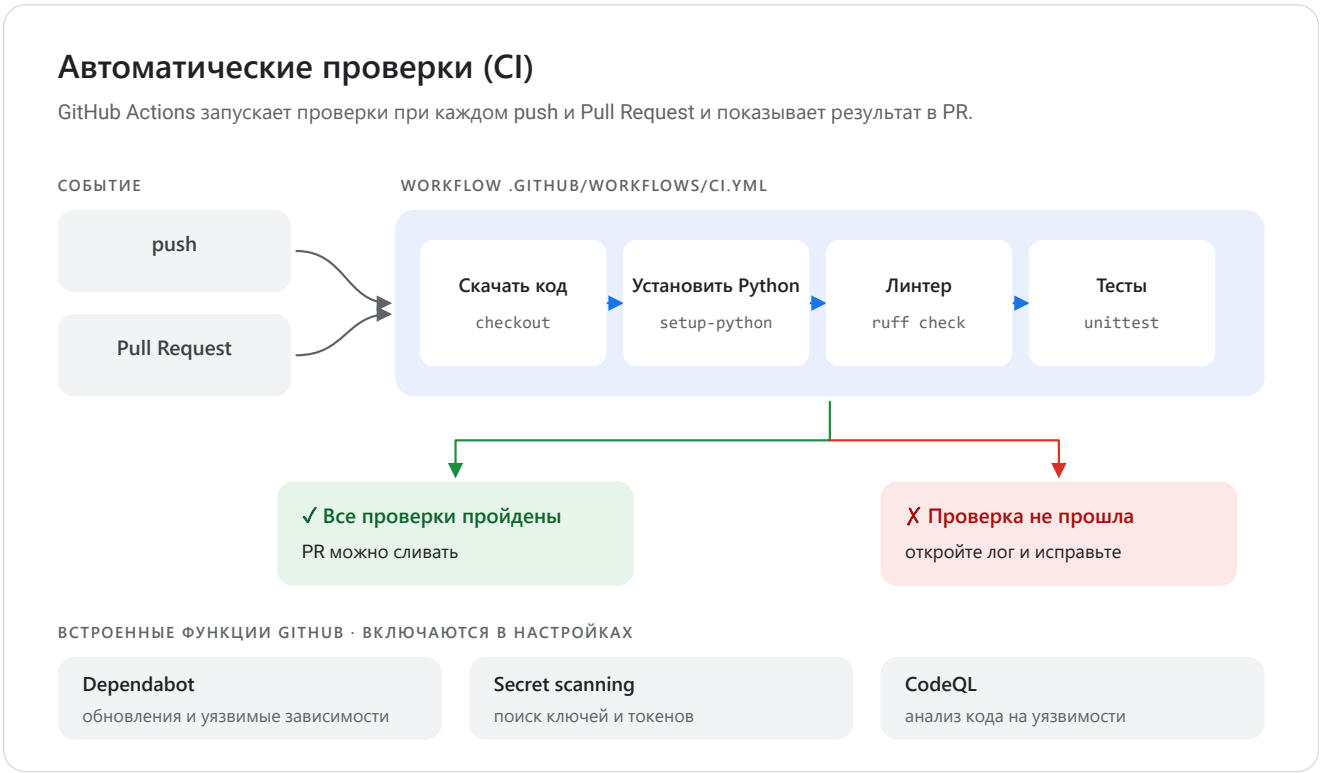
**Время:** 2–3 часа. **Понадобится:** репозиторий на GitHub с кодом и тестами — например, учебный проект examples/tempconv.

> **ПРИМЕЧАНИЕ**
> Доступность функций GitHub и их стоимость зависят от тарифа, видимости репозитория (публичный или приватный) и настроек организации. Ниже указано состояние на октябрь 2026 года. Перед настройкой сверяйтесь с документацией GitHub — ссылки даны в каждом разделе.

## 11.1. CI и GitHub Actions

> **CI** (*continuous integration*, непрерывная интеграция) — практика, при которой каждое изменение автоматически проверяется: собирается, тестируется, проверяется линтером. Ошибка обнаруживается через минуты после push, а не через неделю.

> **GitHub Actions** — встроенная в GitHub система автоматизации. Она запускает ваши команды на виртуальных машинах GitHub по событиям в репозитории.




| Понятие | Что это | Пример |
|---|---|---|
| **Workflow** | Сценарий автоматизации — файл YAML в папке `.github/workflows/` | `.github/workflows/ci.yml` |
| **Событие** (*event*) | Что запускает workflow | `push`, `pull_request`, расписание (`schedule`), ручной запуск (`workflow_dispatch`) |
| **Задание** (*job*) | Набор шагов на одной машине | `test` |
| **Шаг** (*step*) | Одна команда или готовое действие | `ruff check .` |
| **Раннер** (*runner*) | Виртуальная машина, где выполняется задание | `ubuntu-latest` |
| **Действие** (*action*) | Готовый переиспользуемый шаг | `actions/checkout` — скачать код |

> **YAML** — текстовый формат настроек, где вложенность задаётся отступами из пробелов. Табуляция в YAML запрещена, а неверный отступ ломает файл.

**Стоимость.** Для публичных репозиториев стандартные раннеры GitHub бесплатны. Для приватных в тариф входит ограниченное число минут в месяц (в бесплатном тарифе для личных аккаунтов — 2 000 минут), сверх него — оплата. Источник: GitHub Actions billing.

## 11.2. Пример workflow для проекта на Python

Файл `.github/workflows/ci.yml` из учебного проекта tempconv:

YAML

```yaml
name: CI

on:
  push:
    branches: [main]
  pull_request:

permissions:
  contents: read

jobs:
  test:
    name: Python ${{ matrix.python-version }}
    runs-on: ubuntu-latest
    strategy:
      fail-fast: false
      matrix:
        python-version: ["3.11", "3.12", "3.13", "3.14"]

    steps:
      - name: Скачать код
        uses: actions/checkout@3d3c42e5aac5ba805825da76410c181273ba90b1 # v7.0.1
        with:
          persist-credentials: false

      - name: Установить Python
        uses: actions/setup-python@5fda3b95a4ea91299a34e894583c3862153e4b97 # v7.0.0
        with:
          python-version: ${{ matrix.python-version }}

      - name: Установить проект и инструменты
        run: |
          python -m pip install --upgrade pip
          python -m pip install -e . -r requirements-dev.txt

      - name: Проверить стиль кода
        run: ruff check .

      - name: Запустить тесты
        run: python -m unittest discover -s tests -v
```

Разбор по частям:

| Фрагмент | Значение |
|---|---|
| `name: CI` | Название workflow во вкладке **Actions** |
| `on: push: branches: [main]` | Запускать при push в `main` |
| `on: pull_request:` | Запускать для каждого Pull Request |
| `permissions: contents: read` | Токену workflow разрешено только читать код — принцип минимальных прав (раздел 11.12) |
| `runs-on: ubuntu-latest` | Выполнять на виртуальной машине с Linux |
| `matrix: python-version` | Повторить задание для четырёх версий Python |

| Фрагмент | Значение |
| --- | --- |
| `fail-fast: false` | Не останавливать другие версии, если одна упала |
| `uses: actions/checkout@3d3c...` | Готовое действие, закреплённое **полным хешем коммита**; версия — в комментарии |
| `persist-credentials: false` | Не оставлять токен в настройках Git на раннере после скачивания кода |
| `run:` | Команда, как в терминале |

**Как добавить в свой проект:**

1. Создайте папки `.github/workflows/` и файл `ci.yml`.
2. Замените команды установки, линтера и тестов на команды вашего проекта.
3. Закоммитьте через ветку и откройте Pull Request.
4. Откройте вкладку **Checks** в PR или вкладку **Actions** репозитория — workflow запустится автоматически.

**Значок статуса в README:**

```markdown
[![CI](https://github.com/<владелец>/<репозиторий>/actions/workflows/ci.yml/badge.svg)]
(https://github.com/<владелец>/<репозиторий>/actions/workflows/ci.yml)
```

**Если проверка упала:**

1. Откройте упавший запуск во вкладке **Actions** или по ссылке **Details** в PR.
2. Разверните шаг с красным крестиком и прочитайте последние строки лога.
3. Воспроизведите ошибку у себя той же командой (`ruff check .`, тесты).
4. Исправьте, закоммитьте и отправьте в ту же ветку — проверка запустится заново.

## Вариант для проекта на Node.js

Те же принципы, другие команды:

```yaml
    steps:
      - uses: actions/checkout@3d3c42e5aac5ba805825da76410c181273ba90b1 # v7.0.1
        with:
          persist-credentials: false
      - uses: actions/setup-node@949feb2413d6458794dcd2491c4babbbce0c15c1 # v7.1.0
        with:
          node-version: 24
          cache: npm
      - run: npm ci
      - run: npm run lint --if-present
      - run: npm test
```

`npm ci` устанавливает зависимости строго по `package-lock.json`. Workflow полезен, только если в проекте есть тесты и линтер: пустой workflow «для галочки» ничего не проверяет.

## 11.3. Линтеры, форматтеры и тесты

| Инструмент | Что делает | Примеры |
|---|---|---|
| **Линтер** | Ищет ошибки и подозрительные места в коде без запуска: неиспользуемые переменные, опечатки в именах, опасные конструкции | Ruff (Python), ESLint (JavaScript) |
| **Форматтер** | Автоматически приводит код к единому оформлению | `ruff format` (Python), Prettier (JavaScript) |
| **Тесты** | Запускают код и сверяют результат с ожидаемым | `unittest`, pytest (Python), `npm test` |

Правила:

- запускайте те же проверки локально перед push — так быстрее;
- версии инструментов закрепляйте (`ruff==0.17.0` в `requirements-dev.txt`), чтобы проверки не менялись сами;
- новый код — с тестами; упавший тест — повод разобраться, а не удалить тест.

## 11.4. Защита основной ветки

Правила ветки не дают изменить `main` в обход Pull Request и проверок.

**Как настроить (rulesets):**

1. Репозиторий → **Settings** → в меню слева раздел **Rules** → **Rulesets** → **New ruleset** → **New branch ruleset**.
2. **Ruleset name** — например, `protect-main`. Статус применения (*enforcement status*) по умолчанию **Disabled** — переключите на **Active**.
3. **Target branches** → **Add a target** → **Include default branch**.
4. Отметьте правила:

| Правило | Что даёт |
|---|---|
| **Restrict deletions** | Ветку нельзя удалить |
| **Block force pushes** | Нельзя переписать историю `git push --force` |
| **Require a pull request before merging** | Изменения — только через PR; можно потребовать одобрение (в команде — хотя бы одно) |
| **Require status checks to pass before merging** | Слияние только после успешных проверок CI — добавьте задания из вашего workflow |

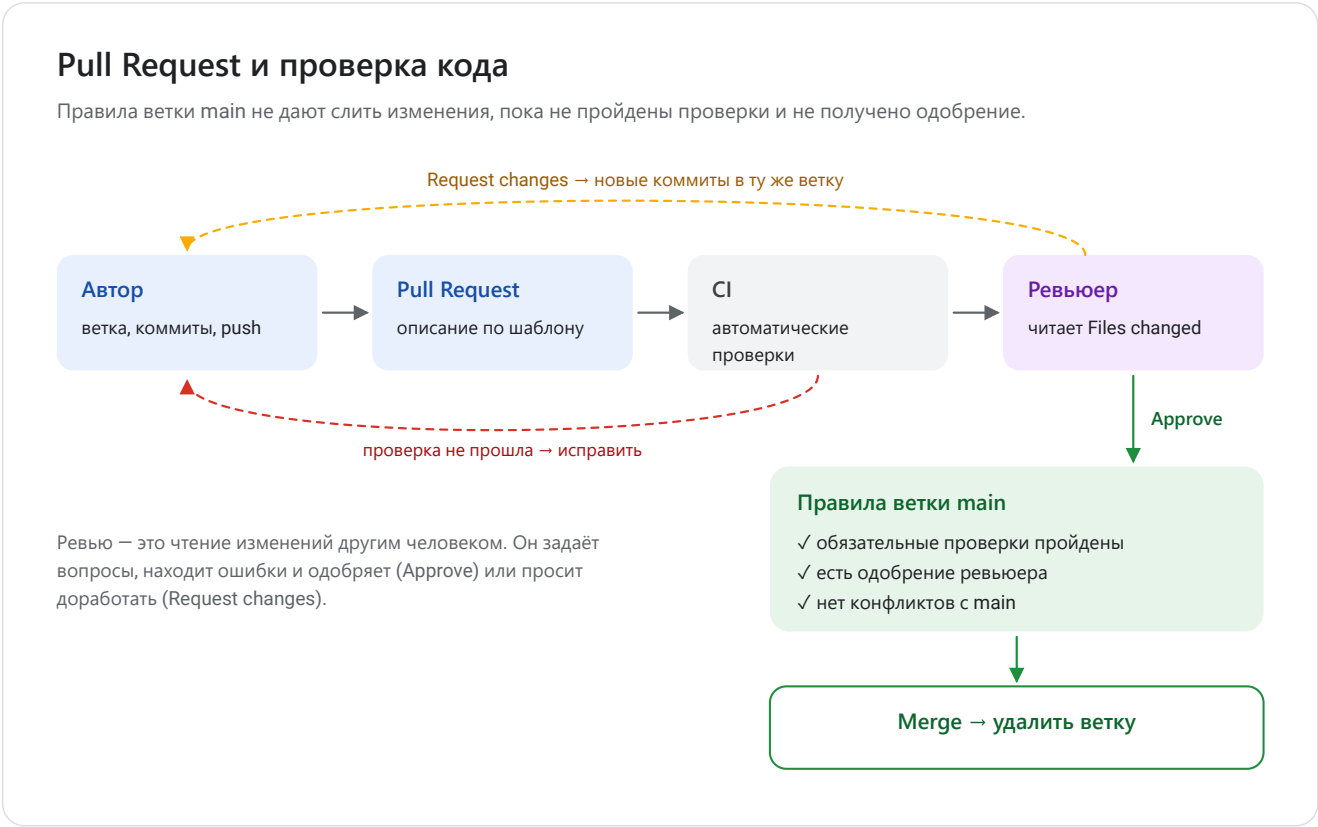
5. **Create**.

Классический вариант — **Settings** → **Branches** → **Add classic branch protection rule** — работает похоже.

**Доступность:** правила и защита веток доступны в публичных репозиториях на бесплатном тарифе; для приватных нужен тариф Pro, Team или Enterprise. Источник: About rulesets.

> **СОВЕТ**
>
> Если вы работаете один, требование одобрения заблокирует вас: одобрить свой PR нельзя. Оставьте число обязательных одобрений равным 0, но сохраните обязательный PR и проверки.

## 11.5. Проверка изменений в Pull Request




Чек-лист безопасности для ревьюера:

☐ нет паролей, токенов, ключей, личных данных — в том числе в тестах и примерах;

☐ новые зависимости обоснованы, существуют в официальном реестре, их лицензии подходят;

☐ изменения в `.github/workflows/` проверены особенно внимательно: workflow выполняется с доступом к секретам репозитория;

☐ пользовательский ввод проверяется; ошибки не раскрывают внутренние данные;

☐ нет отключённых проверок, закомментированных тестов, «временных» обходов;

☐ все автоматические проверки зелёные.

Чтобы изменения в критичных файлах всегда смотрел определённый человек, используйте файл `.github/CODEOWNERS`:

```
# Изменения в workflow и настройках зависимостей — только с одобрения владельца
/.github/ @<логин>
```

## 11.6. Токены, ключи и файлы .env

> **Переменная окружения** — именованное значение, которое операционная система передаёт программе при запуске. Программа читает настройки и секреты из окружения, поэтому их не нужно писать в код.

### Схема .env и .env.example

| Файл | Что в нём | В Git |
|---|---|---|
| `.env` | Настоящие значения, в том числе секреты | **Нет** — в `.gitignore` |
| `.env.example` | Имена переменных и безопасные образцы значений, пояснения | **Да** |

Пример `.env.example`:

```
.ENV
# Ключ API погодного сервиса. Получить: [ССЫЛКА НА СЕРВИС]
WEATHER_API_KEY=замените-на-свой-ключ

# Число знаков после запятой (0–6)
TEMPCONV_PRECISION=2
```

Правила в `.gitignore`:

```
.GITIGNORE
.env
.env.*
!.env.example
```

Чтение переменной в Python:

```
PYTHON
import os

api_key = os.environ.get("WEATHER_API_KEY")
if not api_key:
    raise SystemExit("Задайте переменную WEATHER_API_KEY (см. .env.example)")
```

Задать переменную в PowerShell для текущего окна: `$env:WEATHER_API_KEY = "..."`. Загружать файл `.env` автоматически умеют библиотеки (например, python-dotenv для Python); без них программа видит только переменные окружения.

### Секреты в GitHub Actions

Ключи для workflow храните в **Settings → Secrets and variables → Actions → New repository secret**. В workflow они доступны как `${{ secrets.ИМЯ }}`, а в логах GitHub заменяет их звёздочками.

```yaml
      - name: Вызов API
        env:
          WEATHER_API_KEY: ${{ secrets.WEATHER_API_KEY }}
        run: python scripts/check_api.py
```

Секреты не передаются в workflow, запущенные из Pull Request форков (кроме ограниченного `GITHUB_TOKEN`).

### Правила обращения с токенами

- Создавайте **fine-grained** токены: только нужные репозитории, только нужные права, ограниченный срок.
- Храните токены в менеджере паролей, а не в заметках и файлах проекта.
- Не вставляйте токены в чаты, тикеты и запросы к AI-ассистентам.
- Удаляйте токены, которыми больше не пользуетесь: **Settings** → **Developer settings** → **Personal access tokens**.
- При подозрении на утечку — сразу отзывайте токен (раздел 11.8).

## 11.7. Secret scanning и push protection

> **Secret scanning** — автоматический поиск секретов (ключей, токенов) в репозитории. **Push protection** — проверка в момент `git push`: если в коммите найден секрет, GitHub не примет push.

| Где | Что доступно |
|---|---|
| Публичные репозитории | Secret scanning работает автоматически и бесплатно; push protection на уровне репозитория можно включить в настройках |
| Ваш аккаунт | Push protection для вас лично включена по умолчанию для публичных репозиториев: **Settings** (аккаунта) → **Code security** → **Push protection for yourself** |
| Приватные репозитории организаций | Нужен платный продукт GitHub Secret Protection (тарифы Team и Enterprise Cloud) |

Источник: About secret scanning.

### Если push заблокирован

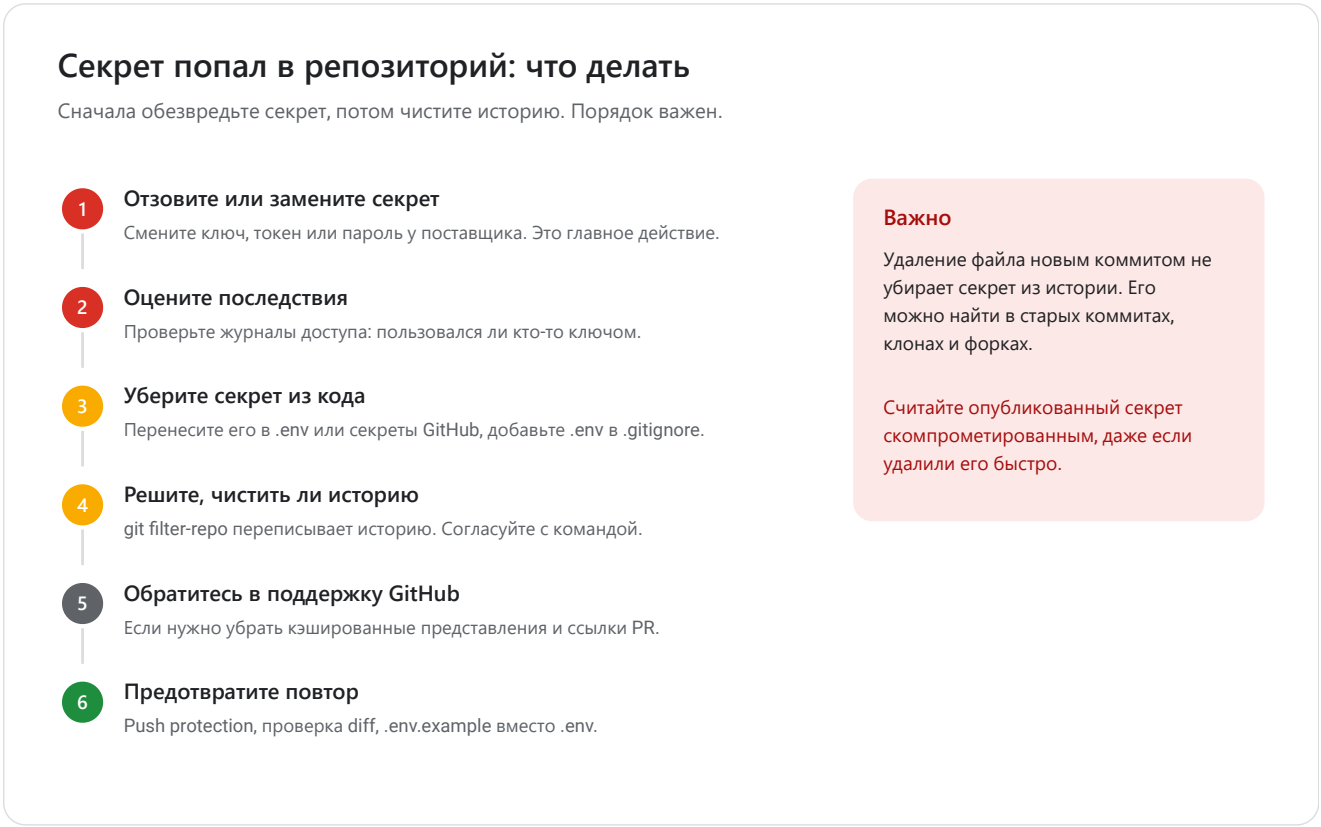
Сообщение содержит тип секрета, коммит и путь к файлу:

```
remote: error: GH013: Repository rule violations found for refs/heads/main.
remote: - GITHUB PUSH PROTECTION
remote:   Push cannot contain secrets
...
```

Что делать:

1. **Если секрет настоящий** — уберите его из кода (перенесите в `.env`). Если он в последнем коммите: исправьте файл, затем `git commit --amend --all --no-edit` и повторите push. Если в более ранних — см. раздел 11.8.
2. **Считайте секрет скомпрометированным,** если он уже был где-то опубликован, и замените его у поставщика.
3. **Обход блокировки** (с указанием причины) допустим только для ложного срабатывания или тестовых данных. Он создаёт предупреждение (*alert*) в репозитории.

## 11.8. Если секрет попал в историю




> **ОПАСНО**
>
> Удаление секрета новым коммитом **не удаляет его из истории**. Старые коммиты, клоны, форки, кэши и резервные копии продолжают его хранить. Если репозиторий публичный — считайте секрет украденным.

**Шаг 1. Отзовите или замените секрет — немедленно.** Смените пароль, удалите ключ API или токен у поставщика и выпустите новый. Это главное действие: старый секрет становится бесполезным, где бы он ни лежал. Часто этого достаточно.

**Шаг 2. Оцените последствия.** Посмотрите журналы доступа поставщика: использовался ли ключ, кем и когда. Проверьте счета, данные, сервисы, к которым был доступ. При утечке персональных данных могут действовать требования закона об уведомлении.

**Шаг 3. Уберите секрет из текущего кода.** Перенесите его в `.env` или секреты GitHub, добавьте `.env` в `.gitignore`, закоммитьте.

**Шаг 4. Решите, нужно ли очищать историю.** Очистка нужна, если секрет нельзя заменить или в истории — чувствительные данные (например, персональные). Очистка переписывает историю — это сложная и рискованная операция:

- меняются хеши всех затронутых коммитов, подписи коммитов теряются;
- участники с доступом к репозиторию должны заново склонировать его — старые клоны нельзя сливать с новой историей;
- форки и чужие клоны вы очистить не сможете;
- правила, запрещающие force push, придётся временно отключить.

Рекомендованный GitHub инструмент — git filter-repo (отдельная программа, версия 2.47 или новее). Установить его можно через Python: `python -m pip install git-filter-repo`. Устаревшую `git filter-branch` документация Git использовать не рекомендует.

> **ВНИМАНИЕ**
>
> Команды ниже **необратимо переписывают историю** на GitHub. Выполняйте их только после шагов 1–3, договорившись с участниками проекта и сохранив резервную копию репозитория. Пошаговую актуальную инструкцию читайте в документации: Removing sensitive data from a repository.

Общая схема:

```powershell
# свежий клон в отдельную папку
git clone https://github.com/<владелец>/<репозиторий>.git cleanup
cd cleanup

# удалить файл из всей истории
git filter-repo --sensitive-data-removal --invert-paths --path config/secrets.json

# отправить переписанную историю (все ветки и теги)
git push --force --mirror origin
```

Чтобы заменить строку внутри файлов, используется параметр `--replace-text` с файлом шаблонов замены.

**Шаг 5. Обратитесь в поддержку GitHub,** если нужно удалить кэшированные представления и ссылки в Pull Request. Поддержка помогает в случаях, когда замены секрета недостаточно.

**Шаг 6. Предотвратите повтор.** Включите push protection, проверяйте `git diff --staged` перед коммитом, держите `.env` в `.gitignore`, используйте `.env.example`.

## 11.9. Зависимости: Dependabot, dependency review и лицензии

> **Уязвимость в зависимости** — известная ошибка безопасности в чужой библиотеке, которую использует ваш проект. Такие уязвимости публикуются в базах данных, например GitHub Advisory Database.

### Dependabot

| Функция | Что делает | Где включить |
|---|---|---|
| **Dependabot alerts** | Предупреждает, если в зависимостях найдена известная уязвимость | **Settings** → раздел безопасности (**Advanced Security**) → **Dependabot alerts** |
| **Dependabot security updates** | Сам открывает PR с обновлением уязвимой зависимости | Там же |
| **Dependabot version updates** | Регулярно предлагает обновления всех зависимостей | Файл `.github/dependabot.yml` |

Функции Dependabot доступны для всех репозиториев на GitHub. Источник: Dependabot alerts.

Пример `.github/dependabot.yml`:

```yaml
version: 2
updates:
  - package-ecosystem: "pip"          # зависимости Python
    directory: "/"                    # где лежит файл зависимостей
    schedule:
      interval: "weekly"              # раз в неделю
  - package-ecosystem: "github-actions"  # версии действий в workflow
    directory: "/"
    schedule:
      interval: "weekly"
```

Для JavaScript используйте `package-ecosystem: "npm"`. Обновления от Dependabot — обычные Pull Request: дождитесь проверок CI, просмотрите изменения и только потом сливайте.

Локальная проверка уязвимостей: `pip-audit` для Python, `npm audit` для Node.js.

### Dependency review — проверка зависимостей в Pull Request

Действие `actions/dependency-review-action` сравнивает зависимости до и после PR и не даёт слить изменение, которое добавляет уязвимую зависимость или зависимость под запрещённой лицензией. Доступно для всех публичных репозиториев и для приватных с GitHub Code Security.

```yaml
name: Dependency review
on: pull_request

permissions:
  contents: read

jobs:
  dependency-review:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@3d3c42e5aac5ba805825da76410c181273ba90b1 # v7.0.1
        with:
          persist-credentials: false
      - uses: actions/dependency-review-action@a1d282b36b6f3519aa1f3fc636f609c47dddb294 # v5.0.0
```

```
        with:
          fail-on-severity: moderate
          allow-licenses: MIT, Apache-2.0, BSD-2-Clause, BSD-3-Clause, ISC, PSF-2.0
```

`allow-licenses` — список разрешённых лицензий: PR с зависимостью под другой лицензией не пройдёт проверку. Список зависит от лицензии и модели распространения вашего проекта. Параметр `deny-licenses` (запрещённые лицензии) в версии 5 объявлен устаревшим — используйте `allow-licenses` (глава 10). Источник: About dependency review.

### Состав зависимостей (SBOM)

Список всех зависимостей проекта можно выгрузить в формате SPDX: **Insights** → **Dependency graph** → **Export SBOM**. Это пригодится для проверки лицензий и для заказчиков, которые требуют перечень компонентов.

## 11.10. CodeQL: поиск уязвимостей в коде

> **CodeQL** — анализатор GitHub, который ищет в исходном коде типичные уязвимости и ошибки: внедрение SQL-кода, небезопасную обработку ввода, утечки данных. Это вид **статического анализа** (*SAST*) — проверки кода без его запуска.

**Как включить (default setup): Settings** → раздел безопасности (**Advanced Security**) → **Code scanning** → **CodeQL analysis** → **Set up** → **Default**. GitHub сам выберет языки и расписание. Результаты появятся в разделе безопасности репозитория (**Code scanning**) и в Pull Request.

**Доступность:** бесплатно для публичных репозиториев; для приватных репозиториев организаций нужен GitHub Code Security.

**Языки:** C/C++, C#, Go, Java/Kotlin, JavaScript/TypeScript, Python, Ruby, Rust, Swift и сами workflow GitHub Actions. Для неподдерживаемых языков используйте другие анализаторы.

Источник: About code scanning with CodeQL.

## 11.11. Закрытые сообщения об уязвимостях

**Private vulnerability reporting** позволяет любому человеку закрыто сообщить об уязвимости прямо через GitHub — без публичного Issue.

1. **Settings** → раздел безопасности (**Advanced Security**) → **Private vulnerability reporting** → **Enable**.
2. В разделе безопасности репозитория появится кнопка **Report a vulnerability**.
3. Сообщения превращаются в черновики рекомендаций по безопасности (*security advisories*), где можно обсудить проблему, подготовить исправление и опубликовать сведения после выпуска.

Функция доступна для публичных репозиториев. Укажите её как канал связи в `SECURITY.md`. Создать политику безопасности через сайт: раздел безопасности репозитория (вкладка **Security**, в новых версиях интерфейса — **Security and quality**) → **Security policy** → **Start setup**.

Источники: Configuring private vulnerability reporting, Adding a security policy.

## 11.12. Минимальные права доступа

> **Принцип минимальных привилегий** — каждый человек, программа и токен получают только те права, которые нужны для их задачи, и только на нужное время.

| Где | Как применить |
|---|---|
| Участники с доступом (*collaborators*) | Добавляйте только тех, кому нужен доступ; удаляйте, когда человек ушёл из проекта. В организациях выдавайте роли по задаче: Read, Triage, Write, Maintain, Admin |
| Аккаунт | Включите двухфакторную аутентификацию; сохраните коды восстановления |
| Токены | Fine-grained, только нужные репозитории и права, срок действия |
| Приложения | Периодически проверяйте **Settings → Applications** и отзывайте доступ ненужным приложениям |
| `GITHUB_TOKEN` в workflow | `permissions: contents: read` на уровне файла; дополнительные права — только заданию, которому они нужны. Для новых репозиториев токен по умолчанию только читает код |
| Ключи развёртывания | Только для одного репозитория и только на чтение, если запись не нужна |

## 11.13. Безопасность GitHub Actions и CI/CD

Workflow выполняет код с доступом к секретам и иногда к праву записи в репозиторий. Поэтому к нему нужны отдельные правила.

> **CI/CD** — непрерывная интеграция и непрерывная доставка (*continuous delivery/deployment*): автоматические проверки и автоматическая выкладка готовой версии.

**1. Закрепляйте сторонние действия полным хешем коммита.** Тег вроде `@v7` можно переставить на другой коммит, а хеш — нет. Документация GitHub называет закрепление по полному хешу единственным способом использовать действие как неизменяемый релиз. Dependabot (экосистема `github-actions`) умеет обновлять такие закрепления.

```yaml
uses: actions/checkout@3d3c42e5aac5ba805825da76410c181273ba90b1 # v7.0.1
```

**2. Проверяйте действие перед использованием.** Кто автор, есть ли у него отметка проверенного создателя, поддерживается ли действие, что оно делает (код открыт — его можно прочитать). Предпочитайте официальные действия (`actions/*`, `github/*`) и действия известных организаций.

**3. Минимальные права токена.** Блок `permissions:` в каждом workflow (раздел 11.12).

**4. Осторожно с `pull_request_target` и `workflow_run`.** Эти события выполняются с правами основного репозитория, даже для PR из форков. Никогда не скачивайте и не запускайте в них код из PR.

**5. Не подставляйте недоверенные данные прямо в команды.** Заголовок PR, имя ветки, текст комментария может прислать кто угодно.

```yaml
# Опасно: заголовок PR подставляется в команду и может её изменить
- run: echo "${{ github.event.pull_request.title }}"

# Безопасно: значение передаётся через переменную окружения
- env:
    PR_TITLE: ${{ github.event.pull_request.title }}
  run: echo "$PR_TITLE"
```

**6. Контролируйте запуск workflow от внешних участников.** В **Settings → Actions → General** можно требовать одобрения перед запуском workflow из Pull Request новых участников.

**7. Защищайте выкладку.** Для развёртывания используйте окружения (**Settings → Environments**) с обязательным одобрением и собственными секретами.

**8. Изменения в `.github/workflows/` — только через ревью** (CODEOWNERS, раздел 11.5).

Источник: Secure use reference.

## 11.14. Чек-лист безопасности репозитория

| Область | Проверка |
|---|---|
| Аккаунт | Включена 2FA, сохранены коды восстановления |
| Секреты | Нет секретов в коде и истории; `.env` в `.gitignore`; есть `.env.example` |
| Защита от утечек | Включены secret scanning и push protection (где доступны) |
| Ветка `main` | Запрещены удаление и force push; обязательны PR и проверки |
| CI | Тесты и линтер в каждом PR; `permissions: contents: read`; действия закреплены хешами |
| Зависимости | Dependabot alerts и version updates; dependency review в PR (где доступен) |
| Код | CodeQL default setup (где доступен) |
| Уязвимости | `SECURITY.md`; включены закрытые сообщения об уязвимостях |
| Доступ | Только нужные участники, токены и приложения |
| Оценка | Для open-source — проверка OpenSSF Scorecard (глава 9) |

Полный чек-лист готовности к публикации — templates/PROJECT_RELEASE_CHECKLIST.md.

> **ВАЖНО**
>
> Ни одна из этих мер не гарантирует безопасность полностью. Автоматические проверки находят известные типы проблем; они не заменяют внимательное ревью, обновление зависимостей и быстрое реагирование на сообщения об уязвимостях.

## Проверьте себя

1. Чем workflow отличается от задания (*job*) и шага (*step*)?

2. Почему действия в workflow закрепляют хешем коммита, а не тегом `@v7`?
3. Вы случайно отправили в публичный репозиторий файл с ключом API и через минуту удалили его новым коммитом. Ключ в безопасности?
4. Какие правила стоит включить для ветки `main` в проекте, где вы работаете один?
5. Чем Dependabot alerts отличаются от Dependabot version updates?

**Ответы**

1. Workflow — весь сценарий (файл YAML), задание — набор шагов на одной машине, шаг — одна команда или действие.
2. Тег можно переставить на другой коммит — например, если репозиторий действия взломают. Хеш коммита неизменен.
3. Нет. Ключ остаётся в истории и мог быть скопирован. Нужно сразу отозвать ключ у поставщика и выпустить новый, затем решить вопрос с очисткой истории.
4. Запрет удаления и force push, обязательный Pull Request (без обязательного одобрения) и обязательные проверки CI.
5. Alerts предупреждают об известных уязвимостях в зависимостях; version updates регулярно предлагают обновить зависимости до новых версий, даже без уязвимостей.

## Что дальше

Пройдите учебный проект «Профессионально оформленный репозиторий» в главе 8 и чек-лист PROJECT_RELEASE_CHECKLIST.md. Термины — в словаре, источники — в sources.md.

ПРИЛОЖЕНИЯ

# Справочные материалы

Словарь терминов, источники, пример оформленного README и шаблоны документов проекта.

Словарь терминов

Источники и полезные ссылки

# Словарь терминов

Термины методички в алфавитном порядке: сначала названия на латинице, затем на кириллице. В скобках — английский вариант и глава, где термин разобран подробно.

## Термины на латинице

**.editorconfig** — файл с правилами форматирования (отступы, кодировка, концы строк), которые редакторы применяют автоматически. *Пример:* `indent_size = 4`. Глава 9.

**.env** — файл с настоящими значениями настроек и секретов для локального запуска. В Git не добавляется. Глава 11.

**.env.example** — образец файла `.env` без секретов: имена переменных и безопасные значения. Хранится в Git. Глава 11.

**.gitattributes** — файл с правилами для Git: какие файлы текстовые, какие двоичные, какие концы строк хранить. *Пример:* `* text=auto`. Глава 9.

**.gitignore** — список файлов и папок, которые Git должен игнорировать. *Пример:* строка `.env`. Глава 6.

**ADR** (*Architecture Decision Record*) — запись архитектурного решения: контекст, решение, последствия. Хранятся в `docs/adr/`. Глава 7.

**AGPL** (*GNU Affero General Public License*) — copyleft-лицензия, которая дополнительно требует предлагать исходный код изменённой программы пользователям, работающим с ней по сети. Глава 10.

**Apache-2.0** — разрешительная лицензия с явной патентной лицензией и правилами о файле `NOTICE`. Глава 10.

**API** (*Application Programming Interface*) — способ, которым одна программа обращается к другой. *Пример:* программа запрашивает погоду у сервиса через его API. Глава 6.

**ASVS** (*Application Security Verification Standard*) — стандарт OWASP с проверяемыми требованиями к безопасности приложений. Глава 9.

**BSD-2-Clause, BSD-3-Clause** — короткие разрешительные лицензии; BSD-3 дополнительно запрещает использовать имена авторов в рекламе без разрешения. Глава 10.

**CHANGELOG** — журнал изменений проекта по версиям для пользователей. Глава 9.

**CI** (*continuous integration*, непрерывная интеграция) — автоматическая проверка каждого изменения: сборка, тесты, линтер. Глава 11.

**CI/CD** — непрерывная интеграция и непрерывная доставка: автоматические проверки и автоматическая выкладка версии. Глава 11.

**CITATION.cff** — файл с правилами цитирования программы в научных работах. Глава 10.

**CLA** (*Contributor License Agreement*) — соглашение, по которому участник предоставляет проекту права на свой вклад (обычно в виде лицензии). Глава 10.

**Claude Code** — AI-ассистент для программирования от Anthropic: работает с файлами проекта, запускает команды и тесты. Глава 7.

**CLAUDE.md** — файл с инструкциями для Claude Code: команды, правила проекта. Глава 7.

**CODE_OF_CONDUCT.md** — кодекс поведения участников проекта. Глава 9.

**CODEOWNERS** — файл, который назначает ответственных ревьюеров для частей репозитория. Глава 11.

**CodeQL** — анализатор GitHub, который ищет уязвимости в исходном коде. Глава 11.

**CONTRIBUTING.md** — правила участия в проекте. Глава 9.

**Conventional Commits** — соглашение о формате сообщений коммитов: `feat: ...`, `fix: ...`. Глава 9.

**Copyleft** (копилефт) — условие лицензии: изменённые версии при распространении должны выходить на тех же условиях и с исходным кодом. Глава 10.

**CRLF, LF** — невидимые символы конца строки: CRLF принят в Windows, LF — в Linux и macOS. Глава 2.

**DCO** (*Developer Certificate of Origin*) — заявление участника, что он вправе отправить код; подтверждается командой `git commit -s`. Глава 10.

**Dependabot** — функция GitHub, которая предупреждает об уязвимых зависимостях и предлагает обновления. Глава 11.

**Detached HEAD** («отсоединённый HEAD») — состояние, когда вы находитесь на конкретном коммите, а не на ветке. Глава 6.

**diff** — построчная разница между двумя версиями файла. *Пример:* `git diff`. Глава 3.

**Fast-forward** («перемотка вперёд») — слияние без нового коммита: ветка просто передвигается вперёд. Глава 5.

**fetch** — скачать новые коммиты с удалённого репозитория, не меняя свои файлы. Глава 3.

**Fork** (форк) — копия чужого репозитория в вашем аккаунте GitHub. Глава 5.

**Git** — бесплатная программа для контроля версий. Глава 1.

**Git Bash** — оболочка в стиле Linux, которая устанавливается вместе с Git для Windows. Глава 2.

**Git Credential Manager** (GCM) — программа для безопасного хранения данных входа; позволяет входить в GitHub через браузер. Глава 4.

**GitHub** — сайт и облачный сервис для хранения Git-репозиториев и совместной работы. Глава 1.

**GitHub Actions** — система автоматизации GitHub: запускает workflow по событиям. Глава 11.

**GitHub CLI** ( `gh` ) — официальная программа GitHub для работы из терминала. Глава 4.

**GitHub Flow** — рабочий процесс: ветка → коммиты → Pull Request → ревью → слияние → удаление ветки. Глава 5.

**GPL** (*GNU General Public License*) — copyleft-лицензия: распространяемая производная программа должна быть под GPL с исходным кодом. Глава 10.

**HEAD** — указатель «вы находитесь здесь»: обычно последний коммит текущей ветки. Глава 3.

**Issue** — задача, сообщение об ошибке или идея в репозитории GitHub. Глава 1.

**Keep a Changelog** — рекомендации по ведению журнала изменений. Глава 9.

**LGPL** (*GNU Lesser General Public License*) — слабый copyleft для библиотек. Глава 10.

**LICENSE** — файл с текстом лицензии проекта. Глава 10.

**main** — основная ветка репозитория со стабильной версией. В старых проектах — `master`. Глава 5.

**Markdown** — простой язык разметки текста. Файлы с расширением `.md`. Глава 4.

**merge** (слияние) — объединение изменений из одной ветки в другую. Глава 5.

**Mermaid** — язык для описания диаграмм текстом; GitHub показывает такие диаграммы в Markdown. Глава 9.

**MIT** — короткая разрешительная лицензия. Глава 10.

**MPL-2.0** (*Mozilla Public License*) — слабый copyleft на уровне файлов. Глава 10.

**NOTICE** — файл с обязательными атрибуциями по лицензии Apache-2.0. Глава 10.

**OpenSSF Scorecard** — инструмент автоматической оценки практик безопасности open-source-проектов. Глава 9.

**origin** — стандартное имя главного удалённого репозитория. Глава 4.

**origin/main** — локальная «закладка», которая помнит, где была ветка `main` на GitHub при последней синхронизации (`fetch`, `pull`, `push`). Глава 4.

**OWASP** — открытое сообщество по безопасности приложений; автор ASVS, SAMM и Top 10. Глава 9.

**PAT** (*personal access token*) — персональный токен доступа вместо пароля для команд Git и API. Глава 4.

**PowerShell** — стандартная командная оболочка Windows. Глава 2.

**Provenance** (происхождение) — проверяемые сведения о том, кто, из чего и как собрал файл. Глава 9.

**pull** — скачать и сразу применить изменения с удалённого репозитория (`fetch` + `merge`). Глава 3.

**Pull Request** (PR) — запрос на слияние изменений из одной ветки в другую с обсуждением и проверкой. Глава 5.

**push** — отправить коммиты в удалённый репозиторий. Глава 3.

**Push protection** — проверка GitHub, которая блокирует push с секретами. Глава 11.

**README.md** — главная страница проекта. Глава 4 и 9.

**rebase** — способ перенести коммиты ветки на новое основание, переписывая их; альтернатива слиянию. В методичке не используется: для начала достаточно `merge`. Глава 5.

**reflog** — локальный журнал перемещений `HEAD`; помогает найти «потерянные» коммиты. Глава 6.

**Release** (релиз) — страница выпуска версии на GitHub, связанная с тегом. Глава 9.

**remote** — удалённый репозиторий. Глава 4.

**REUSE** — практика указывать лицензию и правообладателя для каждого файла. Глава 9 и 10.

**Rulesets** — правила для веток и тегов на GitHub: например, запрет force push и обязательный PR. Глава 11.

**SAMM** (*Software Assurance Maturity Model*) — модель зрелости практик безопасной разработки OWASP. Глава 9.

**SBOM** (*Software Bill of Materials*) — список компонентов программы с версиями и лицензиями. Глава 9.

**Secret scanning** — автоматический поиск секретов в репозитории. Глава 11.

**SemVer** (*Semantic Versioning*) — правила номеров версий `MAJOR.MINOR.PATCH`. Глава 9.

**SLSA** — спецификация уровней защиты цепочки сборки ПО. Глава 9.

**SPDX** — стандарт сведений о лицензиях и составе ПО; даёт идентификаторы лицензий вроде `MIT`. Глава 9 и 10.

**SSH-ключ** — пара ключей для входа на сервер без пароля; публичную часть добавляют на GitHub. Глава 4.

**Staging area** (индекс, область подготовки) — изменения, отобранные для следующего коммита. Глава 3.

**stash** — временное хранилище незакоммиченных изменений. Глава 6.

**Tag** (тег) — постоянная метка на коммите, обычно номер версии. *Пример:* `v1.0.0`. Глава 9.

**Unlicense** — попытка передать код в общественное достояние с запасной разрешительной лицензией. Глава 10.

**upstream** — связанная ветка на удалённом репозитории, с которой работают `git pull` и `git push` без параметров. Глава 4.

**winget** — менеджер программ Windows. Глава 2.

**Workflow** — сценарий автоматизации GitHub Actions в файле `.github/workflows/*.yml`. Глава 11.

**Worktree** (рабочее дерево) — дополнительная папка с другой веткой того же репозитория; позволяет работать над несколькими ветками одновременно. Глава 7.

**YAML** — текстовый формат настроек, где вложенность задаётся отступами. Глава 11.

## Термины на кириллице

**Авторское право** (*copyright*) — права автора на произведение; для программ возникают автоматически при создании. Глава 10.

**Аннотированный тег** — тег с автором, датой и сообщением; используется для релизов (`git tag -a`). Глава 9.

**Архитектура** — устройство программы: из каких частей она состоит и как они связаны. Глава 7.

**Ветка** (*branch*) — независимая линия разработки. Глава 1 и 5.

**Двухфакторная аутентификация** (2FA) — вход с паролем и вторым подтверждением (код из приложения, ключ). Глава 4.

**Зависимость** (*dependency*) — чужая библиотека, которую использует проект. Глава 7 и 10.

**История изменений** — цепочка коммитов от первого до последнего. Глава 1.

**Клонирование** (*clone*) — скачивание репозитория со всей историей. Глава 3.

**Коммит** (*commit*) — сохранённый снимок проекта с автором, датой и сообщением. Глава 1 и 3.

**Контрольная точка** — сохранённое состояние, к которому можно вернуться: коммит, ветка или тег. Глава 7.

**Конфликт слияния** (*merge conflict*) — ситуация, когда обе ветки по-разному изменили одно место файла, и Git просит выбрать вариант вручную. Глава 6.

**Линтер** (*linter*) — программа, которая ищет ошибки и подозрительные места в коде без запуска. Глава 11.

**Лицензия** — разрешение правообладателя использовать программу на определённых условиях. Глава 10.

**Локальный репозиторий** — репозиторий на вашем компьютере. Глава 1.

**Неотслеживаемый файл** (*untracked*) — файл, за которым Git ещё не следит. Глава 3.

**Оболочка** (*shell*) — программа, которая понимает команды в терминале: PowerShell, Git Bash. Глава 2.

**Ответственное раскрытие** — порядок, при котором об уязвимости сначала сообщают разработчикам закрыто, а публикуют после исправления. Глава 9.

**Патент** — исключительное право на изобретение, например на способ обработки данных. Глава 10.

**Переменная окружения** — именованное значение, которое система передаёт программе при запуске. Глава 11.

**Правообладатель** — тот, кому принадлежат исключительные права на произведение. Глава 10.

**Приглашение** (*prompt*) — строка в терминале перед местом ввода команды, например `PS C:\Users\Ivan>`. Глава 2.

**Принцип минимальных привилегий** — давать людям, программам и токенам только необходимые права. Глава 11.

**Публичный интерфейс** (*public API*) — то, на что опираются пользователи программы: команды, функции, форматы. Глава 9.

**Путь** — адрес файла или папки, например `C:\Users\Ivan\projects`. Глава 2.

**Рабочая папка** (*working directory*) — папка с файлами проекта, которые вы редактируете. Глава 3.

**Разрешительная лицензия** (*permissive*) — лицензия с минимумом условий: MIT, BSD, Apache-2.0. Глава 10.

**Раннер** (*runner*) — виртуальная машина, на которой GitHub Actions выполняет задания. Глава 11.

**Ревью кода** (*code review*) — чтение изменений другим человеком перед их принятием. Глава 5.

**Репозиторий** (*repository*) — папка проекта, за которой следит Git, вместе с историей. Глава 1.

**Секрет** — данные, дающие доступ: пароли, ключи API, токены. Глава 6 и 11.

**Система контроля версий** (СКВ, *version control system*) — программа, которая запоминает изменения файлов и позволяет вернуться к любой версии. Глава 1.

**Слияние** (*merge*) — см. merge.

**Совместимость лицензий** — возможность объединить код под разными лицензиями и распространять результат, соблюдая их все. Глава 10.

**Терминал** — окно для управления компьютером текстовыми командами. Глава 2.

**Тест** — программа, которая проверяет, что код работает как задумано. Глава 7 и 11.

**Удалённый репозиторий** (*remote*) — копия репозитория на сервере, например на GitHub. Глава 4.

**Уязвимость** — ошибка, позволяющая нарушить безопасность программы. Глава 9 и 11.

**Форматтер** (*formatter*) — программа, которая автоматически приводит код к единому оформлению. Глава 11.

**Хеш** (*hash*) — уникальный код коммита из цифр и букв, например `3f2a9c1`. Глава 1.

# Источники и полезные ссылки

**Дата проверки всех источников: 11 октября 2026 года.**

Основные источники — официальная документация. Вторичные источники использованы только там, где первоисточник платный или недоступен, и отмечены отдельно. Сайты и интерфейсы меняются: если ссылка не открывается или описание расходится с сайтом, ориентируйтесь на актуальную официальную документацию.

## Версии, на которые опирается методичка

| Компонент | Версия на дату проверки | Источник |
|---|---|---|
| Git | 2.56.0 (28 сентября 2026) | git-scm.com |
| Git для Windows | 2.56.0(2) (5 октября 2026) | gitforwindows.org |
| Git 3.0 | Не выпущен, дата не объявлена | BreakingChanges |
| `actions/checkout` | v7.0.1 | github.com/actions/checkout |
| `actions/setup-python` | v7.0.0 | github.com/actions/setup-python |
| `actions/setup-node` | v7.1.0 | github.com/actions/setup-node |
| `actions/dependency-review-action` | v5.0.0 | github.com/actions/dependency-review-action |
| Python | 3.15 — новейшая; 3.11–3.14 поддерживаются; 3.10 — окончание поддержки 1 октября 2026 | Status of Python versions |
| Ruff | 0.17.0 | pypi.org/project/ruff |

Примеры вывода команд Git получены в Git для Windows 2.55 в октябре 2026 года. В версии 2.56 формулировки могут незначительно отличаться.

## Git

- Документация Git — справочник команд, книга, видео.
- Справочник команд: git-init, git-status, git-add, git-commit, git-log, git-diff, git-clone, git-push, git-pull, git-fetch, git-remote, git-branch, git-switch, git-merge, git-restore, git-revert, git-reset, git-reflog, git-stash, git-clean, git-tag, git-config, gitignore, gitattributes, git-filter-branch (предупреждение о рисках и рекомендация git filter-repo).
- Установка Git для Windows, Git for Windows, параметры установщика (исходный код).
- Release notes Git 2.51.0 — `git switch` и `git restore` больше не экспериментальные.
- git filter-repo — инструмент для переписывания истории.

### Книга Pro Git на русском языке

- Pro Git, 2-е издание

- О системе контроля версий
- Установка Git
- Первоначальная настройка Git
- Создание Git-репозитория
- Запись изменений в репозиторий
- Просмотр истории коммитов
- Операции отмены
- Работа с удалёнными репозиториями
- Работа с тегами
- О ветвлении в двух словах
- Основы ветвления и слияния
- Припрятывание и очистка
- Раскрытие тайн reset
- GitHub: внесение собственного вклада в проекты

## GitHub

### Начало работы и аккаунт

- GitHub Docs
- What is GitHub?
- Creating an account on GitHub
- About mandatory two-factor authentication
- Email addresses reference
- Blocking command line pushes that expose your personal email address
- Git and GitHub learning resources

### Вход из терминала

- Caching your GitHub credentials in Git
- Managing your personal access tokens
- Token authentication requirements for Git operations — отказ от паролей с 13 августа 2021 года.
- Connecting to GitHub with SSH
- gh auth login

### Репозитории и файлы

- Creating a new repository
- About repositories
- Adding locally hosted code to GitHub
- Creating new files
- Adding a file to a repository
- Editing files

- Deleting files in a repository
- Downloading source code archives
- Cloning a repository
- About large files on GitHub
- github/gitignore — шаблоны .gitignore

### Ветки, Pull Request, совместная работа

- GitHub flow
- Creating a pull request
- Pull request reviews
- Pull request merges
- Resolving a merge conflict on GitHub
- About rulesets
- About protected branches

### Файлы сообщества, лицензии, релизы

- Creating a default community health file
- Adding a license to a repository
- Licensing a repository
- Adding a code of conduct to your project
- Configuring issue templates
- Creating a pull request template
- About CITATION files
- About community profiles for public repositories
- Syntax for issue forms
- Customizing your repository's social media preview
- Basic writing and formatting syntax — элемент `<picture>` для картинок под светлую и тёмную тему
- Managing releases in a repository
- Automatically generated release notes
- GitHub Terms of Service — раздел D.5 «License Grant to Other Users» (редакция от 27 апреля 2026 года).

### Безопасность и автоматизация

- GitHub Actions documentation
- Secure use reference
- Securely using pull_request_target
- GitHub Actions billing
- About secret scanning
- About push protection
- Managing user push protection
- About Dependabot alerts

- Configuring Dependabot version updates
- Dependabot options reference
- About dependency review
- About code scanning with CodeQL
- Configuring private vulnerability reporting
- Adding a security policy
- Removing sensitive data from a repository
- Artifact attestations
- About GitHub Advanced Security

### Обучение

- GitHub Skills (адрес skills.github.com перенаправляет сюда): Introduction to GitHub, Communicate using Markdown, Review pull requests, Resolve merge conflicts, Hello GitHub Actions, Secure your repository's supply chain, Introduction to Repository Management.
- GitHub Desktop — приложение с графическим интерфейсом для Git и GitHub.

## Claude Code

- Set up Claude Code
- Best practices
- Memory (CLAUDE.md)
- Checkpointing
- Permission modes
- Settings
- Worktrees
- Commands
- Claude Code GitHub Actions

## Лицензии и авторское право

- Choose a License, сравнительная таблица, No License
- SPDX License List (версия 3.29.0), SPDX License Expressions, спецификация SPDX 3.0.1
- Open Source Initiative — лицензии, OSI FAQ
- GNU: Various Licenses and Comments about Them, GPL FAQ
- GNU GPL v3, GNU LGPL v3, GNU AGPL v3
- Apache License 2.0, ASF: Licensing howto (NOTICE)
- Mozilla Public License 2.0, MPL 2.0 FAQ
- MIT License, BSD 2-Clause, BSD 3-Clause
- The Unlicense, OSI: The Unlicense
- CC0 1.0
- REUSE, спецификация REUSE 3.3, руководство

- Contributor Covenant (версия 3.0); официальный перевод версии 2.1 на русский
- Citation File Format, cffinit
- Developer Certificate of Origin
- U.S. Copyright Office: Copyright and Artificial Intelligence, Registration Guidance (2023), Part 2: Copyrightability (2025)
- Гражданский кодекс РФ, часть четвёртая — ст. 1228, 1259, 1261, 1265, 1286.1, 1295.
- Urheberrechtsgesetz (Германия), § 29
- Regulation (EU) 2024/2847 — Cyber Resilience Act
- OpenChain Project

## Международные стандарты

Тексты стандартов ISO, как правило, платные. Перечни характеристик и процессов в методичке даны по общедоступным описаниям на iso.org и по вторичным источникам, указанным ниже.

- ISO/IEC/IEEE 12207:2026 — Software life cycle processes (редакция 2017 года отменена: iso.org/standard/63712.html)
- ISO/IEC 25010:2023 — Product quality model
- ISO/IEC 25019:2023 — Quality-in-use model
- ISO/IEC/IEEE 29148:2018 — Requirements engineering
- ISO/IEC 5962:2021 — SPDX Specification V2.2.1
- ISO/IEC 5230:2020 — OpenChain Specification
- ISO/IEC 18974:2023 — OpenChain security assurance specification
- ISO: сертификация — ISO не выдаёт сертификаты.
- Вторичные источники: ANSI blog о редакции 12207:2026, arc42 Quality Model: ISO 25010.

## Практики и спецификации сообщества

- Semantic Versioning 2.0.0
- Keep a Changelog 1.1.0
- Conventional Commits 1.0.0
- commitlint config-conventional
- EditorConfig
- Mermaid

## Безопасность ПО

- OpenSSF Scorecard, репозиторий и список проверок, scorecard-action
- OpenSSF Best Practices Badge
- OpenSSF OSPS Baseline
- SLSA, SLSA v1.2
- OWASP SAMM, OWASP ASVS, ASVS на GitHub, OWASP Top 10:2025

- NIST SP 800-218 (SSDF)
- pip-audit

# Пример README учебного проекта tempconv

Консольная программа для перевода температур между шкалами Цельсия, Фаренгейта и Кельвина.

> **ПРИМЕЧАНИЕ**
> Это учебный пример из методички «Git и GitHub с нуля». Замените `OWNER` на свой логин GitHub, а заполнители в квадратных скобках — на свои данные.

## Содержание


- Зачем это нужно
- Возможности
- Демонстрация
- Как это устроено
- Требования
- Установка
- Быстрый старт
- Примеры использования
- Настройка
- Тестирование
- Документация
- Статус проекта и ограничения
- Как сообщить об ошибке или предложить улучшение
- Лицензия


## Зачем это нужно

Рецепты, инструкции к технике и научные статьи используют разные шкалы температуры. Пересчитывать в уме неудобно, а онлайн-конвертер требует браузера. `tempconv` переводит значение одной командой в терминале и подходит для использования в скриптах.

## Возможности

- Перевод между шкалами C (Цельсий), F (Фаренгейт) и K (Кельвин) в любом направлении.
- Регистр не важен: `c`, `C`, `f`.
- Проверка значений ниже абсолютного нуля.
- Настраиваемое число знаков после запятой.
- Понятные сообщения об ошибках и код выхода `2` при неверном вводе.

- Только стандартная библиотека Python — без внешних зависимостей.

## Демонстрация

```
> tempconv 36.6 C F
36.6 °C = 97.88 °F

> tempconv -40 F C
-40 °F = -40.00 °C

> tempconv 1 X C
Ошибка: неизвестная шкала 'X', допустимо: C, F, K
```

## Как это устроено




Любое значение сначала переводится в градусы Цельсия, затем — в целевую шкалу. Подробнее — в docs/architecture.md.

## Требования

- Python 3.11 или новее (python.org).
- Git — чтобы скачать проект.

## Установка

POWERSHELL

```
git clone https://github.com/OWNER/tempconv.git
cd tempconv
python -m venv .venv
.venv\Scripts\Activate.ps1
python -m pip install -e .
```

`python -m venv .venv` создаёт виртуальное окружение — отдельную папку для библиотек проекта. `Activate.ps1` включает его в текущем окне PowerShell. В Linux и macOS вместо этого выполните `source .venv/bin/activate`.

Если PowerShell запрещает запуск скрипта активации, выполните один раз `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`.

## Быстрый старт

```powershell
tempconv 100 C F
```

```
100 °C = 212.00 °F
```

Без установки, из папки проекта (PowerShell):

```powershell
$env:PYTHONPATH = "src"
python -m tempconv 100 C F
```

## Примеры использования

| Команда | Результат |
|---|---|
| `tempconv 0 C F` | `0 °C = 32.00 °F` |
| `tempconv 451 F C` | `451 °F = 232.78 °C` |
| `tempconv 0 K C` | `0 K = -273.15 °C` |
| `tempconv 25 c k` | `25 °C = 298.15 K` |
| `tempconv --version` | `tempconv 1.0.0` |
| `tempconv --help` | Справка по аргументам |

## Настройка

Программа читает переменные окружения:

| Переменная | Значение по умолчанию | Описание |
|---|---|---|
| `TEMPCONV_PRECISION` | `2` | Число знаков после запятой, от 0 до 6 |

Пример для PowerShell:

```powershell
$env:TEMPCONV_PRECISION = "3"
tempconv 1 C F
```

```
1 °C = 33.800 °F
```

Образец всех переменных — в файле .env.example. Программа сама не читает файл `.env`: задавайте переменные в окружении.

## Тестирование

```powershell
python -m pip install -r requirements-dev.txt
ruff check .
python -m unittest discover -s tests -v
```

Те же проверки автоматически запускаются в GitHub Actions при каждом push в `main` и в каждом Pull Request: .github/workflows/ci.yml.

## Документация

- docs/architecture.md — устройство программы.
- docs/adr/ — записи архитектурных решений.
- CHANGELOG.md — история изменений по версиям.
- CONTRIBUTING.md — как предложить изменение.

## Статус проекта и ограничения

Статус: **стабильный учебный проект**, версия 1.0.0. Номера версий следуют Semantic Versioning.

Ограничения:

- поддерживаются только шкалы C, F и K;
- значения округляются только при выводе; при вычислениях возможна погрешность чисел с плавающей точкой;
- нет графического интерфейса.

## Как сообщить об ошибке или предложить улучшение

- Ошибка или идея: откройте Issue и выберите шаблон.
- Изменение кода: прочитайте CONTRIBUTING.md и откройте Pull Request.
- Уязвимость: **не публикуйте её в Issue** — следуйте SECURITY.md.

## Лицензия

Распространяется по лицензии MIT — см. файл LICENSE.

Copyright (c) [ГОД] [ИМЯ ПРАВООБЛАДАТЕЛЯ].

# Шаблоны документов

Шаблоны из папки `templates/`. Пояснения в них записаны служебными комментариями `<!-- ... -->`, поэтому семь шаблонов приведены как исходный текст. Руководство по выбору лицензии и чек-лист публикации — обычные документы. Как применять шаблоны — в главе 9.

## README.template.md

Файл: `templates/README.template.md`

MARKDOWN — ИСХОДНЫЙ ТЕКСТ

```
<!--
ШАБЛОН README.md

Как использовать:
1. Скопируйте файл в корень репозитория под именем README.md.
2. Замените все заполнители в квадратных скобках: [НАЗВАНИЕ], [ВЛАДЕЛЕЦ] и т. д.
3. Удалите разделы, которые не относятся к проекту (они отмечены «по ситуации»).
4. Удалите служебные комментарии — такие, как этот блок. На GitHub они не видны, но засоряют файл.
5. Проверьте, что каждая команда из README работает, если её скопировать.

Пример заполненного README: examples/tempconv/README.md
Подробности: глава 9, раздел 9.3.
-->

# [НАЗВАНИЕ ПРОЕКТА]

<!-- По ситуации: 2–4 полезных бейджа. Значок CI появится после настройки workflow (глава 11). -->
[![CI](https://github.com/[ВЛАДЕЛЕЦ]/[РЕПОЗИТОРИЙ]/actions/workflows/ci.yml/badge.svg)]
(https://github.com/[ВЛАДЕЛЕЦ]/[РЕПОЗИТОРИЙ]/actions/workflows/ci.yml)

<!-- Одно предложение: что делает программа и для кого. -->
[КРАТКОЕ ОПИСАНИЕ: что делает программа, в одном предложении.]

<!-- По ситуации: содержание нужно, если README длиннее двух-трёх экранов. -->
## Содержание

- [Зачем это нужно](#зачем-это-нужно)
- [Возможности](#возможности)
- [Демонстрация](#демонстрация)
- [Как это устроено](#как-это-устроено)
- [Требования](#требования)
- [Установка](#установка)
- [Быстрый старт](#быстрый-старт)
- [Примеры использования](#примеры-использования)
- [Настройка](#настройка)
- [Тестирование](#тестирование)
- [Документация](#документация)
- [Статус проекта и ограничения](#статус-проекта-и-ограничения)
- [Как сообщить об ошибке или предложить улучшение](#как-сообщить-об-ошибке-или-предложить-
улучшение)
- [Лицензия](#лицензия)

## Зачем это нужно
```

````
<!-- Какую проблему решает проект. Кто пользователи. Чем он лучше ручного способа или альтернатив.
2–4 предложения. -->
[ОПИСАНИЕ ПРОБЛЕМЫ И ПОЛЬЗОВАТЕЛЕЙ.]

## Возможности

<!-- 3–7 пунктов. Только то, что уже работает. -->
- [ВОЗМОЖНОСТЬ 1]
- [ВОЗМОЖНОСТЬ 2]
- [ВОЗМОЖНОСТЬ 3]

## Демонстрация

<!--
По ситуации:
- программа с интерфейсом — скриншот: ![Главный экран](docs/images/screenshot.png)
- процесс, который трудно описать, — короткая GIF-анимация (небольшого размера);
- консольная программа — пример вывода в блоке кода;
- веб-приложение — ссылка на демоверсию, если она работает.
Всегда добавляйте альтернативный текст к изображениям.
-->
```text
[ПРИМЕР КОМАНДЫ И ЕЁ РЕЗУЛЬТАТА]
```

## Как это устроено

<!-- По ситуации: для проектов из нескольких частей. Короткое описание и схема в Mermaid.
Подробности — в docs/architecture.md. -->
```mermaid
flowchart LR
    A["[ВХОД]"] --> B["[КОМПОНЕНТ]"]
    B --> C["[РЕЗУЛЬТАТ]"]
```

## Требования

<!-- Операционные системы, версия языка, внешние сервисы, аккаунты. -->
- [НАПРИМЕР: Python 3.11 или новее]
- [НАПРИМЕР: Git]

## Установка

<!-- Пошаговые команды. Укажите, для какой оболочки они (PowerShell, bash). -->
```powershell
git clone https://github.com/[ВЛАДЕЛЕЦ]/[РЕПОЗИТОРИЙ].git
cd [РЕПОЗИТОРИЙ]
[КОМАНДА УСТАНОВКИ ЗАВИСИМОСТЕЙ]
```

## Быстрый старт

<!-- Минимальный сценарий: одна-две команды и ожидаемый результат. -->
```powershell
[КОМАНДА ЗАПУСКА]
```
````

```text
[ОЖИДАЕМЫЙ РЕЗУЛЬТАТ]
```

## Примеры использования

| Команда или действие | Результат |
| --- | --- |
| `[ПРИМЕР 1]` | [РЕЗУЛЬТАТ] |
| `[ПРИМЕР 2]` | [РЕЗУЛЬТАТ] |

## Настройка

<!-- По ситуации: если программа читает переменные окружения или файлы настроек. Никогда не пишите здесь настоящие ключи. -->
| Переменная | По умолчанию | Описание |
| --- | --- | --- |
| `[ИМЯ_ПЕРЕМЕННОЙ]` | `[ЗНАЧЕНИЕ]` | [ОПИСАНИЕ] |

Образец всех переменных — в файле [.env.example](.env.example). Файл `.env` с настоящими значениями не добавляйте в Git.

## Тестирование

```powershell
[КОМАНДА ЗАПУСКА ТЕСТОВ]
```

<!-- Если проверки запускаются в GitHub Actions, укажите это. -->
Те же проверки запускаются автоматически в GitHub Actions для каждого Pull Request.

## Документация

<!-- По ситуации. -->
- [docs/architecture.md](docs/architecture.md) — устройство программы.
- [CHANGELOG.md](CHANGELOG.md) — история изменений.

## Статус проекта и ограничения

<!-- Честно: стабильность, текущая версия, поддерживается ли проект, чего он не умеет. -->
Статус: **[В РАЗРАБОТКЕ / СТАБИЛЬНЫЙ / НЕ ПОДДЕРЖИВАЕТСЯ]**, версия [X.Y.Z]. Версии следуют [Semantic Versioning](https://semver.org/lang/ru/).

Ограничения:

- [ОГРАНИЧЕНИЕ 1]
- [ОГРАНИЧЕНИЕ 2]

## Как сообщить об ошибке или предложить улучшение

- Ошибка или идея — [Issue](https://github.com/[ВЛАДЕЛЕЦ]/[РЕПОЗИТОРИЙ]/issues/new/choose).
- Изменение кода — прочитайте [CONTRIBUTING.md](CONTRIBUTING.md) и откройте Pull Request.
- Уязвимость — **не публикуйте её в Issue**, следуйте [SECURITY.md](SECURITY.md).

## Лицензия

```
<!-- Укажите лицензию, только если файл LICENSE действительно добавлен. Выбор лицензии — глава 10.
-->
Распространяется по лицензии [НАЗВАНИЕ ЛИЦЕНЗИИ] — см. файл [LICENSE](LICENSE).
```

## CONTRIBUTING.template.md

Файл: `templates/CONTRIBUTING.template.md`

MARKDOWN — ИСХОДНЫЙ ТЕКСТ

```
<!--
ШАБЛОН CONTRIBUTING.md

Как использовать:
1. Скопируйте файл в корень репозитория или в папку .github/ под именем CONTRIBUTING.md.
2. Замените заполнители в квадратных скобках.
3. Опишите только те правила, которые вы действительно соблюдаете и проверяете.
4. Удалите служебные комментарии — такие, как этот блок.

GitHub показывает ссылку на этот файл при создании Issue и Pull Request.
Подробности: глава 9, раздел 9.4. Пример: examples/tempconv/CONTRIBUTING.md
-->

# Как участвовать в проекте [НАЗВАНИЕ ПРОЕКТА]

Спасибо, что хотите помочь! Здесь описано, как предложить изменение так, чтобы его было легко
принять.

## Перед началом

- Ошибку или идею сначала обсудите в [Issue](https://github.com/[ВЛАДЕЛЕЦ]/[РЕПОЗИТОРИЙ]/issues):
возможно, над ней уже работают.
- Небольшие исправления (опечатки, документация) можно присылать сразу Pull Request.
- Об уязвимостях сообщайте не в Issue, а по правилам [SECURITY.md](SECURITY.md).
<!-- По ситуации: если в проекте есть кодекс поведения. -->
- Участвуя в проекте, вы соглашаетесь соблюдать [кодекс поведения](CODE_OF_CONDUCT.md).

## Подготовка окружения

<!-- Точные команды для установки инструментов разработчика. -->
```powershell
git clone https://github.com/[ВЛАДЕЛЕЦ]/[РЕПОЗИТОРИЙ].git
cd [РЕПОЗИТОРИЙ]
[КОМАНДЫ УСТАНОВКИ ЗАВИСИМОСТЕЙ РАЗРАБОТЧИКА]
```

## Как внести изменение

1. Создайте ветку от `main`: `git switch -c [ТИП]/[КРАТКОЕ-ОПИСАНИЕ]`, например `fix/menu-typo`.
2. Внесите изменения. Новое поведение сопровождайте тестами.
3. Запустите проверки:

   ```powershell
   [КОМАНДА ЛИНТЕРА]
   [КОМАНДА ТЕСТОВ]
   ```
```

```
4. Обновите документацию (README) и раздел `[Unreleased]` в `CHANGELOG.md`, если изменение заметно
пользователям.
5. Сделайте коммит. <!-- Выберите формат сообщений; например, Conventional Commits. --> Формат
сообщений: [ФОРМАТ, например Conventional Commits: `fix: ...`, `feat: ...`, `docs: ...`].
6. Отправьте ветку и откройте Pull Request. Заполните шаблон описания.

## Правила оформления кода

<!-- Только реальные правила проекта. Лучше всего — те, что проверяются автоматически. -->
- [ИНСТРУМЕНТ ПРОВЕРКИ СТИЛЯ И ЕГО НАСТРОЙКИ]
- [ОТСТУПЫ, КОДИРОВКА — см. .editorconfig]
- [ПРАВИЛА ДОБАВЛЕНИЯ ЗАВИСИМОСТЕЙ]

## Ревью и слияние

<!-- Кто смотрит изменения, какие проверки обязательны, сколько обычно ждать ответа. -->
Pull Request сливается, когда:

- все автоматические проверки прошли;
- изменения одобрил [КТО: сопровождающий проекта / один из CODEOWNERS].

Обычно ответ занимает [СРОК]. Если ответа долго нет — напомните о себе в комментарии.

## Лицензия вклада

<!--
Укажите, на каких условиях принимаются изменения. Варианты:
- «на условиях лицензии проекта» (по умолчанию на GitHub так и есть, если нет отдельного
соглашения);
- DCO: каждый коммит с подписью `git commit -s` (https://developercertificate.org/);
- CLA: отдельное соглашение — готовится с юристом.
-->
Отправляя изменения, вы подтверждаете, что имеете право их передать, и соглашаетесь, что они
распространяются на условиях лицензии проекта ([LICENSE](LICENSE)).

## Вопросы

[КУДА ЗАДАВАТЬ ВОПРОСЫ: Discussions, Issues, чат.]
```

## SECURITY.template.md

Файл: `templates/SECURITY.template.md`

MARKDOWN — ИСХОДНЫЙ ТЕКСТ

```
<!--
ШАБЛОН SECURITY.md

Как использовать:
1. Скопируйте файл в корень репозитория, в .github/ или docs/ под именем SECURITY.md.
2. Укажите РЕАЛЬНЫЙ закрытый канал для сообщений. Лучший вариант на GitHub —
   включить Private vulnerability reporting (глава 11, раздел 11.11).
3. Не обещайте сроков, которые не сможете соблюдать.
4. Удалите служебные комментарии — такие, как этот блок.

GitHub показывает этот файл в разделе безопасности репозитория.
Подробности: глава 9, раздел 9.4. Пример: examples/tempconv/SECURITY.md
```

```
-->

# Политика безопасности

## Поддерживаемые версии

<!-- Для каких версий выпускаются исправления безопасности. Обновляйте таблицу при каждом крупном
релизе. -->
| Версия | Поддерживается |
| --- | --- |
| [X.Y.x] | Да |
| < [X.0] | Нет |

## Как сообщить об уязвимости

**Не создавайте публичный Issue, Pull Request или обсуждение.** Публичное описание до выхода
исправления помогает злоумышленникам атаковать пользователей.

Сообщите закрыто:

<!-- Оставьте только реально работающие каналы. -->
- через GitHub: раздел безопасности репозитория → **Report a vulnerability**;
- по почте: [АДРЕС ДЛЯ СООБЩЕНИЙ О БЕЗОПАСНОСТИ].

Пожалуйста, укажите:

- версию программы и окружение (ОС, версия языка);
- описание уязвимости и её возможные последствия;
- шаги для воспроизведения или пример;
- известные вам способы обойти проблему.

## Что будет дальше

<!-- Реалистичные сроки. Для небольших проектов честно укажите, что их сопровождает один человек. -
->
1. Подтвердим получение в течение [СРОК].
2. Проверим сообщение и сообщим оценку в течение [СРОК].
3. Подготовим исправление и согласуем с вами дату публикации.
4. После выпуска исправления опубликуем сведения об уязвимости<!-- и, по желанию, укажем вас как
автора находки -->.

## Раскрытие информации

Мы придерживаемся координированного раскрытия: детали уязвимости публикуются после выхода
исправления или по согласованию с автором сообщения.

<!-- По ситуации: условия вознаграждения (bug bounty), область действия политики, контакты для
шифрованной переписки. -->
```

## CHANGELOG.template.md

Файл: `templates/CHANGELOG.template.md`

MARKDOWN — ИСХОДНЫЙ ТЕКСТ

```
<!--
ШАБЛОН CHANGELOG.md

Как использовать:
1. Скопируйте файл в корень репозитория под именем CHANGELOG.md.
2. Записывайте изменения в раздел [Unreleased] в каждом Pull Request.
3. При выпуске версии:
   - переименуйте накопленное в раздел новой версии с датой (ГГГГ-ММ-ДД);
   - оставьте сверху пустой раздел [Unreleased];
   - обновите ссылки внизу файла;
   - поставьте тег vX.Y.Z (глава 9, раздел 9.5).
4. Пишите для пользователей: что изменилось для них, а не какие файлы вы правили.
5. Не включайте пустые подразделы. Удалите служебные комментарии — такие, как этот блок.

Типы изменений (Keep a Changelog 1.1.0):
- Added       — новые возможности                        → обычно MINOR
- Changed     — изменения существующего поведения        → MINOR или MAJOR, если несовместимо
- Deprecated  — то, что скоро удалят
- Removed     — удалённые возможности                    → MAJOR
- Fixed       — исправленные ошибки                      → PATCH
- Security    — исправления уязвимостей

Отозванную версию пометьте: ## [0.0.5] - 2026-01-15 [YANKED]
Пример: examples/tempconv/CHANGELOG.md
-->

# Журнал изменений

Все заметные изменения проекта записываются в этот файл.

Формат основан на [Keep a Changelog](https://keepachangelog.com/ru/1.1.0/), проект следует
[Semantic Versioning](https://semver.org/lang/ru/).

## [Unreleased]

### Added

- [НОВАЯ ВОЗМОЖНОСТЬ]

### Fixed

- [ИСПРАВЛЕННАЯ ОШИБКА]

## [X.Y.Z] - [ГГГГ-ММ-ДД]

### Added

- [ЧТО ДОБАВЛЕНО]

### Changed

- [ЧТО ИЗМЕНИЛОСЬ. Если изменение несовместимо — напишите об этом явно: **Несовместимое
изменение:** ...]

### Deprecated

- [ЧТО БУДЕТ УДАЛЕНО В СЛЕДУЮЩИХ ВЕРСИЯХ И ЧЕМ ЗАМЕНИТЬ]
```

```
### Removed

- [ЧТО УДАЛЕНО]

### Fixed

- [ЧТО ИСПРАВЛЕНО]

### Security

- [КАКАЯ УЯЗВИМОСТЬ ИСПРАВЛЕНА; ссылка на рекомендацию по безопасности]

<!-- Ссылки на сравнение версий. Замените [ВЛАДЕЛЕЦ]/[РЕПОЗИТОРИЙ] и номера версий. -->
[Unreleased]: https://github.com/[ВЛАДЕЛЕЦ]/[РЕПОЗИТОРИЙ]/compare/v[X.Y.Z]...HEAD
[X.Y.Z]: https://github.com/[ВЛАДЕЛЕЦ]/[РЕПОЗИТОРИЙ]/releases/tag/v[X.Y.Z]
```

## CODE_OF_CONDUCT.template.md

Файл: `templates/CODE_OF_CONDUCT.template.md`

MARKDOWN — ИСХОДНЫЙ ТЕКСТ

```
<!--
ШАБЛОН CODE_OF_CONDUCT.md

Как использовать:
1. Решите, нужен ли кодекс поведения. Он нужен открытому проекту с внешними участниками.
   Без человека, который рассматривает жалобы, кодекс не работает.
2. Рекомендуемый путь — взять готовый проверенный текст Contributor Covenant
   (актуальная версия 3.0): https://www.contributor-covenant.org/
   На GitHub: Add file → Create new file → имя CODE_OF_CONDUCT.md →
   Choose a code of conduct template. Сверьте версию шаблона с сайтом Contributor Covenant.
   Текст Contributor Covenant 3.0 распространяется по лицензии CC BY-SA 4.0;
   у версий 2.x условия свои — проверьте их на сайте. Сохраните указание на источник.
3. Если пишете собственный кодекс — используйте структуру ниже.
4. Обязательно укажите реальный контакт для жалоб: [КОНТАКТ].
5. Удалите служебные комментарии — такие, как этот блок.

Подробности: глава 9, раздел 9.4.
-->

# Кодекс поведения участников [НАЗВАНИЕ ПРОЕКТА]

## Наши обязательства

<!-- Зачем нужен кодекс: сделать участие в проекте комфортным и безопасным для всех. -->
Мы, участники и сопровождающие проекта, обязуемся сделать участие в нём доброжелательным и
свободным от притеснений для каждого — независимо от опыта, возраста, внешности, национальности,
пола, религии, убеждений и других личных особенностей.

## Ожидаемое поведение

- Уважительное общение, даже при несогласии.
- Конструктивная критика: обсуждаем идеи и код, а не людей.
- Готовность признать ошибку и извиниться.
- Внимание к тому, что лучше для сообщества.
```

```
## Недопустимое поведение

- Оскорбления, унижающие комментарии, личные нападки.
- Притеснения в публичной или личной переписке.
- Публикация чужих личных данных без разрешения.
- Сексуализированные высказывания и изображения.
- Иное поведение, неуместное в профессиональной среде.

## Область действия

<!-- Где действует кодекс. -->
Кодекс действует во всех пространствах проекта: репозиторий, Issues, Pull Request, обсуждения, чаты
[ПЕРЕЧИСЛИТЕ], а также когда человек официально представляет проект.

## Как сообщить о нарушении

<!-- Реальный и закрытый канал. Укажите, кто рассматривает жалобы. -->
Сообщите о нарушении закрыто: [КОНТАКТ ДЛЯ ЖАЛОБ]. Жалобы рассматривает [КТО]. Мы сохраняем
конфиденциальность автора сообщения.

## Последствия нарушений

<!-- Опишите меры по степени тяжести. -->
1. **Предупреждение** — личное сообщение с объяснением, что было не так.
2. **Временное ограничение** участия.
3. **Постоянный запрет** участия в проекте.

## Источник

<!-- Если использовали готовый текст — укажите его. Например: -->
[Если текст основан на Contributor Covenant, укажите версию и ссылку: https://www.contributor-
covenant.org/]
```

# ISSUE_TEMPLATE.md

Файл: `templates/ISSUE_TEMPLATE.md`

MARKDOWN — ИСХОДНЫЙ ТЕКСТ

```
---
name: Сообщение об ошибке
about: Программа работает не так, как ожидается
title: "[Ошибка] "
labels: bug
---

<!--
ШАБЛОН ISSUE (сообщение об ошибке)

Как использовать:
1. Сохраните файл в репозитории как .github/ISSUE_TEMPLATE/bug_report.md
   (имя файла может быть любым, папка — именно .github/ISSUE_TEMPLATE/).
2. Блок между строками --- вверху — настройки шаблона для GitHub:
   name — название в списке шаблонов, about — пояснение,
   title — начало заголовка, labels — метка (должна существовать в репозитории).
3. Для идей создайте второй файл, например feature_request.md, с другими разделами.
4. Файл .github/ISSUE_TEMPLATE/config.yml со строкой «blank_issues_enabled: false»
```

```
   отключает пустые Issue без шаблона.
5. Альтернатива — формы Issue в формате YAML (.yml); на октябрь 2026 года GitHub
   помечает их как публичную предварительную версию.

Служебные комментарии — такие, как этот блок, — не видны в созданном Issue — их можно оставить как
подсказки.
Пример: examples/tempconv/.github/ISSUE_TEMPLATE/
-->

## Что произошло

<!-- Опишите проблему одним-двумя предложениями. -->

## Как воспроизвести

<!-- Пошагово: что вы сделали, чтобы увидеть ошибку. -->
1.
2.
3.

## Что ожидалось

<!-- Каким должен был быть результат. -->

## Что получилось

<!-- Фактический результат. Сообщение об ошибке вставьте в блок кода. Удалите из него пароли и
личные данные. -->
```text

```

## Окружение

- Версия программы:
- Операционная система:
- [ДРУГИЕ ВАЖНЫЕ ПАРАМЕТРЫ, например версия Python или браузер]:

## Дополнительно

<!-- Скриншоты, логи, ссылки. Уязвимости НЕ описывайте здесь — см. SECURITY.md. -->
```

## PULL_REQUEST_TEMPLATE.md

Файл: `templates/PULL_REQUEST_TEMPLATE.md`

MARKDOWN — ИСХОДНЫЙ ТЕКСТ

```
<!--
ШАБЛОН ОПИСАНИЯ PULL REQUEST

Как использовать:
1. Сохраните файл в репозитории как .github/pull_request_template.md
   (также подходит корень репозитория или папка docs/).
2. GitHub автоматически подставит текст в описание каждого нового Pull Request.
3. Пункты чек-листа замените на реальные правила проекта.
```

```
Служебные комментарии — такие, как этот блок, — в описании PR не видны — это подсказки автору.
Пример: examples/tempconv/.github/pull_request_template.md
-->

## Что сделано

<!-- Кратко: какие изменения внесены. -->

## Зачем

<!-- Какую проблему решает. Если есть Issue — укажите его: «Closes #123» закроет Issue
автоматически после слияния. -->

## Как проверить

<!-- Шаги, по которым ревьюер может убедиться, что всё работает. -->
1.
2.

## Тип изменения

- [ ] Исправление ошибки
- [ ] Новая возможность
- [ ] Несовместимое изменение (ломает существующее поведение)
- [ ] Документация
- [ ] Служебное (CI, зависимости, рефакторинг)

## Чек-лист

- [ ] Изменения проверены локально, тесты проходят
- [ ] Добавлены или обновлены тесты
- [ ] Обновлены README и раздел `[Unreleased]` в CHANGELOG.md, если нужно
- [ ] В изменениях нет паролей, ключей, токенов и личных данных
- [ ] Новые зависимости обоснованы, их лицензии подходят проекту
- [ ] Код, написанный с помощью AI, прочитан и понятен автору PR
```

## LICENSE_SELECTION_GUIDE.md

Файл: `templates/LICENSE_SELECTION_GUIDE.md`

Рабочий лист, который помогает правообладателю выбрать лицензию и оформить сведения о сторонних компонентах. Заполните его для своего проекта и сохраните в `docs/licensing.md` или в описании задачи.

> **ВАЖНО**
>
> Это учебный инструмент, а не юридическая консультация. Он помогает сузить выбор и подготовить вопросы. Решение принимает правообладатель. Для коммерческих продуктов, спорных случаев и вопросов о правах на код обратитесь к юристу. Подробное объяснение — в главе 10.

### Часть 1. Права на код

Ответьте до выбора лицензии. Если хотя бы на один вопрос ответ «нет» или «не знаю» — сначала выясните права.

| Вопрос | Ответ |
|---|---|
| Весь код написан мной (нами)? | [да / нет / не знаю] |
| Код написан не в рамках работы по найму или по заказу — или договор разрешает его публикацию? | [да / нет / не знаю] |
| Кто правообладатель (человек или организация)? | [ИМЯ ПРАВООБЛАДАТЕЛЯ] |
| Есть ли соавторы кода (люди, чей код вошёл в проект)? Согласны ли они с выбором лицензии? | [список / согласие получено] |
| Есть ли в проекте чужой код, скопированный из интернета, книг, ответов AI? Известно ли его происхождение и лицензия? | [описание] |
| Есть ли чужие изображения, шрифты, данные, тексты? | [список — см. часть 4] |

## Часть 2. Цели

Отметьте, что для вас важно.

☐ Чтобы код могли использовать кто угодно и где угодно, включая закрытые коммерческие продукты.

☐ Чтобы изменённые версии распространялись с исходным кодом на тех же условиях.

☐ Чтобы это требование действовало и для сетевых сервисов на основе моей программы.

☐ Чтобы закрытые программы могли подключать мою библиотеку, но изменения самой библиотеки оставались открытыми.

☐ Явная защита от патентных претензий участников.

☐ Максимально короткий и простой текст лицензии.

☐ Совместимость с GPL.

☐ Не разрешать использование кода другими.

## Часть 3. Сопоставление целей и вариантов

| Если важно | Изучите | Обратите внимание |
|---|---|---|
| Широкое использование, минимум условий, простота | `MIT`, `BSD-2-Clause` | Нет явной патентной лицензии |
| То же плюс защита имени автора от использования в рекламе | `BSD-3-Clause` | — |
| Широкое использование и явная патентная лицензия | `Apache-2.0` | Пометки об изменениях, файл `NOTICE`; несовместима с GPL-2.0-only |
| Изменённые версии программы должны оставаться открытыми | `GPL-3.0-only` или `GPL-3.0-or-later` | Выберите «только 3.0» или «3.0 и новее» |
| То же для сетевых сервисов | `AGPL-3.0-only` или `AGPL-3.0-or-later` | Многие компании избегают AGPL-зависимостей |
| Библиотека для закрытых программ, изменения библиотеки — открыты | `LGPL-3.0-only` / `-or-later` или `MPL-2.0` | LGPL — на уровне библиотеки, MPL — на уровне файлов |
| Отказаться от прав, насколько возможно | `Unlicense` | Отказ от прав действует не во всех странах; MIT предсказуемее |

| Если важно | Изучите | Обратите внимание |
|---|---|---|
| Не разрешать использование | Открытая лицензия не подходит | Приватный репозиторий; коммерческое соглашение — с юристом |

Выбранная лицензия: **[SPDX-ИДЕНТИФИКАТОР]**

Причина выбора: [ОБОСНОВАНИЕ В 1–3 ПРЕДЛОЖЕНИЯХ]

### Часть 4. Сторонние компоненты и совместимость

Перечислите всё чужое, что входит в проект или в распространяемый продукт.

| Компонент | Тип (библиотека, шрифт, изображение, данные, код) | Версия | Лицензия (SPDX) | Источник (ссылка) | Как используется (подключается / встроен в репозиторий / в сборке) | Совместима с выбранной лицензией? |
|---|---|---|---|---|---|---|
| [НАЗВАНИЕ] | [ТИП] | [ВЕРСИЯ] | [ЛИЦЕНЗИЯ] | [ССЫЛКА] | [СПОСОБ] | [да / нет / уточнить] |

Проверьте:

☐ Лицензия каждой зависимости найдена в её репозитории или реестре пакетов, а не только в автоматическом отчёте.

☐ Нет GPL/AGPL-компонентов, если проект распространяется под разрешительной лицензией или как закрытый продукт (или их использование согласовано с юристом).

☐ Нет материалов с запретом коммерческого использования (например, CC BY-NC) в коммерческом продукте.

☐ Для компонентов, которые требуют сохранять уведомления, уведомления сохранены.

☐ Проверка зависимостей автоматизирована, например `actions/dependency-review-action` с параметром `allow-licenses` (список разрешённых лицензий) (глава 11).

### Часть 5. Оформление

☐ Файл `LICENSE` с официальным текстом в корне репозитория.

☐ Указаны год и правообладатель, если этого требует лицензия: `Copyright (c) [ГОД] [ИМЯ ПРАВООБЛАДАТЕЛЯ]`.

☐ Раздел «Лицензия» в README.

☐ SPDX-идентификатор в манифесте пакета (`pyproject.toml`, `package.json` и т. п.).

☐ По желанию — заголовки `SPDX-License-Identifier` в файлах (практика REUSE).

☐ `NOTICE` — если выбрана Apache-2.0 и есть обязательные атрибуции, или если вы распространяете компонент под Apache-2.0 с файлом `NOTICE`.

☐ `THIRD_PARTY_NOTICES.md` — если в продукт входят сторонние компоненты (шаблон ниже).

☐ Условия приёма вклада описаны в `CONTRIBUTING.md` (лицензия проекта, DCO или CLA).

### Шаблон THIRD_PARTY_NOTICES.md

```markdown
# Сторонние компоненты

[НАЗВАНИЕ ПРОЕКТА] включает следующие сторонние компоненты.
Их лицензии и уведомления приведены ниже.

## [НАЗВАНИЕ КОМПОНЕНТА] [ВЕРСИЯ]

- Источник: [ССЫЛКА]
- Лицензия: [SPDX-ИДЕНТИФИКАТОР]
- Где используется: [ПУТЬ В РЕПОЗИТОРИИ ИЛИ ЧАСТЬ ПРОДУКТА]

[ПОЛНЫЙ ТЕКСТ УВЕДОМЛЕНИЯ ОБ АВТОРСКИХ ПРАВАХ И ЛИЦЕНЗИИ КОМПОНЕНТА]
```

### Официальные тексты лицензий

| Лицензия | Текст |
|---|---|
| MIT | https://opensource.org/license/mit |
| Apache-2.0 | https://www.apache.org/licenses/LICENSE-2.0 |
| BSD-2-Clause | https://opensource.org/license/bsd-2-clause |
| BSD-3-Clause | https://opensource.org/license/bsd-3-clause |
| GPL-3.0 | https://www.gnu.org/licenses/gpl-3.0.html |
| LGPL-3.0 | https://www.gnu.org/licenses/lgpl-3.0.html |
| AGPL-3.0 | https://www.gnu.org/licenses/agpl-3.0.html |
| MPL-2.0 | https://www.mozilla.org/en-US/MPL/2.0/ |
| Unlicense | https://unlicense.org/ |
| Список SPDX | https://spdx.org/licenses/ |
| Помощник выбора | https://choosealicense.com/ |

## PROJECT_RELEASE_CHECKLIST.md

Файл: `templates/PROJECT_RELEASE_CHECKLIST.md`

Пройдите чек-лист перед тем, как сделать репозиторий публичным или выпустить версию. Уровни накопительные: профессиональный уровень включает базовый, open-source — оба предыдущих.

Как отмечать:

- `[x]` — выполнено;
- `[ ]` — не выполнено;
- если пункт осознанно пропущен — оставьте `[ ]` и допишите причину: `— не нужно: нет зависимостей`.

Подробности — в главах 9, 10 и 11.

Проект: **[НАЗВАНИЕ]** · Версия: **[X.Y.Z]** · Дата проверки: **[ГГГГ-ММ-ДД]** · Проверил: **[ИМЯ]**

## Уровень 1. Базовый — небольшой личный проект

### Содержимое

☐ `README.md`: что это, как установить, как запустить.

☐ Команды из README работают, если их скопировать.

☐ `.gitignore` подходит языку и инструментам проекта.

☐ В репозитории нет временных и служебных файлов (`__pycache__`, `node_modules`, `.DS_Store`, логи).

☐ Нет больших двоичных файлов без необходимости (файлы больше 100 МБ GitHub не принимает).

### Безопасность

☐ В коде **и в истории** нет паролей, токенов, ключей, личных данных (`git log -p`, поиск по словам `password`, `token`, `secret`, `key`).

☐ Файл `.env` в `.gitignore`; при наличии настроек есть `.env.example`.

☐ В аккаунте GitHub включена двухфакторная аутентификация.

### Права и лицензия

☐ Вы имеете право публиковать весь код (нет кода работодателя или заказчика без разрешения).

☐ Для публичного кода выбрана лицензия и добавлен файл `LICENSE` с годом и правообладателем, если требуется.

### История

☐ Сообщения коммитов понятны.

☐ `git status` чистый, всё отправлено на GitHub.

## Уровень 2. Профессиональный — документация, тесты, качество, сопровождение

### Документация

☐ README содержит требования, настройку, тестирование, статус и ограничения.

☐ `CHANGELOG.md` по Keep a Changelog; раздел версии заполнен.

☐ `CONTRIBUTING.md` описывает подготовку окружения, проверки и порядок Pull Request.

☐ Архитектура и важные решения описаны в `docs/` (если проект не тривиальный).

### Качество

☐ Есть автоматические тесты; они проходят.

☐ Линтер и (по желанию) форматтер настроены, версии закреплены.

☐ Workflow CI запускается в каждом Pull Request и при push в `main`.

☐ `.editorconfig` и `.gitattributes` добавлены.

### Процесс

☐ Ветка `main` защищена: запрещены удаление и force push; обязательны Pull Request и успешные проверки.

☐ Шаблон Pull Request с чек-листом (для командной работы).

☐ Версия соответствует Semantic Versioning; поставлен аннотированный тег `vX.Y.Z`.

☐ Релиз на GitHub с заметками о выпуске.

#### Безопасность

☐ `SECURITY.md` с реальным закрытым каналом связи.

☐ Dependabot alerts включены; `.github/dependabot.yml` настроен.

☐ В workflow указано `permissions: contents: read`; сторонние действия закреплены полным хешем коммита.

☐ Секреты CI хранятся в **Settings → Secrets and variables**, а не в файлах.

☐ Лицензии зависимостей проверены и совместимы с лицензией проекта.

### Уровень 3. Публичный open-source — сообщество, безопасность, лицензирование, релизы

#### Сообщество

☐ `CODE_OF_CONDUCT.md` с реальным контактом для жалоб и человеком, который их рассматривает.

☐ Шаблоны Issue ( `.github/ISSUE_TEMPLATE/` ) и `config.yml`.

☐ В `CONTRIBUTING.md` указаны условия приёма вклада (лицензия проекта, DCO или CLA).

☐ Описание и темы (*topics*) репозитория заполнены.

#### Безопасность

☐ Включены закрытые сообщения об уязвимостях (Private vulnerability reporting).

☐ Secret scanning и push protection включены.

☐ CodeQL (default setup) включён, если язык поддерживается.

☐ Dependency review в Pull Request (где доступен).

☐ Проверена оценка OpenSSF Scorecard; критичные замечания исправлены или осознанно приняты.

☐ У сопровождающих включена 2FA; доступ есть только у тех, кому он нужен.

#### Лицензирование

☐ SPDX-идентификатор лицензии в манифесте пакета.

☐ `THIRD_PARTY_NOTICES.md` или `NOTICE` для встроенных сторонних компонентов.

☐ По желанию — сведения о лицензии в каждом файле, проверка `reuse lint`.

☐ Для научного ПО — `CITATION.cff`.

#### Релизы

☐ Релизы собираются в CI, а не на личном компьютере.

☐ Заметки о выпуске перечисляют несовместимые изменения и исправления безопасности.

☐ По желанию — аттестации происхождения сборки (SLSA, GitHub Artifact Attestations).

☐ Указано, какие версии поддерживаются ( `SECURITY.md` ).

## Итог

| Уровень | Выполнено | Пропущено осознанно | Не выполнено |
|---|---|---|---|
| Базовый | [ЧИСЛО] | [ЧИСЛО] | [ЧИСЛО] |
| Профессиональный | [ЧИСЛО] | [ЧИСЛО] | [ЧИСЛО] |
| Open-source | [ЧИСЛО] | [ЧИСЛО] | [ЧИСЛО] |

Решение о публикации: **[ПУБЛИКУЕМ / ДОРАБАТЫВАЕМ]**

> **ПРИМЕЧАНИЕ**
>
> Чек-лист — рекомендации методички, а не требования стандарта. Выполнение всех пунктов не гарантирует отсутствия ошибок и уязвимостей и не означает соответствия какому-либо стандарту или сертификации.